

Ein anderes Problem ist die Eindeutigkeit der Namensreferenz, wenn Dateien aus Dateisystemen, die lange Dateinamen erlauben, in Dateisystemen verwendet werden sollen, die nur kurze Namen erlauben. Beispielsweise sind die Namen „MeineAllerKleinsteDatei“ und „MeineAllerGrößteDatei“ in einem Dateisystem, das nur die ersten 8 Buchstaben beachtet, völlig identisch und damit ununterscheidbar. Hier muss eine Methode gefunden werden, die eine eindeutige Zuordnung erlaubt.

#### Beispiel Windows NT Namenskonversion

Da Windows NT auch MS-DOS-Dateisysteme neben dem NT-Dateisystem NTFS verwalten kann, stellt sich hier das Problem, NTFS-Dateien mit max. 215 Zeichen langen Namen in eindeutige MS-DOS-Namen mit max. 12 Zeichen zu konvertieren. Dies wird mit folgenden Algorithmus erreicht:

- (1) Alle Buchstaben, die illegal in MS-DOS sind, werden aus dem NTFS-Namen entfernt, also z. B. Leerzeichen, 16-Bit-Unicodezeichen usw., sowie vorangestellte und abschließende Punkte. Alle Punkte innerhalb des Namens, bis auf den letzten, werden ebenfalls entfernt. Gleich, Semikolon und Komma werden zum Unterstrich. Alle Kleinbuchstaben werden in Großbuchstaben umgewandelt.
- (2) Von der Zeichenkette vor dem Punkt werden alle bis auf die ersten 6 Buchstaben entfernt und dann die Zeichen „~1“ vor dem Punkt eingefügt. Die Zeichenkette nach dem Punkt wird nach 3 Zeichen abgeschnitten und die gesamte Zeichenkette zu Großbuchstaben konvertiert.
- (3) Gibt es eine Datei, die den gleichen Dateinamen aufweist, so werden statt „~1“ die Zeichen „~2“ eingefügt. Existiert diese Datei auch schon, so wird „~3“ usw. gewählt, bis der Name noch nicht existiert.

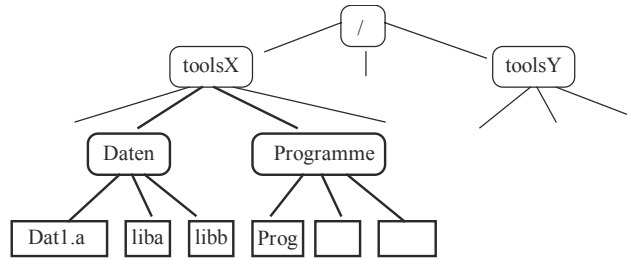
### 4.2.2 Pfadnamen

Durch die hierarchische Ordnung der Dateien ist eine eindeutige Namensgebung möglich: Es reicht, von der Wurzel des Baumes alle Knoten aufzulisten, die auf dem Pfad zu der Datei durchlaufen werden. Der **Pfadname** einer Datei wird aus der Kette der Knotennamen gebildet und enthält als Trennungszeichen zwischen den Knotennamen meist ein spezielles Zeichen, das typisch ist für das Betriebssystem. Beispielsweise ist in Abb. 4.3 für *Brief1.doc* der UNIX-Pfadname *Abteilung5/Rudi/Brief1.doc*, und in Windows NT *Abteilung5\Rudi\Brief1.doc*. Dabei sind durch Querverbindungen durchaus mehrere Namen für dieselbe Datei möglich: *Abteilung5/Rudi/BriefVorlage.dot* ist identisch mit *Firma/Formulare/BriefVorlage.dot*.

Der oberste Knoten im Dateisystem, die Wurzel, wird meist mit einem besonderen Symbol, beispielsweise „/“, bezeichnet. In UNIX ist dies ein einfaches „/“, in Windows NT ist dies „\“.

Um auf dem Pfad nicht nur abwärts in Richtung der Blätter abzustiegen, gibt es meist auch in jedem Verzeichnis ein spezielles Verzeichnis mit dem Namen „...“, das die Bedeutung „darüber liegendes Verzeichnis“ hat und in Pfadnamen mit dieser Bedeutung

**Abb. 4.4** Gültigkeit relativer Pfadnamen



verwendet werden kann. Bei der Interpretation des Dateinamens wird es als Schritt nach oben in Richtung der Dateiwurzel ausgeführt. Ein weiteres Zeichen „.“ bedeutet „dieses Verzeichnis hier“ und ist gleichbedeutend mit dem Verzeichnis, in dem man sich befindet. Beispielsweise lässt sich in [Abb. 4.3](#) die Datei *Brief1.doc* bei *Rudi* auch von *Hans* aus mit dem Pfadnamen *./Rudi/Brief1.doc* ansprechen. Ein solcher **relativer Pfadname** erlaubt es, ganze Dateibäume von Werkzeugen (Programmen) an verschiedene Stellen im Dateisystem zu verschieben, ohne dass dabei die Pfadnamen, die in den Programmen verwendet werden, geändert werden müssen. Beispielsweise ist der relative Pfadname „./Daten/Dat1.a“ der Datei „Dat1.a“ in [Abb. 4.4](#) für das Programm „Prog“ aus dem Ordner „Programme“ immer richtig – egal, ob sich der gesamte Dateibaum unter „toolsX“ oder „toolsY“ befindet.

### 4.2.3 Beispiel UNIX: Der Namensraum

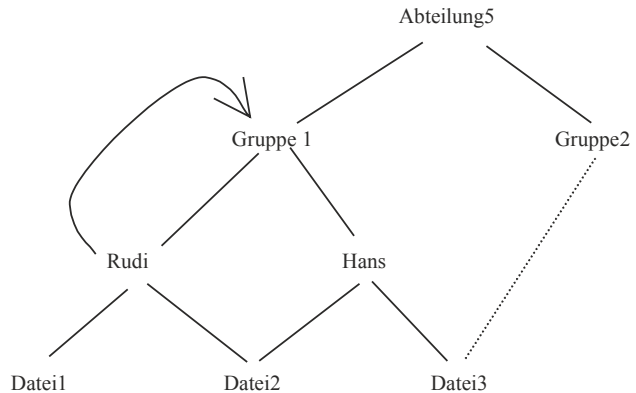
In UNIX wird ein hierarchisches Dateisystem verwendet, ähnlich wie in [Abb. 4.2](#) gezeigt, und danach der Namensraum gebildet. Zusätzlich zu der Baumstruktur kann man aber auch Querverbindungen (*links*) zwischen den Dateien mit dem „ln“-Kommando herstellen.

Für das Verhalten der Namenshierarchie muss man unterscheiden, ob die Verzeichnisse mit den Dateien auf demselben Laufwerk untergebracht sind oder nicht. Sind sie es, so ist eine direkte Querverbindung möglich (*physical link*, **hard link**). Sind sie dagegen auf unterschiedlichen Laufwerken untergebracht, so kann man nur eine logische Querverbindung (**symbolic link**) vornehmen, bei der unter dem Dateinamen im Verzeichnis auf die Originaldatei, repräsentiert mit ihrem Pfadnamen, verwiesen wird.

Der Unterschied in den Querverbindungen macht sich bemerkbar, wenn mehrere Prozesse oder Benutzer auf eine Datei bzw. ein Verzeichnis zugreifen, um es zu löschen. Betrachten wir dazu das Dateisystem in [Abb. 4.5](#), gezeichnet ohne die Unterscheidung der Dateinamen durch ovale und viereckige Formen.

Seien die Verzeichnisse „Gruppe1“ und „Gruppe2“ auf verschiedenen Massenspeichern (Laufwerken) untergebracht, so ist im Unterschied zur Querverbindung *Rudi/Datei2* zu *Hans/Datei2*, die dieselbe Datei bezeichnen (*hard link*), die Querverbindung von *Gruppe2* zu *Datei3* (gestrichelt in [Abb. 4.5](#)) nur als eine logische Verbindung (*symbolic link*) mit der Namensreferenz *./Gruppe1/Hans/Datei3* möglich.

**Abb. 4.5** Querverbindungen und Probleme in UNIX



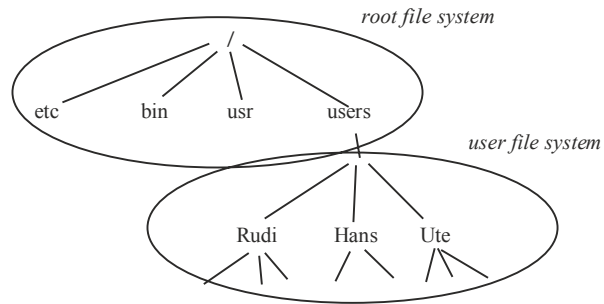
Wird nun in UNIX eine Datei gelöscht, so bleibt sie so lange erhalten, wie mindestens ein *physical link* darauf existiert. Wird also in unserem Beispiel die Datei *Rudi/Datei2* gelöscht, so bleibt sie noch unter dem Namen *Hans/Datei2* erhalten und kann verwendet werden. Wird dagegen *Hans/Datei3* gelöscht, so ist die entsprechende Datei auch tatsächlich völlig gelöscht: Die Namensreferenz (*symbolic link*) im Verzeichnis *Gruppe2* ist zwar noch vorhanden, aber beim Zugriff auf die Datei über diese Namensreferenz erfolgt sofort eine Fehlermeldung: Eine Datei mit diesem Namen gibt es nicht mehr.

Die Entscheidung im UNIX-Betriebssystem, eine Datei vollständig zu löschen oder nicht, hängt also von einem Zähler, der Zahl der *hard link*-Referenzen, ab. Dieser UNIX-Mechanismus ist zwar in einer Multiprozessumgebung ein einfaches Mittel, kann aber auch zu Fehlern führen. Nehmen wir an, wir befinden uns in [Abb. 4.5](#) auf Verzeichnis *Rudi* und erstellen einen *hard link* auf *Gruppe1*. Aus dem azyklischen Graphen wird so ein zyklischer Graph. Danach gehen wir nach ganz oben im Dateisystem, auf die Wurzel *Abteilung5*. Da das Löschen von Verzeichnissen, die nicht leer sind, verboten ist (um Fehler zu verhindern), können wir *Gruppe2* nicht löschen. Anders dagegen bei *Gruppe1*: Hier gibt es noch eine zweite Verbindung (vom Verzeichnis *Rudi*), so dass es zulässig ist, eine davon zu löschen. Dies führt allerdings zu der Situation, dass nun ein Verzeichnis *Gruppe1* mit Unterverzeichnissen und Dateien existiert, das nicht mehr benutzt werden kann, Platz wegnimmt und trotzdem nicht gelöscht werden kann. Dies lässt sich nicht verhindern: Zwar sind Querverbindungen möglich, aber die Prüfung auf Zyklen und Zerlegung des Dateigraphen bei jeder Löschoperation ist zu aufwendig und wird deshalb nicht gemacht. Aus diesem Grund sind in neueren UNIX-Versionen zu Verzeichnissen nur noch *symbolic links* möglich.

Für das Erzeugen und Löschen von Dateien gibt es in UNIX übrigens einen ähnlichen Mechanismus: Erzeugt ein Prozess eine Datei und öffnen dann andere Prozesse diese Datei, so verschwindet sie nicht beim Löschen durch den Erzeuger. Sie existiert so lange, bis der letzte Prozess die Prozedur `close()` aufruft und damit der Referenzzähler null wird.

Das gesamte hierarchische Dateisystem zerfällt normalerweise in verschiedene Teilgraphen, die auf unterschiedlichen Laufwerken ihre Dateien haben. Beim Starten des Systems (*bootstrap*) wird das Dateisystem des Laufwerks als Grundsystem geladen. Aufbauend auf

**Abb. 4.6** Die Erweiterung des Dateisystems unter UNIX



den Wurzelknoten (*root*) kann man nun weitere Dateisysteme anderer Laufwerke einhängen (Systemaufruf `mount()`), wobei der Wurzelknoten jedes Unterdateisystems auf den Namen des angegebenen Verzeichnisses (*mount point directory*) abgebildet wird. In Abb. 4.6 ist ein solcher zusammengesetzter Dateibaum gezeigt.

#### 4.2.4 Beispiel Windows NT: Der Namensraum

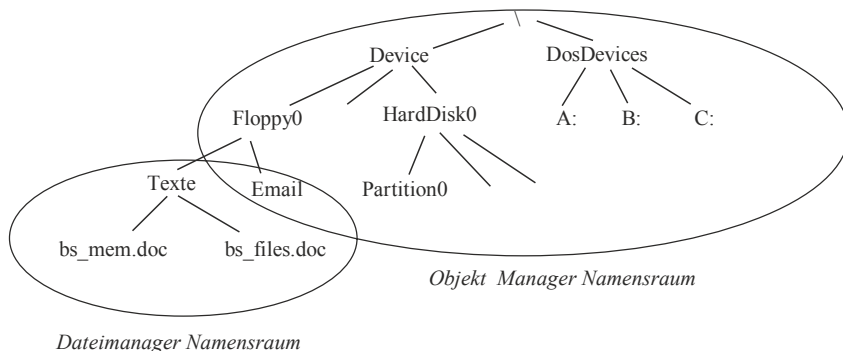
Der Namensraum in Windows NT umfasst alle globalen Objekte, unabhängig davon, ob es reine Dateien (*files*) oder auch andere Objekte wie Kommunikationskanäle (*named pipes*), Speicherbereiche (*shared memory*), Semaphore, Ereignisse oder Prozesse sind. Alle Objekte mit einem globalen Namen sind dabei den gleichen Schutzmechanismen für den Zugriff unterworfen.

Das hierarchische Namenssystem beginnt mit dem Wurzelknoten „\“, in dem Objekte (Dateien etc.) oder Objektverzeichnisse existieren. Objektverzeichnisse sind Objekte, die als Attribute (Variablen und Konstanten) die Namen der Objekte enthalten; als Methoden sind Erzeugen, Öffnen und Abfragen des Verzeichnisses möglich. Dies ist nicht nur im *kernel mode*, sondern auch im *user mode* möglich, so dass nicht nur der Betriebssystemkern, sondern auch das OS/2 und POSIX-Subsystem Verzeichnisse anlegen können. Für jede Objektart (Datei, Prozess etc.) gibt es spezielle Versionen der drei Methoden, die von unterschiedlichen Kernmodulen (I/O-Manager, Prozessmanager usw.) zur Verfügung gestellt werden.

Dabei ist es sowohl möglich, den globalen Namensraum aus unterschiedlichen Objektverzeichnissen aufzubauen, als auch (wie in UNIX) mit logischen Namen Querverbindungen zu schaffen.

##### Beispiel Namensräume in Windows NT

Im Namensraum des Objektmanagers existieren verschiedene Objekte, siehe Abb. 4.7. Eines davon, Objekt „A:“, ist eine logische Querverbindung (*symbolic link object*) zu dem Dateisystemobjekt „Floppy0“. Mit jedem Objekt ist auch eine „Durchsuchen“-Methode spezifiziert. Möchte nun der Editor die Datei `A:\Texte\bs_files.doc` öffnen, so fragt er beim Objektmanager nach diesem Objekt. Der Objektmanager durchsucht nun seinen



**Abb. 4.7** Ergänzung der Namensräume in Windows NT

Namensraum, wobei jedes Objekt mit seiner „Durchsuchen“-Methode untersucht wird. Kommt er nun zu „A:“, so wird die spezielle *symbolic-link-parsing*-Methode (s. [Abb. 4.7](#)) verwendet:

Die Zeichenkette „A:“ wird ersetzt durch die Zeichenkette „\Device\Floppy0“ und dann dem Objektmanager übergeben. Dieser arbeitet den Pfadnamen erneut ab, bis er auf das Dateiojekt „Floppy0“ stößt. Die „Durchsuchen“-Methode hierfür arbeitet nun auf dem Namensraum des Dateisystem-Managers und durchsucht den Pfad mit speziellen, auf das Dateisystem der Floppy zurechtgeschnittenen Prozeduren, bis schließlich ein Dateiojekt für „bs\_files.doc“ zurückgegeben werden kann.

Mit diesem Mechanismus ist es in Windows NT möglich, so unterschiedliche Dateisysteme wie das MS-DOS FAT-System, das *High-Performance-File-System* HPFS von OS/2 und das *NT-File-System* NTFS einheitlich zu integrieren. Auch Spezialsysteme wie z. B. das CD-ROM Dateisystem oder das verschlüsselte Dateisystem Encrypting File System EFS aus Windows 2000 lassen sich so elegant integrieren.

Für das Löschen einer Datei, also eines Objekts, das von mehreren Prozessen benutzt wird, gelten ähnliche Mechanismen wie bei UNIX. Bedingt durch die Implementierung gibt es allerdings zwei Referenzzähler: einen Zähler für die Anzahl der *object handles*, die an Benutzerprozesse ausgegeben wurden, und einen für die Anzahl der Zeiger, die an das Betriebssystem anstelle von Objekten für einen schnellen Zugriff vergeben wurden. Sinkt der *user*-Referenzzähler für die Namensreferenzen auf null, so wird das Objekt im Namensraum gelöscht, so dass kein Prozess mehr darauf zugreifen kann. Allerdings bleibt es so lange noch im Speicher erhalten, bis auch der Zähler für die Referenzen des Betriebssystems ebenfalls auf null gesunken ist. Erst dann wird der Speicher freigegeben und wiederverwendet. Mit diesem Mechanismus wird verhindert, dass ein Objekt, das von mehreren Prozessen bearbeitet wird, aus Versehen von einem gelöscht wird, bevor die anderen damit ebenfalls fertig sind.

Der doppelte Referenzzähler verhindert auch, dass beispielsweise ein Prozessobjekt von einem Prozess erzeugt wird und dann mit dem Terminieren des Erzeugerprozesses

zu Ende geht, obwohl noch ein Kindsprozess davon einen Bezug zum Objekt hat. Die noch erhaltene Referenz des Betriebssystems zum ablaufenden zweiten Prozess sichert die Existenz des Dateiobjekts bis zum expliziten Ableben des letzten Bezugs.

Probleme mit rückbezüglichen Verzeichnisstrukturen werden dadurch vermieden, dass alle Querverbindungen in Verzeichnissen nur über *symbolic links* hergestellt werden. Die Ungültigkeit von Querverbindungen zu gelöschten Objekten wird erst bemerkt, wenn sie referiert werden und müssen vom aufrufenden Programm behandelt werden.

### 4.2.5 Aufgaben

#### Aufgabe 4.2-1 Isoliertes Verzeichnis

Angenommen, ein Verzeichnis „Test“ sei über */root/ ...* zu erreichen. Legen Sie ein Unterverzeichnis darin an. Legen Sie einen *hard link* auf das übergeordnete Verzeichnis an. Es existiert nun ein zirkulärer *link*. Versuchen Sie jetzt, das Verzeichnis *Test* aus dem *root*-Pfad zu löschen. Was erreichen Sie?

#### Aufgabe 4.2-2 Namenskonversion

Im vorangehenden Beispiel zur Namenskonversion wurde der Algorithmus für Windows NT beschrieben, um einen langen Namen in einen kurzen, eindeutigen Dateinamen zu konvertieren.

- Wie viele lange Dateinamen, die in den ersten 6 Buchstaben übereinstimmen, lassen sich so eindeutig mit nur 8 Buchstaben kodieren?
- Wie könnte man das Schema abändern, um mit 8 ASCII-Buchstaben eine größere Anzahl langer Dateinamen eindeutig zu kodieren? Warum, meinen Sie, haben die Implementatoren diese Möglichkeit nicht gewählt?

#### Aufgabe 4.2-3 Extension und Inhalt

ASCII – Dateien haben die gemeinsame Eigenschaft, dass sie lesbaren Text enthalten. Warum kann bei diesen Dateien die Dateinamenserweiterung (Extension) wichtig sein? Wann kann bei Dateien darauf verzichtet werden?

#### Aufgabe 4.2-4 Pfadnamen

- Was sind die Vor- und Nachteile von relativen gegenüber absoluten Pfadnamen? Wo/wann sollte man welchen Typ einsetzen? Vollziehen Sie dies z. B. anhand eines Compilersystems nach!
- Wie viele Platten-Leseoperationen werden benötigt, um das Änderungsdatum von einer Datei */home/os/source/aufgabe6/musterlösung.pdf* auszulesen?

Nehmen sie an, dass der Indexknoten für das Wurzelverzeichnis im Speicher liegt, sich jedoch keine weiteren Elemente entlang des Pfades im Speicher befinden. Nehmen sie außerdem an, dass alle Verzeichnisse jeweils in gesonderten Plattenblöcken gespeichert sind.

---

## 4.3 Dateiattribute und Sicherheitsmechanismen

In dem Verzeichnis, in dem der Name einer Datei steht, wird meist auch zusätzliche Information über die Datei aufbewahrt. Dies sind neben der Länge der Datei (in Bytes oder „Blöcken“ (Seiten)), ihrem Erzeugungs- und Modifikationsdatum auch verschiedene Attribute, die logische Werte („Flags“), beispielsweise „verborgen“ (*hidden*) oder „Systemdatei“ (*system*), sein können, wie der Erzeuger und der jetzige Besitzer der Datei.

Eine besondere Art der Statusinformation sind die sicherheitsrelevanten Angaben wie die Zugriffsrechte von Personen und Programmen auf die Datei. Ähnlich wie bei den Basismechanismen des Speicherschutzes versucht man, durch gezielte Maßnahmen die Fehlfunktionen von Programmen sowie bewusste Angriffe von Benutzern auszuschließen. Die vom POSIX-6-Komitee vorgeschlagenen Maßnahmen beinhalten:

- das Prinzip, für die Erfüllung einer Aufgabe nur die geringstmöglichen Rechte einzuräumen (**least privilege**)
- die Zugriffskontrolle durch diskrete Angaben zu ergänzen (**discretionary access control**), also z. B. durch Zugangs- oder Kontrolllisten (**Access Control Lists ACL**), in denen alle Leute enthalten sind, die Zugriff auf diese Datei haben, sowie ihre genau spezifizierten Rechte. Dies sind zusätzliche Spezifikationen der normalen Zugriffsrechte.
- eine verbindliche Zugangskontrolle, unabhängig vom Erzeuger (**mandatory access control**). Der Zugang zu dem Objekt sollte nur von Prozessen mit größeren Rechten erfolgen. Leider wurde dies 1997 zurückgezogen.
- die Aufzeichnungen von Zuständen des Objekts (**audit trail**), um bei missbräuchlicher Verwendung die Ursachen und Personen herausfinden zu können.

Demgegenüber implementieren Betriebssysteme nur sehr beschränkt die obigen Forderungen.

### 4.3.1 Beispiel UNIX: Zugriffsrechte

In UNIX gibt es drei verschiedene Zustandsvariablen (Flags): *Lesen* (R), *Schreiben* (W) und *Ausführen* (X). Alle Dateien, auch die Verzeichnisse, haben einen solchen Status. Bei Verzeichnissen bedeutet

r = Lesen der Dateiliste erlaubt

w = Ändern der Dateiliste, z. B. durch Kopieren einer Datei „an diese Stelle“, ist möglich

x = Durchsuchen der Dateiliste erlaubt.

Bei den Zugriffsberechtigten unterscheidet UNIX drei Kategorien: Eigentümer (*owner*), Gruppenmitglied (*user group member*) und alle anderen (*others*).

#### Beispiel Anzeige der Zugriffsrechte

Ein „normales“ Kommando `ls` zeigt alle Dateien und ihre Zugriffsrechte außer denen, die mit dem Zeichen „.“ beginnen. Diese sind damit verborgen. Lässt man sich mit dem Kommando `ls -al` alle Dateien eines Verzeichnisses anzeigen, so erhält man beispielsweise den Ausdruck

```
drwxr-xr-x  brause  512    Apr23  15:55  .
drwxr-xr-x  off     512    May17  17:53  ..
-rw-r--r--  brause 44181  Apr23  15:56  data1.txt
```

In der ersten Spalte sind die Zugriffsrechte aufgeführt. Nach dem ersten Zeichen, das die Art der Datei aufführt (`d` = *directory*, `l` = *link*, `-` = einfache Datei), sind für die drei Kategorien von Benutzern *owner*, *group*, *others* jeweils die drei Zugriffsrechte aufgeführt, wobei für „Recht nicht gewährt“ ein „-“ notiert ist. Die übrigen Spalten zeigen den Benutzernamen, Dateigröße, Erzeugungsdatum sowie den Dateinamen an. Beispielsweise besagt der erste Eintrag, dass eine Datei namens „.“, also das aktuelle Verzeichnis, dem Benutzer „brause“ gehört und nur von ihm beschrieben werden kann. Es ist 512 Bytes lang und am 23. April um 15 Uhr 55 erzeugt worden. Es kann (ebenso wie das darüber liegende Verzeichnis „..“) von allen gelesen und benutzt werden.

Zusätzlich gibt es noch die Möglichkeit in UNIX, für die Ausführung eines Programms die Zugriffsrechte des Eigentümers an das Programm zu binden statt an den Ausführenden (*userId*) oder seine Gruppe (*groupId*). Damit können die Funktionen von Systemprogrammen auch von „normalen“ Benutzern ausgeführt werden, wenn dies ausdrücklich erwünscht ist (*set user Id*, *set group Id* Status). Wird ein spezieller Status (*sticky bit*) bei einem Verzeichnis gesetzt, so kann ein normaler Benutzer die Dateien von anderen Benutzern in diesem Verzeichnis nicht löschen oder umbenennen.

In verschiedenen UNIX-Versionen (z. B. MAC OS, OpenVMS, Linux, BSD, Solaris) gibt es *Access Control Lists*, die aber sehr unterschiedliche Mechanismen haben und deshalb in Netzwerken bei der Kopplung der Rechner Probleme bereiten können. Aus diesem Grund gibt es eine verstärkte Aktivität der X/Open-Gruppe, UNIX auch bei den Sicherheitsnormen auszubauen und zu vereinheitlichen.



### 4.3.2 Beispiel Windows NT: Zugriffsrechte

Das Dateisystem in Windows NT wird über einen objektorientierten Mechanismus verwaltet. Unabhängig von den Zugriffsrechten, die für jedes Objekt im Namensraum von Windows NT gelten (wie etwa Prozesse, Semaphore, Speicherbereiche usw.), gibt es noch spezielle Eigenschaften für **Dateiobjekte**:

#### *Attribute*

- Dateiname
- Typ des Geräts, auf dem die Datei liegt
- *Byte offset*: aktuelle Position in der Datei
- *share mode*: Status (read/write/delete) der Datei für andere während der Benutzung
- *open mode*: Art und Weise der Dateioperation (synchron/asynchron, mit/ohne Cachepuffer, sequentiell/beliebiger Zugriff, ...)
- *file disposition*: temporäre oder dauerhafte Datei

#### *Methoden*

CreateFile (), OpenFile (), ReadFile (), WriteFile (), CloseFile (), abfragen/setzen von Dateiinformatoren, abfragen/setzen von erweiterten Attributen, sichern/freigeben der Dateilänge, abfragen/setzen von Geräteinformationen, abfragen eines Verzeichnisses, ...

Jedes Dateiobjekt ist nur eine Kopie der Kontrollinformation einer Datei; es kann also auch mehrere Dateiobjekte geben, die dieselbe Datei referieren. Deshalb gibt es globale Informationen, die in der Datei selbst (und nicht im Dateiobjekt) gespeichert sind und auch dort nur verändert werden können.

Jede **Datei** besitzt verschiedene Dateiattribute, die als Datenströme variabler Länge implementiert sind. Diese sind vor allem

- *Standardinformation* wie
  - Erzeugungsdatum und -zeit, Datum und Zeit des letzten Zugriffs und der letzten Änderung,
  - die aktuelle Dateilänge,
  - logische Dateiattribute der Werte JA/NEIN wie z. B. Systemdatei, verborgene Datei, Archivdatei, Kontrolldatei, nur Lesen, komprimierte Datei usw.
- *Dateiname (file name)*, wobei dies bei bestehenden *hard links* auch mehrere sein können, unter anderem auch der MS-DOS-Kurzname.
- *Sicherheitsdaten (security descriptor)*: ACL mit Eigentümer, Erzeuger und möglichen Benutzern der Datei. Dabei ist ab Windows 2000 der Sicherheitsdeskriptor ausgelagert, um eine zentrale Sicherheitsinformation für mehrere Dateien zu ermöglichen

und so bei konsistenter Behandlung vieler Dateien mit gleichen Rechten Platz für die Beschreibung zu sparen.

- *Dateiinhalt*: Dieses Attribut enthält die eigentlichen Daten; bei Verzeichnissen ist darin eine Indexstruktur mit den verzeichneten Dateien gespeichert.
- Interessanterweise werden damit die eigentlichen Daten zu einem von vielen Attributen. Da der Benutzer auch weitere Attribute erzeugen kann, sind somit auch weitere, zusätzliche Datenströme möglich. Im Unterschied zum Hauptdateinamen, z. B. „MeineDatei.dat“, werden die Nebendaten vom Programmierer mit einem zusätzlichen Namen referiert, der durch einen Doppelpunkt getrennt ist, etwa „MeineDatei.dat : MeinKommentar“. Dies schafft vielfältige Möglichkeiten, Zusatzinformationen an eine Datei zu hängen (etwa den Namen des Bearbeitungsprogramms, den Kontext bei der letzten Bearbeitung usw.), ohne die Hauptdaten zu verändern. Dabei wird für jeden Datenstrom eine eigenständige Statusinformation geführt wie aktuelle und maximale, allozierte Länge, Semaphore (*file locks*) für Dateiabschnitte usw.

Die logischen Attribute werden beim Setzen oder Zurücksetzen auch durch die assoziierten Methoden unterstützt. Beispielsweise wird eine Datei, die das Attribut „komprimiert“ zugewiesen bekommt, automatisch sofort komprimiert. Dies gilt auch für einen ganzen Dateibaum.

Die Sicherheitsmechanismen in Windows NT sind etwas stärker differenziert als in UNIX. Zu jeder Datei gibt es eine detaillierte Zugriffsliste (*Access Control List ACL*), in der die Zugriffsrechte aller Benutzer aufgeführt sind. Zusätzlich zu den Standardnamen wie „Administrator“, „System“, „Creator“, „Guest“, „Everyone“, ... kann man weitere Benutzer und ihre Rechte spezifizieren. Dabei kann man entweder spezifizieren, was der Benutzer darf und/oder was er nicht darf.

Dieses Schutzsystem ist nicht auf Dateien beschränkt, sondern wird generell bei allen globalen Objekten im Namensraum von Windows NT angewendet und einheitlich geführt. Dabei wird auch ein *auditing* (Aktionsprotokollierung) für die Zugriffe auf Dateien und Verzeichnisse unterstützt. In neueren Windows NT Versionen (Windows 2000) ist außerdem ein netzwerkübergreifender Verzeichnisdienst nutzbar (*active directory*), der alle Objekte im Namensraum verwaltet und damit für die Administration einen kontrollierten, einheitlichen, netzweiten Zugang zu allen Ressourcen (Dateien, Drucker, etc.) ermöglicht.

### 4.3.3 Aufgaben

#### Aufgabe 4.3-1 Objektorientierte Dateiverwaltung

Angenommen, man wollte die Dateiverwaltung in UNIX objektorientiert gestalten. Welche Attribute und Methoden sind für ein UNIX-Verzeichnisobjekt (*directory*) nötig? Welche für ein Dateiojekt?

### Aufgabe 4.3-2 Access Control List

Was sind die Vor- und Nachteile einer *Access Control List*, die jeweils beim Benutzer für alle Dateien angelegt wird, gegenüber einer, die jeweils bei der Datei für alle Benutzer abgespeichert wird?

## 4.4 Dateifunktionen

Es gibt sehr viele Arten von Operationen, die auf Daten – und damit auch auf Dateien – möglich sind. In diesem Abschnitt wollen wir einige davon betrachten, die typisch sind für Dateien auf Massenspeichern.

### 4.4.1 Standardfunktionen

In den meisten Betriebssystemen gibt es einige Grundfunktionen, mit denen Dateien gelesen und geschrieben werden können.

Dies sind

- *Create File*  
Anlegen einer Datei. Üblicherweise werden als Parameter sowohl der Name (eine Zeichenkette) als auch die Zugriffsarten (Schreiben/Lesen, sequentiell/wahlfrei) spezifiziert. Der Aufrufer erhält eine Dateireferenz zurück, mit der er bei allen weiteren Funktionen auf die Datei zugreifen kann. Eine solche Dateireferenz (*file identifier*, *file handle*) kann eine Zahl sein (Index in einem internen Feld von Dateieinträgen) oder ein Zeiger auf eine interne Dateistruktur.
- *Open File*  
Beim Öffnen einer bestehenden Datei werden verschiedene Datenstrukturen initialisiert, so dass weitere Zugriffe schneller gehen. Hierzu gehören das Abprüfen der Zugriffsrechte sowie die Einrichtung von Puffern und Zugriffsstrukturen.
- *Close File*  
Schließen einer Datei. Dies ermöglicht, die Verwaltungsinformationen der Datei auf dem Massenspeicher zu aktualisieren und anschließend den Platz der Datenstrukturen der Dateiverwaltung im Hauptspeicher wieder freizugeben.  
Ähnlich wie bei der Interprozesskommunikation in [Abschn. 2.4.1](#) kann man das Paar `OpenFile/CloseFile` dadurch ersetzen, dass man anstelle der festen Kommunikationsverbindung eine verbindungslose Kommunikation erlaubt: Alle Zugriffe werden in der Reihenfolge ihres Eintreffens ohne vorhergehendes `OpenFile ()` durchgeführt. Dies ist allerdings für eine lokale Datenorganisation nicht so effizient: Anstelle den Dateizugriff nur einmal einzurichten (Prüfen der Zugriffsrechte, Aufsuchen der Dateiblöcke, ...), muss man dies bei jedem Zugriff durchführen, was einen unnötigen Verwaltungsaufwand bedeutet.

- *Read File/Write File*

Als Parameter erhält dieser Systemaufruf üblicherweise die Dateireferenz sowie einen Puffer und die Anzahl der Bytes, die gelesen bzw. geschrieben werden sollen. Gibt es nur eine (default) Datei und entspricht die Zahl der Bytes der Puffergröße, so reicht ein Parameter, der Zeiger zum Datensatz.

- *Seek File*

Eine einfache, sequentiell organisierte Datei besitzt eine Position (Index), an der gerade gelesen oder geschrieben wird. Diese Dateiposition wird nicht nur intern als Verwaltungsinformation aktualisiert, sondern kann in manchen Dateisystemen auch vom Programm gesetzt werden. Von einem sequentiellen Bearbeiten, z. B. von Magnetbändern (*sequential access*), kommt man so zu dem wahlfreien Zugriff (*random access*), wie dies beim Arbeiten mit Magnetplatten und CD-ROM üblich ist.

Je nach Betriebssystem werden diese Systemdienste unterschiedlich vom System unterstützt. Dies umfasst sowohl die Frage, ob das Lesen/Schreiben gepuffert ist oder ob das Benutzerprogramm dies selbst tun muss, als auch die Reaktion des Systems auf einen anormalen Zustand, etwa wenn keine Daten gelesen oder geschrieben werden können, weil beispielsweise beim `Read ()` noch keine Daten verfügbar sind oder beim `Write ()` das Laufwerk noch nicht bereit ist. Meist ist das `Read ()` in diesem Fall für den Benutzerprozess blockierend (vgl. Prozesskommunikation, [Abschn. 2.4.1](#)), das `Write ()` aber nicht. Stattdessen werden die Daten gepuffert; bei Pufferüberlauf oder Laufwerksdefekt wird eine Fehlermeldung zurückgegeben. Üblicherweise gibt es noch ergänzende Funktionen wie `DeleteFile ()`, `RenameFile ()`, `CopyFile ()`, `AppendFile ()`, `Flush-Buffer ()` usw., die aber stark vom Betriebssystem abhängig sind.

#### 4.4.2 Beispiel UNIX: Dateizugriffsfunktionen

In UNIX liefern die Systemaufrufe `fd=creat(name,mode)` und `fd= open(name,mode)` eine Zahl zurück, den *file descriptor* `fd`. Diese Zahl ist die Referenz für alle anderen Zugriffe wie

- `read (fd, buffer, nbytes)` liest *n* Bytes in einen Puffer,
- `write(fd, buffer, nbytes)` schreibt *n* Bytes aus einem Puffer,
- `close(fd)` schließt eine Datei.

Dieser *file descriptor* ist der Index in einem Feld (*file descriptor table*) aus Einträgen (Zeigern), die sich auf Dateistrukturen beziehen. Die Zahl der Dateien und damit die Zahl der möglichen Einträge (also die maximale Zahl für `fd`) ist fest und wird bei der Übersetzung des Betriebssystems angegeben und bestimmt so die Größe der Verwaltungsstrukturen für Dateien, die jeweils in der *user structure* eines Prozesses abgespeichert werden.

Der *file descriptor* spielt eine wichtige Rolle in UNIX. Standardmäßig wird vor dem Starten eines Prozesses (Programms) der *file descriptor* 0 für die Eingabe (*stdin*), 1 für die Ausgabe (*stdout*) und 2 für die Ausgabe bei Fehlern (*stderr*) reserviert. Dies macht man sich in UNIX zunutze, um Programme miteinander zu einer gemeinsamen Funktion zu verbinden. In [Abschn. 2.4](#) lernten wir das Kommunikationskonstrukt *pipe* kennen. Um das Beispiel, bei dem mehrere Prozesse in einer Informationskette

```
Programm1 | Programm2 | ... | ProgrammN
```

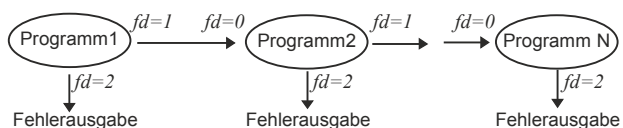
durch die *shell* aufgesetzt werden zu realisieren, setzt der Elternprozess mehrere Kindsprozesse an und erzeugt für jeweils eine Kommunikationsverbindung eine *pipe*. Dann initialisiert der Elternprozess die Dateideskriptoren für jeden Prozess derart, dass für den *pipe*-Eingang *fd*=1 im Sendeprozess und den *pipe*-Ausgang *fd*=0 im Empfängerprozess gilt. In [Abb. 4.8](#) ist dies gezeigt.

Die Zuordnung der Dateideskriptoren ist allerdings nur Konvention; jeder Prozess kann über gezieltes Schließen und Öffnen der Dateien auch andere Zuordnungen der Dateideskriptoren erreichen: Beim Öffnen wird immer der freie Dateideskriptor mit der kleinsten Nummer vom System zuerst benutzt.

#### 4.4.3 Beispiel Windows NT: Dateizugriffsfunktionen

Die Grundfunktionen für Dateizugriffe sind, wie bereits gesagt, als Methoden von Dateiattributen konzipiert. Zu den üblichen, oben aufgezählten Methoden kommen noch weitere Funktionen für Dateien wie `FlushBuffer()` sowie die typischen Verzeichnisoperationen (lese, schreibe, durchsuche Verzeichnis usw.) hinzu. Allen gemeinsam ist ein interessantes Konzept: Alle Operationen, welche die Dateistruktur auf dem Massenspeicher verändern, sind als *atomare Transaktionen* implementiert. Dazu gibt es einen speziellen Dienst, den *log file service* LFS, der bei jedem Dateizugriff (Transaktion) vom Kern aufgerufen wird und spezielle *log records* für das Dateisystem schreibt. Bevor die eigentlichen Dateien auf dem Massenspeicher durch eine Transaktion geändert werden, müssen die *log records* aller Operationen für eine Transaktion davon auf den Massenspeicher geschrieben werden. Dieses als *write-ahead logging* bezeichnete Konzept ermöglicht es, bei einem Systemabsturz (Stromausfall etc.) das Dateisystem wieder sauber aufzusetzen. War der Absturz vor dem Hinausschreiben des log-Puffers, so sind alle Operationen vergessen und müssen erneut durchgeführt werden. War der Absturz zwischen log-I/O-Abschluss und Datei-I/O-Abschluss, so muss beim Wiederanlaufen geprüft werden, welche der notierten

**Abb. 4.8** Das pipe-System in UNIX



Operationen bereits durchgeführt worden sind und welche noch nicht. Je nach Lage werden dann die noch fehlenden Operationen für eine Transaktion noch durchgeführt oder widerrufen, so dass am Ende jede Transaktion entweder vollständig oder gar nicht durchgeführt wurde. Der genaue Ablauf wird in (Custer 1994) geschildert.

Dieses Konzept hat aber einige wichtige Annahmen, die nicht in jedem Fall unbedingt erfüllt sein müssen. So muss die Übermittlung der log-Daten fehlerfrei sein – wird die log-Datei inkonsistent, weil der Datentransfer fehlerhaft war, so wird sie unbrauchbar. Die Annahme besteht also darin, dass ein passiver Defekt das System in einen sicheren Zustand (*fail save*) bringt (z. B. vollständige Datenunterbrechung durch Betriebssystemabsturz etc.), nicht aber ein aktiver Fehler (z. B. defekter DMA-Datentransfer) die Daten korrumpiert.

#### 4.4.4 Strukturierte Zugriffsfunktionen

Die bisher genannten Zugriffsfunktionen sind sehr einfacher Art und bedeuten nur eine Trennung der physikalischen, gerätespezifischen Zugriffsalgorithmen von den logischen Dateioperationen. In vielen Systemen gibt es nun Operationen, die darauf aufbauend einen strukturierten Zugriff auf Dateien ermöglichen, der mehr an der logischen Organisation der Daten in der Datei als an deren Implementierung orientiert ist. Es gibt mehrere Standardmodelle für eine Datenstrukturierung:

##### Sequentielle Dateien (sequential files)

In der Vergangenheit verarbeitete man mit Computern lange Datenlisten, die als sequentiell angeordnete Sätze (*records*) organisiert waren. Da diese Datensätze bei umfangreichen Dateien (z. B. Kartei des Einwohnermeldeamts, Datei der Rentenversicherung etc.) auf Magnetbändern existierten und hintereinander bearbeitet wurden, reichte diese Organisationsform lange Zeit aus. Das Dateimodell von PASCAL beispielsweise geht von solchen sequentiellen Lese- und Schreiboperationen ganzer Datensätze mittels der Operationen `put ()` und `get ()` aus.

Möchte man allerdings in einer anderen Reihenfolge als der abgespeicherten auf die Daten zugreifen, so dauert dies sehr lange.

##### Wahlfreie Dateien (random access files)

Diesen Nachteil vermeidet eine Dateiorganisation, die einen Zugriff auf alle sequentiell angeordneten Datensätze in beliebiger Reihenfolge zulässt. Eine effiziente Realisierung dieses Konzepts wurde erst mit dem Aufkommen von Festplattensystemen als Massenspeicher möglich. Fast alle Dateisysteme ermöglichen diese Art des Zugriffs durch eine zusätzliche Positionsangabe (*offset*) beim Zugriff auf die Datei.

Obwohl sowohl bei sequentiellem als auch bei wahlfreiem Dateizugriff eine Zugriffsart durch die andere realisiert werden kann, ist es nicht sehr sinnvoll, auf eine davon zu verzichten. Sequentielle Dateien beispielsweise kann man schnell auf mehreren hintereinander folgenden Spuren einer Magnetplatte ablegen und lesen, für wahlfreie Dateien ist dies nicht von Vorteil.

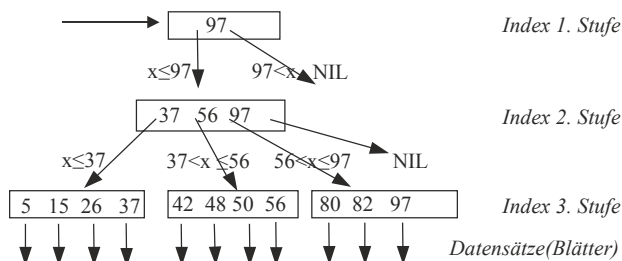
Liegt allerdings eine inhaltliche Organisation der Datei vor, so sind beide Methoden langsamer als eine, die auf diese Organisation Rücksicht nimmt und sie effizient mit den Betriebsmitteln implementiert. Aus diesem Grund gibt es speziell für Datenbanksysteme weitere Zugriffsmethoden.

### Indexsequentielle Dateien (index-sequential files)

Diese Dateien bestehen aus Datensätzen, die nach einem Kriterium (*Schlüssel*) geordnet sind und denen am Anfang der Datei ein Index (Inhaltsverzeichnis) vorangestellt ist. In diesem Indexverzeichnis sind Verweise auf den Schlüssel (Typ von Einträgen bei einer Einwohnerdatei, z. B. Einwohnernamen, Geburtstag usw.) derartig strukturiert und aufgeführt, dass man schnell auf den Teil der Datei zugreifen kann, der den interessanten Wertebereich des Schlüssels beherbergt. Da eine solche Struktur typisch für alle Dateisysteme ist und deshalb sehr häufig vom Betriebssystem in den Datenstrukturen von Verzeichnissen (*directory*) verwendet wird, wollen wir uns dies genauer ansehen. In Abb. 4.9 ist eine mögliche Indexstruktur, eine Baumstruktur, am Beispiel einer Einwohnerdatei gezeigt, die nach einem Schlüssel (dem Alter) geordnet ist. Die Schlüssel (Altersangaben) seien 5, 15, 26, 42, 48, 56, 80, 82, 97, 37, 50. Die eigentlichen Datensätze werden über die Pfeile aus der 3. Stufe erreicht und sind nicht abgebildet.

Der jeweils größte Schlüssel (größte Zahl) eines Dateiabschnitts (viereckiger Behälter in Abb. 4.9) aus dem Index 3. Stufe ist als ein Index im Behälter der 2. Stufe nochmals vermerkt. Entsprechend sind als Indizes der 1. Stufe jeweils die größten Schlüssel eines Behälters der 2. Stufe aufgeführt, usw. Sucht man den Datensatz eines bestimmten Schlüssels  $x$ , so muss man von der Wurzel des Baumes, hier: von den Indizes der zweiten Stufe ausgehend das Intervall bestimmen, in dem sich der gesuchte Schlüssel befindet, und sich dann sequentiell durch die Hierarchie nach unten bewegen, bis man den Schlüssel des Datensatzes gefunden hat. Dann kann man den Datensatz mit Hilfe der Informationen im daran hängenden Blatt zielgerichtet aus der Datei lesen. Ein solcher Baum aus drei Ebenen kann direkt für eine Dateispeicherung eingesetzt werden: Die oberste Ebene ordnet dem Index Zylindernummern einer Festplatte zu, die zweite Ebene für den Index die Spurnummern innerhalb des Zylinders und die unterste Ebene die Sektoren bzw. Datenabschnitte (*records*) auf einer Spur selbst. Auf der obersten Ebene notieren wir neben jedem Schlüssel genau eine Zylindernummer. Damit vereinbaren wir für jeden Zylinder den maximalen Schlüssel, dessen Daten im Zylinder gespeichert werden. Mit einer solchen Zuordnung wird der Zugriff schnell: dem

**Abb. 4.9** Baumstruktur für zweistufigen, index-sequentialen Dateizugriff



großen Indexabstand in der obersten Stufe entspricht dem großen zeitlichen Unterschied beim Zugriff auf verschiedenen Zylinder; der geringe Abstand bei den Blättern entspricht der geringe (zeitliche) Abstand beim Lesen oder Schreiben benachbarter Sektoren einer Spur. Eine solche Dateispeicherung wird als **index-sequentielle** Datei bezeichnet.

Bei der Zuordnung einer solchen Baumstruktur auf die physikalischen Gegebenheiten (Spuren, Zylinder, Sektoren) bekommt man leicht Probleme, wenn man versucht, Datensätze herauszunehmen oder neue zusätzlich einzugliedern. Beispielsweise schafft die Eingliederung des Satzes mit dem Schlüssel „41“ in [Abb. 4.9](#) ziemliche Probleme, da der Behälter (reservierter Speicherabschnitt, *container*) eine maximale Kapazität von vier Schlüsseln hat und bereits vier Schlüssel enthält. Die deshalb nötigen Hilfsstrukturen für die Angliederung der Zusatzdaten „wuchern“ unkontrolliert bei häufigen Änderungen der Daten und erfordern bald ein komplettes Neuschreiben der Datei. Dies tritt besonders bei Datenorganisationen mit sehr vielen Schlüsseln und häufigen Änderungen auf, etwa wenn das Indexverzeichnis (*directory*) ein Inhaltsverzeichnis des Dateisystems darstellt und als Schlüssel die Dateireferenzen (Namen, Dateinummern) des Massenspeichers enthält. Für ein effizientes Dateisystem ist es also unumgänglich, die Schlüssel im Indexverzeichnis nach einem anderen Schema zu organisieren, das sowohl eine schnelle Suche als auch flexibles Hinzufügen und Herausnehmen von Schlüsseln gestattet.

## B-Bäume

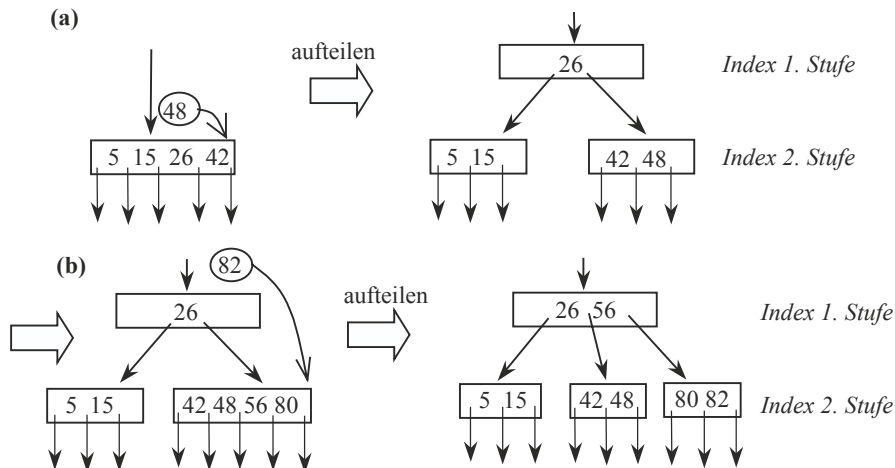
Dazu führen wir zwei Änderungen ein. Zuerst notieren wir jeden Schlüssel in den Knoten des Baumes nur einmal. Dies spart Platz und Suchzeit: haben wir einen Schlüssel gefunden, so müssen wir nicht weitersuchen. Zum anderen speichern wir bei dem Schlüssel auch direkt die Referenzen zu den *records*. Die Blätter des Baumes sind hier nicht die eigentlichen Datensätze, auf diese wird ja direkt von den Schlüsseln verwiesen. Stattdessen dienen die Blätter als potentielle Verzweigungen nur als Platzhalter und enthalten keine Information.

Nun können wir uns einen Baum über folgende Regeln bilden:

- Jeder Knoten (Behälter) enthält maximal  $m$  Verzweigungen und damit  $m - 1$  Schlüssel.
- Wird ein Schlüssel in einen bereits vollen Behälter eingeführt, so wird stattdessen der Behälter in zwei gleiche Teile geteilt und der mittlere Schlüssel wird in einen Behälter eine Ebene höher verlagert.

Betrachten wir unser obiges Beispiel mit maximal 4 Schlüsseln pro Schlüsselbehälter ( $m = 5$ ) und den Schlüsseln 5, 15, 26, 42, 48, 56, 80, 82, 97, 37, 50. Wir gehen von einem leeren Schlüsselbehälter aus und füllen ihn mit den Schlüsseln 5, 15, 26, 42 bis er voll ist. Der nächste Schlüssel (hier „48“) bringt den Behälter zum Überlaufen. Um dies zu verhindern wird nun der Inhalt des Behälters in zwei Teile geteilt und auf zwei Behälter verteilt, wobei ein Schlüssel in der Mitte abgezweigt werden muss, um bei der Suche auf den richtigen Behälter zu verweisen. Dieser Schlüssel wandert nach oben in einen Behälter als Index 1. Stufe. Bei ungeradem  $m$  gibt es immer einen mittleren Schlüssel (sonst nehmen





**Abb. 4.10** Der Aufbau des B-Baums mit  $m = 5$

wir den letzten der linken Hälfte), in diesem Fall ist es der Schlüssel 26. Es entsteht eine zweischichtige Hierarchie, siehe [Abb. 4.10a](#).

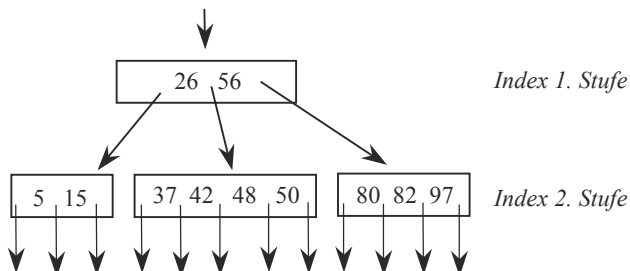
Die weiteren Schlüssel 56 und 80 können direkt eingegliedert werden, da der Behälter maximal  $m - 1 = 4$  Schlüssel enthalten kann. Wird der Schlüssel 82 eingeführt, so muss wieder geteilt werden und Schlüssel „56“ geht nach oben verschoben, siehe [Abb. 4.10b](#). Nun können die restlichen Schlüssel 97, 37 und 50 problemlos eingegliedert werden. Der fertige Baum ist in [Abb. 4.11](#) zu sehen.

Mit der neuen Baumstruktur haben wir zusätzlich zum schnelleren Zugriff auch einen weiteren Vorteil gewonnen: Wir können leicht zusätzliche Schlüssel einfügen.

Dabei funktioniert folgender Einfügealgorithmus bei Überlaufen eines Behälters:

- Teile den Schlüsselbehälter mit  $k$  Schlüssel auf in zwei Behälter mit den Schlüssel  $S_1 \dots S_{\lceil k/2 \rceil - 1}, S_{\lceil k/2 \rceil + 1} \dots S_k$  auf.
- Den Schlüssel  $S_{\lceil k/2 \rceil}$  in der Mitte verschiebe nach oben in die vorige Stufe.
- Falls dort der Schlüsselbehälter überläuft, verfähre dort genauso.

**Abb. 4.11** Der fertige Baum des Beispiels mit  $m = 5$



Die Löschoption von Schlüsseln ist analog invers.

Man beachte, dass wir den Baum so errichtet haben, dass auf derselben Ebene jeweils ungefähr die gleiche Zahl von Verzweigungen pro Behälter abgehen (der Baum ist „ausgewogen“). Dabei achten wir darauf, dass die Schlüsselbehälter gut gefüllt sind, also bei maximal  $m$  Verzweigungen oder  $m - 1$  Schlüsseln mindestens die Hälfte  $\lceil m/2 \rceil$  der möglichen Verzweigungen auch tatsächlich genutzt werden. Nur bei der Wurzel kann der Fall auftreten, dass nur ein Schlüssel (bzw. zwei Verzweigungen) vorhanden ist, obwohl  $m$  wesentlich größer sein kann.

Eine solche Datenstruktur hat einen Namen: Es ist ein **B-Baum**, siehe Knuth (1998) oder Comer (1979). Formal gesehen ist ein B-Baum ein Baum, bei dem

- (1) Die Wurzel mindestens 2 und maximal  $m$  Äste hat,
- (2) jeder Ast bzw. Knoten (bis auf die Wurzel und Blätter) in minimal  $k = \lceil m/2 \rceil$  und maximal  $k = m$  Äste verzweigt,
- (3) jeder Knoten  $k - 1$  Schlüssel (Indizes) enthält,
- (4) alle Blätter auf derselben Ebene sind. Allerdings sind sie uninteressant, da sie keine Schlüssel enthalten, und erfüllen nur Stellvertreterfunktion für Schlüssel oder potentielle Verzweigungen.

Ein solcher Baum ist „von der **Ordnung**  $m$ “. Es lässt sich zeigen, dass wir bei insgesamt  $N$  Schlüsseln im gesamten Baum genau  $N + 1$  Blattverzweigungen in der untersten Stufe haben.

Ein B-Baum kann schnell durchlaufen werden: Die Wurzel hat mindestens 2 Verzweigungen und jede Verzweigung in der nächsten Stufe wieder mindestens  $\lceil m/2 \rceil$  Verzweigungen, so dass auf jeder der  $n$  Stufen mindestens  $2 \cdot \lceil m/2 \rceil \cdot \lceil m/2 \rceil \dots = 2 \cdot \lceil m/2 \rceil^{n-1}$  Verzweigungen vorhanden sind. Die Zahl der Verzweigungen auf der  $n$ -ten Stufe ist gleich der Anzahl der Blattverzweigungen, also bei  $N$  Schlüsseln genau  $N + 1$ . Bei der Mindestanzahl von Verzweigungen auf jeder Stufe ist die Zahl  $n$  von Stufen die nötig sind, um alle Schlüssel im Baum unterzubringen, maximal. Es gilt auf der  $n$ -ten Stufe

$$2 \cdot \lceil m/2 \rceil^{n-1} = N + 1 \text{ oder } n - 1 = \log_{\lceil m/2 \rceil} (N + 1) / 2$$

Mehr Stufen sind also nicht möglich, da wir immer nur die minimale Schlüsselzahl pro Container und damit pro Stufe bedacht haben. Allerdings ist auch bei dieser extrem dünn mit Schlüsseln besetzten Baumstruktur ein dramatischer Zeitgewinn gegeben: Bei einer Containergröße von  $m = 200$  und insgesamt  $N = 2$  Millionen Schlüsseln benötigen wir lediglich  $n - 1 = \log_{100}(10^6) = \log_{100}(100^3) = 3$ , also  $n = 4$  Stufen, um aus 2 Millionen Schlüsseln eine Datei zu finden! Die B-Bäume tendieren also dazu, selbst bei großer Anzahl von Dateneinträgen sehr flach zu bleiben.

### B\*-Bäume

Es gibt verschiedene Varianten des B-Baumes. Beispielsweise kann man auf jeder Stufe den Wert für  $m$  verschieden wählen. Dies ändert im Prinzip die Algorithmen nicht, sondern

nur die resultierende Baumstruktur. Im Folgenden wollen wir nun eine andere wichtige Möglichkeit betrachten, die sich davon abhebt und auch meist von Betriebssystemen und Datenbankorganisationen verwendet wird.

Die Grundidee zur Verbesserung der einfachen Baumstruktur durch den B-Baum bestand darin, nicht nur volle „Behälter“ (Abschnitte) zu verwenden, sondern bei zusätzlichen Datensätzen einen Abschnitt in zwei neue zu zerteilen und die Änderungen im Index der nächsten Stufe zu reflektieren. Die Abschnitte sind dann durch Zeiger dynamisch miteinander verbunden. Die Füllung der zwei neuen Behälter kann dabei allerdings sehr gering sein. Um dies zu verbessern und die Behälter nicht so oft teilen zu müssen, können wir nun Schlüssel des zu vollen Knotens zu einem weniger vollen Nachbarn „hinüberfließen“ lassen, so einen Ausgleich erzielen und die Schlüsselbehälter besser auslasten. Diese Idee lässt sich in folgendem Algorithmus formulieren:

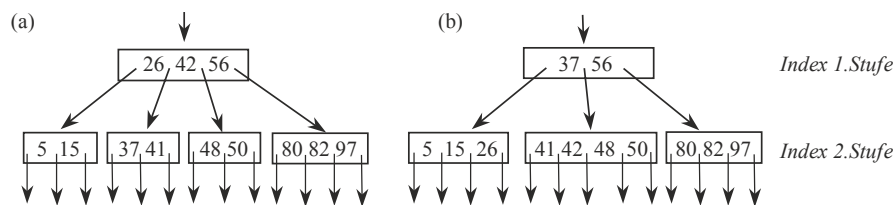
- Hat ein Nachbarbehälter mit  $k < m - 1$  weniger Schlüssel, so verteile die  $t = k + m$  Schlüssel gleichmäßig auf beide Behälter. Dazu nehme den dazugehörigen Schlüssel der nächst-höheren Stufe hinzu, teile davon  $\lfloor t/2 \rfloor$  Schlüssel für den linken Behälter ab, verschiebe den Schlüssel mit Index  $i = \lfloor t/2 \rfloor + 1$  in die nächst-höhere Stufe und gebe die restlichen  $\lceil t/2 \rceil$  Schlüssel in den rechten Behälter.

Aber auch in dem Fall, wenn kein Hinüberfließen möglich ist, können wir eine Verbesserung erreichen:

- Haben beide Nachbarbehälter  $m - 1$  Schlüssel, so fasse einen Schlüsselbehälter mit  $m - 1$  Schlüsseln, den überlaufenden Behälter mit  $m$  Schlüsseln sowie den dazugehörigen Schlüssel der nächst-höheren Stufe zu einer einzigen Schlüsselmenge zusammen. Diese enthält nun die Schlüssel  $S_1, \dots, S_{2m}$ .
- Teile diese Menge zu *drei* Behältern auf. Jeder Behälter enthält dann abzüglich der beiden Schlüssel für die nächst-höhere Ebene  $(2m - 2)/3$  Schlüssel. Da dies nicht immer ganzzahlig ist, modifizieren wir die Anzahl zu jeweils  $\lfloor (2m - 2)/3 \rfloor$ ,  $\lfloor (2m - 1)/3 \rfloor$  und  $\lfloor (2m)/3 \rfloor$  Schlüsseln, wobei zwei Schlüssel  $S_A$  und  $S_B$  mit den Indizes  $A = \lfloor (2m - 2)/3 \rfloor + 1$  und  $B = A + \lfloor (2m - 1)/3 \rfloor + 1$  nach oben in die nächst-höhere Indexstufe wandern.

Jede der drei Partitionen ist nun nicht mehr halb, sondern mit  $(2m - 2)/3 = (m - 1)(2/3)$  mindestens zu  $2/3$  voll mit Schlüsseln, was eine Verbesserung gegenüber dem B-Baum ist. Ein solcher Baum wird als **B\*-Baum** bezeichnet. Dabei ist die Definition der Struktur nicht an die Operationen darauf gebunden: Solange die Baumcharakteristik (Baumdefinition) nicht verändert wird, sind alle Optimierungsoperationen zulässig, etwa das „Hinüberfließen“ auch beim B-Baum.

Für unser Beispiel ist das Resultat vom Einfügen des Schlüssels „41“ in den Baum aus Abb. 4.11 in der folgenden Abb. 4.12 gezeigt. In (a) ist dies für den B-Baum und in (b) für den B\*-Baum bei  $m = 5$  illustriert. Man sieht, dass in (a) ein neuer Behälter geschaffen werden musste, während in (b) ein „Überfließen“ dies verhinderte.



**Abb. 4.12** Einfügen des Schlüssels „41“ im (a) B-Baum und (b) im B\*-Baum

Formal unterscheidet sich ein B\*-Baum in seiner Definition vom B-Baum durch folgende, geänderte Bedingung 1)

- 1a) Jeder Ast bzw. Knoten (bis auf die Wurzel und Blätter) verzweigt in minimal  $k = (2m - 1) / 3$  und maximal  $m$  Äste.
- 2b) Die Wurzel hat mindestens 2 und maximal  $2 \lfloor (2m - 2) / 3 \rfloor + 1$  Verzweigungen. Damit kann die Wurzel beim Überfließen in zwei Behälter zu je  $\lfloor (2m - 2) / 3 \rfloor$  Schlüssel (plus ein Schlüssel als neue Wurzel) geteilt werden.

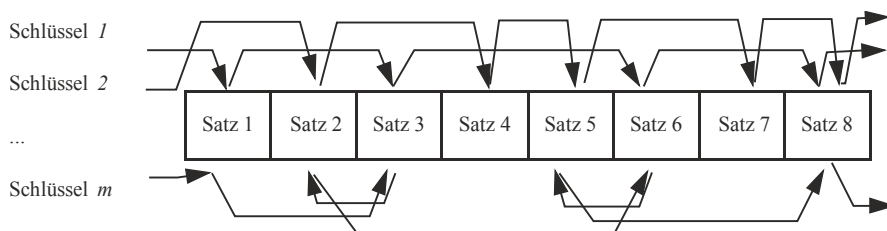
Es gibt noch weitere Variationen des B-Baumes (s. Comer 1979), auf die aber hier nicht weiter eingegangen werden soll.

### Mehrfachlisten (multi-lists), Invertierte Dateien (inverted files)

Bei der Dateiorganisation von Mehrfachlisten werden die Informationen über die (sequenziell) angeordneten Datensätze nach unterschiedlichen Schlüsseln geordnet. Für jeden Schlüssel werden die Referenzen zu den Datensätzen, in denen der Schlüssel auch enthalten ist, als Index an den Anfang der Datei geschrieben. In Abb. 4.13 ist dies für Mehrfachlisten gezeigt.

Dabei sind die Listen, mit dem Kopfeintrag startend, nach dem Index des Satzes geordnet, siehe Schlüssel 1 und 2. Man kann sie aber auch nach auf- oder absteigenden Werten des Schlüssels ordnen, so dass die Verzeigerungsstruktur in Abb. 4.13 unübersichtlicher wird. Dies ist mit der Liste der Sätze für Schlüssel  $m$  (1,3,2,6,5,8) gezeigt.

Fassen wir alle Zeiger eines Schlüssels jeweils im Index am Anfang der Datei zusammen und schreiben sie nicht in die Datensätze, so wird dies als **invertierte Datei** bezeichnet.



**Abb. 4.13** Mehrfachlisten einer sequentiellen Datei

Die Bezeichnung „invertiert“ resultiert aus der Tatsache, dass wir den Schlüsselwert nicht aus dem Satz erschließen, sondern umgekehrt den Satz aus dem Schlüssel.

Invertierte Dateien eignen sich besonders gut für Abfragen, die sich auf bestimmte Schlüssel beziehen („welche Einwohner sind älter als 80 Jahre“).

Die effiziente Implementierung der Dateiorganisation der obigen Strukturen wurde in den Betriebssystemen von Großrechnern durchgeführt. Aus diesem Grund wurde UNIX, das nur die allereinfachste Art des Dateizugriffs (wahlfreier Zugriff) implementiert, jahrelang als Computersystem für Datenbankanwendungen ausgeschlossen, da man in den Anwenderprogrammen nicht auf den höheren Operationen aufbauen konnte. Die Zwischenschicht zwischen den komplexen Operationen und den einfachen Grundmechanismen von wahlfreiem `Read/WriteFile ()` des Betriebssystems musste jeder Anwender selbst schreiben, was die Portierung derartiger Datenbanken auf UNIX in hohem Maße hemmte. Inzwischen spielen derartige Überlegungen nur noch in sehr speziellen Anwendungen eine Rolle, da neue, schnelle Prozessoren und Laufwerke den Leistungsverlust einer solchen zusätzlich nötigen Zwischenschicht mehr als ausgeglichen haben.

#### 4.4.5 Gemeinsame Nutzung von Bibliotheksdateien

Zu den von den Anwendungsprogrammen genutzten Funktionen eines Betriebssystems gehören nicht nur die Systemaufrufe des Kerns, sondern auch Systemfunktionen, die in Systembibliotheken enthalten sind. Diese Funktionen bieten grundlegende, für viele Anwendungen benötigte Prozeduren und Datenstrukturen, ohne sie gleich in den Kern zu verlagern.

Die Systembibliotheken werden naturgemäß von vielen Prozessen gleichzeitig benutzt. Sie werden deshalb gern nur einmal in den Speicher geladen und mit Speichermechanismen allen Programmen in Form von *shared pages* (s. Abschn. 3.3) zur Verfügung gestellt. In Unix ist dies vorzugsweise die *C-Library*, in Windows NT die *Dynamic Link Libraries DLL*.

Die Vorteile des Konzepts sind:

- Es wird weniger Hauptspeicher und Massenspeicher benötigt, da die Bibliothek nur einmal vorhanden ist und nicht als Kopie an jeder Applikation hängt
- Die Wartung wird erleichtert, da ein Fehler nur in einem kleinen, zentralen Bibliotheksmodul verbessert werden muss und nicht in vielen, großen Applikationen.

Allerdings sind diese Vorteile in der Praxis nicht immer gegeben:

- Massenspeicher und Hauptspeicher werden tendenziell immer billiger. Da auch durch die große Anzahl unterschiedlicher Bibliotheksmodule meist nicht alle Module von allen Prozessen benutzt werden, ist dieser Aspekt nur für sehr wenige, zentrale Module gültig, nicht aber für die Masse der Anwendungsbibliotheken.

- Jedes Bibliotheksmodul hat eigene Adressreferenzen, die beim Laden im Hauptspeicher entsprechend der Lage im virtuellen Adressraum einheitlich gesetzt werden. Meist sind die Module so kompiliert, dass sie mit Adresse Null anfangen, so dass die virtuelle Adressabbildung nur die höheren Adressbits berührt. Bei vielen Bibliotheksmodulen ist es nun nicht immer möglich, dass alle Prozesse denselben Adressraumteil für dieses Modul zur Verfügung haben, so dass im Endeffekt der Lader doch wieder eine Kopie des Moduls auf einen anderen Adressplatz laden muss und der Speichervorteil wegfällt.
- Die Wartung ist auch erleichtert, wenn das Bibliotheksmodul zentral ist; es kann durchaus als Kopie mehrfach in Applikationen benutzt werden.

Außerdem treten bei gemeinsam genutzten Bibliotheksdateien einige Probleme zusätzlich auf (Windows NT: „DLL-Hölle“):

- Manche Module sind Teil einer Gruppe. Ersetzt man ein Modul durch eine neuere Version, so müssen alle anderen ebenfalls durch eine dazu passende Version ersetzt werden. Wird dies nicht beachtet, so können schwere Fehler unklarer Ursache daraus resultieren, beispielsweise, weil das alte Modul vorher nichtdokumentierte Seiteneffekte benutzte oder tolerant gegenüber Datenfehlern war, das neue Modul dagegen dies nicht leistet.
- Veränderte Schnittstellen (Zahl oder Typ der Argumente), die laufzeitmäßig nicht überprüft werden, führen ebenfalls zu Fehlern unklarer Ursache.

Aus diesen Gründen verzichtet man tendenziell darauf, jede Applikationsbibliothek zentral zur Verfügung zu stellen, sondern belässt die Bibliotheksmodule als Kopie lokal bei der Applikation. Gibt es eine neue Bibliotheksversion, so funktioniert die Applikation immer noch unproblematisch mit der lokalen alten Version.

#### Beispiel Windows NT *DLL management*

Da es in Windows 2000 rund 2800 Systembibliotheken gibt, ist die korrekte Verwaltung sehr wichtig. Im Unterschied zu früheren Versionen, bei denen jedes beliebige Anwendungsprogramm seine DLLs in die Systemordner schreiben durfte und damit leicht Instabilitäten des Systems hervorrufen konnte, ist dies in Windows 2000 strikt verboten und werden vom *Windows File Protection* (WFP)-Mechanismus überwacht. Alle Systembibliotheken sind dort registriert und werden automatisch durch eine Sicherheitskopie aus einem Ordner „%windir%\System32\dlldata“ ersetzt, wenn sie unbefugt abgeändert werden und keine korrekte digitale Signatur mehr tragen. Nur korrekt signierte Bibliotheken, wie sie etwa durch *Service Packs* bereitgestellt werden, sind hier erlaubt.

Nicht-Systembibliotheken werden grundsätzlich lokal bei der Applikation abgespeichert. Lokale DLLs, werden zuerst geladen, auch wenn eine entsprechende Systembibliothek existiert.

Eine Prozess-spezifische Ausführung von Bibliotheken funktioniert erst dann nicht mehr, wenn globale Datenstrukturen verwaltet werden müssen. Dies ist in Windows 2000 bei etwa 20 zentralen DLL des Betriebssystems der Fall, etwa „kernel32.dll“, „user32.dll“ und „ole32.dll“, die speziell registriert sind.

Ab Windows Vista/Server 2008 wurde dieser WFP-Mechanismus ersetzt durch den *Windows Resource Protection* (WRP)-Mechanismus. Dabei werden zusätzlich zu den wichtigen Systemdateien auch Teile der Registry geschützt. Der Mechanismus garantiert, dass Originaldateien nur von einem *TrustedInstaller* geändert werden und ersetzt alle Originaldateien im Bedarfsfall aus dem Ordner „%windir%\winsxs\Backup“ für den Neustart. Die einzigen Instanzen, die diese Originaldateien installieren oder ändern dürfen, sind die vom *TrustedInstaller* installierten Windows *service packs*, *hot fixes*, *upgrades* oder *update files*.

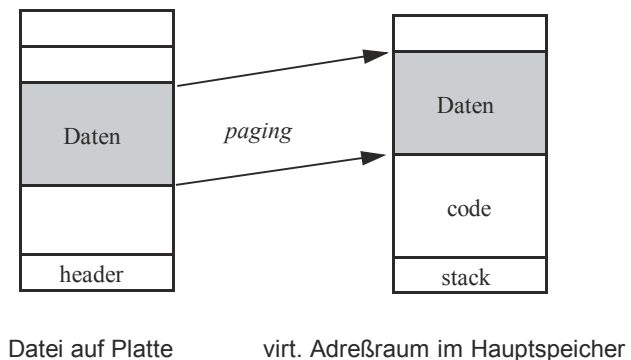
Zur Überprüfung und Wiederherstellung der Integrität der Systemdateien genügt es, ein Prüfprogramm *sfc* (*system file checker*) mit der Option *-scannow* im Administratormodus (Administratorkonsole) laufen zu lassen.

#### 4.4.6 Speicherabbildung von Dateien (*memory mapped files*)

Dateien mit wahlfreiem Zugriff ähneln in ihrem Zugriffsmodell stark dem normalen Hauptspeicher mit wahlfreiem Zugriff (RAM). Diese Analogie kann man weiter treiben: Dateien auf Massenspeichern kann man als Fortsetzung des normalen Hauptspeichers auf einem Massenspeicher ansehen. Deshalb liegt es nahe, einen weiteren Schritt zu tun und konsequenterweise eine direkte Verbindung von einer Datei, die in Abschnitte der Länge einer Seite unterteilt ist, mit einem Bereich des Hauptspeichers herzustellen (**memory mapped file**), der ebenfalls in Seiten untergliedert ist.

Eine solche Verbindung wird mit Systemaufrufen erreicht, in denen die Seiten einer Datei als Ersatz von Hauptspeicherseiten in den virtuellen Adressraum eines Prozesses abgebildet werden. In Abb. 4.14 ist dies illustriert.

**Abb. 4.14** Abbildung von Dateibereichen in den virtuellen Adressraum



Die Realisierung eines solchen Mechanismus ist relativ einfach: Als Ort, auf den die Seiten des Prozesses ausgelagert werden, wird der Ort der Datei auf dem Massenspeicher angegeben. Dies hat verschiedene Vorteile:

- *Schnelligkeit*  
Die Seite der Datei wird nur dann tatsächlich gelesen, wenn sie benutzt wird; unnötiges Kopieren von Daten entfällt.
- *automatische Pufferung*  
Bei den meisten Programmen, die Dateien benutzen, muss eine interne Pufferlogik geschrieben werden, um nur wenige, effiziente Lese-/Schreiboperationen durchführen zu müssen. Dies entfällt bei der Speicherabbildung: Die gesamte Pufferung wird mit dem *Paging*-Mechanismus automatisch und effizient vom Betriebssystem durchgeführt.

Zusätzliche Semaphoroperationen erlauben es, solche Speicherbereiche auch mit mehreren Prozessen gemeinsam zu benutzen.

Aus diesen Gründen gibt es einen solchen Mechanismus in vielen Betriebssystemen. Er findet allerdings dort seine Grenzen, wo eine bestehende Datei nicht nur gelesen und beschrieben wird, sondern sich zusätzlich in ihrer Länge ändert. Dies ist mit der Speicherabbildung nicht möglich.

---

#### Beispiel UNIX *Memory mapped files*

In Linux gibt es die Systemaufrufe

- `mmap ()` stellt eine Abbildung von virtuellem Adressraum in den Bereich einer Datei her.
- `munmap ()` beendet die Abbildung. Wurde der Speicherinhalt verändert, so werden die veränderten Seiten auf die Datei zurückgeschrieben.
- `msync ()` aktualisiert die Datei aus dem Speicher.

Zusätzlich ist es in HP-UX-UNIX möglich, Semaphore für den Speicherbereich zu erzeugen (`msem_init ()`), die Bereiche zu sperren (`msem_lock`) oder freizugeben (`msem_unlock`) sowie die Semaphore wieder zu entfernen (`msem_remove`).

---

#### Beispiel Windows NT *Memory mapped files*

Für die Abbildung zwischen Speicher und Dateisystem sind VM-Manager und I/O-Manager gemeinsam zuständig. Durch den Aufruf `CreateFileMapping ()` wird ein Objekt erzeugt, das auch von anderen Prozessen unter dem angegebenen Namen mit `OpenFileMapping ()` geöffnet werden kann. Dabei kann ein beliebiger Bereich (1 Byte bis 2 GB) deklariert werden. Teilbereiche daraus können mit `MapViewOfFile ()` in den virtuellen Adressraum abgebildet werden. Mit dem Aufruf `FlushViewOfFile ()` kann der Dateibereich aktualisiert und mit `UnmapViewOfFile ()` wieder geschlossen werden.



Dieser Mechanismus lässt sich auch zur Prozesskommunikation nutzen, indem man als gemeinsamen Speicherbereich eine Datei wählt, die mit dem *file mapping*-Mechanismus direkt von mehreren Prozessen bearbeitet werden kann. Für die Synchronisation derartiger Zugriffe sind keine speziellen Funktionen vorgesehen, sondern die Verwendung normaler Semaphore.

#### 4.4.7 Besondere Dateien (*special files*)

Es gibt verschiedene Mechanismen des Betriebssystems, die sich mit Hilfe des Dateisystems elegant umsetzen lassen, ohne aber „echte“ Dateien zu erzeugen oder zu behandeln.

Eine der bekanntesten Erweiterungen ist die UNIX-Modellierung von physikalischen Geräten als sogenannte **special files**. Jedem Namen einer Datei, die mit dem Status eines *special file* gekennzeichnet wird, ist ein physikalisches Gerät zugeordnet. Alle Zugriffe und Statusänderungen für diese symbolische Datei wirken sich stattdessen direkt auf dem Gerät aus. Wird beispielsweise auf solch eine Datei geschrieben, so werden in Wirklichkeit die Daten zu dem Gerät, beispielsweise einem Terminal, transportiert und dort auf dem Bildschirm dargestellt. Öffnet ein Prozess das *special file* des Terminals zum Lesen, so empfängt er alle Zeichen, die auf der Tastatur geschrieben werden.

Der Vorteil einer solchen Modellierung liegt in der Tatsache, dass so einerseits der effiziente, direkte Zugriff der Benutzer auf die physikalischen Eigenschaften von Geräten möglich wird, andererseits aber auch die Schutz- und Verwaltungsmechanismen des Betriebssystems wirken und so der Zugriff geregelt und kontrolliert eingeschränkt werden kann.

---

##### Beispiel UNIX *Special files*

In UNIX existiert ein Verzeichnis mit dem Pfadnamen */dev*, in dem Dateien für verschiedene Geräte existieren. Es gibt zwei Arten von Gerätedateien: sequentielle Dateien (*character-oriented special files*), die für alle zeichenorientierten Geräte wie Terminals, Bandgeräte, Drucker usw. eingesetzt werden, und wahlfreie Dateien (*block-oriented special files*), bei denen beliebige Speicherabschnitte (**Blöcke**) wahlfrei angesprochen werden können, beispielsweise Magnetplatten, Disketten usw. Jedes dieser *special files* wird mit Hilfe eines Systemaufrufs `mknod()` erzeugt und spricht beim Aufruf (`open()`, `close()`, `read()`, `write()`) spezielle Prozeduren an, die vom Gerätetreiber zur Verfügung gestellt werden. Beispielsweise können auf das gerade benutzte Terminal mit dem Dateinamen */dev/tty* Buchstaben ausgegeben und gelesen werden. Auch symbolische Geräte, etwa ein Zufallszahlengenerator */dev/random* als Eingabe oder ein Mülleimer */dev/null* für die Ausgabe, können eingebunden werden.

Man beachte, dass mit diesem Mechanismus dasselbe Gerät unter verschiedenen *special file*-Namen verborgen sein kann. Beispielsweise kann ein Magnetband als *character special file* wie eine kontinuierliche, sequentielle Datei behandelt werden. Unter einem

anderen Namen kann man es aber auch als *block-oriented special file* ansprechen und damit direkt auf einen der sequentiell gespeicherten Blöcke zugreifen, also einen Block mit der Nummer 15 vor einem Block der Nummer 5 lesen. Für diese Operation müssen intern noch zusätzlich Positionierungsbefehle im Gerätetreiber verwendet werden.

Bis auf einige wenige Geräte wie Drucker und Terminal, die kein *block device* emulieren können, ist es deshalb möglich, alle Geräte sowohl als *character special files* als auch als *block special files* anzusprechen. Da diese Schnittstelle nur Zugriff auf die reinen Daten ohne Struktur und Verwaltungsinformation gestattet, werden diese *special files* auch als **raw devices** bezeichnet.

Um ein auf einer Platte existierendes Dateisystem zu nutzen, muss vorher mit dem `mount ()`-Befehl der Wurzelknoten auf ein Verzeichnis des existierenden Dateisystems abgebildet werden. Alle Dateien, die vorher in dem Verzeichnis waren, sind damit erst einmal überlagert von dem neuen Dateisystem und deshalb nicht mehr zugänglich.

Die *special files* stellen in UNIX aus Benutzersicht die einzige Möglichkeit dar, auf physikalische Geräte zuzugreifen. Alle Statusänderungen (wie Änderung der Übertragungsgeschwindigkeit und Modus bei seriellen Leitungen) müssen durch den speziellen Systemaufruf `ioctl ()` unter Angabe des entsprechenden *special file* durchgeführt werden. Möchte man eine spezielle Hardwareeigenschaft nutzen, beispielsweise eine höhere Dichte beim Schreiben von Magnetbändern oder das automatische Zurückspulen nach dem `close ()`-Aufruf, so erzeugt der Administrator in UNIX mit `mknod ()` einen neuen *special file*, der dem Treiber die entsprechenden Parameter beim Aufruf übergibt.

---

#### Beispiel Windows NT *Virtual files und Special files*

Die Designer von Windows NT lernten von den existierenden Betriebssystemen und übernahmen den Mechanismus der *special files* von UNIX und ermöglichen den Zugriff auf Geräte, Netzwerkverbindungen etc. mittels normaler Dateioperationen. Jedem Gerät entspricht eine **virtuelle Datei** (*virtual file*), die wie alle Objekte bestimmte, festgelegte Methoden hat, die man ansprechen kann und die auf entsprechenden Dienstleistungsprozeduren der Gerätetreiber (s. nächstes [Kap. 5](#)) beruhen. Dabei teilten sie die Verwaltung des Namenraums unterschiedlichen Managern zu. Die Verwaltung der Objekte untersteht dem allgemeinen Objektmanager; die Dateisysteme, die sich auf den Geräten befinden, obliegen dagegen dem I/O- und dem Dateimanager des Kerns.

### 4.4.8 Aufgaben zu Dateikonzepten

---

#### Aufgabe 4.4-1 OpenFile

Ein Betriebssystem kann Dateioperationen auf zwei verschiedene Arten durchführen: Entweder muss der Benutzer zum Dateizugriff die Datei vorher öffnen (Normalfall), oder dies geschieht automatisch beim ersten Zugriff auf die Datei.

Im ersten Fall sieht ein Dateizugriff dann z. B. wie folgt aus:

**Open** ( ) ... **read** ( ) ... *weitere beliebige I/O-Ops* ... **Close** ( )

Im anderen Fall hätte man nur die Dateizugriffe (I/O-Ops); die Datei würde dann bei Programmende geschlossen. Welche Vor- und Nachteile besitzen die beiden Varianten?

---

#### Aufgabe 4.4-2 **write**-Befehl

Es gibt den Befehl *write*, um in Dateien zu schreiben. Gibt es solch einen Befehl auch für Verzeichnisse? Wenn ja, welchen, wenn nein, warum nicht?

---

#### Aufgabe 4.4-3 (**copy**-Befehl)

Jedes Betriebssystem kennt Befehle zur Verwaltung von Dateien. Versetzen Sie sich in die Lage eines Betriebssystemarchitekten, der einen *copy*-Befehl schreibt.

- a) Implementieren Sie den *copy*-Befehl zum Kopieren einer Datei des Betriebssystems mit den Bibliotheksfunktionen zur Dateiverwaltung (*read*, *write*, ...).
- b) Welche Änderungen müssten für den *move*-Befehl vorgenommen werden? Reicht es, erst zu kopieren und dann die Datei im Quellverzeichnis zu löschen? (Denken Sie an Schutzmechanismen!)

Wie müsste der *copy*-Befehl aussehen, um ganze Verzeichnisse mit ihren Unterverzeichnissen zu kopieren? Implementieren Sie ihn.

---

#### Aufgabe 4.4-4 Indexbaum

Gegeben sei eine hierarchisch verzweigte Indexliste, die als mehrstufige Übersetzung von logischer zu physikalischer Blocknummer dient. Sei  $m$  die Anzahl der Verzweigungen (Adressen) eines jeden Blocks einer  $n$ -stufigen Übersetzung. Wir haben  $D$  adressierbare Speichereinheiten auf der  $n$ -ten Stufe der Indexliste. Man gehe davon aus, dass im Mittel  $m/2$  Einträge pro Block durchsucht werden müssen, wobei  $m$  eine reelle Zahl sei.

Berechnen Sie das optimale  $m$ , bei dem die mittlere Suchzeit  $t$  minimal ist.

---

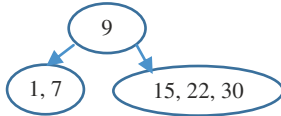
#### Aufgabe 4.4-5 **B\*-Bäume**

Zeigen Sie: Für die Aufteilung von  $2m-2$  Schlüsseln in drei Teile gilt

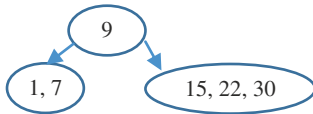
$2m - 2 = \lfloor (2m - 2)/3 \rfloor + \lfloor (2m - 1)/3 \rfloor + \lfloor (2m)/3 \rfloor$ . Untersuchen Sie dafür die drei Fälle, wenn jeweils  $2m - 2$ ,  $2m - 1$  oder  $2m$  glatt durch 3 teilbar ist.

**Aufgabe 4.4-6 B-Bäume vs. B\*-Bäume**

1. Zeichnen Sie den Baum, der sich für  $m = 4$  (4 Verzweigungen pro Container) nach Einfügen der Schlüssel 7, 22, 9, 30 ergibt
  - a) in die leere Wurzel eines B-Baumes
  - b) in die leere Wurzel eines B\*-Baumes
2. Zeichnen Sie den Baum, der sich für  $m = 4$  nach Einfügen der Schlüssel 16 und 3 ergibt
  - a) in folgenden B-Baum



- b) in folgenden B\*-Baum

**Aufgabe 4.4-7 I/O**

Warum wird in UNIX zwischen der Standardausgabe und der Standardfehlerausgabe unterschieden, wenn doch beide *per default* auf den Bildschirm gehen?

**Aufgabe 4.4-8 echo auf special devices**

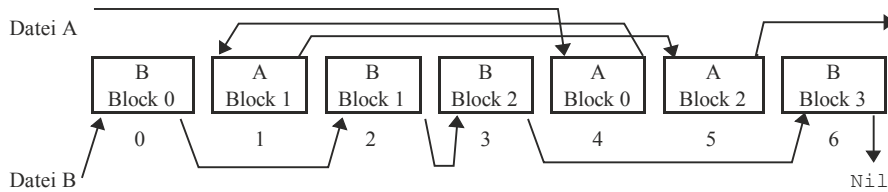
Was passiert, wenn Sie in UNIX einen Text mit dem Programm `echo` auf `/dev/tty` oder `/dev/null` ausgeben?

## 4.5 Implementierung der Dateioorganisation

Die grundsätzlichen Probleme der Implementierung einer Speicherverwaltung haben wir schon in [Abschn. 3.1](#) kennengelernt. Auch bei der Organisation des Speicherplatzes auf Magnetbändern und Festplatten können wir diese Strategien verwenden, um freien und belegten Platz zu verwalten.

### 4.5.1 Kontinuierliche Speicherzuweisung

Die Grundstruktur einer solchen Methode ist relativ einfach: Zu Beginn des gesamten Speicherplatzes steht ein Index (Verzeichnis), in dem alle Dateien aufgeführt sind, und der



**Abb. 4.15** Speicherung von Dateien mittels Listen

Platz, wo die Datei steht. Die Datei selbst ist kontinuierlich in einem Stück hintereinander geschrieben.

Dieses Konzept ist historisch gewachsen in einer Zeit, in der für jede Datei ein eigenes Magnetband reserviert war und die Dateiverzeichnisse Übersichten an Stahlschränken mit Reihen von Magnetbändern waren. Da dieses Konzept für Dateiveränderungen viel zu inflexibel ist, wird es nur noch benutzt, um viele kleine Dateien (z. B. Objekte) in einer größeren Datei (z. B. Objektbibliothek) unterzubringen.

#### 4.5.2 Listenartige Speicherzuweisung

Eine Alternative zu einem kontinuierlichen Speicherabschnitt besteht darin, den Speicher in gleich große Abschnitte (*Blöcke*) zu unterteilen und alle Blöcke einer Datei mit Zeigern zu einer Liste zu verbinden.

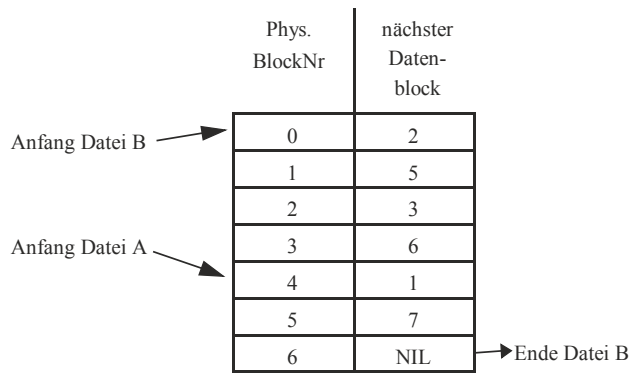
Der Vorteil einer solchen Methode liegt darin, dass auch bei einem defekten Verzeichnis (irrtümliche Löschung usw.) die gesamte Datei wiederhergestellt werden kann, wenn die Dateiblöcke doppelt verzeigert und die Dateiinformationen (Name, Zugriffsrechte etc.) auch am Anfang der Datei enthalten sind.

Der Nachteil des Ansatzes liegt im ineffizienten Zugriff auf die Datei: Möchte man den 123. Block lesen, so muss man alle 123 Blöcke vorher lesen, um durch die Liste bis zum Block 123 vordringen zu können. In [Abb. 4.15](#) ist ein solcher Ansatz gezeigt.

#### 4.5.3 Zentrale indexbezogene Speicherzuweisung

Den Nachteil der Dateilisten kann man vermeiden, indem man die Informationen über die Sequenz der Blocknummer, also die Liste aufeinanderfolgender Blöcke, nicht auf das Speichermedium verteilt, sondern in einem zentralen Block zusammenfasst. Die zentrale Indexstruktur ist für unser Beispiel dann eine Liste, in der jeder Eintrag die Blocknummer des darauffolgenden Blocks enthält, siehe [Abb. 4.16](#).

Abb. 4.16 Listenverzeichnis



Beispiel Belegungstabelle von Disketten bei MS-DOS

Unter MS-DOS ist jede Diskette in Sektoren (Datenblöcke) der Länge 512 Byte eingeteilt. Abgesehen vom Sektor 0, dem Bootstrap Sektor, ist in den Sektoren 1–5 eine Belegungstabelle (*File Allocation Table FAT*) untergebracht, von der eine Sicherheitskopie in den Sektoren 6–10 existiert. Die FAT besteht aus Einträgen der Länge 12 BITS oder 3 Hexadezimalzahlen der Werte  $000_H - FFF_H$ , so dass in  $5 \cdot 512 = 2560$  Bytes maximal  $2560/1,5 = 1706$  Einträge Platz finden. Jeder Eintrag steht für eine Speichereinheit der Diskette. Jede Speichereinheit, genannt *Cluster*, ist 1024 Bytes (2 Sektoren) lang, so dass eine MS-DOS-Diskette bei dieser Kodierung maximal 1,7 MB Kapazität erreichen kann und jede Datei mindestens 1 KB lang ist.

Jeder Tabelleneintrag hat über die reine Tatsache der Belegung hinaus folgende Bedeutung:

- $000_H$  freier Cluster
- $001_H \dots FF0_H$  Cluster belegt; Eintrag = Nummer des nächsten Clusters der Datei
- $FF1_H \dots FF7_H$  unnutzbarer, defekter Cluster (Oberflächenfehler des Materials)
- $FF8_H \dots FFF_H$  letzter Cluster einer Datei

Da die beiden ersten Einträge zur Verwaltung genutzt werden und die Daten bei Sektor 18 anfangen, ergibt sich bei 2 Sektoren pro Cluster die erste Sektornummer eines Clusters zu

$$\text{Sektornummer} = (\text{Index des Clustereintrags} - 2) \cdot 2 + 18$$

Der normale Tabelleneintrag ermöglicht es, unabhängig von der Tatsache der Belegung auch den nachfolgenden Cluster einer Datei zu bestimmen, so dass die Angabe eines Startclusters genügt, um eine Datei vollständig von der Diskette zu lesen. Es ist also sinnvoll, über die statische *belegt*-Information hinaus die Einträge zu verketten. Allerdings schränkt diese Kodierung die mögliche Anzahl der verwalteten Cluster ein; auch

bei beliebig langer FAT, die über die 5 Sektoren hinausgehen würde, kann man nur bei maximal  $2^{12} = 4096$  Index bzw. Clustereinträgen nur 4 MB an Clusterplatz verwalten.

Aus diesem Grund ist bei Festplatten das MS-DOS-Tabellenformat geändert: Der Clusterindex ist eine 16-Bit-Zahl. Dies bedeutet allerdings auch, dass mit  $2^{16} = 65.536$  Clustern der gesamte Plattenplatz abgedeckt werden muss. Bei einer 2-GB-Platte, einer durchaus üblichen Größe für PCs, bedeutet dies eine Clustergröße = Speicherplatz/Clusteranzahl von  $2^{31}/2^{16} = 2^{15} = 32.768$  Bytes. Da die meisten Dateien im Bereich von 1 KB sind, verschenkt man also im Normalfall 31 KB von 32 KB – eine sehr schlechte Speicherplatzausnutzung. Deshalb wird eine solche physische Platte bei MS-DOS in mehrere logische Platten unterteilt, die jeweils mit einem eigenen Dateisystem kleinere Clustergrößen zulassen.

Allerdings hat diese Methode das Problem, dass bei großen Dateisystemen die zentralen Listen sehr groß werden können. Da das gesamte Verzeichnis im Hauptspeicher sein muss, um es durchsuchen zu können, wird dauerhaft Platz im Hauptspeicher belegt, ohne ihn tatsächlich zu benutzen.

*Beispiel:* Bei 2 GB =  $2^{31}$  Byte Festplattenplatz und einer Blockgröße von 1 KB =  $2^{10}$  Byte muss eine Indexliste von mindestens  $2^{31}/2^{10} = 2^{21} = 2$  Millionen Einträgen geführt werden. Jeder Eintrag muss dabei alle anderen 2 Mill. Einträge referieren können, so dass 3 Byte pro Eintrag nötig sind. Dies bedeutet, dass für eine zentrale Indexliste immer 2 Mill. mal 3 Byte = 6 MB Hauptspeicher reserviert sein müssen – ohne dass die Einträge auch alle benutzt werden!

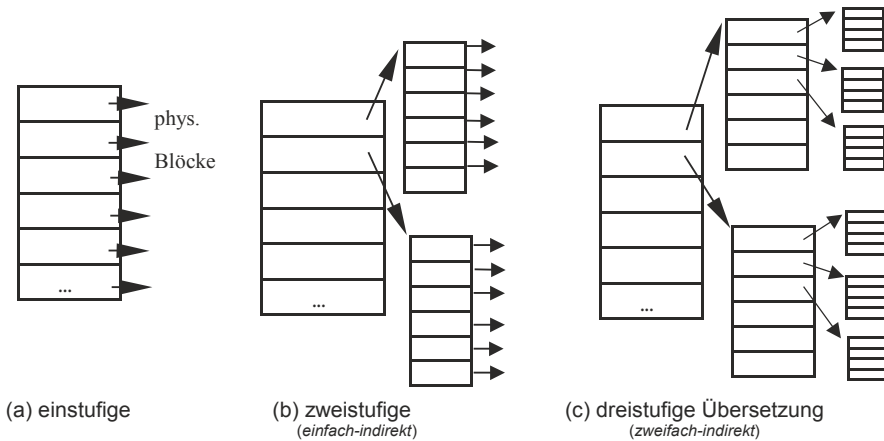
Aus diesen Gründen wurde beim Nachfolgesystem FAT 32, das auch in Windows 2000 enthalten ist, ein 32-Bit Datenwort als Clusterindex verwendet und so auch kleineren Dateilängen Rechnung getragen.

4.5.4 Verteilte indexbezogene Speicherzuweisung

Eine wichtige Alternative zu einer zentralen Indexdatei für den gesamten Massenspeicher besteht darin, für jede Datei eine eigene Indexliste zu führen und sie jeweils in einem Block abzuspeichern, vgl. Abb. 4.17. Nur wenn die entsprechende Datei tatsächlich angesprochen wird, muss der dazugehörende Indexblock eingelesen werden.

Abb. 4.17 Verteilung der Dateiindexliste auf die Dateiköpfe

| Datei A | Datei B |
|---------|---------|
| 4       | 0       |
| 1       | 2       |
| 5       | 3       |
| 7       | 6       |
| ...     | NIL     |



**Abb. 4.18** Mehrstufige Übersetzung von logischer zu physikalischer Blocknummer

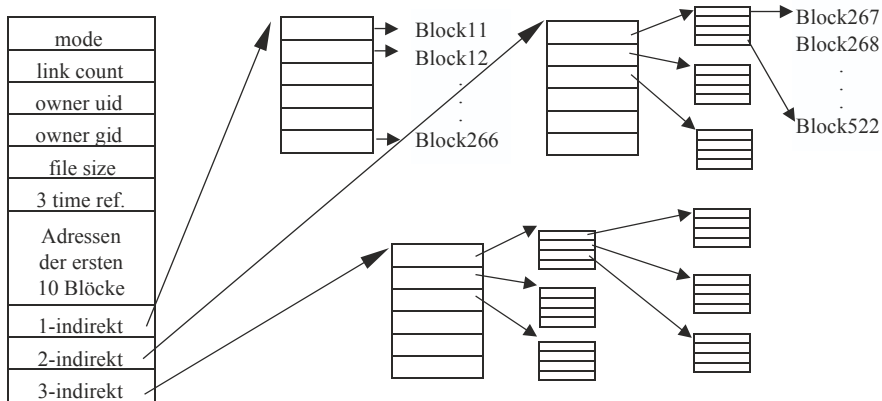
Die Indexlisten werden natürlich länger, je länger die dazugehörigen Dateien werden. Nun beobachtet man in existierenden Dateisystemen, dass sehr viele Dateien mit wenigen Blöcken und wenige Dateien mit vielen Blöcken existieren. Aus diesem Grund kann man den Hauptindexblock so auslegen, dass er für die meisten Dateien ausreichend ist. Für die wenigen Dateien, die mehr Indexplatz benötigen, müssen wir einen weiteren Indexblock anlegen, dessen logische Adresse am Ende der Indexliste steht. Sind die Dateien sehr groß, so wird auch dies zu langsam, da dann alle Indexblöcke sequentiell von der Platte gelesen werden müssten, um die physikalische Adresse des gesuchten Blocks zu finden. Es ist stattdessen sinnvoll, die verzeigten Indexlisten hierarchisch zu organisieren und so die Übersetzung der logischen Blockadresse der Datei zur physikalischen Adresse des Geräts zu beschleunigen. In [Abb. 4.18](#) ist das Prinzip dafür verdeutlicht, das stark an die mehrstufige Übersetzung von virtuellem zu physikalischem Adressraum erinnert, vgl. [Abschn. 3.3](#).

#### 4.5.5 Beispiel UNIX: Implementierung des Dateisystems

Das bekannteste Betriebssystem, das die zuletzt erwähnte Methode verwendet, ist UNIX. Hier ist eine spezielle Variante im Gebrauch, die versucht, in einem Indexblock sowohl die relevante Information für kurze als auch für lange Dateien zu halten.

Dazu besteht der erste Indexknoten (**i-node**) aus einem allgemeinen Teil, in dem Namen usw. vermerkt sind. Danach folgen die ersten 10 Adressen der Datenblöcke, falls die Datei kleiner als 10 Blöcke (10 kB) ist. Ist sie größer, so müssen die weiteren Blöcke extra untergebracht werden. Dazu folgt ein Zeiger zu einem Block mit bis zu 256 Einträgen von Blocknummern der Platte. Reichen diese Angaben für  $10 + 256 = 266$  kB Plattenplatz nicht aus, so wird ein Zeiger zu einem zweifach- und danach, wenn diese weiteren  $2^8 \cdot 2^8 = 2^{16}$  Blöcke = 67 MB nicht ausreichen, zu einem dreifach-indirekten Übersetzungsbaum eingeführt. In [Abb. 4.19](#) ist dies für ein Beispiel übersichtsweise gezeigt.





**Abb. 4.19** Die Struktur einer i-node in UNIX

Der oben angedeutete Inhalt einer *i-node* ist in konkreten Systemen modifiziert und erweitert mit wesentlich mehr internen Angaben, z. B. in System V mit der Semaphorinformation zur Sperrung der Datei, Platz für Zeiger, wenn die *i-node* als freier Speicherplatz in die doppelt verkettete *free*-Liste gehängt wird, Einträgen für die `mount()`-Tabelle, Statusinformation für den Zugriff im Netzwerk usw. Auch die doppelt- und dreifach-indirekte Tabelle ist in System V nicht vorhanden, dafür aber Zeiger zur Fortsetzung der *i-node* in weiteren Blöcken.

Jeder Datei ist also genau eine *i-node* zugeordnet, auch den Dateiverzeichnissen (*directory*) selbst als einer gespeicherten Datei. Die *i-node*-Nummern werden sequentiell vergeben, ihre korrespondierende, aktuelle Lage auf dem Massenspeicher (phys. Blocknummer) ist in einem speziellen Block (*super node*) zusammengefasst. Wird die *super node* zerstört, so ist das Dateisystem unbrauchbar.

Die eindeutige Schlüsselreferenz einer Datei ist die *i-node*-Nummer, auch wenn mehrere Referenzen (Dateinamen) für dieselbe Datei in verschiedenen Verzeichnissen existieren.

Ein Verzeichnis ist deshalb sehr einfach aufgebaut: Es enthält nur die *i-node*-Nummer der Datei und den Namen (sowie Länge des Namens und des Eintrags), mehr nicht. Alles andere, wie Zugriffsrechte usw., ist im ersten Block der Datei gespeichert. Da jedes Verzeichnis eine Länge besitzt, die ein Vielfaches der Blockgröße (also einer schnell transportierbaren Speichereinheit) ist, können Verzeichnisse schnell und effizient gelesen und gespeichert werden. Die zusätzliche Angabe der Namenslänge bzw. Eintragsgröße (= auf Wortgrenze erweiterte Namenslänge) erlaubt es, auch kurze Namenseinträge effizient in einem Verzeichnis zu speichern.

Die Grundstruktur des UNIX-Dateisystems ist also zur schnelleren Suche wie ein Baum aufgebaut und liegt fast allen anderen Dateisystemen zugrunde. In der heutigen Zeit sind darüber hinaus noch weitere Dateisysteme für Unix hinzugekommen, die in vielen Punkten weit über das Grundsystem hinausgehen. So wurde von Oracle seit 2007 das *btrfs* (*B-tree file system*) als freie Software sowohl für Linux als auch für Windows freigegeben und bei *facebook* weiterentwickelt. Es ist als balancierter Baum (B-Baum, s. Seite 210) mit den

Datenelementen in den Blättern (B<sup>+</sup>-Baum) aufgebaut und kann so besonders schnell (in logarithmischer Zeit) durchsucht werden, wachsen und schrumpfen. Außerdem nutzt es die *copy on write*-Idee, um Datenblöcke bei Kopieroperationen erst dann neu zu schreiben, wenn sie verändert wurden. Ansonsten wird nur eine Referenz zu den Originaldaten statt der Kopie verwendet. Die Unterstützung von RAID-Datenreplikation, Prüfsummen, inkrementelle Datensicherung, Datenkompression, online-Defragmentierung und dynamische i-nodes macht es auch für kommerzielle Anwendungen interessant.

Die hauptsächliche Motivation, ein neues Dateisystem einzuführen, ist vor allem die maximale Größe einer einzelnen Datei sowie die maximale Größe des gesamten Dateisystems (Anzahl der Dateien, Anzahl der Ordner, Schachtelungstiefe, Gesamtzahl speicherbarer Datenblöcke). Für *btfs*, das bereits als Nachfolger des Unix-Standardbetriebssystems *ext3* bzw. *ext4* gehandelt wird, kann eine Datei maximal 16 Exbibyte ( $2^{64}$  Byte),  $2^{64}$  Dateien in  $2^{64}$  Bytes Gesamtspeicher enthalten. Dies sollte für die meisten Anwendungsfälle ausreichend sein.

#### 4.5.6 Beispiel Windows NT: Implementierung des Dateisystems

Die Grundeinheit zur Dateiverwaltung ist in Windows NT ein *volume*. Dies ist eine logische Massenspeichereinheit; sie kann sowohl aus einer einzigen Festplatte als auch aus mehreren, auf verschiedenen Platten existierenden Speicherbereichen bestehen, die nach außen zu einer logischen Einheit zusammengefasst wurden.

Jedes *volume* besteht prinzipiell nur aus einer Sorte Daten: aus Dateien (*files*), die durchnummeriert sind. Jede Datei hat eine eindeutige 64-Bit-Nummer (*file reference number*); sie besteht aus einer 48-Bit-Dateinummer und einer zusätzlichen 16-Bit-Sequenznummer. Die Sequenznummer wird nach jedem Löschen der zur Dateinummer gehörenden Datei inkrementiert, so dass man zwischen der Referenz zu einer inzwischen gelöschten und der aktuellen Datei unterscheiden kann, auch wenn sie die gleichen Dateinummern referieren.

Die Vorteile des Konzepts, alle Informationen als Dateien zu präsentieren, sind:

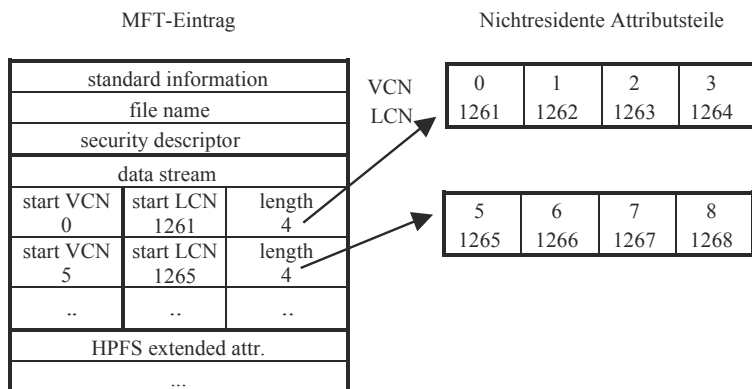
- einfacher, konsistenter Dateizugriffsmechanismus, sogar auf die *boot*-Information.
- Die Sicherheitsinformationen sind getrennt nach Verwaltungsteilen (Tabellen, Objekte) und damit besser anpassbar an die inhaltlichen Notwendigkeiten der jeweiligen Verwaltungsfunktion.
- Werden Plattenteile unbrauchbar, so können die Verwaltungsinformationen unsichtbar für den Benutzer auch auf andere Plattenteile verlagert werden.

Jedes *volume* enthält eine Datei, die zentrale Tabelle *Master File Table* MFT, in der alle Dateien verzeichnet sind einschließlich ihrer selbst, siehe [Abb. 4.20](#).

Der Einträge für die MFT selbst enthält alle Daten (Start, Länge etc.) der MFT als Datei. Eine Sicherheitskopie ist ebenfalls als Datei eingetragen. In [Abb. 4.21](#) ist ein solcher Eintrag visualisiert. Die einzelnen Dateiattribute sind in [Abschn. 4.3.2](#) näher erläutert. Die Zuordnung der virtuellen, bei 0 beginnenden linear aufsteigenden virtuellen

|          |                                                                                                                                                                                                                                                                    |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Datei 0  | <i>Master File Table</i>                                                                                                                                                                                                                                           |
| Datei 1  | <i>Sicherheitskopie der MFT</i> für die Metadateien, in Plattenmitte lokalisiert                                                                                                                                                                                   |
| Datei 2  | <i>log file</i> : Alle Operationen, die die NTFS-Struktur ändern, werden hier verzeichnet und garantieren so atomare Dateitransaktionen.                                                                                                                           |
| Datei 3  | <i>volume file</i> : Statusinformationen des <i>volume</i> wie Name, NTFS-Version usw. Ist das <i>corrupted</i> -Bit gesetzt, so muß das <i>chkdsk</i> -Programm durchgeführt werden, das das Dateisystem auf Konsistenz prüft und festgestellte Fehler repariert. |
| Datei 4  | <i>attribute definition table</i> : Hier sind alle Attributstypen verzeichnet, die im <i>volume</i> vorkommen, sowie ihr Status, beispielsweise ob sie indiziert oder wiederhergestellt werden können.                                                             |
| Datei 5  | <i>root directory</i> : Name des Wurzelverzeichnisses, z. B. „\“                                                                                                                                                                                                   |
| Datei 6  | <i>bitmap file</i> : Belegungstabelle der <i>volume</i> -Speichereinheiten (Cluster), vgl. Abschn. 3.1.1                                                                                                                                                           |
| Datei 7  | <i>boot file</i> : Diese Datei wird durch das Formatierungsprogramm erzeugt und enthält den <i>bootstrap</i> -Code von Windows NT und die physikalische Geräteadresse der MFT-Datei.                                                                               |
| Datei 8  | <i>bad cluster file</i> : Verzeichnis der unbrauchbaren <i>volume</i> -Cluster                                                                                                                                                                                     |
| ~        | ...                                                                                                                                                                                                                                                                |
| Datei 16 | Normale Benutzerdateien und Verzeichnisse                                                                                                                                                                                                                          |

**Abb. 4.20** Die Reihenfolge der ersten 16 Dateien, der NTFS-Metadaten Files



**Abb. 4.21** Die Struktur eines MFT-Eintrags

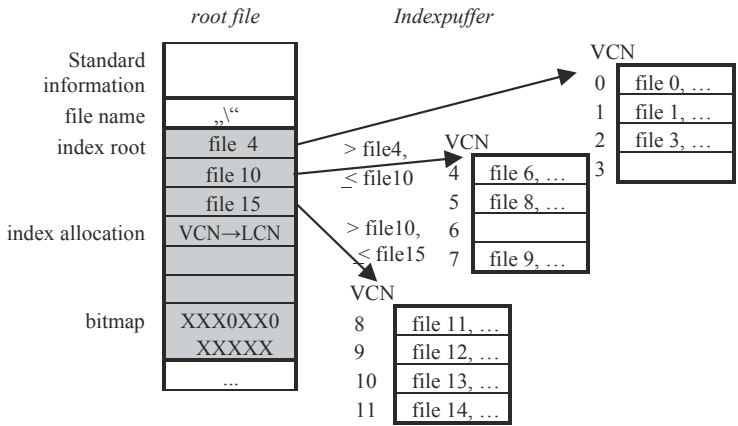
Clusternummern VCN zu den logischen Clusternummern LCN des Geräts, auf dem die Daten liegen, ist ebenfalls in der Dateibeschreibung enthalten. Da der Speicherplatz normalerweise nicht zusammenhängend ist, liegen die einzelnen Speicherplatzstücke als Einträge in der Liste vor. Die eigentlichen Daten werden dabei als „erweiterte, nichtresidente Attribute“ betrachtet. Reicht der reservierte Platz im Eintrag für diese Liste nicht aus, etwa weil die Datei zu stark fragmentiert ist, so ist im *data stream* die Information hinterlegt, in welchen zusätzlichen Clustern die weiteren Listeneinträge stehen.

Existieren Speicherblöcke, die nur aus Nullen bestehen, und ist als Standardattribut „komprimiert“ für die Datei angegeben, so werden die leeren Blöcke nicht gespeichert. Stattdessen werden bei der Zählung der virtuellen Cluster die entsprechenden Clusterblöcke einfach übersprungen, so dass in einer VCN-Sequenz „Löcher“ existieren können, siehe die VCN- und LCN-Bezeichnungen der Datenblöcke in [Abb. 4.21](#).

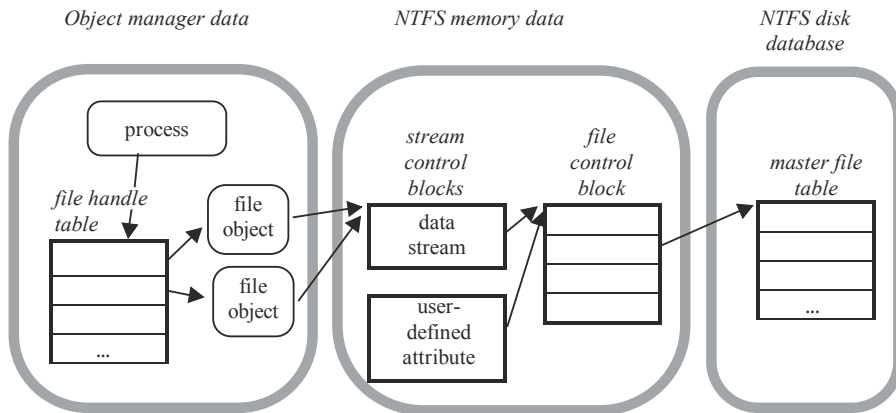
Beim Lesen der Daten wird dies vom NTFS-Dateisystem bemerkt und statt dessen entsprechende, mit Nullen initialisierte Blöcke zurückgegeben. Enthalten die Blöcke von „komprimiert“-Dateien keine Nullen, so wird ein allgemeiner Kompressionsalgorithmus angesprochen, der aber auf Schnelligkeit und nicht auf Kompressionsleistung optimiert wurde. Bei einem Verzeichnis (*directory*) besteht der Datenstrom aus einem Index mit Dateinamen, deren Puffer in Form eines B\*-Baumes (wie bei HPFS in OS/2) organisiert sind, vgl. [Abschn. 4.4.4](#).

Dieser Index besteht aus drei Teilen: zum einen aus mehreren, in lexikalischer Reihenfolge indizierten Namen (*index root*) und ihren Angaben, zum zweiten aus den Zuordnungstabellen VCN→LCN der Zusatzpuffer (nichtresidente Daten) (*index allocation*) und zum dritten aus den Belegungstabellen (*bitmaps*) dieser Puffer. In [Abb. 4.22](#) ist als Beispiel die Datei des Wurzelverzeichnisses (*file reference number 5* der MFT) gezeigt.

Jeder Eintrag (hier: „file x“) besteht aus der *file reference number*, dem Namen, Zeitstempel und Dateilänge, die von der MFT hierher kopiert wurden. Damit ist zwar ein zusätzlicher Synchronisationsaufwand verbunden, der sich aber durch kürzere Suchzeiten



**Abb. 4.22** Die Dateistruktur des Wurzelverzeichnisses



**Abb. 4.23** Datenstrukturen für eine NTFS-Datei

in Verzeichnissen lohnt. Die Dateinamen sind, wie erwähnt, in lexikalischer Reihenfolge indiziert. Jeder Dateiname verweist auf einen Indexpuffer, der Einträge mit kleinerem Index (z. B. „file1“, „file2“, „file3“ vor „file4“) enthält. Ein solcher B\*-Baum wächst dabei eher in die Breite als in die Tiefe und erlaubt so schnelle Suchoperationen.

Die Zusatzpuffer enthalten die Dateiangaben in gepackter Form; bei der erstmaligen Benutzung (*mount*) des Dateisystems muss die Datenstruktur im Speicher ausgepackt werden. Dadurch ist pro Puffer von 2 KB oder 4 Cluster mit je 512 Byte die Speicherung von ca. 15 Dateinamen möglich.

In der folgenden [Abb. 4.23](#) ist ein Gesamtüberblick über die Datenstrukturen gegeben, die zur Benutzung einer NTFS-Datei nötig sind.

Dabei kann man drei Bereiche unterscheiden: die Datenstrukturen, die vom Objektmanager für einen Prozess im Speicher aufgebaut werden, die Datenstrukturen, die vom NTFS-Treiber im Speicher errichtet werden und die Strukturen, die der NTFS-Treiber auf dem eigentlichen Massenspeicher zur Verwaltung einrichtet. In der Abbildung ist jeder Bereich grau umrandet.

Die maximale Dateigröße, Anzahl der Dateien und maximale Dateisystemgröße hängen stark von der jeweiligen Betriebssystemversion ab. Derzeitig unterstützt Windows NT Version 10 eine maximale Dateigröße von  $2^{28}$  Cluster, also 16 TiB ( $2^{44}$  Byte) bei einer Clustergröße von 64 KiB =  $2^{16}$  Byte. Die maximale Größe des gesamten Dateisystems umfasst  $2^{64}$  Cluster bei einer Clustergröße von 1 KiB, implementierungsbedingt allerdings nur 256 TiB ( $2^{48}$  Byte), d. h.  $2^{32}$  Cluster bei einer Clustergröße von 64 KiB ( $2^{16}$  Byte). Bei kleineren Clustergrößen ergeben sich entsprechend kleinere Maximalwerte.

Seit Windows Vista wurde das Dateisystem generell auf mögliche atomare Transaktionen (*Transactional NTFS*, TxF) erweitert und basiert auf dem ebenfalls eingeführten *Kernel Transaction Manager* (KTM), der generell für alle atomaren Aktionen im System zuständig ist, etwa auch Dateitransaktionen innerhalb von Netzwerken. Da der TxF-Service allerdings sehr wenig genutzt wird, rät Microsoft von dessen Benutzung durch Anwendungen ab, da er zukünftig wegfallen könnte.

## 4.5.7 Aufgaben zu Dateimplementierungen

### Aufgabe 4.5-1 i-Nodes und Bitmaps

- a) Zwei Informatikstudentinnen, Caroline und Leonore, diskutieren über Indexknoten (I-nodes). Caroline behauptet, dass Speicher so schnell und billig geworden ist, dass es sinnvoll ist, eine Kopie der I-node in den Cache des Prozesses zu laden und dort direkt damit zu arbeiten, und nicht bei jedem Zugriff auf die Datei umständlich das Original auf der Platte zu benutzen.

Leonore ist anderer Meinung.

Wer hat Recht?

- b) Das UNIX – Dateisystem hat 1 KiB – Blöcke und 4 Byte Plattenadressen. Was ist die maximale Dateigröße, falls die I-nodes zehn direkte Einträge und jeweils einen einfach-, einen doppelt und einen dreifach-indirekten Eintrag besitzen?
- c) Nachdem eine Plattenpartition formatiert worden ist, sieht der Anfang einer Bitmap zur Verwaltung der freien Speicherblöcke so aus: 1000 0000 0000 0000, wobei der erste Block durch das Wurzelverzeichnis belegt sei. Das System sucht nach freien Blöcken, indem es immer mit der niedrigsten Blocknummer beginnt.

Nachdem die Datei A geschrieben worden ist und nun sechs Blöcke belegt, sieht die Bitmap folglich so aus: 1111 1110 0000 0000. Stellen Sie die Bitmap nach jeder der folgenden weiteren Aktionen dar:

- Die Datei B wird geschrieben, B benötigt fünf Blöcke.
- Die Datei A wird gelöscht.
- Die Datei C wird geschrieben, C benötigt acht Blöcke.
- Die Datei B wird gelöscht.

### Aufgabe 4.5-2 Speicherverwaltung von Dateien

Die zusammenhängende Speicherung von Dateien führt zu Plattenfragmentierung. Handelt es sich dabei um interne oder externe Fragmentierung?

## Literatur

- Comer D.: The Ubiquitous B-Tree. ACM Computing Surveys 11, 121–137 (1979)
- Custer H.: Inside the Windows NT File System. Microsoft Press, Redmond, Washington 1994
- Knuth D.: The Art of Computer Programming, Vol. 3: Sorting and Searching. Addison-Wesley, Reading, MA, 2<sup>nd</sup> ed., 1998

Inhaltsverzeichnis

5.1 Die Aufgabenschichtung ..... 222

5.1.1 Beispiel UNIX: I/O-Verarbeitungsschichten ..... 224

5.1.2 Beispiel Windows NT: I/O-Verarbeitungsschichten ..... 227

5.2 Gerätemodelle ..... 229

5.2.1 Geräteschnittstellen ..... 230

5.2.2 Initialisierung der Geräteschnittstellen ..... 231

5.2.3 Plattenspeicher ..... 231

5.2.4 SSD-RAM-Disks ..... 237

5.2.5 Serielle Geräte ..... 239

5.2.6 Aufgaben zu I/O ..... 240

5.3 Multiple Plattenspeicher: RAID's ..... 240

5.3.1 RAID-Ausfallwahrscheinlichkeiten ..... 246

5.3.2 Aufgaben zu RAID ..... 248

5.4 Modellierung und Implementierung der Treiber ..... 249

5.4.1 Beispiel UNIX: Treiberschnittstelle ..... 250

5.4.2 Beispiel Windows NT: Treiberschnittstelle ..... 252

5.4.3 I/O Treiber in Großrechnern ..... 255

5.4.4 Aufgaben zu Treibern ..... 255

5.5 Optimierungsstrategien für Treiber ..... 256

5.5.1 Optimierung bei Festplatten ..... 256

5.5.2 Pufferung ..... 257

5.5.3 Synchrone und asynchrone Ein- und Ausgabe ..... 259

5.5.4 Aufgaben zur Treiberoptimierung ..... 261

Literatur ..... 261

Die Leistungsfähigkeit eines Computersystems hängt nicht nur vom Prozessortyp und von der Wortbreite (16, 32 oder 64 Bit) ab, sondern im wesentlichen von der Geschwindigkeit,

mit der die Daten zwischen den Ein- und Ausgabegeräten (Massenspeicher, Netzwerkanschluss etc.) und dem Prozessor/Hauptspeichersystem transferiert werden können. Im Unterschied zu meist rein wissenschaftlichen Anwendungen, wo die Rechenleistung (Millionen *FLießkomma-Operationen pro Sekunde* MFLOPS) gefragt ist, bestehen die Anforderungen normaler Rechanlagen aus einer bunten Mischung sehr verschiedener Programmarten: Rechenteile, Datenbankapplikationen, Verwaltungsaufgaben usw. Deshalb verwenden alle leistungsmessenden Programme (*benchmark-Programme*) Aufgaben, deren Abarbeitung stark vom Gesamtsystem aus Prozessor, Speicher, Massenspeicher und Datentransferarchitektur bestimmt ist.

Dabei spielt die Datenein- und -ausgabe eine wichtige Rolle. Aus diesen Gründen sehen wir uns nun nach der Prozessor- und Speicherverwaltung in den vorigen Kapiteln die Ein- und Ausgabearchitektur von Betriebssystemen näher an.

---

## 5.1 Die Aufgabenschichtung

In Betriebssystemen waren früher die Wechselbeziehungen zwischen Anwenderprogramm und Ein- und Ausgabegeräten sehr innig – jeder Anwenderprogrammierer setzte sein eigenes, „effizientes“ System auf, um den Datenfluss zwischen seiner Anwendung und dem Peripheriegerät zu steigern. Dieser Ansatz führte aber nicht nur dazu, dass jeder sein eigenes, geräteabhängiges Programm hatte, sondern auch zu Fehlern und Überschneidungen, wenn mehrere Programme auf dasselbe Gerät zugreifen wollten (z. B. MS-DOS). Da dies in *multi user*-Umgebungen nicht mehr tragbar war, wurden die gerädetypischen Programmteile abgetrennt und als eigene Module (**Treiber**) ins Betriebssystem integriert. Dies fördert nicht nur die Portabilität eines Programms über unterschiedliche Rechnerarchitekturen hinweg und hilft, Fehler zu vermeiden, sondern erspart auch dem Anwendungsprogrammierer Arbeit.

Die Grundaufgabe eines Treibers ist es also, alle gerätespezifischen Initialisierungsschritte und Datentransfermechanismen vor dem Anwenderprogramm hinter einer einheitlichen, betriebssystemspezifischen Schnittstelle zu verbergen. In unserer Notation aus [Kap. 1](#) ist der Treiber also eine *virtuelle Maschine*; er vermittelt zwischen Betriebssystem und physikalischem Gerät.

Die Aufgaben eines Treibers sind auf die Initialisierung der Datenstrukturen und des Geräts sowie Schreiben und Lesen von Daten begrenzt. Zusätzlich kommen aber noch weitere Aufgaben hinzu, die nur mit dem Betriebssystem zusammen durchgeführt werden können:

- Übersetzung vom logischen Programmiermodell zu gerätespezifischen Anforderungen
- Koordination der schreibenden und lesenden Prozesse für das Gerät
- Koordination verschiedener Geräte gleichen Typs
- Pufferung der Daten
- usw.



Diese zusätzlichen Aufgaben kann man in einer weiteren Softwareschicht zusammenfassen, so dass im Allgemeinen mehrere Schichten virtueller Maschinen oder Treiber zwischen dem Anwenderprozess und dem physikalischen Gerät liegen. In Abb. 5.1 ist dies illustriert.

Die Einführung einer Schichtung erlaubt es, zusätzliche Aufgaben für die Datenbearbeitung in Form von Extraschichten in die Bearbeitungsreihenfolge einzufügen. Beispielsweise sieht ein Plattentreiber die Platte als ein Speichergerät an, dessen Speicheradressen durch eine Vielzahl verschiedener Parameter wie Laufwerks-, Sektor-, Plattennummer usw. bestimmt ist. Er übersetzt die Schreib- und Leseanforderungen, die von einem einfachen, linearen Modell von 0..N Speicheradressen ausgehen, in die Adressierlogik des Plattenspeichers. Dieser Umsetzung von logischer zu physikalischer Adresse kann man nun einen weiteren Treiber vorschalten: Die Umsetzung von einer logischen, relativen Adresse innerhalb einer Datei zu der logischen, absoluten Adresse des Speichergeräts, auf dem sich die Datei befindet, wird ebenfalls von einem Dateitreiber durchgeführt.

Die Problematik, für ein neues oder existierendes Gerät einen passenden Treiber zu entwickeln, sollte nicht unterschätzt werden. Fast alle neueren Betriebssysteme leiden nicht nur unter dem Problem, zu wenige Anwendungen zu haben, sondern insbesondere darin, dass die Treiber neuer Geräte meist nur für die am meisten verkauften Betriebssysteme entwickelt werden und damit die Verbreitung des Betriebssystems behindern. Es ist deshalb für jedes Betriebssystem wichtig, eine einfache Schnittstelle für Treiber (oder noch besser: ein Entwicklungssystem) öffentlich zur Verfügung zu stellen.

Eine interessante Initiative bildet in diesem Zusammenhang die seit 1994 im Unix-Bereich agierende *Uniform Driver Interface* UDI-Initiative aus mehreren großen Firmen. Sie versuchen, eine betriebssystemunabhängige, plattformneutrale Treiberschnittstelle zu entwickeln, um die Verwendung neuer Hardware auf möglichst vielen Systemen zu beschleunigen. Das UDI-Konzept ist in Abb. 5.2 visualisiert.

Die UDI-Spezifikation besteht dabei zum einen aus einer Programmierbeschreibung des UDI-Treibers und zum anderen aus einer plattformunabhängigen Ablaufumgebung, die nach oben eine Schnittstelle zum Betriebssystem hat und nach unten zur Hardwareplattform und damit dem eigentlichen Treiber eine einheitliche Umgebung anbietet. Zwar müssen sowohl Betriebssystemschnittstelle als auch Hardwareabhängigkeiten für die Ablaufumgebung programmiert werden, aber nur einmal für jedes Betriebssystem und nur einmal für jede Hardware. So wird die Vielfalt aller Kombinationen aus Hardware und Betriebssystem vermieden, der Programmieraufwand sinkt und die Zuverlässigkeit und Wartbarkeit erhöht.

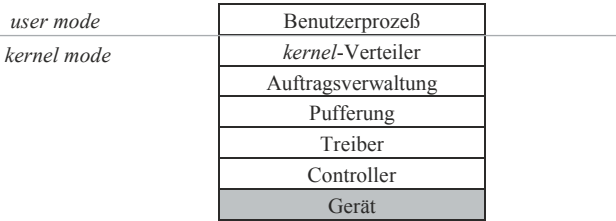
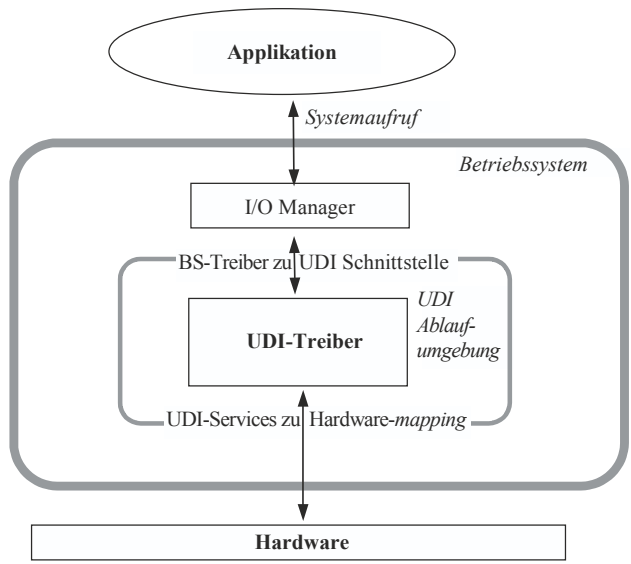


Abb. 5.1 Grundsichten der Geräteverwaltung

**Abb. 5.2** Die Schichtung und Schnittstellen des UDI-Treibers



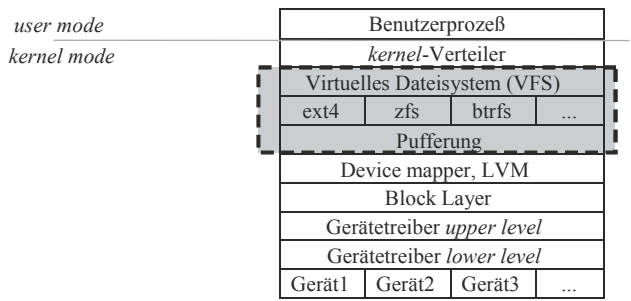
5.1.1 Beispiel UNIX: I/O-Verarbeitungsschichten

Auch Unix ist nach verschiedenen Gesichtspunkten in Schichten eingeteilt. An dieser Stelle sollen zwei Schichtungskonzepte vorgestellt werden: Das Konzept des virtuellen Dateisystems und das der Datenströme und Filter (*streams*-Konzept).

**Dateisystemtreiber**

Unter Unix gibt es verschiedene Dateisysteme. Aus diesem Grund gab es schon früh das Bestreben, den Zugriff auf Dateien mit Standardfunktionen einer Dateisystemschnittstelle durchzuführen und die eigentliche Implementierung auf nachfolgende Schichten zu verlagern. Ein gutes Beispiel dafür ist die Schichtung in Linux, gezeigt in [Abb. 5.3](#). Die Dateisystemschiicht, grau schraffiert in der Zeichnung, besteht selbst wieder aus Unterschichten: dem virtuellen Dateisystem VFS, das die Verwaltung der Dateinformationen (*i-nodes*, *i-node cache*, Verzeichniscache) und die Anbindung an die Benutzerprozesse

**Abb. 5.3** Die Dateisystem-schichtung unter Linux



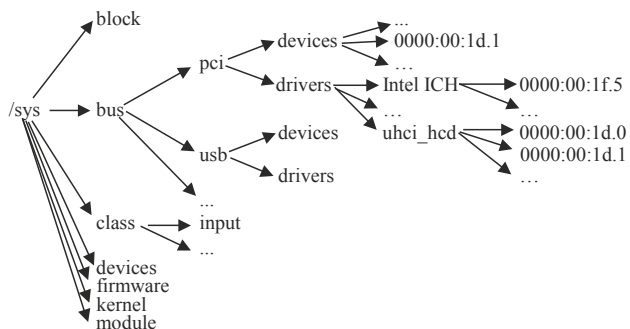
vornimmt, und den eigentlichen Dateisystemen wie das ext4 fs, btrfs, etc. die unterschiedlichen Strategien der Implementierung von Dateioperationen (Lesen, Schreiben) und Verzeichnissen durch Allokation und Freigabe von Blöcken bereitstellen. Eine gemeinsame Pufferung dient allen Dateisystemen.

Ab Version 2.6 gibt es in Linux ein spezielles Dateisystem: das System-Dateisystem. Es implementiert den Gedanken, alle rechner-spezifische Hardware-daten, die eigentlich nur in den Treibern stehen, dem Administrator einheitlich in Form einer Baumstruktur zur Verfügung zu stellen, siehe [Abb. 5.4](#). Dazu wird das virtuelle, systemeigene Dateisystem SysFS verwendet, das üblicherweise auf das Verzeichnis/sys gemountet wird. Nach dem *mount* lassen sich die Daten bequem als Dateien lesen und verändern. In *sys* sind die wichtigsten Äste des SysFS gezeigt. Die einzelnen Äste des Baumes sind Kategorien, nach denen die Informationen gegliedert sind. Beispielsweise ordnet die Kategorie „Klassen“ alle Geräte nach traditionellen Funktionen wie „Eingabegeräte“, „Netzwerke“ oder „Konsolen“. Die Kategorie „Busse“ wiederum zeigt als Attribute bzw. Dateien die Art und Weise, wie die Geräte mit dem System verbunden sind. Hier sind sowohl der serielle USB-Bus als auch der parallele PCI-Bus zu finden mit ihren angeschlossenen Geräten bzw. Karten. Bei „Geräten“ sind auch virtuelle Geräte möglich.

Alle Hardwareeinheiten sind dabei einheitlich in Form von Kernelobjekten (*kobject*) definiert, so dass formal nicht nur ein USB-Gerät im laufenden Betrieb entfernt werden kann (*hot pluggable*), sondern im Extremfall auch ein Prozessor in einem Multiprozessorsystem.

Die Zusammenfassung und Management der physikalischen Blöcke von Geräten und ihre Abbildung auf logische Geräte (virtuelle Geräte) wird im *btrfs* durch eine Zwischenschicht, den *Logical Volume Manager LVM*, erreicht. Er befindet sich in der *device mapper*-Schicht (s. [Abb. 5.3](#)) und ermöglicht damit die Beschreibung der Dateien mittels logischer Blöcke eines einzigen Massenspeichers, der dann dynamisch mittels Tabellen auf echte Blöcke mehrerer echter Festplatten abgebildet werden können. Dies ermöglicht es, zur Laufzeit bei Bedarf neue Geräte mit neuem Speicher hinzuzufügen und damit das Dateisystem konsistent zu erweitern oder fehlerhafte Geräte zu ersetzen. Auch die transparente Parallelisierung von Dateizugriffen auf verschiedene physikalische Geräte wird so möglich, so dass *thrashing* automatisch abgemildert werden kann.

**Abb. 5.4** Beispiel für einen SystemFS-Dateibaum



## Das stream-System

Die grundsätzliche Schichtung des UNIX-Kerns wurde in [Abb. 1.7](#) gezeigt. Zusätzlich zu den normalen Schichten ist es im *stream system*, einer UNIX-Erweiterung in System V (HP-UX, SUN-OS, ...), möglich, andere Bearbeitungsstufen („Treiber“) in den Bearbeitungsablauf einzufügen. In [Abb. 5.5a](#) ist dies für das Terminalsystem gezeigt, in [Abb. 5.5b](#) für eine Festplatte.

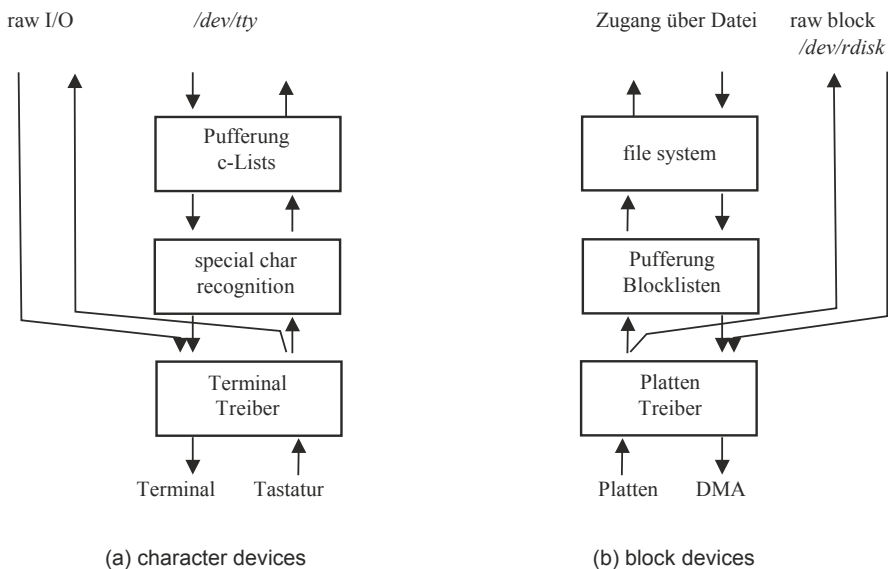
Beim zeichenorientierten Gerät in [Abb. 5.5a](#) ist ein Treiber (*special char recognition*) in den Verarbeitungsweg eingeschoben, der besondere Buchstaben, die als Kommandos dienen (z. B. DEL zum Löschen des letzten Buchstabens, Control-C zum Abbruch des laufenden Prozesses usw.), sowie besondere Zeichenkonversion (z. B. 6 Leerzeichen für ein TAB-Zeichen usw.) erkennt und entsprechende Aktionen veranlasst.

Ein gesonderter Zugang (*raw interface*) erlaubt es, ohne eine „höhere“ Verarbeitung des Zeichenstroms (ohne lokales Echo, ohne Control-C usw.) direkt Zeichen zu senden und zu empfangen. Dieser Weg ist besonders für serielle Datenverbindungen zwischen Computern interessant zum Zwecke des Datenaustauschs, da hier alle Zeichen als Daten aufgefasst werden müssen und nicht als Buchstaben oder Kontrollsignale.

## Blocktreiber-Zwischenschicht

Unter Linux (V4.0) wie auch unter anderen Unix-Derivaten (NetBSD, DragonFlyBSD) gibt es noch einige Unterschichtungen, speziell für Block-orientierte Ein- und Ausgabe.

Zum einen gibt es die Zwischenschicht des *device mappers*, siehe [Abb. 5.3](#). Diese Schicht hat mehrere mögliche Funktionalitäten. So bildet sie die Verbindung von logischen zu physikalischen Blöcken mittels des LVM, kann aber auch beispielsweise den



**Abb. 5.5** Das Stream-System in UNIX

Datenstrom (den Block) verschlüsseln (*dm-crypt*), ein Software-RAID erzeugen und verwalten (*dm-raid*), Pufferfunktionen bereitstellen (*dm-cache*) oder den Systemzustand festhalten und abspeichern. Interessanterweise ist jede Funktion „stackable“, also dynamisch hintereinander aufrufbar. Die Zwischenschicht kann also mehrmals hintereinander sich aufrufen und dann durchlaufen werden, bevor die nächste Schicht angesprochen wird.

Eine weitere Zwischenschicht ist der *block layer*. Hier werden die Blöcke gesammelt und mit Hilfe von mehreren Warteschlangen auf Anfrage (Interrupt!) den eigentlichen Gerätetreibern zur Verfügung gestellt.

Den Abschluss bildet die Zwischenschicht der allgemeinen (*upper level*) Gerätetreiber, die je nach Typ die Gemeinsamkeiten der jeweiligen Gerätetypen verwalten. Sie spricht dann mithilfe der *low level*-Treiber aus der letzten Zwischenschicht die Firmen-spezifischen Treiber an, die dann tatsächlich die Hardware bedienen.

5.1.2 Beispiel Windows NT: I/O-Verarbeitungsschichten

Die Schichtung in Windows NT ist dynamisch und hängt vom jeweiligen Systemdienst ab. In Abb. 5.6 ist links die einfache Schichtung für serielle Geräte gezeigt, rechts die multiple Schichtung für Massenspeicher.

Dateisystemtreiber

Alle Betriebssystemaufrufe für Ein- und Ausgabe werden im I/O-Manager zu Aufträgen in Form eines *I/O Request Package* IRP gebündelt und weitergeleitet. Jedes dieser Auftragspakete enthält im Kopf die Folge der Adressaten sowie den Datenplatz, so dass ein Auftrag in der Schichtung von oben nach unten und mit den Ergebnissen aktualisiert wieder zurück an den Benutzerprozess wandert. Dabei werden – je nach Auftrag – unterschiedlich viele Stationen bzw. Schichten durchlaufen. In Abb. 5.6 ist dies visualisiert.

Im linken Bild ist eine einfache Auftragsfolge dargestellt, im rechten Bild die Auftragsfolge für ein virtuelles Speichergerät, bei der die dynamische Schichtung („stackable“) ausgenutzt wird. Hier wird zuerst auf Basis der Dateiangabe die Anfrage nach einem Dateistück

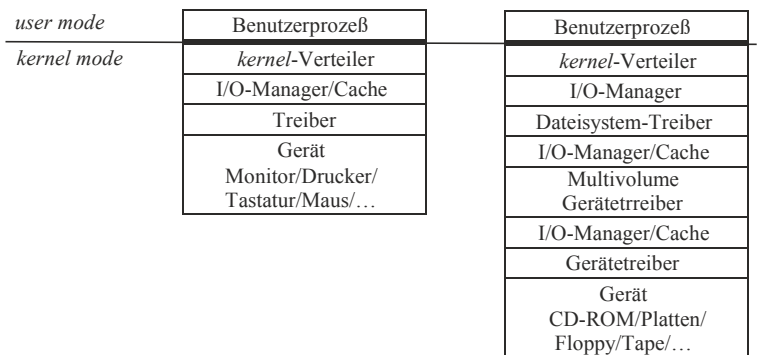


Abb. 5.6 Einfache und multiple Schichtung in Windows NT

vom Dateisystemtreiber in die Anfrage nach einer relativen Blocknummer innerhalb der Datei und dann in die absolute Blocknummer des Speichergeräts umgesetzt. Danach wird dieser Auftrag an den Treiber weitergeleitet, der multiple Geräte bedient. Dieser übersetzt die logische Blocknummer des virtuellen Geräts in die logische Blocknummer eines der Geräte, etwa einer Festplatte, und schickt dieser Festplatte den Auftrag. Am Schluss übersetzt der Gerätetreiber den Auftrag für einen logischen Block des Geräts in einen Auftrag für eine physische Adresse, etwa einen Zylinder und einen Sektor. Nach dem Abarbeiten dieses Auftrags geht das Ergebnis zurück zum Ursprung: Alle Aufträge werden in der inversen Reihenfolge beendet, bis der Dateisystemtreiber das gewünschte Datenstück an den Benutzerprozess liefern kann.

Die Existenz der Treiber ist dynamisch: Im Unterschied zu älteren UNIX-Versionen kann man in Windows NT (wie auch bei den modernen Unix-Versionen) während des Betriebs neue Treiber einklinken oder herausnehmen, um bestimmte Aufgaben zu erfüllen.

### **Dateisystemtreiber und bad cluster mapping**

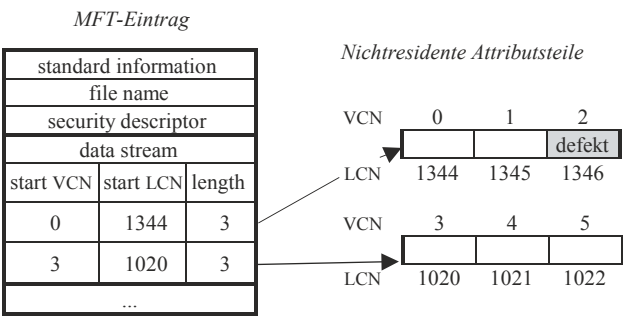
Da auch die Dateisystemverwaltung selbst wieder nur ein Treiber ist, können in Windows NT mit mehreren Treibern auch mehrere Dateisystemarten parallel existieren. So gibt es jeweils einen Treiber für Windows NT-NTFS, einen für MS-DOS-FAT und einen für OS/2-HPFS. Alle Zugriffsoperationen für Dateisysteme sind als Methoden eines Dateisystemtreibers implementiert. Sie unterscheiden sich nicht nur in den Datenstrukturen und Dienstleistungen, die sie mit Hilfe der darunter liegenden Gerätetreiber auf den Massenspeichern errichten, sondern auch in der Reaktion auf auftretende Fehler. Treten beispielsweise in MS-DOS FAT und in OS/2 HPFS nicht korrigierbare Fehler durch defekte Speichereinheiten (Blöcke oder Cluster) auf, die nicht von den darunter liegenden Gerätetreibern, beispielsweise dem fehlertoleranten Treiber *FtDisk* (oder den mikroprozessorgesteuerten Gerätecontrollern z. B. bei SCSI-Geräten), durch Reserveeinheiten abgefangen werden können, so wird bei Fehlern nur ein „corrupted“-Bit gesetzt. Beim nächsten initialen Gebrauch des Dateisystems (*mount*) wird dann ein extra Hilfsprogramm („check disk“) *chkdsk* aufgerufen, das im Dateisystem auf dem Massenspeicher dauerhaft die entsprechenden Speichereinheiten in Tabellen (*bad cluster mapping*) einträgt und von der weiteren Verwendung ausschließt.

Im Gegensatz dazu wird dies bei NTFS dynamisch durchgeführt. Dazu wird die Zuordnung der virtuellen Cluster zu den logischen Clustern, die wir im letzten [Abschn. 4.5.4](#) kennen gelernt haben, entsprechend modifiziert. Betrachten wir dazu das Beispiel in [Abb. 5.7](#), bei dem ein Cluster defekt wurde und vom darunter liegenden Gerätetreiber nicht auf Reserveeinheiten abgebildet werden konnte.

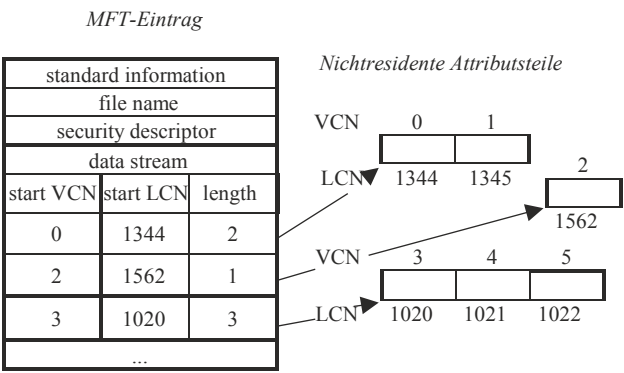
Der NTFS-Treiber kennzeichnet dazu den betreffenden Cluster als „*bad*“, weist die Daten des defekten Clusters, sofern sie durch Redundanz im Massenspeichersystem (Spiegelplatten und RAID-Systeme, s. unten) rekonstruiert werden konnten, im laufenden Betrieb einem der noch freien, logischen Cluster zu und vermerkt dies in der Dateibeschreibung, s. [Abb. 5.8](#). Existiert keine Redundanz bzw. Sicherheitskopie des defekten Clusters, so wird eine Fehlermeldung („Read/Write Error“) nach oben weitergegeben.

Gibt es für NTFS keine freien, verfügbaren Cluster auf dem *volume*, so wird auch hier prinzipiell das betreffende „*corrupted*“-Bit des *volume* gesetzt, und beim nächsten *mount* wird automatisch das *chkdsk*-Programm gestartet.

**Abb. 5.7** Ausfall eines Clusters in NTFS



**Abb. 5.8** Ersatz eines Clusters in NTFS



Konnten die Daten des defekten Clusters rekonstruiert werden und gibt es keine Ersatzcluster, so wird außerdem bei jedem I/O des Clusters eine Warnung an den I/O-Manager (bzw. Benutzer) gegeben, da nun ohne redundante Daten keine Fehlertoleranz mehr existiert und der Inhalt des defekten Clusters bei jedem Lesen aus den Sicherheitskopien erschlossen werden muss.

## 5.2 Gerätemodelle

Die Entwicklung von Treibern für ein Betriebssystem muss das Verhalten von physikalischen Geräten auf ein vom Betriebssystem erwartetes Verhalten (*Schnittstelle*) abbilden. Dazu ist es nötig, dass der Programmierer etwas mehr von den Geräten weiß. Da viele Geräte ein sehr ähnliches Verhalten haben, lohnt es sich, die typischsten Modelle zu betrachten, um die charakteristischen Parameter zu verstehen.

Dazu können wir grob zwischen zwei Arten von Geräten unterscheiden: Geräte mit *wahlfreiem* Zugriff, die Adressierungsinformation benötigen, und Geräte mit *seriellem* Datentransfer ohne Adressinformation. Beide Gerätearten erhalten ihre Aufträge über eine spezielle Hardwareschnittstelle.

5.2.1 Geräteschnittstellen

Im Unterschied zu früher gibt es bei der Systembus-orientierten Rechnerarchitektur keine speziellen Hardwarekanäle und spezielle, dafür vorgesehene Prozessorbefehle, sondern alle Geräte können einheitlich wie Speicher unter einer Adresse im Adressraum des Hauptspeichers angesprochen werden (*memory-mapped I/O*).

Dazu werden von einer speziellen Platine (**Controller**) im Rechner interne Speicherbereiche (Register, Puffer) zur Gerätekontrolle auf den Adressbereich des Hauptspeichers abgebildet, siehe [Abb. 5.9](#).

Allerdings kennt die Intel 80X86-Architektur sowohl direkte Adressierung mittels spezieller I/O-Befehle (I/O-Ports), als auch den memory-mapped I/O-Ansatz. Aus diesem Grund werden die für den Zugriff benötigten Adressen unterschiedlich notiert. Unter Linux sind die aktuell benutzten I/O-Portadressen der Controller, unter `/proc/ioports` zu finden und die *memory-mapped* Speicherbereiche unter `/proc/iomem`.

In [Abb. 5.10](#) sind die Reservierungen im Adressraum eines Standard-PC gezeigt.

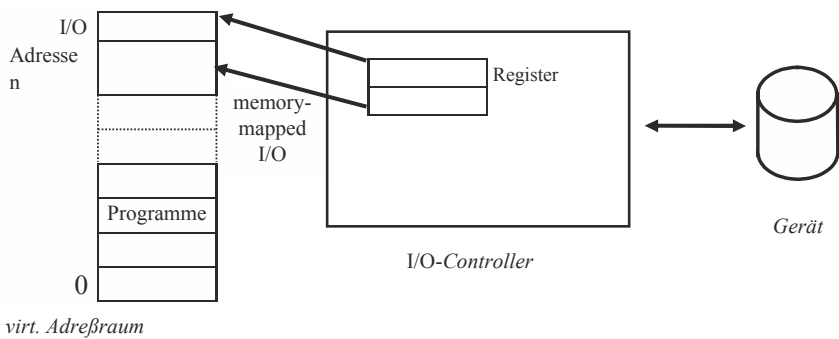


Abb. 5.9 Adressraum, Controller und Gerät

Abb. 5.10 I/O-Port-Adressen der Standardperipherie eines PC

| I/O ports (hexadezimal) | Gerät                    |
|-------------------------|--------------------------|
| 000-00F                 | DMA Controller           |
| 020-021                 | Interrupt Controller     |
| 040-043                 | Timer                    |
| 200-20F                 | Game controller          |
| 2FB-2FF                 | Serial ports (secondary) |
| 320-32F                 | Hard disk controller     |
| 378-37F                 | Parallel port            |
| 3D0-3DF                 | Graphics controller      |
| 3F0-3F7                 | Floppy disk controller   |
| 3F8-3FF                 | Serial ports (primary)   |



### 5.2.2 Initialisierung der Geräteschnittstellen

Die Register der Geräte sind anfangs mit unspezifischen Daten belegt. Insbesondere die Information, welcher DMA-Kanal und welcher Interrupt von welchem Gerät benutzt werden soll, fehlen. Auch die Adressen der Befehls- und Kontrollregister können sich bei den Geräten überschneiden und müssen deshalb in einem Rechnersystem koordiniert werden. Diese Koordination wird bei jedem Neustart des Rechners durchgeführt und ist eine der frühesten Aufgaben des Betriebssystems.

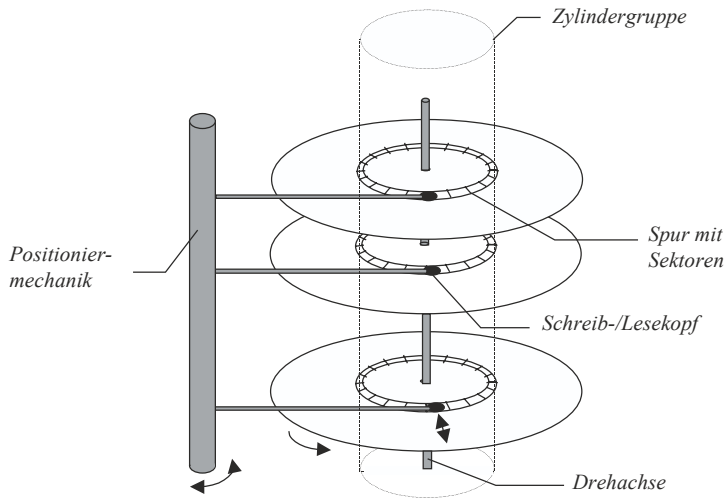
Diese Aufgabe wird bei neueren Controllern vereinfacht durch vordefinierte Protokolle. Ein sehr bekanntes Protokoll ist das „Plug-and-Play“ (PnP) -Protokoll. Hierbei identifiziert das Betriebssystem zunächst alle beteiligten Geräte und fragt dann bei allen Controllern an, welche Ressourcen (DMA, IRQ, Registeradressen, etc.) benötigt werden. Bei PCI-Bus-Controllern ist dies durch eine Standardschnittstelle vereinfacht, bei der sowohl der Hersteller der Karte als auch der Chips, Geräte- und Versionsnummer sowie die Geräteklasse direkt als Zahlen aus der Karte gelesen werden können. In entsprechenden Geräte-Beschreibungsdateien (INF-Dateien in Windows NT) sind dann für diese Geräte alle wichtigen Daten (mögliche Registeradressen, Interrupts usw.) hinterlegt. Feste, nicht-konfigurierbare Geräte (*legacy devices*) müssen aber vorher manuell angegeben werden. Danach werden die Ressourcen nach einer Strategie verteilt und dies den Geräten mitgeteilt. Da dies ein langwieriger Prozess ist, wird meist die Tabelle verwendet, die beim letzten Start gültig und auf Platte gespeichert war (Windows 98, Windows NT) bzw. die vom Computer Mainboard Betriebssystem (BIOS) errechnet und in dessen nicht-flüchtigen ESCD -Bereich abgespeichert wurde (Linux).

Betrachten wir nun als typischen Vertreter aus der Gruppe der wahlfreien Geräte den Plattenspeicher.

### 5.2.3 Plattenspeicher

Das Modell eines Plattenspeichers steht für eine ganze Modellgruppe aller Speichermedien, die eine sich drehende Scheibe verwenden. Dazu gehören neben den Festplattenspeichern auch CompactDisk (CD-R, CD-RW, DVD-R, DVD-RW), Disketten (31/2 Zoll, SuperDisk) und Wechsell Plattensysteme. Alle haben gewisse Modelleigenschaften gemeinsam. Betrachten wir als Beispiel den Festplattenspeicher genauer.

Der übliche magnetische Plattenspeicher besteht aus einer Aluminiumscheibe, die mit einer hauchdünnen Magnetschicht, z. B. Eisenoxid, überzogen ist. Wird nun auf einem langen Arm ein winziger Elektromagnet (**Schreib-/Lesekopf**) darauf gebracht und die Scheibe in Rotation versetzt, so kann der Kopf auf einer kreisförmigen Bahn (**Spur**) die Eisenoxidpartikel verschieden magnetisieren („schreiben“). Umgekehrt rufen die kleinen, magnetischen Partikel beim Vorbeigleiten an dem Elektromagneten einen kleinen Induktionssimpuls in der Spule hervor. Bei diesem „Lesen“ der magnetisch fixierten Information wird jede Magnetisierungsänderung als Bit interpretiert, so dass auf jeder Spur eine



**Abb. 5.11** Plattenspeicheraufbau

Bitsequenz gespeichert werden kann. Da eine Spur sehr viel Information speichern kann, unterteilt man eine Spur nochmals in Untereinheiten gleicher Größe, genannt **Sektoren**.

Nun muss man nur noch den Arm kontrolliert über die Scheibe zwischen Mittelpunkt und Rand bewegen, um viele derartige Spuren schreiben und lesen zu können.

Das Grundmodell eines derartigen Plattenspeichers bleibt immer gleich, ganz egal ob man als Plattenbelag Eisenoxid, Chromdioxid oder ein optisch aktives Medium (CD-ROM, DVD) verwendet, und egal ob man die Platten flexibel aus Kunststoff mit Eisenoxidfüllung macht (*floppy disk*) oder mehrere davon mit jeweils einem Kopf von oben und von unten an der Scheibe übereinander anordnet, siehe [Abb. 5.11](#).

Sind mehrere Platten übereinander auf dieselbe Drehachse montiert, so werden alle Köpfe normalerweise von derselben Mechanik zusammen gleichartig bewegt. Alle Köpfe befinden sich also immer auf einer Spur mit der gleichen Nummer, aber verschiedenen Platten. Alle Spuren mit der gleichen Spurnummer liegen übereinander und bilden die Mantelfläche eines imaginären Zylinders, siehe [Abb. 5.11](#). Die Menge der Spuren mit derselben Nummer wird deshalb auch als **Zylindergruppe** bezeichnet. Man beachte, dass diese Notation von gleichzeitig lesbaren Spuren an Bedeutung verliert, je mehr Spuren auf der Platte existieren und je ungenauer deshalb die Positionierung auf allen Spuren gleichzeitig wird. Bei den mehr als 2000 Zylindern moderner Laufwerke muss deshalb jede Spur unabhängig von den anderen Spuren eines Zylinders betrachtet werden.

Bei allen Konstruktionen muss ein Kopf mechanisch bewegt werden, um eine Spur zu lesen. Möchte man genau in der Spurmitte aufsetzen, den Spuranfang und die Abschnitte der Spur erkennen, so sind außerdem Hilfsinformationen wie regelmäßige Impulse (Formatierung) sowie spezielle Justierinformation (extra *Servospuren* an der Plattenunterseite) nötig. Für den Modellierer eines Plattenlaufwerks und Programmierer der Kontrollmechanismen sind folgende Parameter des Plattenmodells wichtig:

- Es existiert eine mittlere Zugriffszeit  $t_s$  (*average seek time*) pro Spur (ca. 8–10 ms bei Magnetplatten, ca. 100 ms bei CD-ROM, 250 ms bei Floppy Disks usw.).
- Bei gegebener fester Umdrehungsgeschwindigkeit muss man auf einer Spur eine bestimmte Zeit  $t_d$  (*rotational delay*) warten, bis der gewünschte Sektor unter dem Kopf erscheint. Im schlimmsten Fall ist dies die Zeit  $t_r$  für eine ganze Drehung.
- Die maximale Transferrate (*Datenrate*) ist gegeben durch die Schreibedichte (Bits pro mm), die Rotationsgeschwindigkeit und die Spurlänge.

Dabei erhöhen zusätzliche Platten linear die Speicherkapazität bei gleicher Zugriffszeit, dagegen erhöhen zusätzliche Spuren mit größerem Radius die Kapazität bei wachsender Zugriffszeit. Zusätzliche Köpfe erniedrigen die Zugriffszeit bei gleicher Kapazität; im Extremfall ist auf jeder Spur ein extra Kopf, so dass keine Kopfbewegung mehr stattfindet (*fixed head disks*).

Das obige Modell eines Plattenzugriffs unterscheidet also zwischen dem mittleren Zeitbedarf  $t_s$ , den Kopf auf die richtige Spur zu bewegen (*average seek time*), dem Zeitbedarf  $t_d$ , um den richtigen Sektor zu finden (*average rotational delay*), und dem Zeitbedarf  $t_T$ , die Daten beim Lesen/Schreiben in einem Stück zu transferieren (*transfer time*). Die erste Verzögerung wird durch die Kopfmekanik verursacht, die zweite durch die Umdrehungsgeschwindigkeit der Platte und der Länge der Spur ( $m$  Bytes pro Spur) und die dritte durch die Dichte der Information auf der Spur.

Die gesamte Zugriffszeit  $T$  in Abhängigkeit von der zu übertragenden Information ist somit

$$T(k) = t_s + t_d + t_T$$

und mit  $t_d = t_r/2$  für  $k$  Bytes, die auf  $k/m$  Spuren im günstigsten Fall in einem Stück gelesen werden

$$T(k) = t_s + \frac{t_r}{2} + \frac{k}{m} t_T \quad \text{lineares Modell} \quad (5.1)$$

Dies ist linear in  $k$ . Die *Datentransferrate*  $k/t_T$  ist dann allerdings eine nicht-lineare Funktion, die überproportional mit  $k$  ansteigt, so dass sich eine große, in einem Schritt (*atomar*) transferierbare Speichereinheit (Sektorgröße, Blockgröße, Seitengröße) anbietet, wie sie auch von der Optimierung des Hauptspeicherverbrauchs durch Seitentabellen gefordert wird, siehe [Abschn. 3.3.6](#).

Das lineare Modell der Zugriffszeit hat in der Praxis allerdings ziemliche Probleme, die mit entsprechenden Verbesserungen des Modells fast vollständig eliminiert werden können. Dabei ergibt sich eine Rangordnung der Modellfehler, siehe (Ruemmler und Wilkes 1994):

- Die Köpfe werden am Anfang einer Bewegung beschleunigt und konvergieren dann zu einer Bewegung mit konstanter Geschwindigkeit, siehe [Abb. 5.12](#). Bei kurzen Distanzen befindet der Kopf sich immer in der Beschleunigungsphase, bei langen Distanzen in der gleichförmigen Bewegung. Kurze Distanzen sind deshalb immer proportional

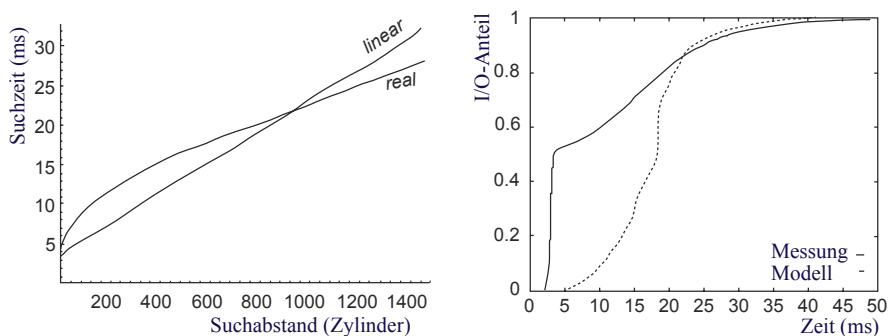
zur quadratischen Zeit, lange eher zur linearen Zeit. Untersuchungen zeigten einen dadurch bedingten Fehler des rein linearen Modells von ca. 15 %.

- Bei der mittleren Zugriffszeit  $t_s$  stellt sich immer die Frage: Wie wird diese Zeit ermittelt? Was wird als Verteilung der Positionsaufträge vorausgesetzt? Ein uniforme Verteilung (was in der Praxis nicht vorkommt) oder ein reelles Dateisystem unter Last? Eine nutzungabhängige Messung ergab einen Fehler der konstanten mittleren Suchzeit von 35 %.
- Entscheidend für die Leistungsfähigkeit ist aber der Einbau eines Puffers. Ohne Puffer ist die Übertragungsgeschwindigkeit an die Leistungsfähigkeit des Busses gebunden, der die Platte an das Gesamtsystem anschließt. Ist die Platte schneller als der Bus, so muss die Platte warten – bei jedem Verzug um eine Umdrehung. Ein Puffer kann die Gesamtleistung stark steigern (über 114 %), siehe [Abb. 5.12](#) rechts. Die gestrichelte Linie zeigt die Leistung eines Modells ohne, die durchgezogene Linie mit Puffer.

Es gibt außerdem ein Problem beim Plattenmodell: Alle Spuren einer Platte haben unterschiedliche Länge. Ist nun die Magnetisierungsdichte und damit die Zahl der Bits pro  $\text{mm}^2$  auf der Oberfläche überall gleich, so hat jede Spur eine andere Zahl von Bits; je länger die Spur, um so mehr sind es. Da aber die Zeit für eine Umdrehung und damit für eine Spur immer gleich ist, haben wir bei jeder Spur eine unterschiedliche Datenrate (Bits pro Sek.). Dies stellt sowohl für die Empfangselektronik ein zusätzliches Problem dar, da dann der Bittakt für jede Spur extra definiert werden muss, als auch für die Software des Treibers, da die Zahl der Speichereinheiten (Sektoren) auf jeder Spur unterschiedlich ist.

Um den zusätzlichen Hardware- und Softwareaufwand zu vermeiden, kann man deshalb alle Spuren in die gleiche Anzahl von Sektoren unterteilt. Dabei können die innersten Spuren nur bis zur Grenze ihrer Kapazität beschrieben werden; existierende Unregelmäßigkeiten der Beschichtung ergeben zusätzlich Fehler. Deshalb werden die innersten Spuren bei manchen Medien nicht genutzt (z. B. *floppy disk*: nur 80 Spuren statt 82); die Mehrheit der Spuren hat dafür durch die geringere Ausnutzung eine höhere Störsicherheit.

Bei modernen Festplatten geht man einen Kompromiss zwischen beiden Konzepten ein: Die Festplatte ist in ca. 20–40 Bereiche (*Zonen*) unterteilt. In jeder Zone haben die Spuren die gleiche Anzahl von Sektoren und der Treiber kann die gleiche Geometrie ausnutzen.



**Abb. 5.12** Modellierungsfehler bei Suchzeit und Transferzeit (nach Ruemmler [94](#))

Verlässt der Kopf eine Zone und setzt in einer anderen Zone auf, so werden aus einer Tabelle des Treibers die korrespondierenden Parameter eingelesen. Dabei ändert sich auch die Datentransfergeschwindigkeit: In den äußeren Zonen werden bei konstanter Umdrehungsgeschwindigkeit mehr Sektoren pro Sekunde gelesen als in den inneren Zonen.

Ohne auf weitere Modellierungsmöglichkeiten einzugehen, die das Modell noch besser an die Realität anpasst, können wir nun eine abschließende Bewertung abgeben:

- 1) Eine gute Modellierung kann den Modellierungsfehler stark verkleinern. Dabei bedeutet „Modellierung“ ein Programm, was möglichst viele mechanische Merkmale typischer Plattenlaufwerke umfasst. Auch müssen parallele Controlleraktionen wie mechanische Sektorsuche, Cache, Busarbitrierung zum Hauptspeicher und DMA-Datentransfer auf dem Bus berücksichtigt werden. Ein Betriebssystemtreiber, der das Gesamtverhalten aus Mechanik, Controller und Systembus berücksichtigt, kann zweifelsohne den Durchsatz stark steigern.
- 2) Allerdings ist es aus der Sicht des Betriebssystemarchitekten nicht sinnvoll, sich dieses gerätespezifische Wissen anzueignen und danach den Gerätetreiber zu optimieren. Jede Anwendungsart und -ort eines Betriebssystems ist anders: Hauptspeichergroße, CPU-Zyklen, Busart und Busdurchsatz wechseln in verschiedenen Rechnern bei gleichem Betriebssystem. Besonders wichtig: Dies gilt auch für die Geräteausführung. Es ist deshalb weder sinnvoll, alle Geräteparameter des Treibers mühsam an das neue Gerät in der aktuellen Rechnerkonfiguration anzupassen, noch finanziell vertretbar, für jedes neue Modell eines Herstellers bei jedem Anwender einen eigenen Treiber zu schreiben.

Eine Zugriffsoptimierung für Platten sollte also nicht im Betriebssystem stattfinden, sondern in dem eingebetteten System des Plattencontrollers. Stattdessen sollte vom Betriebssystem nur ein einfaches, klares Modell eines virtuellen Geräts ohne spezielle Zeiten berücksichtigt werden, etwa das eines großen, linearen Speicherfeldes; die eigentliche detailreiche, effiziente Modellierung sollte dagegen beim Hersteller erfolgen. Er allein kennt alle internen mechanischen Konstruktionsdetails und kann mit Hilfe des internen Controllers die Schnittstelle zum Betriebssystem auf die internen Einzelschnittstellen zu Motorelektronik und Cache effizient abbilden. Dabei sollte dieser interne Treiber (Kontrollprogramm) adaptiv seine Betriebsparameter an die jeweils aktuelle Lastverteilung (Datendurchsatz, Reihenfolge der Suchaufträge, ...) beim Betrieb anpassen.

Man sollte also auf Herstellerseite die Geräteeigenschaften im gerätespezifischen Controller bzw. oder im Treiber kapseln und dem Betriebssystem nur durch eine genormte Schnittstelle (Gerätemodell) zur Verfügung stellen, ähnlich wie ein abstrakter Datentyp.

---

#### **Beispiel UNIX Zugriffsoptimierung bei Platten**

Zur Leistungsoptimierung der Dateisysteme kann man sich die Tatsache zunutze machen, dass bei einer gemeinsamen Positionierungsmechanik alle Köpfe immer auf

derselben Spur und die Speicherblöcke der Spuren eines Zylinders, wenn der Kopf einmal auf diesem Zylinder ist, schnell zugreifbar sind. In Berkeley-UNIX erhält jede Zylindergruppe deshalb eine extra *super node* und damit eine extra Verwaltung. Wenn neue *i-nodes* und Dateiblöcke alloziert werden, so geschieht dies automatisch vorrangig auf der jeweiligen Zylindergruppe. Dadurch wird das Lesen und Schreiben von Dateien sehr beschleunigt, da nach dem Zugriff auf die *i-node* alle weiteren Operationen keine Zeit mehr zum Positionieren des Schreib-/Lesekopfes benötigen.

Vom Gerätemodell wird also hier die Tatsache benutzt, dass Zylinder und Spuren existieren. Die absoluten Zugriffs- und Transferzeiten je nach Modell bleiben unberücksichtigt.

### Geräteschnittstelle

Üblicherweise gibt es bei einem wahlfreien Gerät wie einer Festplatte folgende Register:

- **Statusregister** (*read only*)  
Der Speicherinhalt dieses Dateiwortes wird vom Controller aktualisiert und gelesen. Es dient als Statusregister, jedes Bit hat eine besondere Bedeutung. Beispiel: Bit 4 = 1 bedeutet, dass das Lesen beendet wurde und das Ergebnis im Datenpuffer gelandet ist. Bit 8 = 1 heißt, das ein Lesefehler dabei aufgetreten ist.
- **Befehlsregister**(*write only*)  
In dieses Register wird der Code für einen Befehl (Schreiben/Lesen/Formatieren/Positionieren ...) geschrieben, der ausgeführt werden soll. Das Hineinschreiben des Befehlscodes in dieses Register lässt sich als Prozeduraufruf interpretieren, bei dem die Parameter aus dem Inhalt der anderen Register bestehen.
- **Adressregister**  
Sie enthalten die Speicherzellen-Adresse auf dem Gerät (Gerät, Spur/Zylinder, Sektor, Platte, Kopf ...) und die Zahl der zu transferierenden Bytes. Die Angaben setzen dabei eine Information über die Geometrie der Festplatte voraus, also die Anzahl der verfügbaren Köpfe, Sektoren usw. Diese Information muss nicht echt sein, sondern kann auch nur das Modell widerspiegeln, das der Controller nach außen hin als Schnittstelle repräsentiert; die tatsächliche Kopfzahl, Spurzahl, Zylinder- und Sektorenzahl kann intern ganz anders sein.
- **Datenpuffer**  
Alle Daten, die auf diese Adresse geschrieben werden, landen in einem internen Puffer in der Reihenfolge, in der sie geschrieben werden. Umgekehrt kann der Puffer hier sequentiell ausgelesen werden. Meist sind Ein- und Ausgabepuffer getrennt, so dass ein Puffer nur gelesen und der andere nur geschrieben werden kann.
- Ein **Interruptsystem**, das nach der Ausführung eines Befehls einen Interrupt auslösen kann.

Besitzt das Gerät sehr viele Parameter, die man einstellen kann, so benötigt man viele Register, und es geht sehr viel Adressraum für den Prozess dafür verloren. Bei kleinen Adressräumen ist es deshalb üblich, dass nur eine Adresse existiert, in die man den Zeiger

(die Adresse) zum Auftragspaket oder auch sequentiell ganze Datenpakete (Aufträge) mit genau festgelegten Formaten schreiben muss.

Ein wichtiger Mechanismus für den Datenaustausch mit dem Gerät ist das Verschieben von ganzen Datenblöcken zwischen dem Gerätepuffer des Controllers und dem Hauptspeicher. Dies wird prozessorunabhängig mit speziellen Chips, (*Direkt Memory Access* DMA) durchgeführt. Die DMA-Chips existieren dabei nicht nur im Speichersystem, sondern auch auf den Controllerplatinen und können unabhängig voneinander arbeiten (*DMA-Kanäle*).

#### 5.2.4 SSD-RAM-Disks

Für das Verhalten eines Massenspeichers im Betriebssystem ist nur der Treiber verantwortlich: Er bildet eine virtuelle Maschine. Man kann deshalb an die Stelle des realen Massenspeichers auch einen Bereich des Hauptspeichers als virtuelle Platte verwenden, ohne dass dies im Betriebssystem auffällt und anders behandelt werden müsste.

Eine solche **Solid State Disk (SSD)** oder **RAM-Disk** ist dadurch natürlich sehr schnell; für die darauf befindlichen Dateien gibt es keine mechanischen Verzögerungszeiten beim Schreiben und Lesen. Man muss nur bei der SDRAM-Variante vor dem Ausschalten des Rechners alle Dateien auf echten Massenspeicher kopieren, um sie dauerhaft zu erhalten; bei der FLASH-Speicher-Ausführung ist dies nicht nötig.

Allerdings stellt sich hier prinzipiell die Frage, wozu man einen Teil des sowieso chronisch zu kleinen Hauptspeichers als Massenspeichergerät deklarieren soll. Effiziente Strategien für Cache und *paging*, verbunden mit der direkten Abbildung von Dateibereichen auf den virtuellen Adressraum, senken ebenfalls die Zugriffszeit und verwalten dynamisch den kleinen Hauptspeicher.

Die Antwort auf diese Frage hängt stark vom benutzten System ab.

- Möchte man Systeme schneller machen, deren Hauptspeichergöße ausgeschöpft ist, etwa ein 32-Bit Betriebssystem mit 4 GB, so ermöglicht zusätzlicher Hauptspeicher als RAM-Disk einen schnellen Prozessauslagerungsbereich (*swap file*). Dies gilt auch, wenn generell ein besonders schnelles Dateisystem nötig ist. SSD ermöglichen ein ca. 10-fach schnelleres System bei ca. 100-fach schnelleren Datentransferraten. Damit erhalten wir ein System mit den Vorteilen größerer Cachebereiche, ohne auf ein 64-Bit-System wechseln zu müssen.
- Ist der Rechner (z. B. Notebook) portabel, so ermöglicht eine SSD einen störungsfreien Betrieb auch bei starken Erschütterungen, etwa bei Autofahrten oder anderen *outdoor*-Anwendungen.
- In solch einem Rechner wird bei einer SSD auch nur 20 % des Platzes gegenüber einer Festplatte benötigt.
- Eine RAM-Disk ist auch sinnvoll, wenn man mit fester, vorgegebener Software arbeiten muss, die temporäre Dateien erzeugt und auch durch zusätzlichen Hauptspeicher



dies nicht vermeiden kann. Ein Beispiel dafür sind Compiler: Liegen die Zwischendateien des Kompilervorgangs auf einer RAM-Disk, so erhöht sich die Kompiliergeschwindigkeit erheblich.

- Allerdings kann man auch auf ein solches RAM-Dateisystem verzichten, wenn der *page-out-pool* groß genug ist, um alle Dateiblöcke für das Schreiben zwischenzulagern. Der Lesezugriff auf die temporäre Datei erfolgt dann über diese Blöcke, die nicht von einem Massenspeicher geholt werden müssen und deshalb genauso schnell sind wie eine RAM-Disk.

Möchte man allerdings eine FLASH-SSD als Systemplatte verwenden, so gibt es einige Dinge zu beachten:

- Werden Dateien gelöscht, so reicht es für Magnetplatten aus, nur die Verwaltungsinformationen (*i-nodes*, *directories*) zu löschen; der Inhalt wird bei der nächsten Speicherung nebenbei überschrieben. Dies ist bei FLASH-Speichern so nicht möglich. Stattdessen muss eine Speicherzelle vor dem Neubeschreiben explizit gelöscht werden. Wurde dies nicht getan, so dauert das Schreiben doppelt so lange: erst muss die Zelle gelöscht und dann neu beschrieben werden.

Um diese Verzögerung zu vermeiden, kann das Betriebssystem einen expliziten TRIM-Befehl an das Laufwerk übermitteln, der das Löschen bewirkt. Dazu muss allerdings die gegenüber dem ATA-Interface verbesserte Schnittstelle AHCI (*Advanced Host Controller Interface*) auf der Hauptplatine existieren und im BIOS aktiviert sein. Weiterhin muss das Betriebssystem diesen Befehl beherrschen, was standardmäßig erst ab Windows 7, Linux ab Kernel 2.6.33 und MacOS 10.7 im Kern implementiert ist.

- Beherrscht der SSD-Controller bereits ein *garbage collection*, so ist der TRIM-Befehl nicht mehr nötig, da dann bereits Schreibanweisungen für belegte Blöcke intern als Löschanweisungen interpretiert werden. Werden die neuen Daten in Reserveblöcken oder Cachebereichen zwischengespeichert, so kann parallel dazu das Löschen durchgeführt werden.
- Die SSD erlauben nur eine begrenzte Zahl an Schreiboperationen pro Zelle, meist 3000–100.000, maximal bei industriellen SSD ca. 5 Millionen. Würden wir eine Zelle oft zum Schreiben verwenden, so wäre bei einem Schreibzyklus von 1 ms die Zelle in ca. 16 Minuten an ihrer Grenze von 1 Mill. Zyklen angekommen und damit defekt. Damit eignen sich kurzlebige SSDs nicht für häufiges Schreiben etwa von Auslagerungsdateien.
- Eine Lösung für dieses Problem bietet eine interne Nutzungsverteilung über sog. *wear level*-Algorithmen an: Die Zuordnung der logischen Blocknummern zu den echten physikalischen Blocknummern geschieht im internen SSD-Controller und verändert sich immer dann, wenn ein Block überschrieben wird. Anstelle den Block neu zu beschreiben, wird stattdessen ein bisher unbenutzter Block verwendet. Nach außen hin ändert



sich nichts, aber intern wird so der benutzte Bereich gleichmäßig verteilt. Damit wird die Nutzungsdauer der SSD zwar erhöht, bleibt aber endlich.

- Möchte man deshalb eine FLASH-SSD als Festplattenersatz für eines Systempartition (in Windows: C:/) einsetzen und eine möglichst lange Lebensdauer erreichen, so sollte man in Windows den Systemrettungsdienst abschalten, den/user-Ordner, in dem alle temporären user-Daten in C: abgelegt werden, auf eine Magnetplatte verschieben, den *hibernate*-Modus abschalten, die Auslagerungsdatei auf Magnetplatten verlagern, die für SSDs sinnlose automatische Defragmentierung deaktivieren, sowie alle vorausschauenden, aber nun unnötigen Lesedienste wie *SuperFetch*, *Prefetch*, *Indexdienst*, *ReadyBoost* und *Bootoptimierung* abschalten.

Im Allgemeinen rechnet man mit einer Verringerung von ca. 45 % der Kosten pro GB pro Jahr bei den Festplatten; bei den SSD (z. B. NAND-Flash) dagegen von über 50 %. Es ist deshalb abzusehen, wann Festplatten nicht mehr in Rechnern verwendet werden, sondern nur noch optimierte, langlebige FLASH-Speicher.

Werden die SSDs durch schnelle Datenverbindungen direkt über den Speicherbus an die CPU angebunden, so ergibt sich ein neuer Trend: die Zusammenführung von Hauptspeicher und Massenspeicher zu einer einheitlichen Speicherarchitektur.

### 5.2.5 Serielle Geräte

In [Kap. 4](#) bemerkten wir schon, dass auf fast alle Geräte sowohl sequentiell als auch wahlfrei zugegriffen werden kann, bis auf die „echten“ seriellen Geräte wie Tastatur und Drucker, bei denen dies normalerweise nicht möglich ist. Die Bezeichnung „serielle Geräte“ bezieht sich also auf die Art und Weise, wie Daten übergeben werden müssen: jedes Zeichen hintereinander, ohne Adressinformation für das Gerät.

Meist geschieht das mit geringer Geschwindigkeit (wenigen KB pro Sekunde). Dies sind die klassischen Randbedingungen von Terminals, langsamen Zeilendruckern und Tastaturen. Allerdings gibt es auch sehr schnelle serielle Verbindungen, die nicht zeichenweise vom Treiber bedient werden, sondern größere Datenblöcke zur Übertragung über DMA zur Verfügung gestellt bekommen. Ihre Ansteuerung wird dann mit Hilfe einer Mischung aus Techniken für Treiber von zeichen- und blockorientierten Geräten durchgeführt.

#### Geräteschnittstelle

Die Schnittstelle zum Controller von seriellen Geräten ist (ebenso wie die bei wahlfreien Geräten) mittels Speicheradressen ansprechbar:

- Ein **Kontrollregister** sorgt für Statusmeldungen, die Übertragungsgeschwindigkeit (z. B. 1200, 2400, 4800, 9600, 19.200 Baud, gemessen in Daten- + Kontrollbits pro Sekunde), und den Übertragungsmodus „synchron“ oder „asynchron“. Bei *asynchroner*

Übertragung werden in unregelmäßigen Abständen Daten (Zeichen) übermittelt; bei *synchroner* Übertragung folgen alle Zeichen in festem zeitlichen Abstand und Format.

- Ein **Eingaberegister** enthält das zuletzt empfangene Zeichen, das vom Treiber dort ausgelesen wird.
- In das **Ausgaberegister** wird das zu sendende Zeichen vom Treiber geschrieben; unmittelbar darauf wird es vom Controller gesendet.
- Ein **Interruptsystem** löst nach jedem gesendeten bzw. empfangenen Zeichen einen Interrupt aus.

Die meisten seriellen Geräte arbeiten außerdem mit einer **Flusssteuerung**, die im Kontrollregister gesetzt wird. Üblich ist entweder eine Hardwaresteuerung über extra Drähte (RS232-Norm), oder aber es werden spezielle Zeichen (Control-S, Control-Q, genannt XON und XOFF) zur Steuerung gesendet. Droht beim Empfänger der Empfangspuffer überzulaufen, so sendet er XOFF und veranlasst damit den Sender, mit dem Senden aufzuhören. Ist der Puffer leer, sendet der Empfänger XON und signalisiert so dem Sender, weiterzusenden. Allerdings funktioniert diese Mechanik nur dann, wenn der Empfänger sein Signal rechtzeitig sendet, so dass der Sender auch genügend Zeit hat, das Kontrollzeichen zu empfangen, zu interpretieren und im Treiber umzusetzen, bevor der Empfängerpuffer überläuft.

### 5.2.6 Aufgaben zu I/O

#### Aufgabe 5.2-1 I/O memory mapping

- a) Was ist *I/O-memory mapping* und wozu dient es?
- b) Informieren Sie sich über Systemuhr (*Real Time Clock*) und finden Sie durch geeignete Befehle auf Windows und Linux die I/O-Adressen und den dazugehörigen Interruptvektor heraus.

## 5.3 Multiple Plattenspeicher: RAIDs

Eine weitverbreitete Methode, die Plattenkapazität zu erhöhen, besteht darin, mehrere kleine Platten gemeinsam als eine große virtuelle Platte zu verwalten. Haben wir für jede Platte ein extra Laufwerk, so muss man pro Laufwerk nur noch eine geringere Kapazität vorsehen und kann dafür kleine, preiswerte Laufwerke einsetzen. Ein solches System wird als RAID (**R**edundant **A**rray of **I**ndependent **D**isks) bezeichnet und ist außerordentlich beliebt bei großen kommerziellen Datenbanken. Im Gegensatz dazu wird der klassische Fall einer einzigen Festplatte als SLED (**S**ingle **L**arge **E**xpensive **D**isk) bezeichnet.

Für das Betriebssystem ist diese neue Hardwarevariante transparent: Der Treiber zeigt nur ein einziges, virtuelles, schnelles Laufwerk großer Kapazität; die Realisierung als

Plattensammlung ist für die Schnittstelle zu den höheren Betriebssystemschichten unwichtig. Diese Basiseigenschaft wird als **RAID-linear** bezeichnet.

Ein weiterer Vorteil multipler Plattenspeicher kann genutzt werden, wenn mehrere unabhängige Prozesse oder *threads* im Betriebssystem existieren. Ihre Unabhängigkeit und damit die mögliche parallele, schnellere Abarbeitung wird behindert, wenn die an sich unabhängigen Daten alle auf einem gemeinsamen Massenspeicher versammelt sind, dessen Abschnitte nicht unabhängig voneinander gelesen werden können. Setzen wir nun mehrere Laufwerke anstelle eines einzigen mit festgekoppelten Köpfen (s. Abb. 5.11) ein, so können die Daten der unabhängigen Prozesse auch auf unabhängigen Plattenspeichern abgelegt werden, und damit wird der Datenzugriff für die Prozesse erheblich schneller. In Abb. 5.13 ist eine solche Aufteilung von Daten auf multiple Laufwerke (dargestellt als Säulen) zu sehen. Dazu wird der gesamte logische Speicherplatz in Abschnitte (*stripes*) aufgeteilt. Korrespondierende, gleich große Abschnitte verschiedener Laufwerke werden vom Treiber als gemeinsamer Speicherbereich (**Streifen**) angesehen und bilden einen einheitlichen Speicherbereich für eine Prozessgruppe. Der Treiber im Betriebssystem hat dabei die Aufgabe, die Verteilung und Zusammenfassung der Daten vorzunehmen und dazu die Abbildung virtueller Speichereinheiten auf die logischen Einheiten unterschiedlicher Laufwerke durchzuführen.

#### Beispiel Windows NT Plattenorganisation

In diesem Betriebssystem kann bei der initialen Formatierung von logischen Plattenbereichen (*Partitionen*) die Option angegeben werden, die Plattenbereiche als *Streifen* zu organisieren.

Eine andere, ähnliche Option besteht darin, mehrere Partitionen beliebiger Größe von unterschiedlichen Platten zu einer neuen logischen Platte (*volume*) zusammenzufassen. Der virtuelle Plattenadressraum kann so unregelmäßig auf unterschiedliche Platten verteilt werden.

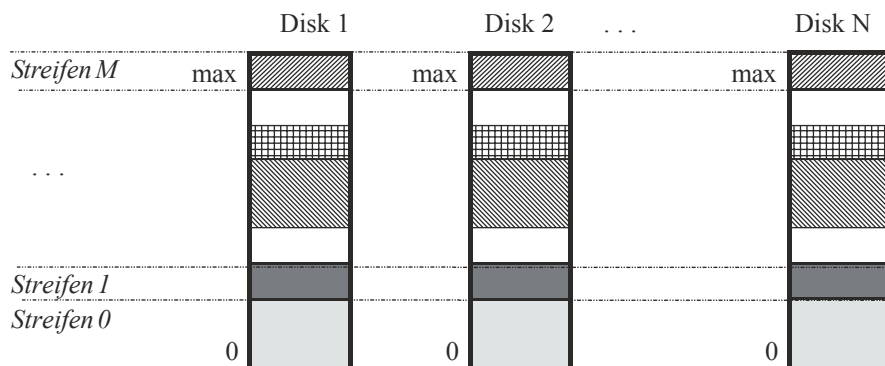


Abb. 5.13 Neugruppierung der Plattenbereiche in Streifen

Machen wir den Streifen sehr schmal, z. B. 4 Byte oder auch einen Block groß, so wird der gesamte Datenpuffer, aufgeteilt in sequentielle Abschnitte der Länge 4 Byte bzw. ein Block, sehr schnell auf das Plattenfeld verteilt geschrieben oder gelesen. Eine solche schnelle, aber nicht ausfallsichere Konfiguration wird als **RAID-0**-System bezeichnet und gern für die großen Datenmengen in der Video- und Bildproduktion eingesetzt.

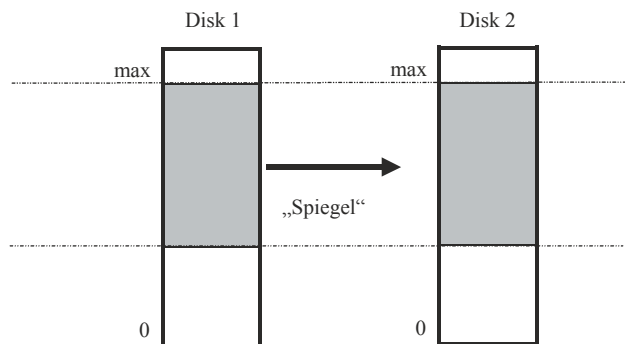
Systeme aus multiplen Laufwerken haben neben der höheren **Zugriffsschnelligkeit** und größeren **Speicherkapazität** noch einen weiteren wichtigen Vorteil: Mit ihnen kann man Ausfälle von Platten tolerieren. Für diese Art von Fehlertoleranz, der **Ausfalltoleranz**, sieht man für die Daten einer Platte eine exakte Kopie auf einer Platte eines anderen Laufwerks vor; die andere Platte ist bezüglich ihrer Daten wie ein Spiegel (*mirror*) der einen Platte aufgebaut. In [Abb. 5.14](#) ist das Schema einer solchen **Spiegelplattenkonfiguration** gezeigt.

Allerdings muss diese Kopie ständig aktualisiert werden. Der Treiber, der eine solche Konfiguration bedienen kann, muss dazu bei jedem Schreibvorgang auf Platte 1 eine Kopie auf Platte 2 bewirken, was eine zusätzliche Belastung des Betriebssystems und der I/O-Hardware bewirkt. Eine solche ausfallsichere, aber ineffiziente Konfiguration von Spiegelplattenpaaren wird als **RAID-1**-System bezeichnet. Führen wir den Gedanken der Spiegelplatten in die Streifenkonfiguration von [Abb. 5.13](#) ein, so erhalten wir ein schnelles, ausfallsicheres **RAID-0/1** System, das in Finanzanwendungen (Lohnbuchhaltung) eingesetzt wird.

Man kann versuchen, den Zeitverlust bei den Lesevorgängen der Spiegelplatten wieder zu kompensieren. Verfügt der Treiber über den augenblicklichen Zustand der Platten, so kann er zum Lesen diejenige der beiden Platten auswählen, deren Lesekopf der gewünschten Spur am nächsten ist; der andere Plattenkopf kann bereits für den nächsten Auftrag positioniert werden oder in anderen, nicht gespiegelten Partitionen schreiben oder lesen.

Den großen Platzbedarf der replizierten Platten für ein schnelles, ausfallsicheres RAID-0/1-System kann man durch einen Trick drastisch senken. Dazu wird jedes Bit eines Datenwortes auf eine extra Platte geschrieben – für ein 32-Bit-Wort benötigt man also 32 Laufwerke. Dies verteilt erst mal die Daten, spart aber noch keinen Platz. Dann wird jedes Wort mit einer speziellen Signatur versehen, beispielsweise einem 6-Bit-Zusatz, so dass

**Abb. 5.14** Spiegelplattenkonfiguration zweier Platten



im Endeffekt jedes Datenwort von einem Controller auf 38 Laufwerke verteilt wird. Dies bedeutet nicht nur einen enormen Datenfluss, sondern ermöglicht auch eine große Ausfalltoleranz ohne die Gesamtheit aller Daten zu replizieren. Da nicht jede Bitkombination der 38-Bit-Datenworte möglich ist, kann man auch bei mehreren ausgefallenen oder defekten Platten auf den korrekten 32-Bit-Code schließen, s. MacWilliams und Sloane (1986).

Beispiel Fehlerkorrektur durch Paritätsbildung

Angenommen, wir wollen die Zahlen 10, 7, 3 und 12 abspeichern und garantieren, dass bei Verlust einer der Zahlen die fehlende wieder rekonstruiert werden kann. Wie können wir dies erreichen?

Dazu bilden wir bei  $n$  Bits, die unabhängig voneinander abgespeichert werden, als Kontrollinformation ein  $(n + 1)$ -tes Bit, das Paritätsbit  $p$ , nach der folgenden XOR-Funktion  $\oplus$  zweier Binärvariablen  $a$  und  $b$ :

| a b | $p=a\oplus b$ | $p\oplus b$ | Es folgt direkt aus der Tabelle<br><br>$a \oplus 0 = a$<br><br>$a \oplus a = 0$<br><br>und somit $p\oplus b = a\oplus b\oplus b = a\oplus 0 = a$ |
|-----|---------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 1 | 0             | 1           |                                                                                                                                                  |
| 1 0 | 1             | 1           |                                                                                                                                                  |
| 0 1 | 1             | 0           |                                                                                                                                                  |
| 0 0 | 0             | 0           |                                                                                                                                                  |

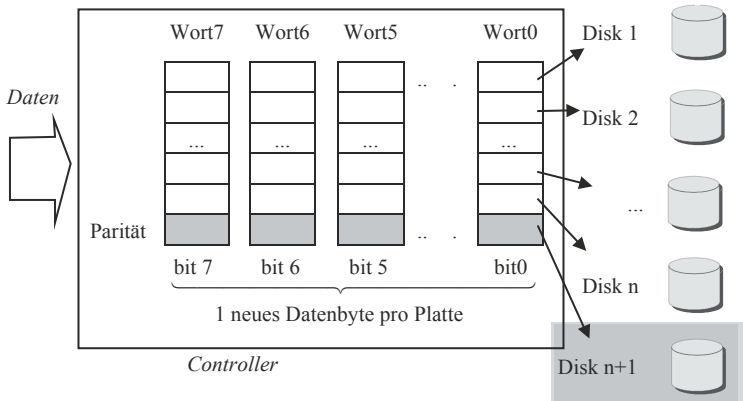
Mit  $p\oplus b = a$  wissen wir nun, dass bei bekannter Parität  $p$  der Ausfall von einem Bit  $a$  toleriert werden kann, da wir  $a$  direkt aus  $p$  und  $b$  wiederherstellen können. Dies gilt natürlich auch, wenn  $b$  selbst wiederum aus vielen Termen besteht, beispielsweise

$$b = b_1 \oplus b_2 \oplus \dots \oplus b_{n-1}$$

Bei  $n$  Binärvariablen, aus denen wir die Parität bilden, kann also eine davon mit Hilfe der anderen und der Parität wieder erschlossen werden. Für unsere vier Zahlen von oben bedeutet dies bei Ausfall von Zahl 3:

| Paritätsbildung             | Rekonstruktion              |
|-----------------------------|-----------------------------|
| Zahl 1: 10 = 1 0 1 0        | Zahl 1: 10 = 1 0 1 0        |
| Zahl 2: 7 = 0 1 1 1         | Zahl 2: 7 = 0 1 1 1         |
| Zahl 3: 3 = 0 0 1 1         | Parität: 2 = 0 0 1 0        |
| <u>Zahl 4: 12 = 1 1 0 0</u> | <u>Zahl 4: 12 = 1 1 0 0</u> |
| Parität: 0 0 1 0 = 2        | Ergebnis: 0 0 1 1 = 3       |

Behandeln wir nun 8 Binärvariablen (zusammengefasst in einem Byte) parallel, so gilt dies für jedes Bit in diesem Byte und damit für das ganze Byte. Speichern wir also jedes Bit – auch das Paritätsbit – auf einem anderen Massenspeicher, so können wir beim Ausfall eines Speichers aus den  $n$  verbliebenen Bits das fehlende direkt rekonstruieren und damit den Ausfall des einen Massenspeichers tolerieren. Bilden wir mehrere Fehlerkorrekturbits (ECC, MacWilliams und Sloane 1986), so können wir auch den Ausfall mehrerer Laufwerke tolerieren.



**Abb. 5.15** Datenzuweisung beim RAID-2-System

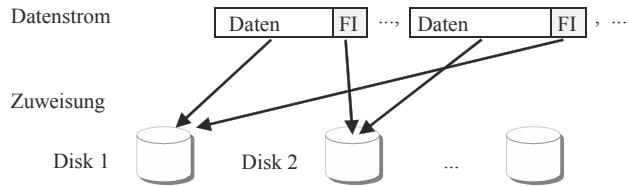
Für ein fehlerkorrigierendes **RAID-2**-System werden die Bits der zu speichernden Datenworte extra behandelt und gespeichert. Die ist in [Abb. 5.15](#) für ein Fehlertoleranzbit (Parity-Bit) dargestellt.

Die Datenworte sind als Säulen gezeigt, die in ihre Bits aufgeteilt werden. Das  $i$ -te Bit aller Worte wird so auf der  $i$ -ten Platte gespeichert. Der Bitstrom für eine Platte lässt sich im Controller wieder in Bytes zusammenfassen, so dass als Speichersysteme handelsübliche Platten genommen werden können. Beim Raid-2-System wird von jedem Wort ein allgemeiner fehlerkorrigierender Code (*Error Correcting Bits ECC*) gebildet, dessen einzelne Bits jeweils auf einer eigenen Platte gespeichert werden. Würden wir die Spindeln der einzelnen Platten zwangssynchronisieren, so ist das Gesamtsystem zwar nicht schneller als ein einzelnes Laufwerk mit  $n$  Platten, aber dafür fehlertolerant.

Natürlich können wir auch die Parität statt über die Säulen über die Zeilen, also über alle  $m$  Worte eines Puffers, bilden. Das sich ergebende Wort aus Paritäten (ECC) wird dann nach jeweils  $m$  Worten auf einer extra Platte gespeichert. In diesem Fall ist die Zahl der Platten nicht gleich der Zahl der Bits pro Wort, sondern  $m$  und damit beliebig; jedes Wort wird nicht in Bits aufgespalten, sondern mit anderen als Teilpuffer (etwa in einem Streifen) auf eine Platte gespeichert. Wir erhalten so ein **RAID-3**-System. Da hier genauso wie bei der reinen Parität von RAID-2 die gesamte Fehlertoleranzinformation aller Blöcke auf einem einzigen Laufwerk gespeichert ist, darf hier nur ein einziges Laufwerk ausfallen. Ein weiterer Nachteil ist die kurze Länge der Fehlertoleranzinformation: Hier sollten wie bei RAID-2 die Spindeln der Laufwerke synchronisiert sein, um die Zugriffszeit  $t_D$  möglichst klein zu halten.

Fassen wir weiter die Datenbits zu größeren Abschnitten zusammen (z. B. zu Blöcken oder *stripes*) und speichern jeden Block wie bei RAID 0 jeweils auf einer separaten Platte, so wird dies als **RAID-4**-System bezeichnet. Da hier zusätzlich bei jedem Schreiben auch die Fehlertoleranzinformation aktualisiert werden muss, dauert das Schreiben länger: RAID-4 bietet keinen Geschwindigkeitsvorteil wie RAID-0. Die Lesegeschwindigkeit entspricht etwa der einer Einzelplatte im Block-Lesemodus.

**Abb. 5.16** Wechselseitige Abspeicherung der Fehlertoleranzinformation



Ein Alternative dazu besteht darin, die häufige gelesene Fehlertoleranzinformation FI für jeden Block bzw. *stripe* extra auf einem anderen Laufwerk zu sichern und so die I/O-Belastung der Aktualisierung auf alle Platten gleichmäßig zu verteilen. Dies ist ein **RAID-5**-System. Das Schema dafür ist in Abb. 5.16 gezeigt. Die Raid-5 Systeme werden gern für Datenbankserver und Internetserver eingesetzt.

Für den Betriebssystemarchitekten besteht die Hauptaufgabe darin, die Komplexität und das Fehlertoleranzverhalten modular in das RAID-Subsystem bzw. den Treiber zu verlagern und das Betriebssystem vom Speichersystem unabhängig zu machen: Für das Betriebssystem sollte nur ein einfaches, virtuelles Speichersystem sichtbar sein.

Bei der Wahl zwischen einer Softwarelösung und einer Hardwarelösung für RAID sollte man neben dem Vorteil einer einheitlichen, HW-unabhängigen reinen SW-Lösung auch ihre Nachteile bedenken: Sie ist sehr aufwändig und stellt viele Probleme, insbesondere das Wiederauflaufen nach einem Hardwarefehler. Aber auch die Hardwarelösungen oder SW/HW-Kombinationen können Probleme bereiten, wenn die RAID-Platten nach einem Defekt durch Controller oder Hardware anderer Hersteller bedient werden sollen. Ursache ist die Konfigurationsdatei (*Configuration Data on Disk COD*), in der bei der Initialisierung des RAID-Systems die Zuordnung der Platten zu den RAID-Einheiten, Angaben zum RAID-Level und ähnliches gespeichert werden und bei jedem Hersteller anderes Format hat. Eine Normung der Datei (*Disk Data Format DDF*) durch die SNIA-Gruppe war deshalb eine wichtige Voraussetzung für die Portabilität und Backup-Management der RAID-Konfigurationen.

#### Beispiel Windows NT Fehlertoleranz durch Spiegelplatten und RAID

Für eine fehlertolerante Dateiverwaltung gibt es in Windows NT einen speziellen Treiber: den FtDisk-Treiber. Dieser wird zwischen den NTFS-Treiber und den eigentlichen Gerätetreiber geschoben; er bildet als Zwischenschicht eine virtuelle Maschine, die alle auftretenden Fehler selbst abfangen soll, unsichtbar und unbemerkt von allen höheren Schichten. Tritt nun ein Fehler (*bad sector*) auf, so werden die Daten des fehlerhaften Blocks von der Spiegelplatte gelesen oder mit Hilfe der Fehlerkorrekturinformation (Paritäts-*stripes*) rekonstruiert. Dann wird für den fehlerhaften Sektor ein neuer Sektor vom Gerät angefordert (*bad sector mapping*) und die Daten darauf geschrieben. Damit existiert wieder die nötige Datenredundanz im System. Gibt es keine freien Sektoren (*spare sectors*) mehr oder kann das Gerät kein *sector mapping*, so wird der wiederhergestellte Datenblock zusammen mit einer Warnung an den Dateisystemtreiber weitergegeben.

Wurde keine fehlertolerante Plattenorganisation eingerichtet, so kann FtDisk den Fehler als „Lese-/Schreibfehler“ nur weitermelden. Es war dann Sache des Dateisystems (oder des Benutzers), darauf sinnvoll zu reagieren.

Man beachte, dass die RAID-Konfigurationen zwar für Ausfalltoleranz sorgen, aber kein regelmäßiges Backup ersetzen. Im Internet gibt es herzerreißende Geschichten von Systemadministratoren, die durch Softwarefehler ganze RAID-Konfigurationen korruptiert erhielten und bitter enttäuscht wurden. Weitere Informationen zu höheren RAID-Stufen sind beispielsweise unter <http://www.acnc.com/raid.html> zu finden.

5.3.1 RAID-Ausfallwahrscheinlichkeiten

Die Ausfallwahrscheinlichkeit eines RAID-Systems lässt sich leicht aus den Ausfallwahrscheinlichkeiten der Einzelkomponenten errechnen. So ist der Ausfall eines RAID0-Systems genau dann gegeben, wenn nur eine der beteiligten Platten  $D_1$  oder  $D_2$  ausfällt. Zum numerischen Errechnen der Ausfallwahrscheinlichkeit des Gesamtsystems stellen wir uns eine Tabelle mit allen Zuständen auf und notieren jeweils die Einzelwahrscheinlichkeiten. Dabei gehen wir davon aus, dass jede Platte unabhängig von der anderen ausfällt. Der Zustand „intakt“ sei mit 1 und „defekt“ mit 0 notiert, wobei die Ausfallwahrscheinlichkeiten von  $D_1$  mit  $P_1$  und von  $D_2$  mit  $P_2$  notiert seien, siehe Abb. 5.17.

Das Gesamtsystem fällt aus, wenn nur eine der beiden Platten oder Beide ausfallen. Die Ausfallwahrscheinlichkeit  $P_j$  des Gesamtsystems ist also  $P(S_3) + P(S_2) + P(S_1)$  oder kürzer  $P_j = 1 - P(S_0) = 1 - [(1 - P_1) (1 - P_2)] = P_1 + P_2 - P_1 P_2$ .

Auf diese Weise lassen sich mit Hilfe einer Tabelle alle RAID-Kombinationen untersuchen.

Wie lange wird das System nun noch leben? Um dies vorherzusagen, müssen wir die vom Hersteller gegebene Angabe der „mittleren Dauer zwischen zwei Ausfällen“ MTBF in Wahrscheinlichkeiten umrechnen. Angenommen, wir haben eine konstante Ausfallrate, also eine konstante Wahrscheinlichkeit des Ausfalls. Diese Annahme entspricht strukturell der Annahme einer konstanten Wahrscheinlichkeit in Abschn. 2.2.7, dass ein Job ankommt. Dort errechneten wir, dass die Wahrscheinlichkeit eines Jobs bei der Exponentialverteilung

Abb. 5.17 Die Zustände des Gesamtsystems und seine Einzelzustände

| Zustand | Disk $D_1$ | Disk $D_2$ | Ausfallwahrscheinlichkeit |
|---------|------------|------------|---------------------------|
| $S_0$   | 1          | 1          | $(1-P_1) (1-P_2)$         |
| $S_1$   | 1          | 0          | $(1-P_1) P_2$             |
| $S_2$   | 0          | 1          | $P_1 (1-P_2)$             |
| $S_3$   | 0          | 0          | $P_1 P_2$                 |



$$P_j(t) = 1 - P_N(t) = 1 - e^{-\lambda t}$$

liegt. In unserem Fall ist die Rate  $\lambda = 1/\text{MTBF}$ , sodass wir

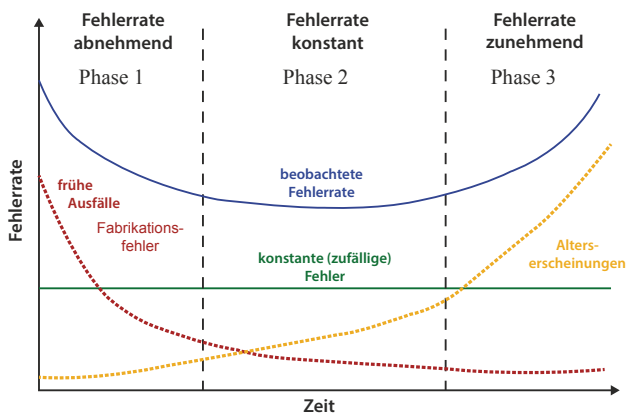
$$P_j(t) = 1 - e^{-t/\text{MTBF}}$$

erhalten.

Ist das immer der Fall? Dazu sehen wir uns die Ausfallwahrscheinlichkeit eines Geräts über die ganze Lebensdauer an. Es ist bekannt, dass sie als „Badewannenkurve“ modelliert werden kann; siehe die blaue, resultierende Kurve in [Abb. 5.18](#).

Wir können dabei drei Phasen erkennen: die erste Phase der Ausfälle durch Werksfehler, die mit der Zeit schnell abnimmt. Dann die zweite Phase des Normalbetriebs mit ungefähr konstanter Ausfallrate und am Ende der Lebensdauer die dritte Phase mit exponentiellem Anstieg der Ausfallwahrscheinlichkeit durch Alterung. Die Angabe der MTBF und damit all unsere Berechnungen beruhen also auf dem „Normalbetrieb“ in Phase 2 und sind nur eine Approximation der Realität für unsere Zwecke. Wir müssen also den Herstellerangaben kritisch gegenüberstehen; sie verdeutlichen nur einen Trend und geben keine stark belastbare Aussage für den augenblicklichen Zeitpunkt. Trotzdem können wir für unsere Modellrechnungen zum Vergleich verschiedener Systeme die MTBF und damit die Wahrscheinlichkeiten der verwendeten Komponenten einfach übernehmen.

Eine besondere Situation tritt ein, wenn wir einen Ausfall als Folge eines anderen Ausfalls haben, etwa die Überlastung eines Systems als Folge eines Ausfalls einer Komponente. Hier ist es günstig, mit bedingten Wahrscheinlichkeiten zu rechnen.



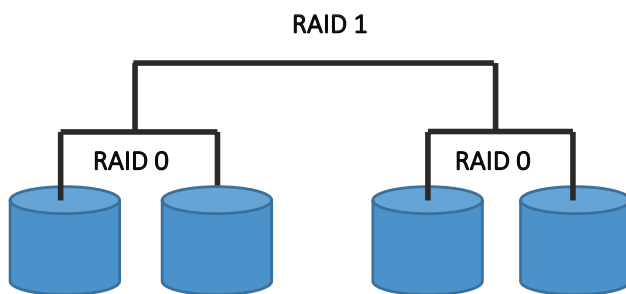
**Abb. 5.18** Entwicklung der Ausfallwahrscheinlichkeit eines Geräts

### 5.3.2 Aufgaben zu RAID

#### Aufgabe 5.3-1 RAID-Ausfall

Die Wahrscheinlichkeit, dass eine Festplatte in einem gegebenen Zeitraum einen Fehler (Head-Crash) aufweist betrage  $p$ .

- Bestimmen Sie die Wahrscheinlichkeit, dass im Beobachtungszeitraum Daten verloren gehen, für eine Konfiguration mit zwei gleichen Platten für ein RAID-0-System.
- Bestimmen Sie die Wahrscheinlichkeit, dass im Beobachtungszeitraum Daten verloren gehen, für eine Konfiguration mit vier gleichen Platten für ein RAID-01-System, wobei jeweils zwei RAID-0 Platten mittels *striping* zu einem Subsystem verbunden sind und beide Subsysteme mittels RAID 1 gespiegelt werden.



#### Aufgabe 5.3-2 RAID und NON-RAID

Angenommen, Sie haben ein System  $S_1$  aus 5 Laufwerken, die jeweils eine Kapazität von 1TB haben und mit der Wahrscheinlichkeit  $p = 0,1 \%$  ausfallen. Sie sind als RAID 5 zusammengeschaltet.

Außerdem haben Sie noch ein einzelnes Laufwerk  $S_2$  von 4 TB, das mit  $p_2 = 0,001 \%$  ausfällt.

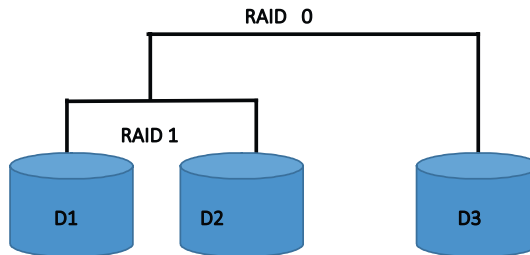
- Welches System hat mehr Speicherkapazität für Anwenderdaten und warum?
- Welches System ist zuverlässiger als das andere und warum?
- Wenn beide Systeme gleich viel kosten, welches von beiden sollte man kaufen, und warum?

#### Aufgabe 5.3-3 RAID-Ausfallzeit

Gegeben sei folgendes System aus drei Festplatten:

| Platte | Kapazität | MTBF        | Aktuelles Alter |
|--------|-----------|-------------|-----------------|
| D1     | 4 TB      | 630.720 h   | 3 Jahre         |
| D2     | 6 TB      | 858.480 h   | 2 Jahre         |
| D3     | 8 TB      | 1.156.320 h | 1 Jahr          |

mit folgendem Aufbau



- Geben Sie an, welche Datenmenge darin abgespeichert werden kann, und erklären Sie, wie diese Zahl zustande kommt.
- Das genannte System wird von jetzt an 3 Jahre lang genutzt. Berechnen Sie die Wahrscheinlichkeit, dass es während dieser Zeit zu einem Datenverlust kommt.  
Es gelten folgende Annahmen:
  - Beim Auftreten von Fehlern gelten keinerlei Abhängigkeiten zwischen den Festplatten.
  - Während der bereits erfolgten Betriebszeit traten keinerlei Fehler auf.

#### Aufgabe 5.3-4 RAID-Folgefehler

Leider trifft eine wichtige Voraussetzung für die Berechnung der Ausfälle meist nicht zu: Die einzelnen Platten fallen nicht unabhängig voneinander aus.

Angenommen, Sie haben einen RAID5-Verbund von drei Platten D1, D2 und D3, wobei D1 und D2 im selben Schacht stecken. Dies bedeutet, dass neben den Einzelausfallwahrscheinlichkeiten  $P_1$ ,  $P_2$  und  $P_3$  auch eine bedingte Wahrscheinlichkeit  $P_a$  existiert, mit der nach dem Ausfall einer Platte kurz darauf die andere Platte auch ausfällt, etwa durch Überhitzung.

Wie hoch ist die Ausfallwahrscheinlichkeit  $P$  des Gesamtsystems als Funktion von  $P_1$ ,  $P_2$ ,  $P_3$  und  $P_a$ ?

## 5.4 Modellierung und Implementierung der Treiber

Das Schreiben eines Gerätetreibers erfordert traditionellerweise ein erhebliches Grundwissen vom Programmierer, sowohl über das Gerät und den Controller als auch über die intimen Einzelheiten aus dem Arbeitsleben des Betriebssystems. Die meisten Fehler in Betriebssystemen resultieren deshalb auch aus neuen Treibern, die mit neuen Peripheriegeräten in das Betriebssystem eingegliedert werden und im *kernel mode* ungehindert Folgefehler produzieren können. Auch der Angriff auf ein System wird gern über präparierte Treiber eingeleitet.

Vor diesem Hintergrund ist es daher sehr wichtig, dass die Betriebssystemhersteller klare, explizite Schnittstellen für die Gerätetreiber bekanntgeben. Auch die Vorgabe und Existenz höherer Treiber, die alle geräteunabhängigen Verwaltungsaufgaben (Führen der Auftragslisten, Verwalten des Cachepuffers usw.) erledigen, hilft sehr bei der Treibererstellung. Ein weiterer Vorteil ist es, wenn man neue Treiber direkt in ein System einhängen kann, ohne den Kern neu binden zu müssen und das System dazu stillzulegen. Derartigen Forderungen wird sehr unterschiedlich von den Betriebssystemen entsprochen.

Ein weiteres Problem ist die Behandlung von Fehlern. Auch hier gibt es wenige gekennzeichnete Schnittstellen.

### 5.4.1 Beispiel UNIX: Treiberschnittstelle

Traditionellerweise lassen sich UNIX-Treiber nicht nachladen, sondern werden übersetzt und mit dem Kern statisch verbunden. Diese Einschränkung ist allerdings bei modernen Unix-Versionen (z. B. Linux, HP-Unix, aber nicht bei OpenBSD) aufgehoben, obwohl dynamisch ladbare Treiber ein potentielltes Sicherheitsrisiko darstellen.

Jeder Treiber implementiert eine Reihe von mehr oder weniger festgelegten Prozeduren, die in einer einheitlichen Tabelle zusammengefasst werden. Da sie in C geschrieben werden, sind die Treiber einigermaßen portabel. Alle Prozeduren der Treiber werden als Adressen in einer `RECORD`-Struktur (*conf.h*) pro Gerät definiert und in einer zentralen Tabelle *conf.c* angeordnet, die auch mit einem Programm *config* erzeugt werden kann. Jedem Gerät (Controller) wird eine eindeutige Nummer (*major device number*) zugeordnet. Wird nun eine der Prozeduren aufgerufen, so geschieht dies nur über die zentrale Tabelle. Jeder Tabelleneintrag besteht aus einer `RECORD`-Struktur, in der die Adressen der Treiberprozeduren des Geräts aufgelistet sind. Der Index des Tabelleneintrags entspricht der Nummer des Geräts.

Diese zentrale, in Linux dynamisch aufgebaute Tabelle, ist die einzige Aufrufverbindung vom Betriebssystem zu den Treibern. Sie ist in zwei Abschnitte nach blockorientierten (*block device switch* `bdevsw`) und zeichenorientierten Geräten (*character device switch* `cdevsw`) geordnet.

#### Beispiel Auszug aus *conf.c* eines SUN-OS für block- und character devices

```
struct bdevsw    bdevsw[] = {
    { tmopen,    tmclose, tmstrategy,    tmdump,    /*0*/
      0,    B_TAPE},
    { nodev,    nodev,    nodev,    nodev,    /*1*/
      0,    B_TAPE},
    { xyopen,    nulldev, xystrategy,    xydump,    /*2*/
      xysize,    0},    ... };
struct cdevsw    cdevsw[] = {
```

```

{ cnopen,    cnclose,  cnread,    cnwrite,    /*0*/
  cniocctl,  nulldev,  cnselect,  0,
    0,        0,        },
{   nodev,    nodev,  nodev,    nodev,    /*1*/
  nodev,    nodev,  nodev,    0,
  &swcinfo,    0,        },
{   syopen,  nulldev,  syread,    sywrite,    /*2*/
  syiocctl,  nulldev,  syselect,  0,
    0,        0,        }, ... };

```

Die blockorientierten Geräte haben hier die synchronen Aufrufe `open`, `close`, `strategy`, `dump`, `psize` und `flags`, die zeichenorientierten Geräte die Aufrufe `open`, `close`, `read`, `write`, `ioctl`, `reset`, `select`, `mmap`, `stream`, `segmap`. Die asynchronen Interrupt-Service-Routinen sind nicht hier, sondern in einem gesonderten Assemblermodul eingetragen.

Nicht jedes logische Gerät hat auch eine Implementierung der möglichen Treiberprozeduren: Existiert keine Implementierung, so wird dies mit `nodev` angegeben und führt beim Aufruf zu einer Fehlermeldung; soll einfach „nichts“ passieren, so wird dies mit `nulldev` abgefangen. Kann ein Gerät sowohl zeichenorientiert als auch blockorientiert angesprochen werden, so sind die entsprechenden Treiberprozeduren des einen Geräts in beiden Tabellen vermerkt.

Beim Auftritt von Fehlern (Gerät nicht eingeschaltet, Spur defekt, Lesefehler etc.) wird die entsprechende Fehlernummer in den Eintrag der *user structure* des Benutzerprozesses geschrieben. Bemerkt der Treiber eine Situation, die er absolut nicht mehr beherrscht (falsche Pufferadressen etc.) und die nicht ignoriert werden kann, so bleibt als letzte Möglichkeit, die Prozedur *panic* aufzurufen, die einen Fehlertext ausdruckt und das gesamte Betriebssystem anhält.

Bei bestimmten Prozeduren, die vom Benutzerprozess aus aufgerufen werden (wie `open()`, `close()`, `read()`, `write()`), stehen als Argumente und zur Fehlerrückmeldung bestimmte Einträge der *user structure* (`u.u_offset`, `u.u_count`, `u.u_error` etc.) zur Verfügung.

Die Prozeduren (Dienstleistungen) der Treiber sind für ein Gerät `xx`, z. B. `xx = mt` (Bandgerät) oder `xx = rk` (Plattenlaufwerk):

- **XX\_init**  
Eine Prozedur zur Initialisierung des Treibers und des Geräts. Diese wird beim Systemstart ausgeführt.
- **XX\_read, XX\_write** *character devices*  
Eine Prozedur zum direkten Lesen und Schreiben von Daten im *raw*-Modus.
- **XX\_open, XX\_close**  
Eine Prozedur, die bei `open()`, und eine, die bei `close()` aufgerufen wird. Diese Prozeduren dienen hauptsächlich dazu, die Geräte bei jeder Benutzung initialisieren zu

können, und haben gerätespezifische Bedeutung. Beispielsweise kann ein Magnetband bei `mt_close` das Band zum Anfang zurückspulen, damit man es entnehmen kann, ein Drucker einen Seitenvorschub durchführen usw.

- **XX\_ioctl** *character devices*

Dieser Aufruf erhält die Parameter, die man beim Systemaufruf `ioctl ()` angegeben hat, und dient zur Einstellung des Geräts, beispielsweise der Übertragungsgeschwindigkeit und Modus bei seriellen Geräten.

- **XX\_strategy** *block devices*

Diese Prozedur wird zum Lesen und Schreiben von Einzelblöcken (*block device*) verwendet und besorgt neben der Übersetzung von logischer Blocknummer zu physikalischer Speicheradresse zusätzlich noch eine spezielle Lese- und Schreibstrategie, um den Durchsatz zu verbessern, siehe [Abschn. 5.1.1](#).

Die Information, ob es sich beim übergebenen Auftragsblock um einen Lese- oder Schreibauftrag handelt, ist im Kopf des Blocks vermerkt. Da die `strategy`-Prozedur auch asynchron aus einem Interrupt-Handler heraus aufgerufen werden kann, um den nächsten Block dem Gerät zu übergeben, sind alle Argumente und Fehlerrückmeldungen nur im Kopf der Blockstruktur (*buf.h*) enthalten, da der Treiber nicht auf den Benutzerprozess zugreifen kann, um sich die Parameter zu holen. Die Blöcke entsprechen also einer Nachricht an den Treiber, in der Auftrag und Parameter vermerkt sind.

- **XX\_intr**

Auch die Interrupt-Service-Routine gehört in UNIX zum Treiber und ist in C geschrieben. Sie setzt voraus, dass bereits alle Register gerettet wurden (durch eine vorgeschaltete Assembleroutine), und transferiert nur die Daten bzw. setzt den nächsten DMA-Transfer mit Hilfe der verketteten Auftragsliste auf.

Es gibt noch weitere Prozeduren, die aber von der jeweiligen UNIX-Version abhängig sind und Spezialfunktionen erfüllen, siehe Egan und Teixera ([1990](#)).

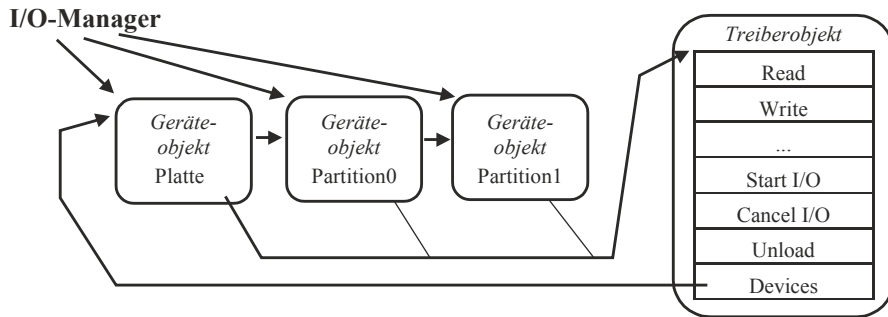
## 5.4.2 Beispiel Windows NT: Treiberschnittstelle

In Windows wird versucht, für alle Varianten (Windows SC, Windows CE 3.0, Windows 98, Windows NT 5.0 bzw. Windows 2000) das gleiche Treibermodell zu verwenden, um einen existierenden Treiber in allen Betriebssystemen nutzen zu können. Das Win-32-Treibermodell WDM stellt eine Bibliothek von Standardroutinen für Ein- und Ausgabe sowie kompatible Gerätetreiber für Windows 98 bzw. Windows 2000 zur Verfügung. Es gibt drei verschiedene Treiberklassen: Eingabegeräte (*Human Interface Device* HID) wie Maus, Tastatur, Datenhandschuhe etc., Standbildgeräte wie Scanner, sowie Geräte

für Datenströme wie Video und Audio. Für jede Klasse gibt es einen Klassentreiber, der die Grundfunktionalität implementiert, und einen gerätespezifischen Treiberzusatz („Minitreiber“).

Die Treiber in Windows NT müssen bestimmte Konventionen einhalten. Dazu gehören folgende Prozeduren, vgl. Nagar (1997):

- *Eine Initialisierungsprozedur (load driver)*  
Sie wird vom I/O-Manager ausgeführt, wenn der Treiber ins Betriebssystem geladen wird, und erzeugt im wesentlichen ein Objekt, unter dem der I/O-Manager das Gerät erkennt und in dem die folgenden Prozeduren referiert sind.
- *Eine Abschlussprozedur (unload driver)*  
Diese Prozedur gibt alle Systemressourcen wie Speicher, Puffer usw. frei, so dass der I/O-Manager sie aufräumen kann.
- *Eine Gruppe von Service-Funktionen (dispatch routines)*  
wie Schreiben, Lesen oder andere Funktionen des Geräts. Diese Funktionen werden von den Ein- und Ausgabebefehlen (*I/O Request Package* IRP) angesprochen.
- *Eine Prozedur, um den Datentransfer zu starten (start I/O)*  
sowie mindestens eine Prozedur, um den Datentransfer *abzubrechen (cancel I/O)*. Die Ausführung und Auswahl dieser Prozedur wird sicher davon abhängen, wie weit der Datentransfer schon gediehen ist.
- *Eine Interrupt-Service-Routine (ISR)*  
Das Interruptsystem von Windows NT gibt die Kontrolle bei einem Geräte-Interrupt an diese Prozedur. Da dies auf hoher Priorität geschieht, wird von der ISR erwartet, dass sie so bald wie möglich den ISR-Aufruf in einen speziellen Prozeduraufruf (*Deferred Procedure Call* DPC) umwandelt und diesen Aufruf als Arbeitspaket in die Warteschlange der DPCs einhängt. Die DPCs laufen unter geringer Priorität ab, so dass andere Interrupts diese unterbrechen können.
- *Eine zur ISR gehörende DPC-Prozedur*  
Die DPC-Prozeduren werden aufgerufen, wenn die Priorität der laufenden Prozesse unter die definierte DPC-Priorität gefallen ist. Die DPC-Prozedur erledigt die Hauptarbeit der ISR, insbesondere den Datentransfer zu vervollständigen und den nächsten Auftrag aus der Warteschlange einzuleiten.
- *Eine Prozedur, um den Datentransfer abzuschließen (completion routine)*  
Diese Prozedur kann von einem Treiber im IRP angegeben werden, so dass sie nach der Funktion eines tieferen Treibers zum Abschluss aufgerufen wird. Die Prozedur gibt Informationen über Erfolg, auftretende Fehler oder Abbruch an höhere Treiber (z. B. das Dateisystem) weiter und ermöglicht diesem einen adäquaten Abschluss des Auftrags.
- *Eine fehlernotierende Prozedur (error logging)*  
Diese gibt die Fehlerinformationen an den I/O-Manager weiter, der sie in eine Fehlerdatei schreibt.



**Abb. 5.19** Die Verbindung von Geräte- und Treiberobjekten

Beim Laden des Treibers wird nicht nur ein Treiberobjekt (*driver object*) für den Zugriff des I/O-Managers erzeugt, sondern auch für jedes Gerät bzw. jede Gerätefunktion ein Geräteobjekt (*device object*). Die Geräteobjekte stellen dabei den Bezugspunkt für die Benutzung der Treiber dar. Beispielsweise wird beim Öffnen der Datei `\Device\Floppy\Texte\bs_files.doc` (s. [Abb. 4.7](#)) der Name `\Device\Floppy0` abgetrennt: Er stellt den Namen des Geräteobjekts dar, das den Bezugspunkt für den I/O-Manager darstellt. Der I/O-Manager, der vom Dateisystem beauftragt wird, reicht das IRP weiter über das Geräteobjekt an den dafür zuständigen Treiber. Dazu sind alle Geräteobjekte der physikalischen, logischen und virtuellen Geräte durch einen Zeiger auf ihr Treiberobjekt mit dem Gerätetreiber verbunden; umgekehrt sind sie auch Teil einer Liste des Treiberobjekts, mit der der I/O-Manager beim Beenden des Treibers (*unload driver*) nachprüfen kann, welche Geräteobjekte davon betroffen sind. In [Abb. 5.19](#) ist diese Verbindung illustriert.

Besondere, allgemein benutzte Funktionen wie beispielsweise die Kontrolle des PCI-Bus sind in spezielle Treiber ausgelagert, die vor oder nach dem eigentlichen Gerätetreiber aufgerufen werden. In diesem Sinne wirken sie ähnlich den Filtern im Unix *stream*-System.

Die Gerätetreiber haben bei ihren Funktionen verschiedene Nebenbedingungen zu beachten. Einer der wichtigsten Faktoren ist die Tatsache, dass der Betriebssystemcode auf verschiedenen Prozessoren eines Multiprozessorsystems ausgeführt werden kann.

Dies erzwingt die Koordination beim Zugriff auf die Daten, die systemweit (*global data*) oder mit anderen Instanzen zusammen (*shared data*) genutzt werden. Dies bedeutet für einen Treiber, dessen Code gleichzeitig auf mehreren Prozessoren abgearbeitet wird, dass er seinen Zugriff auf die Register eines Gerätes durch einen *spin lock* absichern muss. Auch der Zugriff auf die Datenstruktur des Treibers (Auftragsliste etc.) muss so abgesichert werden.

Im Unterschied zu Monoprozessorsystemen muss auch beim Interruptsystem darauf Rücksicht genommen werden. Auch die Interrupt-Service-Routine, die auf höchster Priorität läuft, kann nicht davon ausgehen, alleinigen Zugang zu den Geräteregistern, Auftragslisten



usw. zu haben – eine Kopie des Treiber auf einem anderen Prozessor mit geringerer Priorität kann gleichzeitig auf die Daten zugreifen. Deshalb gibt es im Kern zusätzlich zu den (*busy wait*) *spin lock*-Prozeduren spezielle *kernel*-Synchronisationsprozeduren.

Eine weitere Aufgabe für Treiber ist die Reaktion auf den Zusammenbruch der Versorgungsspannung (*power failure*). Hier wird vom Netzteil vorausgesetzt, dass es die Versorgung nach dem *power failure*-Signal noch einige Millisekunden aufrecht erhalten kann, in denen wichtige Daten gesichert werden können. Ein Treiber muss also bei seinen Aktionen atomare Teile unterscheiden, die nicht gestört werden dürfen, um die Datenintegrität zu wahren. Vor solch einem Abschnitt muss er sich versichern, dass kein *power failure*-Zustand vorliegt, dann den *power failure*-Interrupt sperren, die eigenen Daten schreiben und dann den Interrupt wieder freigeben. Im Falle eines *power failure* dagegen muss das Gerät in einen kontrollierten, bekannten Zustand versetzt werden, bevor die Spannung zusammenbricht.

Ist das System mit einem batteriegestütztem RAM-Bereich ausgestattet, in den alle wichtigen Prozessdaten (Prozesskontext) gerettet werden konnte, so ist es möglich, nach dem Wiederherstellen der Versorgungsspannung die Programmausführung fortzusetzen.

Das Schreiben eines Treibers für Windows NT ist nicht einfach und erfordert eine genauere Auseinandersetzung mit der Materie, siehe Oney (1999) oder Viscarola und Mason (1999).

### 5.4.3 I/O Treiber in Großrechnern

Im Allgemeinen haben Großrechner eine solche Rechengeschwindigkeit, dass man sie nicht mit langsamem I/O ausbremsen will. Anstelle von Treibern besitzen sie deshalb meist Kommunikationskanäle zu dedizierten I/O-Rechner („control units“), die nur für I/O zuständig und dafür optimiert sind. Damit ist die I/O-Funktionsschicht komplett vom Großrechner ausgelagert. Dies entspricht einer Software-Hardware-Migration wie wir sie in Kap. 1 kennen gelernt haben. Beispielsweise wird der Linux CFQ I/O-Scheduler nicht auf dem Hauptrechner, sondern auf den I/O-Rechner (*mainframe control unit*) ausgeführt.

Das Netzwerk aus Kommunikationskanälen vermittelt nicht nur zwischen mehreren I/O-Rechnern und mehreren Großrechnern, sondern ermöglicht auch die Anbindung zusätzlicher Netzwerke für Speicherung und *backup*. Außerdem wird der I/O meist noch auf dem Weg vom Netzwerk zu den CPUs des Großrechners in einem L2 Cache gepuffert, so dass einem schnellen Zugriff der CPUs auf I/O-Daten nichts mehr im Wege steht.

### 5.4.4 Aufgaben zu Treibern

#### Aufgabe 5.4-1 Treiber

- a) Was ist ein Treiber und was der Unterschied zwischen einer abstrakten Maschine, einem Treiber und einem Gerätetreiber?

- b) In welchem Systemmodus laufen Treiber? Warum?
- c) Recherchieren Sie: Was ist ein FUSE-Treiber?
- d) Wie wird der dazu gehörende Treiber bei dem Interrupt eines Geräts ausgewählt?

---

## 5.5 Optimierungsstrategien für Treiber

Im vorigen Abschnitt kamen wir zu der Erkenntnis, dass eine Aufgabentrennung sehr sinnvoll ist zwischen Hersteller eines Geräts und Betriebssystem. Alle gerätespezifischen Dinge sollten möglichst im Treiber oder Controller gekapselt sein und der spezifische Treiber sollte vom Hersteller mitgeliefert werden. Dies ist die Theorie. Leider ist es aber in der Praxis oft so, dass für dieses spezielle Betriebssystem oder das neue Gerät kein Treiber verfügbar ist, so dass wir uns selbst des Problems annehmen müssen. In diesem Fall können wir einige Konzepte früherer Kapitel auch hier anwenden. In diesem Abschnitt soll dies am Beispiel des rotierenden Massenspeichers näher ausgeführt werden. Dabei steht die Bezeichnung „Treiber“ allgemein für alle gerätespezifischen Module, also etwa auch für die Software des eingebetteten Gerätecontrollers.

Angenommen, wir haben einen Treiber für ein Gerät geschrieben, indem wir uns durch die Bits und Bytes des Gerätecontrollers durchgehangelt und die kargen Unterlagen des Herstellers intelligent mit detektivischem Spürsinn interpretiert haben – welche Strategien können wir nun einem „höheren“, möglichst geräteunabhängigen Treiber implementieren, um für eine ganze Gerätekategorie den Zugriff zu optimieren?

### 5.5.1 Optimierung bei Festplatten

Es existiert eine Reihe von Schedulingalgorithmen, um die Reihenfolge der Blöcke möglichst „gut“ den Schreib-Leseköpfen von Festplatten anzubieten. Dabei stellt sich aber die Frage: Lohnt sich überhaupt der Einsatz komplizierter Schedulingalgorithmen? Studien zeigen, dass die normale Arbeitsliste nur einen einzigen Auftrag enthält (Lynch 1972). Dies bedeutet: Alle Strategien sind in diesem Fall gleich gut wie die einfache FCFS-Strategie.

Die Strategien zur Abarbeitung der Block-Arbeitsliste sind nicht die einzige Optimierungsmöglichkeit der Zugriffszeit. Auch die Entscheidung, wo die Platte logisch anfängt, kann den Zugriff schneller machen: Bildet man den logischen Block 0 mit dem zentralen Index nicht auf (Sektor 0, Spur 0) am Rand der Platte ab, sondern auf die mittlere Spur, so ist der mittlere Weg des Kopfes bei ungepuffertem Zugriff zu allen Spuren kleiner als vom Rand aus und der Zugriff damit deutlich schneller. Wird dagegen der zentrale Index als Kopie im Hauptspeicher gehalten, so ist nur noch die Folge der Positionierungsbefehle entscheidend und nicht mehr die Lage des zentralen Index.

Die bisher besprochenen Mechanismen der Leistungssteigerung im Treiber müssen allerdings alle im Kontext des Controllers gesehen werden. Prinzipiell kann leicht der Fall auftreten, dass alle Optimierungsmaßnahmen des Treibers vom Controller, der meist mit eigenem Mikroprozessor ausgestattet ist, durch eigene Optimierungsstrategien ausgebremst werden. Beispielsweise benötigen moderne, schnelle Mikroprozessorgesteuerte Controller kein Scheduling; sie haben ihre eigenen Strategien. Auch haben manche Controller die Möglichkeit, defekte Blöcke durch Ersatzblöcke auf speziellen, normal nicht zugänglichen Spuren zu ersetzen. Diese spezielle Abbildung tritt immer dann in Kraft, wenn ein defekter Block angesprochen wird, und ist transparent nach außen, kann also nicht vom Treiber bemerkt werden. Damit erweisen sich alle Bemühungen des Treibers, die Kopfbewegung zu minimieren, als obsolet: Die dazwischenliegenden Bewegungen zur Ersatzspur lassen keine Optimierung von außen mehr zu. Es ist deshalb sehr sinnvoll, die Schnittstellen zwischen Treiber und Controller vom Hersteller aus systematisch zu durchdenken und vorauszuplanen. Beispielsweise kann man bestimmte geräteabhängige Optimierungen wie Kopfbewegung und Sektorsuche dem Controller zuweisen und dort kapseln; nach außen hin sind nur die Leistungen (Speicherung von Blöcken mit logischen Adressen etc.) sichtbar, die unabhängig von der implementierten Elektronik und Mechanik erbracht werden.

### 5.5.2 Pufferung

Eine wichtige Leistungsoptimierung bei Massenspeichern, die für starken Durchsatz benötigt werden (z. B. bei Datenbankanwendungen), kann man mit einem Puffer (Daten-cache) erreichen, der auf verschiedenen Ebenen eingerichtet werden kann.

Auf der Ebene des Dateisystemtreibers ist es üblich, die wichtigsten, am meisten benutzten Blöcke einer Datei zu puffern, wobei man ihn in zwei verschiedene Arten (Schreib- und Lese-puffer) unterteilen kann, jeden mit eigener Verwaltung.

Auf unterster Treiberebene kann man größere Schreib- und Leseinheiten puffern, beispielsweise eine ganze Spur. Nachfolgende Schreib- und Leseoperationen beziehen sich meist auf aufeinanderfolgende Sektornummern und können mit der gepufferten Spur wesentlich schneller durchgeführt werden.

Allerdings hat die Pufferung auch hier die gleichen Probleme wie in [Abschn. 3.5](#) beschrieben. Bei der Verwaltung muss sichergestellt werden, dass beschriebene Blöcke und Sektoren auch für die Anfragen anderer Prozesse verwendet werden, um die Datenkonsistenz zu garantieren.

Wichtig ist dabei die Synchronisierung der Pufferinhalte mit dem Massenspeicher, bevor das Rechnersystem abgeschaltet wird. Bei Versorgungsspannungsausfall (*power failure*) muss dies sofort von den gepufferten Treibern eingeleitet werden.

Auch hier muss man sorgfältig die Schnittstelle zum Controller beachten. Manche Controller haben bereits einen Cache intern integriert; hier ist es sinnlos, auch auf Treiber-ebene einen Cache anzulegen.

---

**Beispiel UNIX Pufferung**

Die Pufferung der seriellen *character files* wird über Listen aus Zeilen (c-list) durchgeführt. Aus diesem Grund wird ein RETURN(Zeilende/Neue Zeile)-Zeichen benötigt, um einen Text von der Tastatur einzulesen. Besondere Statusbits dieses *special file* erlauben es allerdings auch, für eine reine Zeicheneingabe (Cursor-Tasten etc.) sofort jedes Zeichen ohne Pufferung zu lesen oder das lokale Echo eines Zeichens auf dem Monitor zu unterdrücken.

Das Puffersystem für die *block devices* benutzt als Speichereinheiten die Blöcke. Für jeden Treiber gibt es eine Auftragsliste von Blöcken, die gelesen bzw. geschrieben werden sollen. Die Liste aller freien Blöcke ist doppelt verzeigert und in einem Speicherpool zentral zusammengefasst. Außerdem gibt es zwei Zugangswege zu den *block devices*: zum einen über einen Dateinamen und damit über das Dateisystem und über die `xx_strategy`-Prozedur und zum anderen über den *special file* als *raw device* über die `xx_read/ write`-Prozedur.

Da durch die Pufferung der *i-nodes* das gesamte Dateisystem bei einem Netzausfall hochgradig gefährdet ist, werden in regelmäßigem Takt (alle 30 Sek.) von der `sync ()`-Prozedur alle Puffer auf Platte geschrieben und die Datenkonsistenz damit hergestellt. Dies wird auch beim Herunterfahren des Systems (*shut down*) durchgeführt.

---

**Beispiel Windows NT Pufferung**

Zur Verwaltung des Ein- und Ausgabecache gibt es einen speziellen *Cache Manager*. Dieser alloziert dynamisch Seiten im Hauptspeicher und stellt ein *memory mapping* zwischen den Hauptspeicherseiten und einer Datei her. Die Anzahl der so erzeugten *section objects* ist dynamisch: Sie hängt sowohl vom verfügbaren Hauptspeicher als auch von der Zugriffshäufigkeit auf die Dateiteile ab. Dies geschieht dadurch, dass diese Seiten des Cache-Managers wie die Speicherseiten eines normalen Prozesses vom *Virtual Memory Manager* verwaltet werden: Die Anzahl der im Hauptspeicher sich befindenden Seiten wird über den normalen *working set*-Mechanismus und den *paging*-Mechanismus (s. [Abschn. 3.3](#)) reguliert.

Für die Pufferung der seriellen Ein- und Ausgabe mussten spezielle Mechanismen entwickelt werden, um die Ergebnisse der Programme aus dem Multi-tasking non-preemptiven Windows 3.1 (16 Bit) auf dem Multi-tasking preemptive Windows NT zu erhalten. Für jedes Gerät gibt es nun in Windows 3.1 eine einzige I/O-Warteschlange, auch für serielle Geräte. Werden in verschiedene Fenster Eingaben (Zeichen, Mausklicks etc.) gegeben, so werden sie als jeweils eine Eingabeeinheit in den Eingabepuffer gestellt. Üblicherweise liest und schreibt ein Prozess unter Windows 3.1 beliebig lange (*non-preemptive*), bis er die für ihn bestimmte Eingabe abgearbeitet hat, und wird dann beim Lesen auf den Eingabepuffer blockiert, wenn die weitere Eingabe nicht für ihn ist. Der zum Fenster gehörende Prozess wird dann vom Windows-Manager aktiviert und liest seinen Pufferanteil ein.

Gehen wir nun zu einer preemptiven Umgebung über, bei der ein Prozess sofort deaktiviert werden kann, wenn seine Zeitscheibe abgelaufen ist, so führt dies bei nur einem Eingabepuffer zu Problemen – der neu aktivierte Prozess liest fehlerhaft die für den deaktivierten Prozess bestimmten Daten. Dies ist auch der Fall, wenn der Prozess „abstürzt“, also fehlerhaft vorzeitig terminiert. Aus diesen Gründen hat in Windows NT jeder *thread* seine eigene Eingabewarteschlange – ungelesene Eingabe für einen *thread* verbleibt beim Prozess und wird nicht vom nächsten *thread* versehentlich gelesen; das System wird robust gegenüber fehlerhaften Prozessen.

Wie kann man zwei derart unterschiedliche Systeme wie Windows 3.1 und Windows NT miteinander integrieren? Die Logik der non-preemptiven Prozesse aus Windows 3.1 ähnelt sehr der Logik von Leichtgewichtsprozessen (*threads*). Die Idee ist nun, den Mechanismus der *threads* in Windows NT dafür zu nutzen. Dazu behandelten die Designer von Windows NT das Windows 16-Bit-Subsystem (WOW), das die 16-Bit-Tasks als eigene *threads* gestartet hatte, wie einen einzigen Prozess. Diesem Prozess wird zwar regelmäßig der Prozessor entzogen (wie allen anderen NT Prozessen auch), bei der Prozessorrückgabe erhält aber der letzte laufende *thread* automatisch wieder die Kontrolle – als ob kein Prozesswechsel stattgefunden hätte. Somit wird genau das Verhalten erreicht, das im alten Windows/DOS System üblich war.

Allerdings ist es auch möglich, die 16-Bit-Applikationen in getrennten Adressräumen als eigene Prozesse ablaufen zu lassen. In diesem Fall verhalten sie sich natürlich nicht mehr zwangsläufig wie früher unter Windows 3.1, so dass dies nicht bei allen alten Applikationen sinnvoll ist.

### 5.5.3 Synchrone und asynchrone Ein- und Ausgabe

Bei der Ein- und Ausgabe ist es im Programm üblich, abzuwarten, bis ein Betriebssystemaufruf erfolgreich abgeschlossen wurde (**synchrone** Ein- und Ausgabe). Nun dauert es aber meistens eine gewisse Zeit, bis die Ein- oder Ausgabe durchgeführt wurde. Diese Zeit könnte das Programm besser mit anderen, ebenfalls wichtigen Arbeiten nutzen. Gerade in Programmen, die aus verschiedenen, unabhängig arbeitenden Teilen (*threads*) bestehen, ist nicht einzusehen, warum alle *threads* blockiert werden, obwohl nur einer auf eine Ein- oder Ausgabe wartet.

Eine Möglichkeit, die Blockierung zu verhindern, besteht in der Anerkennung der *threads* als eigenständige Leichtgewichtsprozesse durch das Betriebssystem wie in MACH oder Windows NT. Bei ihnen wird generell nur der aufrufende *thread* blockiert und nicht die anderen eines Prozesses. Dies ist aber nicht immer im Betriebssystem gegeben.

Eine andere Möglichkeit dafür bietet **asynchrone** Ein- und Ausgabe. Der Systemaufruf leitet dabei die Ein- und Ausgabeoperation nur ein; das Ergebnis muss von einem *thread* später mit einem speziellen Befehl abgeholt werden. Diese Art von Systemaufrufen stellt besondere Anforderungen an das Betriebssystem, da sowohl der Auftrag als auch das Ergebnis unabhängig vom beauftragenden Prozess zwischengespeichert und verwaltet werden muss.

**Beispiel UNIX Asynchrones I/O**

In manchen UNIX-Versionen gestatten spezielle Systemaufrufe die Umschaltung auf nicht-blockierendes Schreiben und Lesen. Bei Rücksprung aus der Lese- bzw. Schreibprozedur wird die Zahl der gelesenen bzw. geschriebenen Bytes zurückgegeben. Ist sie null, so konnte nicht gelesen bzw. geschrieben werden.

Eine andere Möglichkeit bietet das `signal ()`-System in UNIX. Üblicherweise gibt es ein eigenes Signal (SystemV-Linie: `SIGPOLL`, BSD-Linie: `SIGIO`), das beim Auftreten des gewünschten I/O-Ereignisses an den Prozess gesendet wird. Setzt er vorher entsprechende Prozeduren auf, die dann aufgerufen werden, so lässt sich der asynchrone I/O wie ein Interrupt behandeln. Alternativ dazu kann der Prozess auch nach etwas Aktivität in eine Pause (`sigpause ()`) gehen und dort auf das Signal warten.

**Beispiel Windows NT Asynchrones I/O**

Die Art und Weise, wie hier die Ein- und Ausgabe durchgeführt wird, hängt von einem Parameter ab, der beim `WriteFile ()`, `ReadFile ()`, `CreateFile ()` etc. angegeben wird.

Im Normalfall durchläuft der Systemaufruf alle Schichten (Systemservice, I/O-Manager, Gerätetreiber, Transfer-Interrupt, I/O-Manager, Rücksprung) und hält bei einem `ReadFile ()`-Aufruf den Prozess so lange an, bis die gewünschten Daten vorliegen. Wird dagegen ein Parameter „overlapped“ angegeben, so geht der Kontrollfluss nach dem Absetzen und Einleiten des Datentransfers sofort zurück an den Aufrufer. Der Prozess kann weiterarbeiten und andere Dinge tun. Um die gewünschten Daten abzuholen, wird ein Aufruf `wait(fileHandle)` abgesetzt, der den aufrufenden Prozess (*thread*) so lange blockiert, bis die gewünschten Daten vorliegen. Das Objekt `fileHandle` wird in den „signalisiert“-Zustand versetzt und der Prozess damit aufgeweckt. Nun kann er erneut den Systemaufruf `ReadFile ()` ausführen und endlich die Daten lesen.

Allerdings muss man darauf achten, dass nicht ein zweiter *thread* dasselbe `fileHandle` benutzt, um einen asynchronen I/O einzuleiten und abzuwarten – das Signal an das `fileHandle` weckt beide *threads* auf, auch wenn nur einer der beiden Aufträge ausgeführt wurde. Liest oder überschreibt nun ein *thread* seinen Puffer in der falschen Annahme, dass er schon transferiert sei, so resultieren falsche Daten bei der Ein- und Ausgabe. Ein Ausweg aus dieser Situation ist die Benutzung von eigenen Ereignisobjekten oder APCs (*Asynchronous Procedure Call*) für jeden *thread*.

Eine andere Alternative, das Problem zu umgehen, besteht in der Anwendung der Prozeduren `ReadFileEx ()` und `WriteFileEx ()`, die als Parameter noch die Angabe einer Abschlussprozedur (*completion routine*) gestatten. Unabhängig davon erhält der aufrufende *thread* die Kontrolle sofort zurück und kann weiterarbeiten. Danach muss der *thread* sich in einem Wartezustand begeben, beispielsweise durch den Aufruf von `SleepEx ()`, `WaitForSingleObjectEx ()` oder `WaitForMultipleObjectsEx ()`.

Ist das Lesen bzw. Schreiben asynchron dazu beendet, so wird der *thread* aufgeweckt und die angegebene Abschlussprozedur aufgerufen, die weitere Schritte veranlassen kann.

## 5.5.4 Aufgaben zur Treiberoptimierung

### Aufgabe 5.5-1 I/O-Dienste

Welche Dienste bietet der BS-Kern einem I/O-Subsystem bzw. Geräten? Nennen Sie mindestens zwei.

### Aufgabe 5.5-2 Pufferung

Betrachten Sie folgendes Schichtenmodell:

|                      |
|----------------------|
| Dateisystem          |
| Multi-device-Treiber |
| Gerätetreiber        |
| Controller           |
| Gerät                |

Welche Vor- und Nachteile ergeben sich, falls Pufferung in jeweils eine der Schichten eingeführt wird? Unterscheiden Sie dazu die vier Fälle beim Übergang von einer Schicht zur anderen. Achten Sie insbesondere auf Dateninkonsistenzen bei parallelen Aktionen.

## Literatur

- Egan J., Teixeira T.: UNIX Device-Treiber. Addison-Wesley, Bonn 1990
- Lynch W. C.: Do Disk Arms Move? Performance Evaluation Review, ACM Sigmetrics Newsletter 1, 3–16 (1972)
- MacWilliams F., Sloane N.: The Theory of Error-Correcting Codes. North-Holland, Amsterdam 1986
- Nagar R.: Windows NT File System Internals – Building NT File System Drivers. O'Reilly & Assoc., Sebastopol, CA 1997
- Oney W.: Programming the Microsoft Windows Driver Model, Microsoft Press, Redmond, WA 1999
- Ruemmler C., Wilkes J.: An introduction to disk drive modeling, IEEE Computer 27 (3), pp.17–29, March 1994.
- Viscarola P.G., Mason W.A.: Windows NT Device Driver Development, Macmillan Technical Publishing, Indianapolis, IN 1999

Inhaltsverzeichnis

6.1 Das Schichtenmodell für Netzwerkdienste ..... 269

6.2 Kommunikation im Netz ..... 274

    6.2.1 Namensgebung im Netz..... 275

    6.2.2 Kommunikationsanschlüsse..... 280

    6.2.3 Aufgaben ..... 288

6.3 Dateisysteme im Netz ..... 289

    6.3.1 Zugriffssemantik ..... 289

    6.3.2 Zustandsbehaftete und zustandslose Server..... 291

    6.3.3 Die Cacheproblematik ..... 293

    6.3.4 Implementationskonzepte ..... 295

    6.3.5 Die WebDAV-Dienste..... 299

    6.3.6 Sicherheitskonzepte ..... 300

    6.3.7 Aufgaben ..... 302

6.4 Massenspeicher im Netz ..... 302

6.5 Arbeitsmodelle im Netz..... 305

    6.5.1 Jobmanagement ..... 305

    6.5.2 Netzcomputer..... 306

    6.5.3 Schattenserver ..... 309

    6.5.4 Aufgaben zu Arbeitsmodellen im Netz ..... 313

6.6 Middleware und SOA ..... 314

    6.6.1 Transparenz durch Middleware ..... 315

    6.6.2 Vermittelnde Dienste ..... 316

    6.6.3 Service-orientierte Architektur..... 318

    6.6.4 Aufgaben zu Middleware..... 320

Literatur..... 321



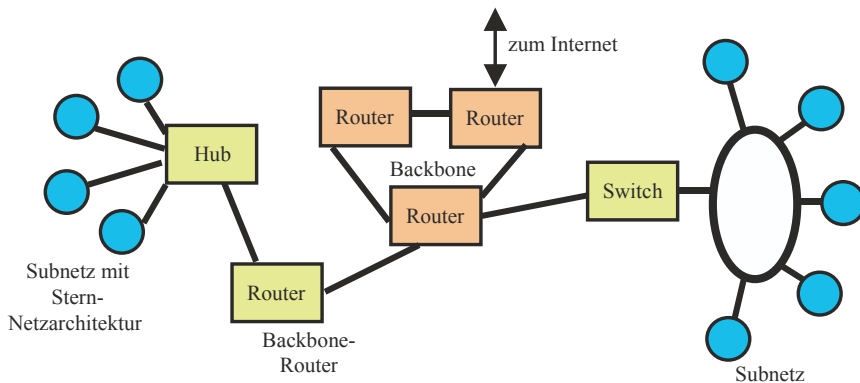
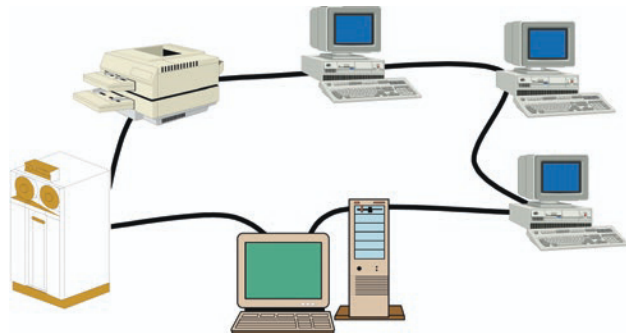
Im Unterschied zu Großrechnern (*main frame*), die zentral bis zu 1000 Benutzer bedienen können, sind die meisten Rechner in Firmen, Universitäten und Behörden nur wenigen Benutzern vorbehalten (*single user*-Systeme). Um trotzdem systemweite Dienste und Daten innerhalb der Firmen oder Institutionen nutzen zu können, sind die Rechner in der Regel vernetzt. Aus diesem Grund sollen in diesem Abschnitt die netztypischen Teile von Betriebssystemen näher betrachtet werden.

Beginnen wir mit einer kurzen Zusammenfassung der Grundbegriffe in Rechnernetzen, um ein gemeinsames Verständnis der Betriebssystemteile zu erreichen. In Abb. 6.1 ist ein solches lokales Netzwerk (Local Area Network LAN) abgebildet wie es sich oft in kleineren Arztpraxen, Unternehmen oder Filialen findet.

Dabei sind die einzelnen Rechner über ein Kabel miteinander verbunden. Üblicherweise werden dazu zwei miteinander verdrehte, isolierte Drähte verwendet (bis 1 GHz) oder für größere Datenmengen Licht, das in Glasfasern geleitet wird. Ist das Signal in einer langen Leitung zu schwach geworden, so muss es durch einen **Repeater** verstärkt werden.

Das Ziel, alle Rechner eines Netzes miteinander zu verbinden, kann man sowohl durch einen Kabelstrang erreichen, der an den Enden miteinander verbunden sein kann (Ringarchitektur), als auch dadurch, dass jeder Rechner mit einem Kabel zu einem zentralen Punkt verbunden ist (Sternarchitektur). In Abb. 6.2 ist eine solche Architektur gezeigt, in

**Abb. 6.1** Ein typisches lokales Netzwerk



**Abb. 6.2** Subnetze und Backbone eines Intranets

der die Rechner als Kreise und die Vermittlungseinheiten als Vierecke gezeichnet sind. Werden die Kabelstücke durch den Apparat des zentralen Punkts zu einem einzigen, elektrisch zusammenhängenden Netzwerk geschaltet, so wird dieser als **Hub** bezeichnet. Ein solches Netz hat zwar physisch eine Sternstruktur, logisch entspricht dies aber einer einzigen, langen Leitung. Solch eine Struktur wird gern verwendet, da sie die zentrale Kontrolle jeden Anschlusses erlaubt: fehlerhaften Rechnern im Netz kann jederzeit „der Stecker gezogen“ werden, ohne die anderen Rechner im Netz zu blockieren.

Der Übergang der Signalinhalte von einem Netz in ein angeschlossenes anderes wird durch ein spezielles Gerät, eine Brücke (**Bridge**) oder ein **Gateway**, ermöglicht. Grundsätzlich betrachtet eine solche Bridge alle Signale und leitet diejenigen ins Nachbarnetz um, die als solche kenntlich gemacht wurden. Schnelle Brücken, die auch noch weitere Funktionen zur Vermittlung beherrschen, werden als **Switch** bezeichnet. Wird die Brücke gezielt angesprochen und beauftragt, die Signale aus dem lokalen Netz, wo sie erzeugt wurden, zum Zielrechner in ein anderes Netz geeignet weiterzuleiten, so spricht man von einem **Router**. Sowohl ein Hub als auch ein Switch kann zu einem Router ausgebaut werden.

Man kann mehrere Netze über Router oder Gateways zusammenkoppeln. In Firmennetzen („Intranet“) bildet die Kopplung über ein spezielles Netz, das keine Drucker oder andere gemeinsam genutzte Geräte (Ressourcen) enthält und nur zur zuverlässigen Verbindung der Subnetze zuständig ist, eine wichtige Geschäftsgrundlage. Es wird deshalb als **Backbone**-Netz bezeichnet im Unterschied zu Netzen, die nichts weiterleiten können, den **Stub**-Netzen. Router, die zu diesem zentralen Daten-Umschlagplatz vermitteln, sind die **Backbone-Router**.

Für die logische Sicht der Nachrichtenverbindungen im Netz können wir zwei Typen unterscheiden: Entweder bauen wir erst eine Verbindung auf und senden dann die Nachrichten (**verbindungsorientierte** Kommunikation), oder aber wir fügen zu den Nachrichten die Empfängeradresse hinzu und erreichen so eine **verbindungslose** Kommunikation.

Für die Realisierung der logisch-verbindungsorientierten Kommunikation kann man jedoch auch wiederum beide Konzepte verwenden: wir können physikalisch eine feste Verbindung errichten und dann senden (z. B. mit einer Telefon-Standleitung) oder nur den physikalischen Weg durch ein Netzwerk über mehrere Rechner durch spezielle Nachrichten initialisieren und dann die Nachrichten auf diesen vorbereiteten Weg schicken. Die exklusive Rechner-Rechner Verbindung ist dann nur logisch vorhanden; tatsächlich können mehrere solche Verbindungen dieselben Leitungen benutzen, ohne sich zu stören.

Für die Realisierung beider logischer Konzepte in einem Netzwerk reicht es, die Nachrichten in Abschnitte (**Datenpakete**) zu zerteilen und mit einer Empfängeradresse versehen als zeitliche Abschnitte der Signale (Signalpakete) in das Netz einzuspeisen. Die Datenpakete enthalten als Zielbezeichnung eine Zahl, die Adresse des Empfängers, sowie die Adresse des Senders. Dazu wird beispielsweise beim Ethernet jedem Netzwerkanschluss (Platine) eines Rechners innerhalb eines LAN eine 32-Bit-Zahl zugewiesen, die MAC (Media Access Control). Dies ist eine Hardwareadresse, die nur einmal auf der Welt existiert und der Platine bei der Herstellung „eingeschnitten“ wird. Damit können nicht

zufällig zwei Platinen mit gleicher MAC in einem Ethernet existieren und es ist immer eindeutig, für wen ein Datenpaket bestimmt ist.

Für kleine Netze reicht dies aus; bei größeren benötigen wir aber eine andere, symbolische Adresse, aus der leicht geschlossen werden kann, in welchem Netz der Zielrechner ist. Da diese Adresse immer gleich sein sollte, egal, ob wir einen Ersatzcontroller in den Rechner stecken oder ob wir gar einen neuen Rechner an die Stelle setzen, muss diese Adresse unabhängig von der Hardware sein und zusammen mit Vorschriften (Protokollen) definiert sein, wie die Adressen zu behandeln sind. Dazu betrachten wir einen Verbindungsaufbau zwischen Rechnern eines Netzes etwas systematischer.

### Client-Server Systeme

Üblich ist dabei die sogenannte Client-Server-Funktionsaufteilung: Spezielle Computer enthalten zentrale Unternehmensdaten (*file server*), dienen als schnelle Rechner (*computing server*), nehmen kritische Datentransaktionen (*transaction server*) vor und steuern besondere Druckgeräte (*print server*). Dies ermöglicht zusätzliche Funktionalität für die Mitglieder einer Arbeitsgruppe:

- *electronic mail*  
Elektronische Nachrichten dienen als Notizzettel und Kommunikationsmittel, zur Terminabsprache und Projektkoordination.
- *file sharing*  
Dokumente und Daten können gemeinsam erstellt und genutzt werden. Dies vermeidet Inkonsistenzen zwischen verschiedenen Kopien und Versionen und hilft dabei auch, Speicherplatz zu sparen.
- *device sharing*  
Der Ausdruck von Grafik und Daten auf beliebigen Druckern im Netz (*remote printing*) ermöglichen die Anschaffung auch teurer Drucker für die ganze Arbeitsgruppe oder alternativ das Einsparen von Investitionen. Dies gilt allgemein für alle Spezialhardware wie *high-speed scanner*, Farblaserdrucker, Plotter usw.
- *processor sharing*  
Durch die Verteilung von Einzelaufgaben eines Rechenjobs auf die Rechner der anderen Gruppenmitglieder kann mit der ungenutzten Rechenzeit der anderen der eigene Job schneller bearbeitet und abgeschlossen werden oder aber Investitionen in mehr Rechengeschwindigkeit gespart werden.

Ist der Unterschied zwischen dem einzelnen Computer und dem Netzwerk für den Benutzer nicht mehr zu sehen, so sprechen wir von einem **verteilten Computersystem**. Die oben genannten Funktionen werden dabei durch ein gezieltes Zusammenspiel mehrerer Betriebssystemteile auf verschiedenen Rechnern erreicht. Die Netzwerkerweiterung kann deshalb auch als eine Erweiterung des Betriebssystems angesehen werden.

Die oben geschilderten Vorteile sind in Netzen zwar möglich; sie werden aber durch die Vielfalt der eingesetzten Rechnermodelle, Betriebssysteme und Programmiersprachen

stark behindert: Die Zusammenarbeit der Rechner leidet unter den unterschiedlichen Netzwerknormen, die in der Hardware und Software existieren. Deshalb wollen wir in diesem Kapitel die wichtige Rolle des Betriebssystems genauer unter die Lupe nehmen und die Aufgaben, Funktionsmodelle und Lösungen näher untersuchen, die ein Netzwerkanschluss für ein Betriebssystem mit sich bringt.

Einen wichtigen Versuch, die inhomogene Landschaft zu vereinheitlichen, stellt dabei das *Distributed Computing Environment* (DCE) der Herstellervereinigung *Open Software Foundation* (OSF) (später *OpenGroup*) dar, das als komplexes Softwarepaket (1 Mill. Codezeilen) verschiedene Lösungen für Client-Server-Arbeitsverwaltung (z. B. *threads*), Dateiverwaltung und Sicherheitsmechanismen enthält.

### Verteilte Betriebssysteme

Ein Betriebssystem, das mit anderen Betriebssystemen über Netzverbindungen gekoppelt wird und jeweils vollständig vorhanden ist, wird als **Netzwerkbetriebssystem** bezeichnet. Man kann nun eine Aufgabe, etwa das Führen von Dateien, innerhalb eines Netzes auch unter den Rechnern aufteilen. Bezüglich dieser Aufgabe spricht man dann von einem **verteilten System**, in unserem Beispiel von einem verteilten Dateisystem. Die Funktionen des verteilten Systems beschränken sich im Fall eines Netzwerkbetriebssystems auf höhere Dienste und benutzerspezifizierte Programmsysteme.

Im Gegensatz dazu befindet sich jede Komponente eines **verteilten Betriebssystems** nur einmal exklusiv auf einem Rechner des Netzwerks. Dabei müssen alle Komponenten bzw. Rechner zusammenarbeiten; es gibt also nur ein Betriebssystem für das gesamte Netzwerk.

Ein solches Betriebssystem verwendet nur die untersten Schichten des Transportprotokolls, um seine auf verschiedene Rechner verteilten Dienste schnell ansprechen zu können. In Abb. 6.3 ist die Schichtung für den Kern eines solchen Systems gezeigt. Der Rest des Betriebssystemkerns, der noch auf jedem Rechner existiert, ist meist als Mikrokern (vgl. Kap. 1) ausgeführt. Ähnlich dem MACH-Kern enthält der Mikrokern nur die allernötigsten Dienste, um Kommunikation und schnelle Grunddienste für Speicherverwaltung und Prozesswechsel durchzuführen.

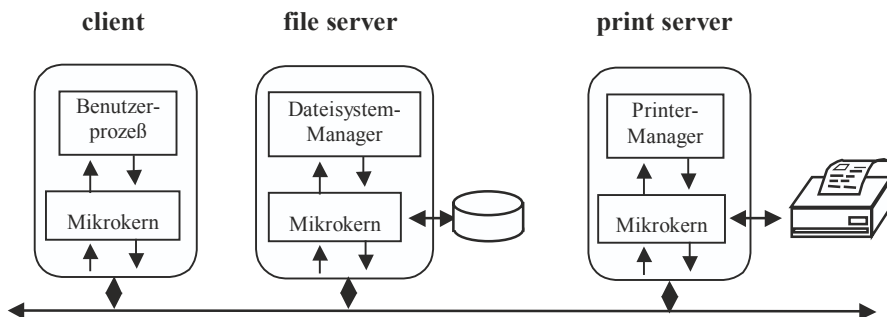


Abb. 6.3 Ein verteiltes Betriebssystem

Alle anderen Dienste wie Dateisystemverwaltung (*file server*), Drucker (*print server*), Namensauflösung (*directory server*), Job Management (*process server*) usw. sind auf anderen, spezialisierten Rechnern lokalisiert. Diese Art von Betriebssystem hat folgende Vorteile:

- *Flexibilität*  
Weitere Dienste (*computing server* etc.) können nach Bedarf im laufenden Betrieb hinzugefügt oder weggenommen werden; das Computersystem kann inkrementell erweitert werden.
- *Transparenz*  
Die Dienste im Netzwerk können erbracht werden, ohne dass der Benutzer wissen muss, wo dies geschieht.
- *Fehlertoleranz*  
Prinzipiell ist so auch Fehlertoleranz möglich; das System kann intern rekonfiguriert werden, ohne dass der Benutzer dies merkt.
- *Leistungssteigerung*  
Da alle Dienste parallel erbracht werden, kann durch zusätzliche Rechner auch höherer Durchsatz erzielt werden.

Fairerweise muss man allerdings hinzufügen, dass verteilte Betriebssysteme durch den größeren Hardwareaufwand auch Nachteile haben, beispielsweise wenn ein Service nur einmal vorhanden ist und gerade dieser Rechner ausfällt: Das ganze Netz ist dann bezüglich dieser Dienstleistung funktionsunfähig.

Ein grundsätzliches Problem verteilter Betriebssystemkerne ist auch der Zeitverlust für einen Service durch die Kommunikation. Für ein leistungsfähiges System lohnt es sich also nur, größere Arbeitspakete „außer Haus“ bearbeiten zu lassen; die Mehrzahl der kleinen Aktionen, die in einem Betriebssystemkern existieren, lassen sich schneller auf demselben Prozessorsystem ausführen.

Ein weiterer Punkt sind die Netzwerkdienste. Auch bei identischen, replizierten Betriebssystemkernen, die autonom alle Betriebssystemfunktionen wahrnehmen können, gibt es spezielle höhere Dienste (wie netzweite Zugriffskontrolle usw.), deren Funktionalität verteilt ist. Da diese Grunddienste bereits teilweise zum Betriebssystem gerechnet werden und im Lieferumfang des Betriebssystems enthalten sind, handelt es sich mehr oder weniger auch um verteilte Betriebssysteme, ohne aber einen Mikrokern zu besitzen.

Aus diesen Gründen ist es müßig, die Streitfrage zu entscheiden, ob ein zentrales (*main frame*) oder ein verteiltes (*Client-Server*) Betriebssystem besser sei: Die meisten Betriebssysteme sind eine Mischung aus einem reinen Netzwerkbetriebssystem, das alles selbst macht und nur lose an andere Systeme anderer Rechner angekoppelt ist, und einem reinen verteilten Betriebssystem, das alle Dienste auf spezialisierte Rechner verlagert. Der Umfang des Kerns zeigt dabei den Übergangszustand zwischen beiden Extremen an.

In [Abschn. 6.5.2](#) ist dieses Thema nochmals am Beispiel des Netzcomputers diskutiert.

---

## 6.1 Das Schichtenmodell für Netzwerkdienste

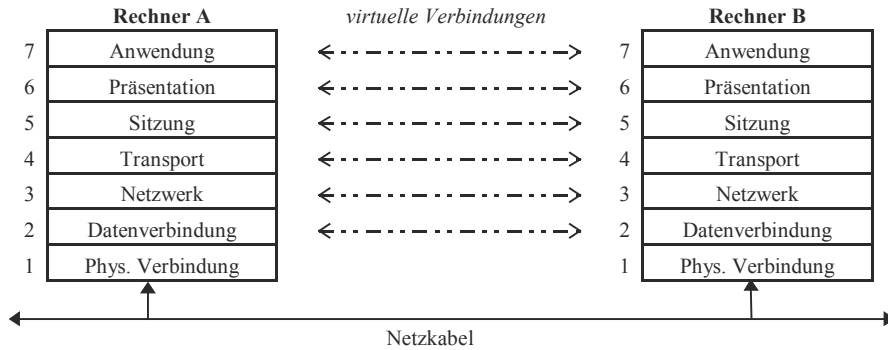
Bei der Vernetzung von Rechnern kommt in erster Linie zum isoliert arbeitenden Computer ein Controller für den Netzwerkanschluss als zusätzliche Platine hinzu. Meist ist die Datenverbindung seriell, so dass nun ein zusätzliches serielles Gerät mit einem Treiber im Betriebssystemkern eingebunden werden muss.

Für die logische Sicht der Nachrichtenverbindungen können wir wieder unser Schema aus [Abschn. 2.4.1](#) verwenden: Entweder bauen wir erst eine Verbindung auf und senden dann die Nachrichten (verbindungsorientierte Kommunikation), oder aber wir fügen zu den Nachrichten die Empfängeradresse hinzu und erreichen so eine verbindungslose Kommunikation. Für die Realisierung der logisch-verbindungsorientierten Kommunikation kann man jedoch auch wiederum beide Konzepte verwenden: Wir können physikalisch eine feste Verbindung errichten und dann senden (z. B. mit einer Telefon-Standleitung) oder nur den physikalischen Weg durch ein Netzwerk über mehrere Rechner durch spezielle Nachrichten initialisieren und dann die Nachrichten auf diesen vorbereiteten Weg schicken. Die exklusive Rechner-Rechner Verbindung ist dann nur logisch vorhanden; tatsächlich können mehrere solche Verbindungen dieselben Leitungen benutzen, ohne sich zu stören. Für die Realisierung beider logischer Konzepte in einem Netzwerk reicht es also, die Nachrichten in Abschnitte (**Datenpakete**) zu zerteilen und mit einer Empfängeradresse versehen in das Netz einzuspeisen.

Normalerweise handelt es sich bei der Netzverbindung der Rechner nicht um eine physikalische Punkt-zu-Punkt-Verbindung, sondern mehrere Computer sind mit einem Kabel verbunden. Um einen bestimmten Rechner anzusprechen, müssen auf dem Kabel nicht nur die Daten, sondern auch die Adressinformation übertragen werden. Diese weitere Verwaltungsinformation (Nachrichtenlänge, Quersumme zur Prüfung auf Übertragungsfehler usw.) wird mit den Daten in einem Nachrichtenpaket zusammengefasst. Aufgabe der Basiselektronik des Controllers ist es, die Übersetzung zwischen dem elektrischen Signal im Kabel und dem logischen Format der Nachrichtenpakete sowohl für das Lesen (Empfangen) als auch für das Schreiben (Senden) durchzuführen. Mit Hilfe dieser Dienstleistung müssen weitere Funktionen wie Sendekontrolle, Aufbau einer logischen Nachrichtenverbindung zu einem anderen Rechner mit Hilfe einer Sequenz von Nachrichtenpaketen usw. aufgebaut werden. Die Folge mehrerer fest vorgegebener Schritte eines Nachrichtenaustauschs für einen Zweck (die Menge der Kommunikationsregeln) wird als **Protokoll** bezeichnet.

Der Aufbau immer höherer Dienste, die sich auf einfache, niedere Funktionen (*Dienste*) stützen, entspricht unserer Strukturierung durch virtuelle Maschinen aus [Kap. 1](#) und ist durch das ISO-OSI-Schichtenmodell standardisiert, s. [Abb. 6.4](#).

Die verschiedenen Schichten, von 7 bis 1 absteigend durchnummeriert, haben dabei folgende Aufgaben:



**Abb. 6.4** Das Open System Interconnect (OSI)-Modell der International Organization for Standardization (ISO, früher International Standards Organization)

- 7) Auf der **Anwendungsschicht** werden benutzerdefinierte Dienste wie Dateitransfer (*FTP*), spezielle Grafikanwendungen, Sicherheitsüberprüfung, Nachrichtenaustausch (*electronic mail*) usw. angeboten.
- 6) Die **Präsentationsebene** formatiert die Daten und legt ihre Kodierung oder ihr Kompressionsformat fest und gruppiert die Daten, beispielsweise in ein anwenderabhängiges RECORD-Format.
- 5) Auf der **Sitzungsebene** wird festgelegt, wer Sender und Empfänger ist (z. B. *remote login*), wie Fehler beim Datenaustausch behandelt werden usw. Sie wird meist vernachlässigt.
- 4) Die **Transportschicht** wandelt nun den Strom von Daten um in Datenpakete, achtet auf die Nummerierung und, beim Empfang, auf die richtige Folge der Pakete. Hier wird auch zum ersten Mal auf die darunterliegende Hardware Rücksicht genommen. Verschiedene Ansprüche an die Fehlertoleranz (TP0 bis TP4) lassen sich hier wählen. Typische Protokollvertreter sind das *Transmission Control Protocol* **TCP** oder das *User Datagram Protocol* **UDP**.
- 3) Auf der **Netzwerkschicht** werden alle Fragen und Probleme behandelt, die mit der Netzwerktopologie und der Vermittlung zusammenhängen. Festlegen des Übertragungsweges, Nachrichtenumleitung bei Nachrichtenverkehrsstatus auf bestimmten Strecken, Maschinenkonfigurationen, Grenzen durch Bandweite und Kabellänge, kurzum, alle zu beachtenden technischen Nebenbedingungen des Netzwerks werden hier behandelt. Für eine verbindungslose Kommunikation wird meist das *Internet-Protokoll* **IP** und das **X.25** Protokoll (Datex-P) verwendet.

Wird auf dieser Schicht eine Flusskontrolle für die Pakete verwendet (z. B. nach IEEE 802.3x), so kann das Überlaufen von Puffern beim Empfänger beispielsweise für UDP verhindert werden.

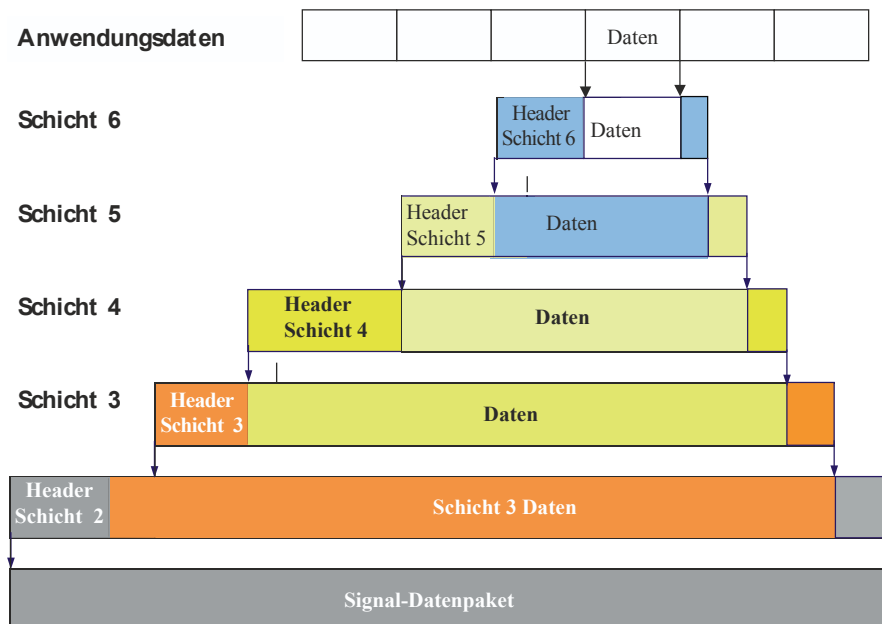
- 2) Die **Ebene der Datenverbindung** unterteilt die großen, unregelmäßigen Datenmengen der Nachrichten in einzelne kleine Pakete (*frames*) von fester Größe bzw. festem Format, sendet sie und wiederholt die Sendung, wenn keine Rückmeldung erfolgt oder

die Prüfsumme (CRC) beim Empfänger nicht stimmt. Sie erfüllt damit die Funktion einer Datensicherung, wobei allerdings die Datenpakete auf dem Weg zum Empfänger sich überholen können. Beispiele: LLC (*Logical Link Control*), LAPD (*Link Access Procedure for D-channels*).

- 1) Die Datenpakete werden zum Senden auf der **physikalischen Ebene** als Bits in elektrische oder optische Impulse umgesetzt und auf das Übertragungsmedium gebracht. Beispiel dafür sind die serielle ITU-T V.24-Spezifikation, der Betrieb von 10 Mbit pro Sekunde auf verdrehten Drähten (*twisted pair*) 10BaseT oder auch die Spezifikation der Übertragung auf Fiberglasleitungen FDDI.

Üblicherweise werden Schicht 1 und 2 durch Hardware behandelt und als *Network Access Layer* bezeichnet. Populäres Beispiel für einen solchen kontinuierlichen Übergang zwischen der physikalischen Anschlussspezifikation und dem unteren Paketprotokoll ist das **Ethernet**.

Die Pakete der Anwendungsdaten, die die einzelnen Schichten mit den untergeordneten Schichten austauschen, bestehen dabei aus einem Kopfteil (Header), der die Kontrollinformation des Datenpakets für die Schicht enthält, den eigentlichen Daten, die von der Schicht mit der jeweils höheren Schicht ausgetauscht werden, und einem Ende (*Tail*). In [Abb. 6.5](#) ist eine solche sukzessive Kapselung der Information durch die verschiedenen Schichten schematisch gezeigt.



**Abb. 6.5** Die Kapselung der Daten und Kontrollinformationen



Jeder einzelne Header hat eine genau festgelegte Länge und Format, bei der die Bedeutung jedes Bits entsprechend dem Protokoll, das die Schicht benutzt, genormt ist.

Die Realisierung und Nutzung dieses Modells ist sehr unterschiedlich und sehr umstritten. Im einfachsten Fall kann man eine Punkt-zu-Punkt-Verbindung zwischen zwei Computern herstellen, indem man die (fast immer vorhandenen) seriellen Anschlüsse, die für Drucker und andere langsame Geräte vorgesehen sind, mit einem Kabel verbindet. In diesem Fall erfüllt der Chip für die serielle Schnittstelle die Funktionen der Schicht der Datenverbindung; der elektrische Ausgangstreiber ist dann Schicht 1. Für einen Datenaustausch kann man dann ein Terminal-Emulationsprogramm verwenden, beispielsweise das populäre Kermit-Programm. Dieses beinhaltet sowohl Befehle zum Senden von Daten (Schicht 7) als auch Mechanismen zur Kodierung in Datenpakete (Schicht 6). Die Schichten 5, 4 und 3 entfallen hierbei, da es nur eine dedizierte Punkt-zu-Punkt-Verbindung ist.

Das Konzept, Verbindungen zwischen Rechnern auf der Anwendungsebene direkt herzustellen (*end-to-end-Strategie*), wird mittels *gateways* implementiert.

**Beispiel UNIX Kommunikationsschichten**

Die einzelnen Schichten sind in UNIX sehr unterschiedlich vertreten. Historisch gewachsen war UNIX nicht für Netzwerke konzipiert. Da aber der Quellcode des Betriebssystems schon von Anfang an für Universitäten frei verfügbar war, wurden wichtige Entwicklungen und Konzeptionen für Netzwerke an UNIX ausprobiert.

Ein interessantes Konzept wurde mit UNIX System V eingeführt: die Anordnung der Gerätetreiber in Schichten. Die Grundidee besteht darin, den gesamten Ein- und Ausgabedatenstrom einer Verbindung (*stream*) durch verschiedene Bearbeitungsstufen (Treiber) fließen zu lassen, s. [Abb. 5.2](#). Dieses *streams*-System ermöglicht über eine normierte Schnittstelle, beliebig Bearbeitungsstufen einzuschieben oder aus dem Strom zu nehmen. Damit ist es leicht möglich, Protokollschichten (als Treiber realisiert) auszutauschen, also auch eine andere als die populäre Kombination TCP und IP als Protokoll zu wählen. In [Abb. 6.6](#) ist die Schichtung gezeigt, die auch einige in [Abschn. 6.2.2](#) erwähnten Dienste enthält.

**Abb. 6.6** Oft benutzte Protokollschichten in UNIX

|   |                  |                                  |
|---|------------------|----------------------------------|
| 7 | Anwendung        | named pipes, rlogin, ...         |
| 6 | Präsentation     | XDS                              |
|   |                  | <i>BS-Schnittstelle: sockets</i> |
| 5 | Sitzung          | <i>ports, IP Adresse</i>         |
| 4 | Transport        | TCP                              |
| 3 | Netzwerk         | IP                               |
| 2 | Datenverbindung  |                                  |
| 1 | Phys. Verbindung | Network Access Layer             |

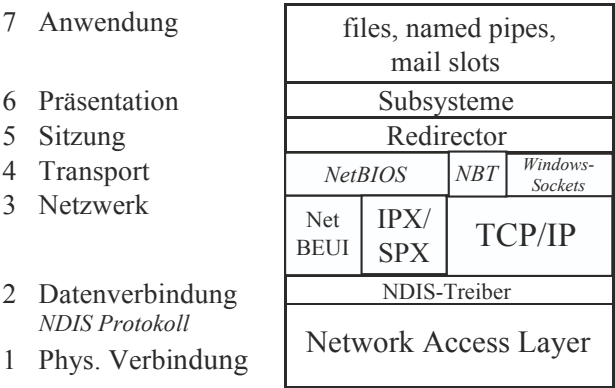
Der Aufruf der Transportdienste im Kern geschieht so über spezielle Systemaufrufe. Eine Applikation kann also entweder auf hoher Ebene spezielle Konstrukte wie *named pipes* usw. (s. [Abschn. 6.2.2](#)) nutzen, oder aber sie kann tief auf den `Socket()`-Betriebssystemdiensten aufsetzen, um eine Kommunikation über das Netzwerk zu erreichen.

**Beispiel Windows NT Kommunikationsschichten**

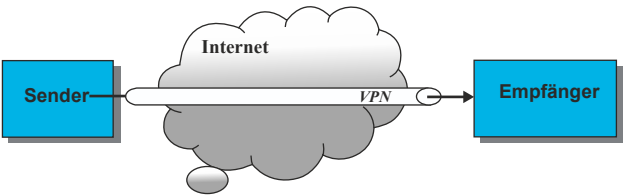
In Windows NT sind verschiedene Netzwerkdienste untergebracht, die eine Kompatibilität sowohl zu den proprietären MS-DOS-Netzwerkprotokollen (MS-Net, *Server Message Block*-Protokoll **SMB**, *Network Basic Input-Output System* **NetBIOS**-Interface, *NetBIOS over TCP/IP* **NBT**) als auch zu den Protokolldiensten anderer Hersteller erlauben sollen. Die in [Abb. 6.7](#) skizzierte Architektur zeigt, dass die im ISO-OSI-Modell vorgesehene Schichtung nur sehr unvollständig befolgt wurde. Die Transportschicht wird durch die alternativen Protokolle NetBEUI (*NETBIOS Extended User Interface*) und IPX/SPX der Fa. Novell und durch das populäre TCP/IP abgedeckt. Sie setzen auf der Standardschnittstelle für Netzwerkcontroller, der *Microsoft Network Driver Interface Specification* **NDIS**, auf.

Die höheren Schichten 5, 6 und 7 werden auch alternativ von der SMB-Schicht abgedeckt, die direkt die Aktionen einer Applikation auf ein anderes Rechnersystem abbildet. Eine Applikation kann also entweder das SMB-Protokoll, den Redirector-Dienst oder aber den direkten Zugang zu den Sockets, NBT und NetBIOS nutzen, um eine Kommunikation zu erreichen.

**Abb. 6.7** OSI-Modell und Windows NT-Netzwerkkomponenten



**Abb. 6.8** Virtual Private Network



Die Konzeption eines solchen Schichtensystems gestattet es, zusätzliche Schichten darauf zu setzen, ohne die Funktionalität darunter zu beeinflussen. So kann man beispielsweise alle Kommunikation vor dem Senden verschlüsseln und beim Empfänger wieder entschlüsseln. Der dabei benutzte Kommunikationskanal kann ruhig abgehört werden; die Lauscher können mit den Daten nichts anfangen. Das Konzept ist in [Abb. 6.8](#) visualisiert. Auf diese Weise kann man sich trotz unsicherer Internetverbindungen ein eigenes, privates Netzwerk (Virtual Private Network VPN) schaffen.

### **Bemerkung**

Bei Netzwerken mit höheren Ansprüchen wie *E-Mail* und gemeinsame Nutzung von Dateien sind die Schichten etwas komplexer. Auch die Netzwerkschichten bieten sich deshalb zur Software-Hardwaremigration an.

In billigen Netzwerkcontrollern ist meist nur ein Standard-Chipset enthalten, das die zeitkritischen Aspekte wie Signalerzeugung und -erfassung durchführt. Dies entspricht den virtuellen Maschinen auf unterster Ebene. Alle höheren Funktionen und Protokolle zum Zusammensetzen der Datenpakete, Finden der Netzwerkrouuten und dergleichen muss vom Hauptprozessor und entsprechender Software durchgeführt werden, was die Prozessorleistung für alle anderen Aufgaben (wie Benutzeroberfläche, Textverarbeitung etc.) drastisch mindert. Aus diesem Grund sind bei neueren Netzwerkcontrollern viele Funktionen der Netzwerkkontrolle und Datenmanagement auf die Hardwareplatine migriert worden; der Hauptprozessor muss nur wenige Anweisungen ausführen, um auf hohem Niveau (in den oberen Schichten) Funktionen anzustoßen und Resultate entgegenzunehmen.

Dabei spielt es keine Rolle, ob die von dem Netzwerkcontroller übernommenen Funktionen durch einen eigenen Prozessor mit Speicher und Programm erledigt werden oder durch einen dedizierten Satz von Chips: Durch die eindeutige Schnittstellendefinition wird die angeforderte Dienstleistung erbracht, egal wie. Es hat sich deshalb bewährt, dass der Controller nicht nur Chips enthält, die die Schichten 1 und 2 abhandeln, sondern auch ein eigenes Prozessorsystem mit Speicher und festem Programm in ROM, das den Hauptprozessor des Computersystems entlastet und Schichten 3 und 4 für ein spezielles Übertragungsmedium (z. B. Ethernet) und ein spezielles Protokoll (z. B. die Kombination *Transmission Control Protocol TCP* mit IP, das **TCP/IP**) implementiert. Diese „intelligenten“ Controller können dann mit Treibern des Betriebssystems auf hohem Niveau zusammenarbeiten.

Für das Betriebssystem ist also wichtig zu wissen, welche der Schichten und Systemdienste im Controller bereits vorhanden sind und welche extra bereitgestellt werden müssen.

---

## **6.2 Kommunikation im Netz**

Öffnet man die einfache Punkt-zu-Punkt-Kommunikationsverbindung und bezieht mehrere Rechner mit ein, so muss man für eine gezielte Verbindung zu einem einzelnen Rechner diesen mit einem Namen oder einer Adresse ansprechen können.

### 6.2.1 Namensgebung im Netz

Es gibt verschiedene Namenskonventionen in Netzen, die je nach Anwendung eingesetzt werden. Dabei können wir grob zwischen den weltweit genutzten Namen in großen Netzen (insbesondere im Internet) und den Namen in eng gekoppelten lokalen Netzen unterscheiden

#### Namen im weltweiten Netz

Im weltweiten Rechnernetz des **Internet** hat sich eine Namenskonvention durchgesetzt, die historisch gewachsen ist. Dazu sind alle Organisationen in den USA in verschiedene Gruppen eingeteilt worden: `com` für Firmen, `edu` für Universitäten und Schulen, `gov` und `mil` für Regierung und Militär, `net` für Netzerkanbieter und `org` für meist gemeinnützige Organisationen. Innerhalb einer solchen Gruppe (*Top Level-Domäne*) wird jedem Gruppenmitglied (**Domäne**) ein Name zugewiesen, z. B. für die Universität Berkeley aus der *Top Level-Domäne* `edu` der Name `berkeley`. Der volle Name für einen Rechner der Universität setzt sich aus dem Domänennamen und dem Rechnernamen zusammen, die mit einem Punkt als Trennzeichen verbunden werden.

#### Beispiel Rechnernamen

Der volle Name des Rechners `OKeeffe` der Universität Berkeley lautet

`okeeffe.berkeley.edu`

wobei Groß- und Kleinschreibung nicht beachtet werden. Für den Rest der Welt wurde eine andere Vereinbarung getroffen: Anstelle des Gruppennamens erscheint die Länderabkürzung. Ein Rechner `diokles` des Fachbereichs Informatik der Universität Frankfurt hat also den Namen

`diokles.informatik.uni-frankfurt.de`

Eine solche symbolische Adresse ist manchmal zu lang und muss nicht eindeutig sein: Für einen Rechner kann es mehrere Namen geben. Es existiert deshalb für jeden Rechner ebenfalls eine eindeutige logische Zahl, die aus der Aneinanderreihung von vier Zahlen je 8 Bit, also einer 32-Bit-Internetadresse (in der IP-Version 4) bestehen.

#### Beispiel Numerische Internetadresse bei IPv4

Der obige Rechner hat auch die numerische Adresse

`141.2.1.2`

Diese 32-Bit Adresse teilt sich dabei auf in eine Netzwerknummer `141.2.` und eine Rechnernummer `1.2`. Die innere Aufteilung der Rechnernummer auf Subnetze und Rechner innerhalb der Subnetze ist dabei variabel, und der Institution überlassen.

Dabei gibt es mit maximal 32 nutzbaren Bits nur 65.535 mögliche Netzadressen – viel zu wenig für die große Menge an neuen Institutionen außerhalb der USA, besonders in Europa und im asiatischen Raum, zu versorgen. Zusätzlich sind einige Adressräume schon vorgelegt: 127.0.0.0 für den lokalen Computer und 10.0.0.0, 172.16.0.0-172.31.0.0 und 192.168.0.0 für die Adressierung in lokalen Netzwerken.

Deshalb wird ein neuer IP-Standard Version 6 (IPv6) mit 128 Bit gefördert – genug Adressen, um auch allen kleinen Geräten wie Mobilrechnern, Handys oder Kühlschränken eine eigene IP-Adresse zuzuweisen. Dies erleichtert auch den Aufbau von besonders sicheren Verbindungen (Virtual Private Networks).

Die IP-Nummern (NetzId) werden zentral von einer Institution (ICANN *Internet Corporation for Assigned Names and Numbers* bzw. für Deutschland: DENIC) genauso wie die Domännennamen vergeben, wobei die RechnerId von der Domänenorganisation (hier: der Universität) selbst verwaltet wird. Dies vereinfacht die Verwaltung und wird gern dazu verwendet, vorhandene Subnetze und ihre Rechner auf die RechnerId abzubilden.

Abgesehen von der logischen Zahl existiert übrigens in jedem Netzwerkcontroller für das TCP/IP-Protokoll auch eine weitere, feste Zahl, die für den Controller einzig ist, fest eingebaut ist und nicht verändert werden kann (im Unterschied zu den logischen Nummern der IP-Adresse). Beim Aufbau einer Verbindung wird zuerst diese physikalische Nummer referiert, dann erst die logische Bezeichnung.

Die Zuordnung von der logischen numerischen Adresse zu der symbolischen Adresse aus Buchstaben muss mit Hilfe einer Tabelle vorgenommen werden. In UNIX-Systemen ist sie unter */etc/hosts* zu finden, in anderen Systemen ist es eine echte Datenbank.

Ein solcher Internetname kann nun dazu verwendet werden, einen Dienst auf einem Rechner anzufordern.

---

### Beispiel Dienste im Internet

Im World Wide Web (**WWW**), einem Hypertext-Präsentationsdienst, setzt sich als Adresse ein *Uniform Resource Locator* (**URL**) aus dem Namen für das Protokoll des Dienstes (z. B. `http://` für das Hypertextprotokoll HTTP), dem Internetnamen des Rechners und dem lokalen Dateinamen zusammen. Für den Dateikopierdienst (File Transfer Protocol; **FTP**) lautet dann die URL der Domäne `uni-frankfurt.de` und der Datei `/public/Text.dat`

```
ftp://ftp.informatik.uni-frankfurt.de/public/Text.dat
```

Man beachte, dass die Groß- und Kleinschreibung zwar unwichtig beim Internetnamen, nicht aber beim Dateinamen im lokalen Dateisystem ist! Der Rechner `uni-frankfurt.de` wird also auch in der Schreibweise `Uni-Frankfurt.DE` gefunden, nicht aber die Datei `/public/Text.dat` in der Schreibweise `/Public/text.dat`. Das Wurzelverzeichnis des so referierten Dateibaums ist willkürlich und wird für jedes Protokoll anders gesetzt. Wird

beispielsweise das Verzeichnis /Data/Web als Wurzel eingetragen, so hat die Datei /Data/Web/images/Background.gif die URL <http://informatik.uni-frankfurt.de/images/Background.gif>.

Die Zuordnung des Namens zur logischen IP-Adresse ist nicht automatisch, sondern muss in einer Namensliste nachgesehen werden. Im lokalen Netz wird dies von einem **name server** erledigt, der entweder die gewünschte Rechneradresse direkt in seiner Liste aller bekannten Rechner stehen hat oder aber bei einem weiteren Rechner nachfragen muss. Die Rechnernamen und ihre Dienste einer ganzen Domäne werden in einem speziellen *Domain Name Server* (**DNS**) geführt, wobei jeder DNS nur die direkt angeschlossenen Computer sowie weitere DNS-Rechner kennt, an die er Anfragen für ihm unbekannte Rechner weiterleitet.

### Namen im regionalen Netz

Beim Zusammenschluss mehrerer lokaler Netze zu einem regionalen Netz (*wide area network* **WAN**), bestehend aus mehreren Domänen, ist meist das Problem zu bewältigen, netzübergreifend und herstellerneutral die Möglichkeiten und Dienste (Drucker, Suchdienst, Rechenkapazität, etc.) anzubieten. Dabei ist nicht nur das Problem zu bewältigen, einen netzübergreifenden, einheitlichen Namen für die Ressource zu finden um sie anzusprechen, sondern überhaupt das Wissen über ihre Existenz (und ihr Verschwinden) zu erhalten.

Eine weitverbreitete Methode dafür ist die Einrichtung eines globalen Namendienstes, beispielsweise des bereits erwähnten DNS. Allerdings sind für eine effiziente Ressourcenverwaltung noch mehr Informationen erforderlich als nur der Namen, bei einem Drucker etwa, ob er PostScript kann, welcher Druckertyp es ist und wieviel Toner zu einem Zeitpunkt noch übrig ist. Für diese Aufgabe wurde der Standard X.500 im Jahre 1988 von der CCITT spezifiziert. Er enthält DAP (*Directory Access Protocol*), das zum Zugriff auf die Informationen dient, DSP (*Directory Service Protocol*), mit dem die Kommunikation zwischen Servern durchgeführt wird, und DISP (*Directory Information Shadowing Protocol*).

Allerdings setzt X.500 auf dem vollständigen ISO/OSI-Schichtenprotokoll auf, was alle Implementierungen sehr langsam und aufwendig machte. Deshalb wurde eine vereinfachte Version, das LDAP (*Lightweight DAP*), sehr begrüßt. Sie nimmt dadurch einen Teil der Last vom Klienten, indem sie direkt auf der TCP/IP-Schicht aufsetzt. Der LDAP-Dienst bietet neben einem einheitlichen Namen auch die Möglichkeit, Objekte und ihre Attribute aufzunehmen.

---

#### Beispiel Windows 2000 Active Directory Service ADS

Der ADS in Windows 2000 setzt auf den Mechanismen und Diensten von LDAP auf und bietet einen Dienst an, bei dem die Namensgebung der Unternetze einheitlich zusammengefasst wird in der Form <NamensID>://<Pfad>, etwa „Unix://Hera/Zentrale“. Der NamensID bezeichnet den hersteller- bzw. systemabhängigen obersten Knoten (*root*)

einer Domäne, der Pfad den in diesem Untersystem gebräuchlichen Pfad. Jedes Untersystem wird als „aktives Objekt“ behandelt, das seine Angaben (Attribute) von selbst aktuell hält. Ein solches „aktives Verzeichnis“ kann auch selbst wieder als Eintrag (Objekt) ein anderes „aktives Verzeichnis“ enthalten, so dass das gesamte System aus Domänen eine Baumstruktur bildet. Die Blätter des Baumes sind die gewünschten Ressourcen. Eine spezielle Schnittstelle ADSI gestattet es, die Ressourcen abzufragen und zu verändern. Wird auf dasselbe Objekt von zwei verschiedenen Seiten geschrieben, so bleibt nur die letzte Änderung erhalten. Sequenznummern des Objektzustandes (USN) gestatten es dem zugreifenden Dienst, Kollisionen zu erkennen.

### Namen im lokalen Netz

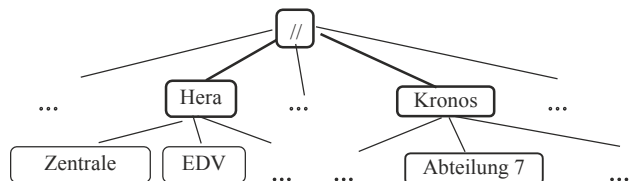
Mit dem Internetnamen haben wir eine Möglichkeit, einen Namen für einen Rechner zu finden, kennengelernt. In einem lokalen Netz (*local area network LAN*), beispielsweise bei mehreren Rechnern, die in einem großen Raum stehen oder über mehrere Räume einer Arbeitsgruppe verteilt sind, ist diese explizite Namenskonvention aber zu inflexibel. Um eine Datei von einem Rechner zu einem anderen zu transportieren, muss man beide Rechnernamen sowie die Pfadnamen der lokalen Dateisysteme angeben. Diese Flexibilität interessiert den Benutzer eigentlich gar nicht; seine Arbeitsgruppe bleibt ja immer gleich.

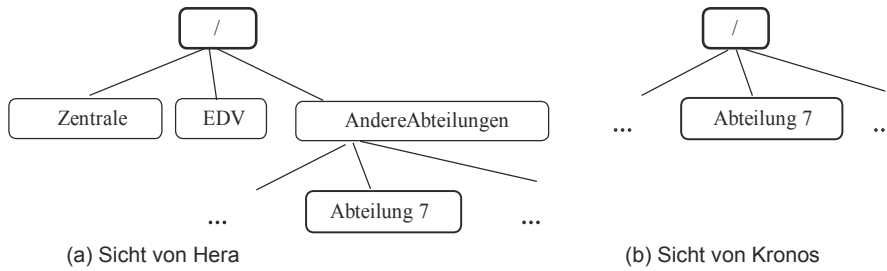
Es ist deshalb sinnvoll, vom Betriebssystem aus Mechanismen bereitzustellen, mit denen der Zugriff auf das Dateisystem eines anderen Rechners transparent gestaltet werden kann. Sind mehrere Rechner in einem lokalen Verbund zusammengeschaltet, so kann man beispielsweise die Dateisysteme von den Rechnern „Hera“ und „Kronos“ zu einem gemeinsamen, homogenen Dateibaum wie in [Abb. 6.9](#) zusammenschließen. Als gemeinsame, virtuelle Wurzel des Dateisystems erscheint das Symbol //; es dient als Schlüsselzeichen für Dateianfragen. Ein solcher Ansatz stellt das Dateisystem auf beiden Rechnern gleichartig dar; alle Benutzer der Rechner referieren den gleichen Namensraum.

Es gibt aber auch andere Ansätze. Angenommen, wir stellen nur eine einseitige Verbindung her und binden den Rechner Kronos in unserem Beispiel an das Verzeichnis „AndereAbteilungen“. Alle Anfragen auf Hera werden nun an Kronos weitergeleitet, dort bearbeitet und an Hera zurückgegeben. Für den Benutzer von Hera bietet sich nun ein Bild des Dateisystems, wie es in [Abb. 6.10a](#) gezeigt ist.

Für den Benutzer von Kronos aber hat sich nichts geändert; er bemerkt nicht in [Abb. 6.10b](#) den zusätzlichen Service, den Kronos für Hera bietet und hat nur Sicht auf das eigene, beschränkte Dateisystem: Die Dateisysteme sind inhomogen zusammengeschaltet.

**Abb. 6.9** Ein homogener Dateibaum im lokalen Netz





**Abb. 6.10** Lokale, inhomogene Sicht des Dateibaums

#### Beispiel UNIX Das DFS-System

Das **Andrew File System** (AFS) der Carnegie-Mellon-Universität unterstützt eine globale, homogene Dateisicht und wurde als *Distributed File System* (DFS) dem DCE-Paket beigelegt. Für jede Datei wird entschieden, ob sie *local* oder *remote* existiert, und alle Zugriffe danach organisiert. Dabei benutzt es Stellvertreterprozesse, um Daten und Protokoll zwischen den Rechnern auszutauschen, vgl. [Abb. 6.22](#).

#### Beispiel UNIX Das NFS-System

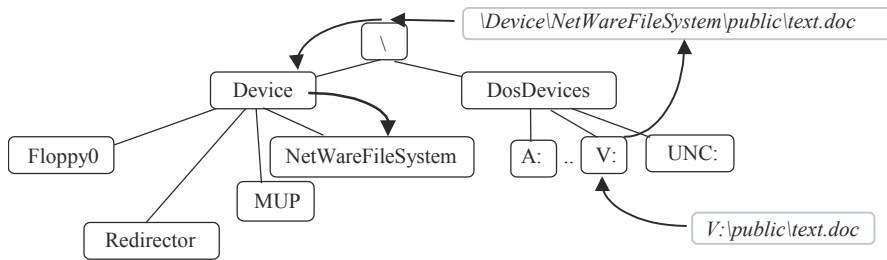
Die Namensgebung im **Network File System** (NFS), das von der Fa. SUN für Berkeley-UNIX entwickelt wurde und inzwischen auch bei anderen UNIX-Versionen verbreitet ist, orientiert sich nach dem zweiten, inhomogenen Modell. Es geht davon aus, dass ein (oder mehrere) Dateiserver im Netz existieren, die jeweils ein Dateisystem zur Verfügung stellen. Möchte ein Rechner ein solches System nutzen, so kann er mit einem speziellen `mount()`-Befehl (der z. B. im Startup-File `/etc/rc` enthalten sein kann) den Dateibaum des Servers unter einem bestimmten Verzeichnis zuweisen.

Da dies auf jedem Rechner anders geschehen kann, ist der Dateibaum auf jedem Rechner möglicherweise unterschiedlich. Beim Einsatz des NFS-Systems ist also sorgfältige Planung des Namensraums nötig.

#### Beispiels Windows NT Namensraum im Netz

In Windows NT gibt es zwei verschiedene Mechanismen, um auf Dateien anderer Rechner zuzugreifen. Der eine Mechanismus bedient sich der *symbolic links*, wie sie bereits in [Abschn. 4.2.2](#) vorgestellt wurden. In dem Verzeichnis der Geräte `\Device` wird für jede Netzdateiart (*MS-Redirector File System*, *Novell NetWare File System*, ...) ein besonderer Treiber eingehängt, dessen *parse*-Methode zum Aufbau einer Netzverbindung und zur Abfrage des Netzdateisystems führt. Setzt man nun am Anfang eine Gerätebezeichnung, z. B. „v:“, auf diesen Netzwerktreiber, so wird er automatisch beim Öffnen einer Datei angesprochen. In [Abb. 6.11](#) ist die Abfolge eines solchen umgeleiteten Dateinamens zu sehen.





**Abb. 6.11** Auflösung von Netzwerk-Dateianfragen

Der zweite Mechanismus benutzt das spezielle Microsoft-Namensformat „Universal Naming Convention UNC“, in dem alle Netzwerknamen mit dem Zeichen „\\“ beginnen. Bemerkt das Windows32-Subsystem, dass ein solcher Name benutzt wird, so wird „\\“ automatisch durch das DOS-Gerät „UNC:“ ersetzt. Von diesem geht ein *symbolic link* zu einem Treiber namens MUP (Multiple UNC Provider). Dieser fragt nun mit einem IRP bei jedem registrierten Netzwerk-Dateimanager an, ob der entsprechende Pfadname erkannt wird. Ist dies bei mehreren der Fall, so entscheidet die Reihenfolge der Registrierung der Dateimanager in der Registrationsdatenbank.

#### Beispiel Auflösung der Dateinamen mit UNC

Angenommen, wir wollen die Datei `\\textserv\\public\\text.doc` öffnen. Dann wird diese Anfrage zu `UNC:\\textserv\\public\\text.doc` und weiter zu `\\Device\\MUP\\textserv\\public\\text.doc` übersetzt. Vom Treiber `\\Device\\MUP` wird nun die Zeichenkette `textserv\\public\\text.doc` an die Treiber `Redirector`, `NetwareFileSystem` usw. weitergegeben und die korrespondierende *parse*-Methode aufgerufen. Jeder Treiber fragt über seine Netzwerkverbindung bei seinem Dateiserver an, ob diese Datei existiert. Wenn ja, öffnet er eine Verbindung dazu.

Im Windows NT-Namensraum ist also wie beim NFS-Dateisystem keine einheitliche Sicht auf die Dateisysteme garantiert. Die Netzwerkverbindungen werden wie beim NFS-System anfangs eingerichtet und garantieren nur einen lokalen Zusammenhang eines Dateiservers zu einem Klienten.

## 6.2.2 Kommunikationsanschlüsse

Die Modelle der Namensgebung in Netzwerken benötigen auch Mechanismen, um sie zu implementieren.

### Ports

Hier hat sich das Konzept der *Kommunikationspunkte* durchgesetzt: Jeder Rechner besitzt als Endpunkt der Netzwerkkommunikation mehrere Kommunikationsanschlüsse. An der

| Dienst  | Portnummer | Protokoll |
|---------|------------|-----------|
| Telnet  | 23         | TCP       |
| FTP     | 21         | TCP       |
| ssh     | 22         | TCP       |
| SMTP    | 25         | TCP       |
| http    | 80         | TCP       |
| https   | 443        | TCP       |
| portmap | 111        | TCP       |
| rwhod   | 513        | UDP       |
| portmap | 111        | UDP       |

**Abb. 6.12** Zuordnung von Portnummern und Diensten

Schnittstelle zur TCP/IP-Schicht sind dies die **Ports** („Türen“), die durchnummeriert sind. Einigen der 16-Bit-Portnummern ist per Konvention (*well known port number*) ein besonderer Dienst zugeordnet, dem man Aufträge geben kann. In [Abb. 6.12](#) ist ein Auszug aus einer Liste von Zuordnungen gezeigt, die man beispielsweise in UNIX in der Datei */etc/services* findet.

Einen solchen Kommunikationspunkt kann man auch mit einem Briefkasten (*mailbox*) vergleichen, wie er zur Kommunikation in [Abschn. 2.4.1](#) eingeführt wurde. Für den Mailbox-Dienst muss das Betriebssystem allerdings zusätzlich zwei Warteschlangen einrichten: eine für die eingehenden Nachrichten und eine für die zu sendenden Nachrichten. Außerdem muss jede Warteschlange mit einem Semaphor abgesichert sein. Die Gesamtheit dieser Prozeduren und Datenstrukturen bildet eine Schicht, die auf den Ports aufsetzt und als Endpunkte einer Interprozesskommunikation angesehen werden kann.

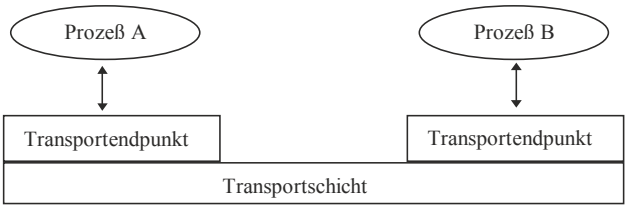
Eine solche Kommunikationsverbindung zwischen zwei Prozessen A und B lässt sich also beschreiben durch ein Tupel

( **Protokoll**, RechnerAdresse von **A**, ProzessId von **A**,  
RechnerAdresse von **B**, ProzessId von **B** )

**Beispiel UNIX Transport Layer Interface TLI**

Mit UNIX System V, Version 3 wurde 1986 das **Transport Layer Interface** (TLI) eingeführt, das aus einer Bibliothek `libnsl.a` von Funktionen (Network Service Library) besteht und auf der Transportschicht aufsetzt. Eine leicht verbesserte Form wurde von X/Open-Normierungsgremium als „Extended Transport Interface XTI“ aufgenommen. Das obige Fünf-Tupel, das die Kommunikationsverbindung spezifiziert, dient hier dazu, um den Prozessen direkt die Kommunikation über sogenannte Transportendpunkte zu ermöglichen. In [Abb. 6.13](#) ist das Modell gezeigt. In TLI kann ein Prozess nach Wahl synchron oder asynchron kommunizieren; in XTI ist dies nur synchron möglich. Da die Transportschicht transparent ist, kann man verschiedene Parameter beispielsweise des TCP/IP-Protokolls nicht beeinflussen. Dies kann sich nachteilig auswirken.

**Abb. 6.13** Kommunikation über Transportendpunkte



**Sockets**

Ein weiteres logisches Kommunikationsmodell ist das **Socket**-Modell, das eine punkt-orientierte Kommunikation organisiert. Es unterscheidet zwischen einem *Server*, der die Kommunikation für eine Dienstleistung anbietet und auf Kunden (*Clients*) wartet, und den Kunden, die eine zeitweilige Kommunikation zu ihm aufbauen.

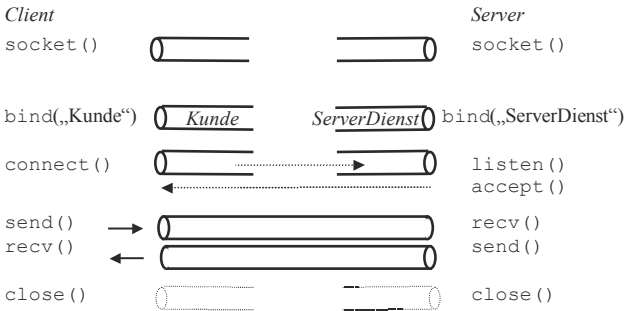
Am Anfang müssen beide, Server und Client, einen Socket mit dem Systemaufruf `socket()` unter einem Namen einrichten. Zusätzliche Prozeduren erlauben es, den Socket an einen Port zu binden. Der Name des aufrufenden Prozesses wird mit einem `bind()`-Systemaufruf im System registriert. Der Prozess tritt damit aus der Anonymität heraus und ist offiziell unter diesem Namen von außen ansprechbar. Der Serverprozess wartet nun mit `listen()`, bis ein Clientprozess mit `connect()` eine Verbindung zu ihm verlangt. Ist der Kunde akzeptiert, so bestätigt der Serverprozess die Verbindung mit `accept()` und die eigentliche Kommunikation kann beginnen. In [Abb. 6.14](#) ist der Ablauf des Kommunikationsprotokolls im Socket-Konzept gezeigt, wobei die Namensgebung mit Etiketten visualisiert ist. Beim Schließen des Sockets mit dem normalen `close()`-Systemaufruf kann noch angegeben werden, ob die Daten im Puffer gesendet oder gelöscht werden sollen.

Derartige Socket-Kommunikationsverbindungen sind auch zwischen den Prozessen auf dem lokalen Computer möglich und stellen damit eine Möglichkeit der Interprozesskommunikation dar, die unabhängig von der Prozesserzeugungsgeschichte (siehe z. B. die Restriktion bei *pipes* in [Abschn. 2.4.1](#)) ist.

**Named Pipes**

Ein weiteres wichtiges Kommunikationsmodell ergibt sich, wenn wir die namenlosen *pipes* aus [Abschn. 2.4.1](#) mit einem Namen versehen (**named pipes**) und sie damit

**Abb. 6.14** Kommunikationsablauf im Socket-Konzept



zu einer speziellen Art von schreib- und lesbaren Dateien machen, auf die auch von außerhalb der Prozessgruppe zugegriffen werden kann. Ist eine solche Datei auf einem vernetzten Rechner eingerichtet, so kann man über das Netzwerk-Dateisystem darauf zugreifen und so eine Interprozesskommunikation auch zwischen Rechnern durchführen.

---

#### Beispiel UNIX *named pipes*

Die Netzwerkfähigkeit von *named pipes* gilt allerdings nicht für das NFS-Dateisystem, da *named pipes* in UNIX als *special file* mit `mknod()` eingerichtet werden. Zu jeder *named pipe* gehört also ein Treiber, der anstelle des Datei-Massenspeichers die Puffer des Betriebssystems zur Zwischenspeicherung und dem Aufbau der FIFO-Datenwarteschlange nutzt.

*Named pipes* lassen sich also meist nur zur Interprozesskommunikation nutzen, wenn alle beteiligten Prozesse auf demselben Rechner sind. Im Unterschied dazu erlaubt die in SYSTEM V innerhalb des *Stream System* eingeführte STREAM von der Art *named pipe* eine mit Namen versehene Kommunikationsverbindung zu einem Prozess auf einem anderen Rechner. Mit den Aufrufen `socketpair()` wird ein bidirektionaler TCP/IP-Kommunikationskanal eröffnet; mit `bind()` erhält er noch zusätzlich einen Namen, unter dem er auf dem anderen Rechner referiert werden kann.

---

#### Beispiel Windows NT *named pipes*

In Windows NT wird der Aufruf `CreateNamedPipes()` zum Erzeugen einer *named pipe* Kommunikationsverbindung ebenfalls über spezielle Treiber abgehandelt. Im Unterschied zum NFS-Dateisystem ist der Zugriff auf dieses im globalen Namensraum angesiedelte Objekt aber auch über das Netzwerk möglich, so dass Prozesse auf verschiedenen Rechnern über eine *named pipe* mit `ReadFile()` und `WriteFile()` kommunizieren können. Üblicherweise wird die *named pipe* auf dem Server erzeugt, so dass alle Clients sie danach für einen Dienst nutzen können.

Eine *named pipe* wird normal als Datei geöffnet, wobei allerdings ein UNC-Name verwendet werden muss der Form

```
\\ComputerName\PIPE\PipeName
```

Lokale *named pipes* benutzen für *ComputerName* das Zeichen „.“

Das *named pipe*-Konstrukt in Windows NT ist unabhängig vom verwendeten Übertragungsprotokoll. Es ist allerdings nur zwischen Windows NT (oder OS/2)-Rechnern möglich im Unterschied zum Socket-Konstrukt, das kompatibel zu UNIX-Sockets ist. Für eine *named pipe*-Verbindung zu UNIX muss dort ein besonderer Prozess, z. B. der *LAN Manager for UNIX* LM/U, installiert sein.

## Mailboxdienste

Im Unterschied zu den bisher betrachteten bidirektionalen Punkt-zu-Punkt-Kommunikationsverbindungen erlauben allgemeine Mailboxdienste das Senden von Nachrichten von einem Prozess an mehrere andere (*multicast* und *broadcast*). Das Kommunikationsmodell dafür entspricht einem Senden eines Briefes an einen Empfänger: Voraussetzung für den Empfang ist die Existenz eines Briefkastens beim Empfänger; ob der Brief auch angekommen ist, bleibt dem Sender aber verborgen, wenn er nicht einen Bestätigungsbrief in den eigenen Briefkasten erhält. Dabei ist der Empfang der Nachrichten beim Server allerdings zusätzlichen Beschränkungen unterworfen, die stark vom darunterliegenden Transportprotokoll abhängen:

- Die Reihenfolge der Nachrichten beim Sender muss nicht die gleiche beim Empfänger sein.
- Der Empfang einer Nachricht ist nicht garantiert.

### Beispiel Windows NT *mailslots*

Der Briefkasten entspricht hier einem **mailslot** beim Server, der mit dem `CreateMailslot (MailBoxName)`-Systemaufruf erzeugt wird. Das Senden von Nachrichten wird beim Client mit dem `CreateFile (MailSlotName)`-Systemaufruf eingeleitet, jede Nachricht mit einem `WriteFile()`-Aufruf verschickt und die Kommunikation mit `CloseFile()` beendet. Der Name eines *mailslot* beim `CreateFile()`-Aufruf hat dabei das Format

$$\text{MailSlotName} = „\ComputerName\mailslot\MailBoxName”$$

Wird für *ComputerName* das Zeichen „.“ verwendet, so wird ein lokaler *mailslot* auf dem Rechner mit dem Namen *MailBoxName* angesprochen; wird ein echter Computername dafür gesetzt, so wird dieser Rechner kontaktiert. Ist *ComputerName* ein Domänenname, so werden die Nachrichten an alle Rechner der Domäne geschickt. Beim Zeichen „\*“ für *ComputerName* wird auf das angeschlossene Netzwerk ein *broadcast* durchgeführt, wobei alle Rechner angesprochen werden, die einen *mailslot* mit dem Namen *MailboxName* haben. Üblicherweise wird dies der Name eines Dienstes sein; der erste freie Dienst wird sich darauf melden.

Da in Windows NT nur unzuverlässige Datagrammdienste für *mailslots* verwendet werden, gelten die obigen Einschränkungen von Briefkästen.

In Windows NT kann außerdem die maximale Länge einer Nachricht, die jeweils mit einem `SendFile()` bzw. `ReadFile()` übermittelt wird, stark vom verwendeten Transportprotokoll abhängen. Beim NetBEUI-Protokoll beträgt beispielsweise die Länge bei Punkt-zu-Punkt-Nachrichten maximal 64 KB, bei *broadcast*-Nachrichten nur maximal 400 Byte.

Allgemein reicht das Mailboxkonzept kaum aus, um Nachrichten strukturiert auszutauschen. Selbst wenn man eine zuverlässige, verbindungsorientierte Transportschicht und

keinen ungesicherten Datagrammdienst für das darunterliegende Transportprotokoll voraussetzt, so muss man trotzdem zusätzliche Regeln (ein zusätzliches Protokoll) einführen, was die Nachrichten bedeuten, wann eine *mailbox* wieder gelöscht werden kann und wie mit unbeantworteten Nachrichten verfahren werden soll. Aus diesen Gründen kann man den *mailbox*-Dienst eher als eine weitere Zwischenschicht für höhere Dienste ansehen.

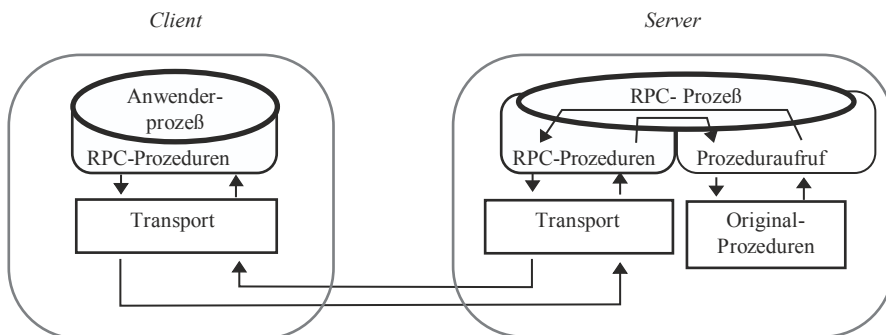
### Remote Procedure Calls

Es gibt in Rechnernetzen verschiedene, netzwerkweite Dienste wie die elektronische Post oder das Drucken von Dateien im Netzwerk. Einer der wichtigsten Dienste, auf den sich viele Programme stützen, ist der Aufruf von Prozeduren auf dem Server durch den Client. Dieser Prozedur-Fernaufwurf (**Remote Procedure Call (RPC)**, *Remote Function Call (RFC)*) funktioniert für den aufrufenden Prozess wie ein einfacher, lokaler Prozeduraufwurf; die Realisierung durch einen Auftrag, der über ein Netzwerk an einen anderen Rechner geht, dort ausgeführt wird und dessen Ergebnisse über das Netzwerk wieder zum aufrufenden Prozess zurückkommen, bleibt dabei verborgen.

Das Konzept eines Prozedurfernaufwurfs ist sehr wichtig. Durch das hohe Abstraktionsniveau kann es gut geräte- und architekturunabhängig eingesetzt werden und hat sich deshalb als ein universeller Mechanismus für Client-Server-Anwendungen bewährt, der auch noch in sehr inhomogenen Netzwerken funktioniert.

Die Implementierung eines RPC benutzt dazu entweder Prozeduren, die den gleichen Namen wie die Originale haben (*stub procedures*), aber statt dessen nur die Argumente verpacken und über eine Transportschicht zum Server schicken. Oder aber es wird nur ein RPC aufgerufen, dem der Name der gewünschten Prozedur als Parameter beigelegt wird.

Beim Server werden die Argumente wieder entpackt und den eigentlichen Prozeduren mit einem normalen Prozeduraufwurf übergeben. Die Ergebnisse durchlaufen in umgekehrter Richtung die gleichen Stationen. In [Abb. 6.15](#) ist das Grundschemata eines RPC gezeigt. Wie bei normalen Prozeduren, so gibt es auch hier zwei Versionen: synchrone RPCs, die den aufrufenden Prozess so lange blockieren, bis die gewünschte Leistung



**Abb. 6.15** Das Transportschema eines RPC

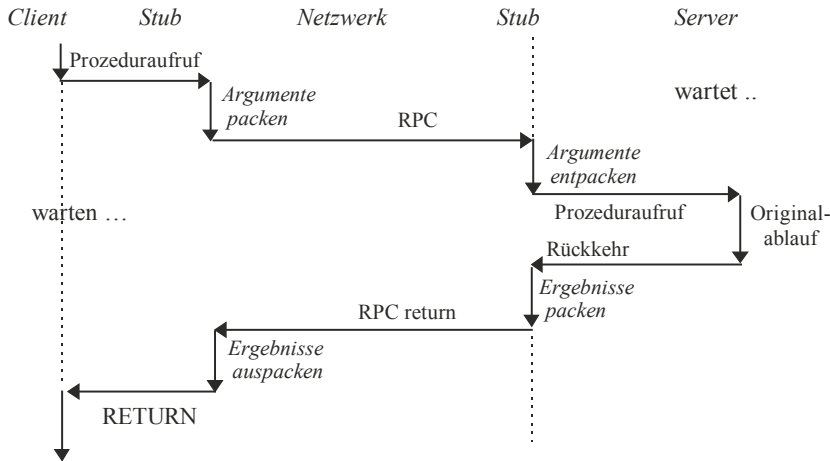


Abb. 6.16 Ablauf eines synchronen RPC

erbracht wurde, und asynchrone RPCs, die den aufrufenden Prozess benachrichtigen, wenn die RPC-Ergebnisse vorliegen. In Abb. 6.16 ist der Ablauf eines synchronen RPC gezeigt.

Ein besonderes Problem bei einem Prozeduraufruf bilden die unterschiedlichen Hard- und Softwaredatenformate, die in den verschiedenen Rechnersystemen verwendet werden. Beginnen wir mit der Hardware. Eine einfache 32-Bit-Integerzahl, bestehend aus 4 Byte, wird in manchen Systemen mit dem höchstwertigen Byte auf der niedrigsten Byteadresse (Format **big endian**) abgelegt, in anderen auf der höchsten Byteadresse (Format **little endian**).

In Abb. 6.17 ist die Bytefolge auf zwei verschiedenen Prozessorarchitekturen gezeigt. Auch das Format von Fließkommazahlen (IEEE 754-1985: Mantisse 23 Bit, Exponent 8 Bit, Vorzeichen 1 Bit) kann verschieden sein, genauso wie die Kodierung von Buchstaben (ASCII-Code oder EBCDIC-Code).

Die Schicht zum Einpacken und Auspacken der Argumente (*data marshaling*) und Ergebnisse muss allen diesen Fragen Rechnung tragen und die Daten zwischen ihrer plattformabhängigen Darstellung und der Repräsentation bei der Kommunikation hin- und hertransformieren, beispielsweise durch eine neutrale Darstellung in XML (*extended markup language*). Ähnliches wird auch durch den Vorgang der „Serialisierung“ in Java erreicht.

Eine weitere Aufgabe besteht in der Anpassung der Daten an die compiler-generierte Aufreihung (*alignment*) auf den Speicheradressen. Manche Prozessoren verweigern den Zugriff auf eine Zahl, wenn sie nicht auf einer geraden Adresse beginnt, oder benötigen

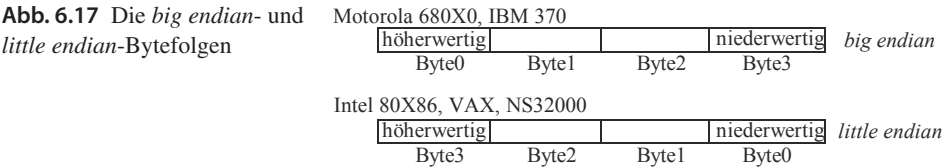


Abb. 6.17 Die big endian- und little endian-Bytefolgen

zusätzliche Zyklen. Aus diesem Grund fügen die Compiler, wenn sie beispielsweise in einem RECORD nach einem Buchstaben eine Zahl speichern wollen, vorher ein Leerzeichen ein. Auch dies muss vom *data marshaling* beachtet werden.

---

#### Beispiel UNIX *Remote Procedure Calls*

Ein RPC in UNIX (nicht zu verwechseln mit dem *remote copy*-Kommando *rcp*) wird mit Hilfe spezieller Bibliotheken verwirklicht (SYSTEM V: in */usr/lib/librpc.a*, sonst in */lib/libc.a* enthalten) und besteht aus zwei Schichten: zum einen aus den RPC-Mechanismen für die Stubprozeduren und zum anderen aus dem Verpacken/Entpacken der Argumente mit Hilfe der *External Data Representation* XDR-Schicht.

Insgesamt sind auch die RPC-Prozeduren wieder in höhere und niedere Aufrufe geschichtet. Die oberste Schicht besteht aus den Aufrufen `registerrpc()`, mit dem beim Server ein Dienst (eine Prozedur) angemeldet wird, der Prozedur `svr_run()`, mit der der Serverprozess sich blockiert und auf einen RPC wartet, und der Prozedur `callrpc()`, mit der der Clientprozess die gewünschte Prozedur auf dem Server aufruft.

Die mittlere Schicht wird von Prozeduren für Client und Server gebildet, um Parameter des Transportprotokolls, Berechtigungsausweise usw. einzustellen (Prozeduren mit dem Präfix `clnt_` und `svc`).

Die unterste Schicht reguliert sehr tiefgehende Details des RPC/XDR-Protokolls und sollte nicht ohne Not direkt angesprochen werden (Präfix `pmap_`, `auth_`, `xdr_`).

Das RPC-System ist in UNIX beispielsweise über das *Network File System* (NFS) realisiert; viele Zusatzdienste zu NFS benutzen RPCs zu ihrer Implementierung.

Einen anderen Zugang bietet das RPC-System im DCE-Softwarepaket, in dem die Dienste in einer *Interface Definition Language* (IDL) abstrakt formuliert werden. Ein Compiler übersetzt dann die Prozeduraufrufe in Stub-Aufrufe und regelt zusammen mit einer Laufzeitunterstützung die Kommunikation und die Anfrage bei einem Verzeichnisdienst aller verfügbaren Server, so dass der aufrufende Client (und damit der Programmierer) vorher nicht wissen muss, auf welchem Rechner im Netz die benötigten Dienste (Dateien, Rechenleistung usw.) zur Verfügung stehen.

---

#### Beispiel Windows NT *Remote Procedure Calls*

Windows NT bietet eine breite Palette von synchronen und asynchronen RPCs, siehe Sinha (1996). Die RPCs können sowohl auf die gleiche Maschine an einen anderen Prozess als auch auf einen bestimmten Server (verbindungsorientierter Aufruf) oder einen bestimmten Service auf einen unbestimmten Server (verbindungsloser Aufruf) zugreifen. Dabei können verschiedene low-level-Mechanismen und Transportprotokolle benutzt werden, s. Abb. 6.18. Die Formatierung bzw. das Verpacken der Argumente für den RPC wird von Stubprozeduren übernommen, die die Daten in das *Network Data Representation* (NDR)-Format übertragen.



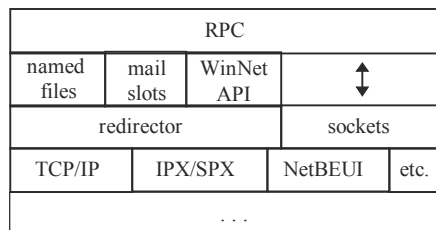
**Abb. 6.18** Die Schichtung der RPC in Windows NT

presentation layer

session layer

transport layer

network layer



Die Stubprozeduren müssen dabei vom Programmierer nicht selbst erstellt werden; ein spezieller Compiler (Microsoft RPC IDL Compiler MIDL) erstellt aus Konfigurationsdateien zusätzlichen Code der mit dem Client bzw. Serverprogramm kompiliert und gelinkt wird und zur Laufzeit spezielle Bibliotheken anspricht. Dadurch wird die Netzwerkverbindung für den Programmierer vollständig transparent. Besondere Protokolle können über ein Präfix vor dem Pfadnamen des gewünschten Service gewählt werden. Beispielsweise wird mit „ncacn\_ip\_tcp: MyServer[2004]“ das TCP/IP-Protokoll benutzt, um beim Rechner *MyServer* an Port 2004 einen Dienst anzufordern.

## 6.2.3 Aufgaben

### Aufgabe 6.2-1 Schichtenprotokolle

In vielen Schichtenprotokollen erwartet jede Schicht ihr eigenes Kopfformat am Anfang einer Nachricht. In jeder Schicht wird beim Durchlaufen die Nachricht als Datenpaket betrachtet, das mit einem eigenen Format und einem eigenen Nachrichtenkopf versehen an die nächste Schicht nach unten weitergereicht wird. Es wäre sicherlich effizienter, wenn es einen einzigen Kopf am Anfang einer Nachricht gäbe, der die gesamte Kontrollinformation enthält, als mehrere Köpfe. Warum wird dies nicht gemacht?

### Aufgabe 6.2-2 Endian

Ein SPARC-Prozessor benutzt 32-Bit-Worte im Format *big endian*. Falls nun ein SPARC-Prozessor die ganze Zahl 2 an einen 386-Prozessor sendet, der das Format *little endian* verwendet, welchen Wert sieht der 386-Prozessor (unter Vernachlässigung der Transportschicht)?

### Aufgabe 6.2-3 Internetadressierung

Gegeben ist die logische Internetadresse 134.106.21.30

- Welche 32-Bit-Zahl ist dies, wenn alle Zahlen innerhalb der logischen Internetadresse die gleiche Anzahl Bits benötigen? Welcher Dezimalzahl entspricht sie?
- Finden Sie heraus, welcher Rechner unter dieser Adresse zu erreichen ist (Hinweis: Benutzen Sie den Netzdienst **nslookup**!).

- c) Angenommen, die angegebene 32-Bit-Zahl sei im *big endian*-Format dargestellt. Welche Dezimalzahl ergibt sich, wenn das *little endian*-Format benutzt wird?
- d) Welche Portnummern verwenden folgende Servicedienste **ftp**, **telnet** und **talk** auf den Rechnern Ihrer Abteilung? (Schauen Sie in UNIX dazu in der Datei */etc/services* nach!)

---

#### Aufgabe 6.2-4 Server und Adressierung im Netz

Recherchieren Sie: Was ist ein DNS-Server und was im Gegensatz dazu ein DHCP-Server, und wofür werden sie benötigt?

---

#### Aufgabe 6.2-5 Remote Procedure Calls

- a) Erklären Sie: Was ist ein RPC? Wann ist die Verwendung von RPCs sinnvoll bzw. wann ist es sinnvoller, auf sie zu verzichten?
- b) Wie werden bei RPCs Parameter und/oder Datenstrukturen übertragen? Wie wird mit diesem Problem in Java umgegangen?
- c) In [Abschn. 6.2](#) wurde die Implementierung von RPCs behandelt. Was muss unternommen werden, um „normale“ (UNIX)-RPCs durch Stubprozeduren bzw. Stubprozeduren durch „normale“ RPCs zu ersetzen?

---

## 6.3 Dateisysteme im Netz

Der Zugriff auf gemeinsame Dateien mit Hilfe eines Netzwerkes ist eine wichtige Arbeitsgrundlage für Gruppen und damit eine der wesentlichen Funktionen der Vernetzung. Es ist deshalb sehr sinnvoll, sich nicht nur über die Tatsache zu freuen, dass so etwas möglich ist, sondern auch einen genaueren Blick darauf zu werfen, wie dies vor sich geht. Deshalb soll in den folgenden Abschnitten ein Überblick über die Konzepte und Probleme verteilter Dateisysteme gegeben und die Verhältnisse bei UNIX und Windows NT betrachtet werden. Zusätzliche Beispiele anderer Dateisysteme findet der/die interessierte Leser/in in der Übersicht von Levy und Silberschatz ([1990](#)).

### 6.3.1 Zugriffssemantik

Betrachten wir nur einen einzigen Prozess im gesamten Netz, so ist ein Netzdateisystem relativ einfach: Statt eine Datei auf dem lokalen Dateisystem anzulegen, zu lesen und zu schreiben, wird dies in Form von Aufträgen an ein anderes Rechnersystem weitergereicht, das diese Aufträge durchführt. So weit, so gut.

Anders sieht die Lage allerdings aus, wenn wir mehrere Prozesse im System zulassen, die alle auf derselben Datei arbeiten: Was passiert dann? Für diesen Fall gibt es mehrere

mögliche Konzepte, die in den verschiedenen Netzdateisystemen implementiert sind und als **Zugriffssemantik** bezeichnet werden. Wir können dabei folgende Fälle unterscheiden:

- *Read Only File*  
Die Datei ist im Netz nur lesbar. In diesem Fall erhalten alle Prozesse Kopien der Datei, die beliebig gelagert und gepuffert werden können: Es entstehen keine Probleme.
- *Operationssemantik*  
Die ausgeführten Operationen der Prozesse auf der Datei verändern diese in der Reihenfolge der sukzessiv ausgeführten Operationen. Wird also ein `read()` von Prozess A, `write()` von Prozess B und folgend ein weiteres `read()` von A initiiert, so erfährt A beim zweiten `read()` die Änderungen, die B vorher auf der Datei vorgenommen hat. Da dies in UNIX-Systemen so implementiert ist, wird dies auch als *UNIX-Semantik* referiert.
- *Sitzungssemantik*  
Arbeiten mehrere Prozesse auf einer Datei, so erhalten alle Prozesse zunächst eine Kopie der Datei, die sie verändern können. Erst wenn ein Prozess die Datei schließt, wird die gesamte Kopie zurückgeschrieben. Dies bedeutet, dass die Version des letzten Prozesses, der die Datei schließt, alle anderen überlagert und auslöscht.
- *Transaktionssemantik*  
Das Konzept der unteilbaren Handlung, die entweder vollständig in der spezifizierten Reihenfolge stattfindet oder gar nicht (*atomic transaction* aus [Abschn. 2.4](#)), lässt sich auch hier anwenden. Öffnen und modifizieren wir eine Datei atomar, so wird vom Netzdateisystem garantiert, dass dies unabhängig von allen anderen Prozessen in einem einzigen, gesicherten Arbeitsgang geschieht. Während dieses Vorgangs ist also die Datei für den Zugriff durch andere gesperrt.

Der konkrete Arbeitsablauf in einer Gruppe hängt stark davon ab, welches der Modelle implementiert ist. Dazu kommt noch das Problem, dass die konkreten Auswirkungen des jeweiligen Modells noch zusätzlich von der Hardware, den implementierten Puffern und Kommunikationsprotokollen abhängen.

Betrachten wir dazu beispielsweise die Operationssemantik. Angenommen, Prozess B schreibt etwas auf die Datei und Prozess A liest von der Datei. Beide Anfragen erreichen den Dateiserver zu einem Zeitpunkt (und damit in einer Reihenfolge), die vom verwendeten Netzwerk, vom Netzwerkprotokoll und von der Last auf den beteiligten Computern abhängt. Ob Prozess A die von B geschriebenen aktuellen Daten liest oder die alte, nicht-aktualisierte Form der Datei erhält, ist also von der Implementierung abhängig, nicht von der Zugriffssemantik.

Problematisch ist, wenn in einem Betriebssystem lokal die eine Semantik, im Netz aber die andere Semantik implementiert ist. Wenn die beteiligten Prozesse teilweise auf demselben Rechner und teilweise auf anderen Rechnern im Netz lokalisiert sind und auf gemeinsamen Dateien arbeiten, muss man die Zusammenarbeit der Prozesse eines Prozesssystems sorgfältig planen, um Fehlfunktionen zu vermeiden.

### 6.3.2 Zustandsbehaftete und zustandslose Server

Bereits bei Kommunikationsverbindungen haben wir danach unterschieden, ob diese durch ein Verbindungsprotokoll eingeleitet (verbindungsorientierte Kommunikation) oder die Daten nur mit einer Adresse versehen direkt transferiert werden (verbindungslose Kommunikation). Eine solche Unterscheidung können wir auch beim Zugriff auf die Dateisysteme im Netzwerk machen. Wird eine Datei geöffnet, bevor darauf zugegriffen wird, so werden wie bei der verbindungsorientierten Kommunikation extra Datenstrukturen im Server errichtet, die Zugriffsrechte werden geprüft und der Zugriff wird registriert. Dem Kunden wird eine spezielle Dateikennung übergeben, so dass alle nachfolgenden `read()`- und `write()`-Operationen mit dieser Kennung sofort bearbeitet werden können.

Der Server hat damit einen bestimmten Zustand errichtet; erst ein Abschluss der Zugriffe mit `close()` bewirkt ein Löschen der Verwaltungsinformation und ein Freigeben des dafür belegten Speichers. Ein solches zustandsbehaftetes Vorgehen hat folgende Vorteile:

- *Schneller Zugriff*

Der nachfolgende Zugriff auf die Dateien geht sehr schnell, da zum einen die Aufträge zum Schreiben und Lesen keine Adress- und Benutzerangaben mehr benötigen, deshalb kürzer sind und schneller über das Netz transportiert werden können, und zum anderen die Angaben nicht mehr nachgeprüft werden müssen, bevor ein Zugriff erfolgen kann.

- *Effizienter Cache*

Da alle Angaben über den Zugriffszustand auf dem Server geführt werden, können effiziente Cachestrategien (*read ahead* etc.) für die Datei den Zugriff beschleunigen.

- *Vermeiden von Auftragskopien*

Durch Führen einer Nachrichtennummer für eine Datei können Kopien desselben Auftrags, die in Vermittlungsrechnern erzeugt werden, aussortiert werden.

- *Dateisperrung möglich*

Für die Transaktionssemantik können Dateien den Zustand „gesperrt“ annehmen. Dies ist besonders bei Datenbanken wichtig.

Allerdings hat ein zustandsbehafteter Server auch Probleme, die dafür sprechen, keine Zustände bei Servern zuzulassen:

- *Client crash*

Wenn ein Client „abstürzt“ und damit der Zugriffsschlüssel für die Datei verlorengeht, bleibt die Datei immer offen, und die Verwaltungsinformation belegt unnötig Platz.

- *Server crash*

Versagt ein Server, so ist der gesamte Zustand der Dateibearbeitung gelöscht. Der Client muss dann erfahren, in welchem Zustand die Datei ist und an der Stelle der

Bearbeitung weitermachen. Dies erfordert verschiedene Maßnahmen von seitens des Client und zusätzlichen Aufwand.

- *Begrenzte, gleichzeitig benutzte Dateienanzahl*

Die Speicherbelegung beim Server für die Zustandsinformation (Tabellen etc.) beschränkt die Anzahl gleichzeitig geöffneter Dateien.

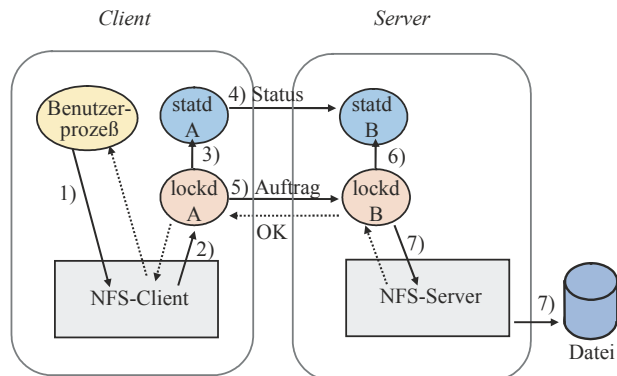
Ein zustandsloser Server ist also im Prinzip *fehlertoleranter* und verlangt keinen zusätzlichen Aufwand zum Aufbau einer Kommunikationsverbindung, hat aber dafür bei zustandsorientierten Diensten, wie Sperren einer Datei oder Eliminieren von Auftragskopien, mehr Probleme.

### Beispiel UNIX NFS-Server und Network Lock Manager

Der am häufigsten benutzte Netzdateiserver NFS war ursprünglich ein zustandsloser Server, wobei jeder Rechner gleichzeitig Server oder Client sein kann. Der `open()`-Aufruf zum Öffnen einer Netzdatei wird auf dem Client in einen `lookup()`-Befehl übersetzt, der auf dem Server eine dem Zugriff erleichternde, interne 32-Byte-Referenz für die Datei ausliest und dem Client zurückgibt. Alle Zustände wie aktuelle Position in der Datei usw. sind auf dem Client gespeichert; die `read()`- und `write()`-Aufrufe senden neben den Daten auch die Server-Dateireferenz, Position in der Datei usw. Für die Implementierung einer Transaktionssemantik muss eine Möglichkeit zum Sperren einer Datei (*Locking*) vorhanden sein, was aber bei einem zustandslosen Server nicht existiert. Deshalb entschloss sich die Firma Sun, drei Jahre nach dem Erscheinen von NFS auch *file locking* für NFS einzuführen. Da die NFS-Protokolle zustandslos waren, wurde dieser Dienst durch einen RPC-Dienst (*Network Lock Manager* NLM) erfüllt, der das komplexe Zusammenwirken mehrerer Protokolle und Dienste erfordert. Dieses Zusammenwirken ist auf verschiedenen UNIX-Systemen verschieden, da teilweise bereits lokale Locking-Mechanismen (z. B. in System V) existieren. Durch die ausschließliche Nutzung von TCP statt UDP ist NFS ab Version 4 nun auch zustandsbehaftet.

Beim Berkeley-BSD-System ist für das *file locking* ein spezieller Prozess (Dämon) *lockd* zuständig, der sich auf jedem Rechner befindet und den Dienst durch Kommunikation erfüllt. In Abb. 6.19 ist ein solcher Vorgang gezeigt.

**Abb. 6.19** Der Vorgang zum Sperren einer Datei



Der Vorgang besteht aus folgenden Schritten:

- (1) Ein Benutzerprogramm auf Rechner A setzt einen Aufruf an das System ab, um einen Bereich einer Datei zu sperren.
- (2) Der *Lock*-Dämon auf Rechner A erhält darauf vom NFS-Client Kernmodul einen *Lock*-Auftrag für Rechner B.
- (3) Der *Lock*-Dämon auf A registriert die Anfrage bei einem speziellen Statusmonitor *statd* auf Rechner A mittels einer RPC-Prozedur.
- (4) Der Statusmonitor *statd* A wendet sich an den korrespondierenden Statusmonitor auf Rechner B. Dieser speichert den Namen von Rechner A in einer Datei */etc/sm*.
- (5) Nun wird der eigentliche Auftrag an den *Lock*-Dämonen B geschickt mittels eines RPC.
- (6) Der *Lock*-Dämon B registriert die Sperrung beim Monitor B,
- (7) und sperrt schließlich die gewünschte Datei. Anschließend wird die erfolgreiche Durchführung dem Auftraggeber angezeigt.

Die zusätzliche Registrierung des *Lock*-Auftrages bei den speziellen Statusmonitoren schützt vor der Gefahr, dass bei einem Systemabsturz eine Datei ewig gesperrt bleibt. Beim Absturz des Servers versucht der Lockdienst grundsätzlich, die vorher existierenden Tabellen wieder aufzubauen, und zwar auf Client und Server gleichermaßen.

Anders ist die Lage, wenn mehrere Clients eine Datei sperren wollten, aber nur einer erfolgreich war. Dieser Client sei nun abgestürzt und würde durch seine exklusive Dateibeleugung alle anderen Interessenten blockieren. Hier setzt die Funktion der Monitoren ein: Bei jeder Sperranfrage prüfen sie, ob der bisher sperrende Prozess noch „am Leben“ ist. Registriert der lokale Monitor das Ableben des sperrenden Prozesses, so wird die frühere Sperrung aufgehoben, und der anfragende Prozess erhält die Datei zum Sperren.

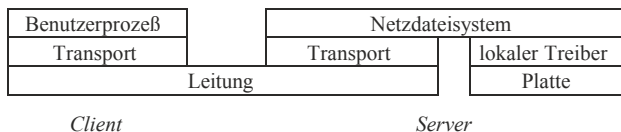
Man beachte, dass ab Version 4 die Protokolle zum Einbinden und Sperren von Dateisystemen ausschließlich über TCP (Port 2049) laufen und damit die erforderlichen Zustände verwalten können, so dass das gesamte System vereinfacht werden konnte.

*Hinweis:* Das Protokoll schützt zwar vor unkontrolliertem Zugriff auf dieselbe Datei, nicht aber vor Verklemmungen im Netzwerk. Obwohl ein Warndienst auf lokaler Rechnerebene sinnvoll und möglich ist, wäre der Aufwand dafür in einem Netz einfach zu hoch und wird deshalb nicht implementiert.

### 6.3.3 Die Cacheproblematik

Bei der Betrachtung der *memory mapped files* (Abschn. 4.4) und der I/O-Verwaltung (Abschn. 5.4) bemerkten wir, dass hier effiziente Pufferstrategien den Datenfluss erheblich beschleunigen können, aber auch eine Menge Probleme verursachen können. Dies ist bei Netzwerkdateisystemen nicht anders. Installieren wir beispielsweise einen Puffer auf einem Rechnersystem mit Prozess A, so können alle Anfragen von Prozess A auf diese Daten schnell beantwortet werden und belasten nicht das Netz. Allerdings werden

**Abb. 6.20** Übertragungsschichten und Pufferorte für Dateizugriffe im Netz



dabei die Änderungen von Prozess B am Original auf dem anderen Rechnersystem nicht berücksichtigt.

Hier treten also die typischen Cacheprobleme auf, wie wir sie in den [Abschn. 3.5](#) und [5.4](#) kennengelernt haben. In [Abb. 6.20](#) ist zur Übersicht ein Bild aller beteiligten Übertragungsschichten gezeigt. Auf jeder Schicht kann ein Puffer implementiert werden; jede Schicht hat dann ihre speziellen Probleme.

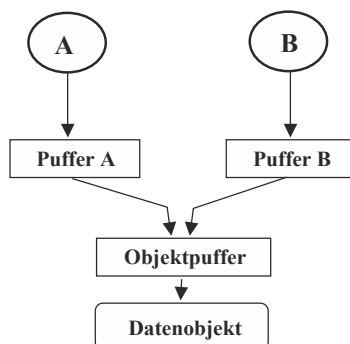
Man beachte, dass Leiter bei hohen Frequenzen auch Daten puffern: Bei  $f = 1$  GHz Übertragungsrate ( $10^9$  Bits/sec) und einer Ausbreitungsgeschwindigkeit der elektrischen Impulse von  $c = 3 \cdot 10^8$  m/sec resultieren  $f/c = 3,3$  Bit/m Information – also 10 kB Speicherung in der Leitung bei einem Abstand von 3 km zwischen den Rechnern.

Entscheidend für das Auftreten der Cache-Konsistenzproblematik ist dabei, ob die Prozesse, die eine Datei gemeinsam bearbeiten, verschiedene Puffer oder denselben Puffer nutzen. In [Abb. 6.21](#) ist die grundsätzliche Situation für zwei Prozesse A und B gezeigt, die auf ein Datenobjekt schreiben wollen.

Nutzen sie verschiedene Puffer, so kommt es zu Konsistenzproblemen: Alles, was in Puffer A landet, ist nicht mit dem verträglich, was in Puffer B steht. Erst wenn die Transportwege sich vereinigen und ein gemeinsamer Weg durch einen gemeinsamen Puffer („Objektpuffer“) vorliegt, gibt es keine Konsistenzprobleme im Normalbetrieb. Dabei ist es unerheblich, ob der einzelne Puffer in Wirklichkeit aus vielen Puffern besteht, beispielsweise Puffer A aus dem Heap von Prozess A zuzüglich dem Betriebssystempuffer auf Rechner A und dem Treiberpuffer des Netzwerks auf Rechner A: Die Problematik bleibt die gleiche. Unkritisch sind jeweils die gemeinsamen Puffer, als meist alle Puffer, die auf der Serverseite vorhanden sind.

Die Hauptstrategien, um die beschriebenen Probleme zu lösen, verwenden eine Mischung bekannter Konzepte.

**Abb. 6.21** Cachekohärenz bei gleichen und unterschiedlichen Puffern



- *Write Through*  
Werden Kopien von einer Datei geändert, so wird der Änderungsauftrag außer an die Kopie im Cache auch an das Original auf dem Server geschickt, um der Operationssemantik zu genügen.
- *Delayed Write*  
Problematisch an der *write through*-Lösung ist allerdings die Tatsache, dass der Cachevorteil für den Netzwerkverkehr nicht mehr vorhanden ist. Dies lässt sich etwas verbessern, wenn man alle `write()`-Aufträge sammelt und in einem Paket zum Server schickt (*delayed write*), was aber wieder die Semantik der Zusammenarbeit verändert.
- *Zentrale Kontrolle*  
Um die Änderungen in den Kopien auf allen anderen Rechnern ebenfalls wirksam werden zu lassen, muss jeweils das lokale Cachesystem mit dem zentralen Server zusammenarbeiten. Dies kann sofort nach jeder Änderung des Originals geschehen, oder aber erst bei tatsächlichem Gebrauch der Daten. In letzterem Fall vergleicht bei jedem Lesen der Cachemanager seine Dateidaten (Versionsnummer, Pufferquersumme etc.) mit der des Servers. Sind sie identisch, erfolgt das Lesen aus dem Cache; wenn nicht, wird der Leseauftrag an den Server weitergegeben.
- *Write On Close*  
Eine Alternative zu der Operationssemantik ist die Sitzungssemantik. Dafür reicht es aus, die gesamte geänderte Kopie der Datei nach einem `close()` des Prozesses an den Dateiserver zurückzuschicken und das Original so zu aktualisieren. Das Problem, dass mehrere inkonsistente Kopien der einen Originaldatei auf verschiedenen Rechnern existieren können, muss dann bei der Sitzungssemantik berücksichtigt werden und nicht mehr beim Cache.

Allgemein lässt sich sagen, dass die Einführung von Cache auf dem Dateiserver in jedem Fall den Durchsatz erhöht, ohne allzu viele Probleme zu verursachen. Die Bereitstellung von Cache auf dem Client-Rechner dagegen ist zwar noch effizienter, bringt aber unter Umständen bei Dateiänderungen Konsistenzschwierigkeiten mit sich. Aus diesem Grund existieren bei verteilten Datenbanken spezielle Sperrprotokolle.

### 6.3.4 Implementationskonzepte

Es gibt verschiedene Ideen, wie man Netzwerkdateisysteme implementieren kann. Die beiden populärsten Möglichkeiten sehen vor, entweder den Dateiservice als eigenständigen Prozess oder aber nur als speziellen Treiber im Betriebssystemkern zu installieren. In Abb. 6.22 ist die erste Konfiguration gezeigt, bei der sowohl auf Client- als auch auf Serverseite Prozesse existieren. Man beachte, dass die Betriebssystemschichten symmetrisch sind; der konzeptionelle Unterschied drückt sich nur in der Art des Sender- bzw. Empfängerprozesses aus.



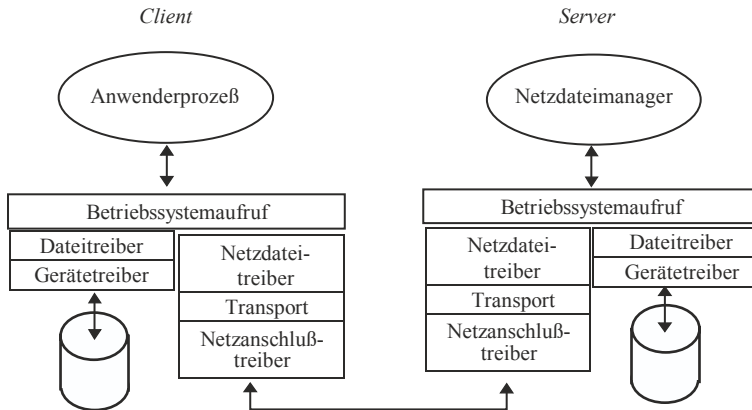


Abb. 6.22 Implementierung eines Netzdateiservers auf Prozessebene

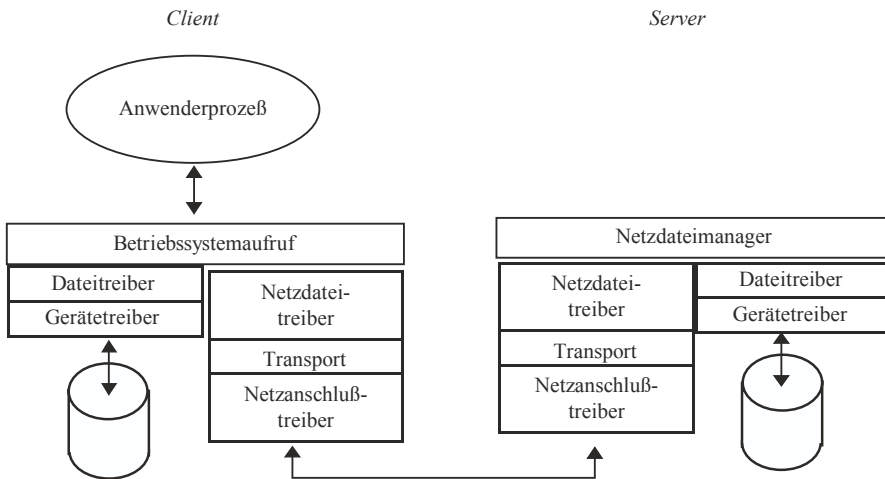


Abb. 6.23 Implementierung eines Netzdateiservers auf Treiberebene

Im Unterschied zu diesem Mechanismus, der auf Interprozesskommunikation beruht, ist in [Abb. 6.23](#) ein Implementierungsschema auf Treiberebene gezeigt. Die Clientseite ist hier genauso gestaltet; auf der Serverseite ist dagegen kein Managerprozess vorhanden. Die Anfragen werden direkt im Betriebssystemkern verwaltet und auf das lokale Dateisystem umgeleitet.

Dieser Ansatz implementiert das Netzwerkserver-Dateisystem auf Treiberebene und verhindert so das aufwendige Kopieren der Daten aus den Systempuffern in den Adressbereich eines Serverprozesses und zurück in die Systempuffer zum Schreiben im lokalen Dateisystem des Servers. Allerdings wird so die Konfiguration asymmetrisch:

Das Netzdateisystem setzt nicht mehr auf einem Stellvertreterprozess und regulären Interprozesskommunikation auf, sondern auf speziellen Schichten (Treibern) des Kerns. Die beiden Konzepte werden unterschiedlich realisiert.

### Beispiel UNIX Das NFS-System

Im NFS-System gibt es zwei Protokolle: eines, das das Einhängen (`mount()`) eines Serverdateisystems unter ein Verzeichnis im Client regelt, und eines, das den normalen Schreib-/Leseverkehr betrifft. Das erste Protokoll wird zwischen zwei Prozessen durchgeführt: dem anfragenden Clientprozess, meist beim Starten des Clientsystems, und einem speziellen Dämon *mountd* auf dem Server. Dazu schickt der *mount*-Prozess des Client eine Nachricht an den *mountd*-Dämon auf dem Server, der sich mit einem Systemaufruf `getfh()` ein *file handle* (Dateisystemdeskriptor) von der NFS-Serverschicht holt und an den Clientprozess zurückgibt. Dieser führt nun damit einen Systemaufruf `mount()` aus und verankert so das Serverdateisystem in der NFS-Clientschicht des Betriebssystemkerns.

Das zweite Protokoll des Schreib-/Lesevorgangs auf Dateien wurde unter Leistungsgesichtspunkten in den Betriebssystemkern verlegt. Obwohl auf dem Server das Programm *nfsd* anfangs als Prozess gestartet wird, läuft es doch bald auf den Systemaufruf `nfs_svc()` in den *kernel mode*, aus dem es nicht mehr zurückkehrt. Dieser Ansatz implementiert das Netzwerkdatsystem auf Treiberebene mit den erwähnten Leistungsvorteilen. In Abb. 6.24 ist die grundsätzliche Architektur gezeigt.

Eine wichtige Rolle spielt dabei die zusätzliche Schicht für ein virtuelles Dateisystem VFS, das die Dateisystemoperationen abstrahiert. Alle Operationen im VFS (System V:

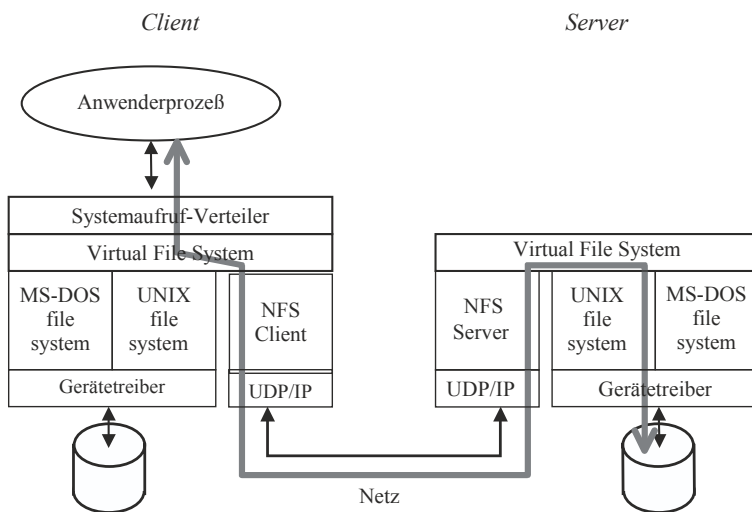
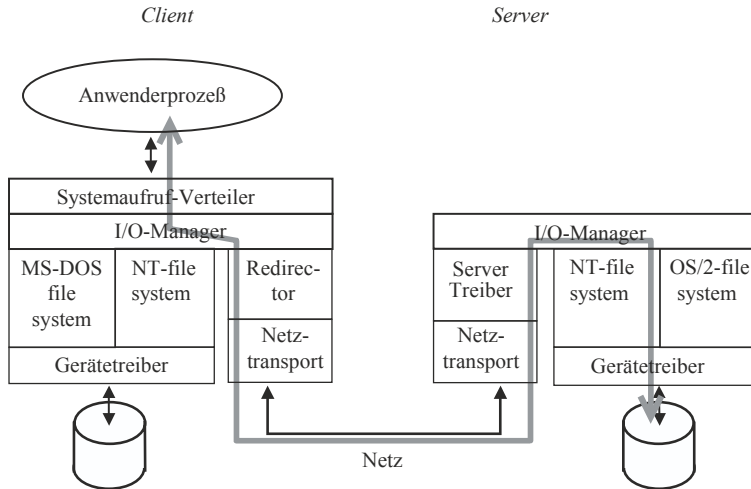


Abb. 6.24 Die Architektur des NFS-Dateisystems



**Abb. 6.25** Implementierung des Netzdateisystems in Windows NT

File System Switch FSS) sind Methoden von Objekten, den *virtual i-nodes* (vnode), und sind je nach Objekts dateisystemspezifisch implementiert. So ist ein Umschalten zwischen MS-DOS-Dateien, UNIX-Dateien und NFS-Dateien problemlos möglich.

#### Beispiel Windows NT Netzdateisysteme

Die Implementierung der Netzdateisystemdienste von Windows NT ist aus den gleichen Gründen wie bei NFS in den Betriebssystemkern gelegt worden und baut damit auf dem Treibermodell auf. In Abb. 6.25 ist eine Übersicht gezeigt. Der *Redirector*-Treiber hat dabei die Aufgabe, die Dateisystemanfragen, die über das Netz gehen, auf die gleiche Art zu behandeln, wie dies bei lokalen Dateiaufträgen geschieht. Dazu muss er Netzverbindungen, die gestört sind, über die Transportschicht wieder aufbauen. Auch die gesamte Statusinformation (welche Dateien wurden geöffnet usw.) wird vom Redirector verwaltet und aktualisiert.

Die Unterstützung der asynchronen Ein- und Ausgabe verlangt dabei besondere Mechanismen. Anstatt einen eigenen Prozess zu erzeugen, der nach der asynchronen Ausführung des Auftrags durch das Netz die Kontrolle erhält und aufgeweckt wird, benutzt Windows NT einen Pool von *threads*, der für die Treiberinitialisierung beim Systemstart verwendet wurde, um die Abschlussprozeduren für ein IRP durchzuführen.

Der *Redirector* wickelt seine Netzwerkkommunikation über das *Transport Driver Interface* (TDI) ab, dessen Kanäle (*virtual circuits*) durch das darunter liegende Transportsystem verwirklicht werden.

Die Dateiserverschicht, die ebenfalls als Treiber installiert ist, ist kompatibel zu den existierenden SMB-Protokollen der MS-Net- und LAN-Manager-Software.

### 6.3.5 Die WebDAV-Dienste

Die bisherigen Implementationskonzepte setzen beim Betriebssystem auf der Ebene der Treiber an, so dass transparent auf Dateisysteme auf anderen Rechnern mittels eines Netzwerks zugegriffen werden kann. Es gibt nun ein Konzept, dass ganz anders auf der obersten Schicht der Anwendungsebene Dateien über das Netz lesen, schreiben und verändern kann: das WebDAV-Konzept. Dabei steht „DAV“ als Abkürzung für „Web-based Distributed Authoring and Versioning“. Das Konzept dahinter ist einfach: Erweitert man das bekannte HTTP-Protokoll im Internet, das von den meisten Webbrowsern verwendet wird, um einige Elemente zum Kopieren (COPY) und Verschieben (MOVE) von Dateien von einem Rechner im Netz zu einem Anderen und ergänzt es mit Befehlen zum Erstellen von Verzeichnissen (MKCOL) und zum Lesen und Schreiben der Dateieigenschaften (PROPFIND und PROPPATCH) sowie Befehlen zum exklusiven Sperren der Dateien (LOCK/UNLOCK), so erhält man die genormten Befehle (RFC 5323, RFC 3253) des WebDAV.

Diese Befehle können nun von verschiedenen Prozessen benutzt werden, um mit Hilfe des WebDAV-Protokolls Dateien direkt aus dem lokalen System übers Netz zu einem Server zu senden oder die Dateien zu empfangen.

Der Vorteil dieser Methode gegenüber tieferen Eingriffen ins Betriebssystem liegt in der leichteren Konfigurierbarkeit: Es ist nur ein erweitertes HTTP-Protokoll zu benutzen und es müssen keine besonderen Ports freigeschaltet werden. Sogar die verschlüsselte Version HTTPS kann verwendet werden.

---

#### Beispiel Windows NT WebDAV

Ab Windows XP ist das Aufsetzen einer WebDAV-Verbindung mit einem Server möglich, der einen WebDAV-Service anbietet. Dazu muss zunächst eine Verbindung mit dem Server erstellt werden. Dies lässt sich in XP in der Netzwerkumgebung mit „Netzwerkressource hinzufügen“ und dem damit aufgerufenen Assistenzprogramm durchführen, bei dem man die http-URL, Benutzer und Passwort für die Verbindung angibt; bei Windows 7 erreicht man dies durch den Aufruf „Netzlaufwerk verbinden“ unter „Extras“ beim Netzwerk- und Freigabecenter. Auf diesem Fenster gibt es einen Link „Verbindung mit einer Website herstellen ...“, der wieder den Assistenten aufruft.

Die Verbindung wird dann symbolisch mit Hilfe eines Ordnersymbols im Explorer angezeigt und kann wie ein „normaler“ Ordner geöffnet und Dateien daraus oder hinein kopiert werden.

Auf Serverseite werden bei Windows die *Internet Information Services* (IIS) eingesetzt, um die Anfragen zu beantworten.

---

**Beispiel OS X WebDAV**

Bei dem Apple-Betriebssystem ist der WebDAV-Zugriff ebenfalls bereits eingebaut und muss nur mit einer neuen Netzwerkverbindung über die http-URL, Benutzer und Passwort eingerichtet werden über den Menüpunkt „Gehe zu“, Untermenü „Mit Server verbinden“ im Finder-Programm.

---

**Beispiel Linux WebDAV**

Beim Benutzer, dem Client, kann man entweder ein Weblaufwerk einbinden (*mounten*), indem man den Open-source Treiber *davfs* installiert (<http://sourceforge.net/projects/dav/>). Oder man verwendet beispielsweise beim Konqueror-Browser direkt die Webadresse mit dem Protokollpräfix „webdav://URL“. Für eine SSL-Verbindung muss „webdavs:“ verwendet werden und Port 443 freigegeben sein.

Auf Serverseite enthält meist der Apache-Server ein DAV-Modul, so dass nur noch das Modul in *httpd.conf* angegeben, ein passendes Verzeichnis erstellt und die Zugriffsrechte auf Serverseite konfiguriert werden müssen.

### 6.3.6 Sicherheitskonzepte

Es ist klar, dass auch im Netzwerk nicht mehr Sicherheit zum Schutz von Dateien möglich ist, als das lokale Dateisystem von sich aus bietet. Alle zusätzlichen Sicherheitsprüfungen müssen extra installiert und durchgeführt werden.

Eine besondere Schwierigkeit besteht darin, Dateisysteme mit verschiedenen Sicherheitsmechanismen und Sicherheitsstufen konsistent miteinander zu koppeln. Beispielsweise macht es nicht nur Schwierigkeiten, Dateien mit einem langen Namen in Groß- und Kleinschreibung von UNIX- und Windows NT-Servern mittels PC-NFS auf ein MS-DOS-Dateisystem mit kurzem Namen (max. acht Buchstaben) und Großbuchstaben abzubilden, sondern die Zugriffsrechte von *Access Control Lists* unter Windows NT lassen sich auch schwer auf die simplen Zugriffsrechte unter UNIX abbilden, wo derartige ACL noch nicht existierten.

Der Übergang von einem Sicherheitssystem in ein anderes mittels eines Netzwerkdateisystems ist deshalb meist nur mit vielen Inkonsistenzen und Widersprüchen zu bewerkstelligen; es ist fast unmöglich, alle Sicherheitslücken zu schließen.

---

**Beispiel UNIX NFS-Sicherheitssystem NIS**

Im NFS-Dateisystem werden alle Benutzer des Systems in einer besonderen Liste (*yellow pages*) geführt, ähnlich wie Handwerker in den gelben Seiten eines Telefonbuches. Diese Liste wird von einem besonderen Prozess, dem **Network Information System** NIS, verwaltet und enthält außer dem Tupel (Benutzer, Passwort) auch Einträge über den aktuellen Codeschlüssel für einen chiffrierten Datenaustausch zwischen

Client und Server, differenzierte Benutzergruppen und ähnliche Mechanismen, die in der speziellen Variante *secure NFS* verwendet werden können.

Für die Anforderung von Diensten mittels eines RPC wird das NFS-System verwendet. Spezifiziert man als Sicherheitsausweis den Modus „UNIX-Authentifikation“, so hat der RPC-Sicherheitsausweis das Format der üblichen UNIX-Zugriffskennung von Benutzer- und Gruppen-Id. Auch wenn beide Systeme, Client und Server, die gleiche Art von Sicherheitsmechanismen haben, so können auch hier Sicherheitslücken existieren. Beispielsweise sollte ein Administrator (*super user*) mit der Benutzer-Id = 0 auf dem Client nicht automatisch auch alle Dateien des Administrators auf dem Server ändern können, auch wenn die Id auf beiden Systemen die gleiche ist. Aus diesem Grund werden alle Anfragen mit Benutzer-Id = 0 auf dem Server in Benutzer-Id = -2 („Datei für externen *super user*“) abgebildet, was mit den normalen Sicherheitsmechanismen weiterbehandelt werden kann.

Das NIS ermöglicht darüber hinaus, generell zentral die Zuordnung von Benutzern zu den Benutzer-Ids auf dem Dateisystem des Servers zu regeln, so dass keine Überschneidungen und Inkonsistenzen von Dateien verschiedener Benutzer auftreten, die lokal auf unterschiedlichen Rechnern jeweils zufälligerweise in */etc/passwd* die gleiche Benutzer-Id erhalten.

Trotz verschiedener Anstrengungen enthält das Sicherheitssystem des NFS noch einige Lücken. Für weitergehende Ansprüche sei deshalb auf das Kerberos-System verwiesen, das im übernächsten Kapitel beschrieben ist.

#### Beispiel Windows NT Sicherheit in Netzdateisystemen

In Windows NT sind die Dateien, wie in [Abschn. 4.3](#) dargelegt, mit Zugangskontrolllisten ACL versehen, die für jeden Benutzer bzw. jede Benutzergruppe die Zugriffsrechte regeln. Diese Sicherheitsmechanismen werden auch auf den Dateiservern bereitgestellt. Dazu muss jeder Benutzer, bevor er eine Datei öffnen kann, in der Benutzerdatei des *Security Account Managers* (SAM) sowohl auf dem Client-Rechner als auch auf dem Server registriert sein. Fehlt die Registrierung oder sind die verschlüsselten Passwörter auf Client und Server nicht identisch, so wird der Zugriff auf die Dateien des Servers prinzipiell verweigert. Aber auch bei erfolgter Registrierung gelten immer noch die Zugriffsbeschränkungen der ACL wie auf dem lokalen Dateisystem.

Man beachte, dass dieses Konzept keine automatische Kontrolle des Client-Administrators über den Server mit sich bringt, so lange beide unterschiedliche Passwörter benutzen.

Im Konzept ab Windows 2000 ist die Sicherheitspolitik grundlegend überarbeitet worden. Hier gibt es nicht nur die Authentifikations- und Verschlüsselungsmechanismen des weiter unten behandelten Kerberos-Systems (s. [Kap. 7](#)), sondern auch das netzübergreifendes Adressierungssystem ADS, bei dem jedes Objekt auf einem Rechner im Netz einer Domäne im Intranetz genau bezeichnet und dessen Zugang systemweit reguliert werden kann.

### 6.3.7 Aufgaben

#### Aufgabe 6.3-1 Zugriffssemantiken

- a) Wollen mehrere Personen gleichzeitig sowohl lesend als auch schreibend auf eine Datei zugreifen, so kann diese je nach verwendeter Zugriffssemantik nach Abschluss der Änderungen unterschiedliche Zustände aufweisen.

Beschreiben Sie den Zustand einer Datei davon ausgehend, dass zwei Personen diese Datei öffnen, Änderungen vornehmen und die Datei dann schließen, für folgende Zugriffssemantiken:

- i) die Operationssemantik
  - ii) die Sitzungssemantik
  - iii) die Transaktionssemantik
- b) Microsoft Office Online erlaubt das gemeinsame Arbeiten an Dokumenten. Welcher der bisher kennengelernten Zugriffssemantiken entspricht die Funktionsweise dieser Anwendung? Begründen Sie Ihre Lösung.

#### Aufgabe 6.3-2 Serverzustände

- a) Man unterscheidet zwischen zustandsbehafteten und zustandslosen Servern. Was sind die Vor- und Nachteile dieser beiden Konzepte?
- b) Können bei NFS-Servern Verklemmungen auftreten? Begründen Sie Ihre Antwort.

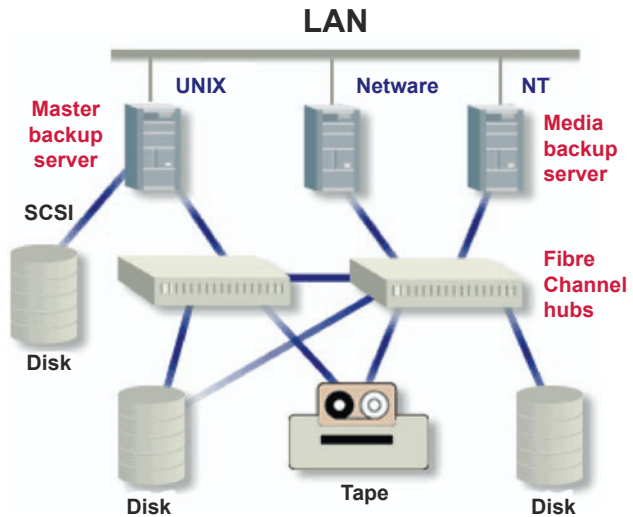
## 6.4 Massenspeicher im Netz

Eines der attraktivsten Vorteile eines Netzes besteht darin, Ressourcen für mehrere Rechner nutzbar zu machen. Ein wichtiges Beispiel dafür bilden die Speicherressourcen. Verbindet man alle wichtigen Massenspeicher mit schnellen Leitungen untereinander und mit den Dateiservern, so wird ein spezielles Netz nur zur Speicherung gebildet. Ein solches Netz wird Speichernetz SN genannt. Alle Server, Netzverbindungen und Speichergeräte lassen sich zu einem Speicherbereich, dem **Storage Area Network** SAN zusammenfassen. In [Abb. 6.26](#) ist ein solches Netz gezeigt.

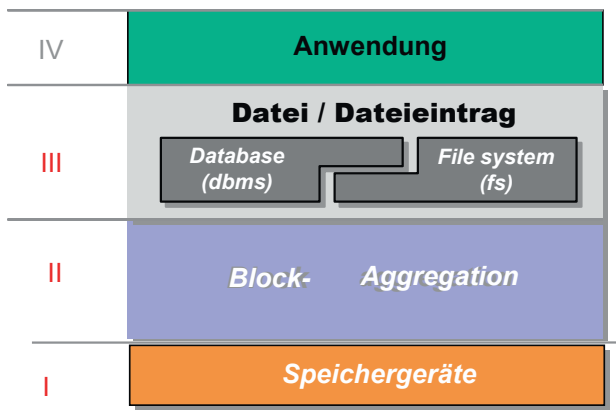
Das SAN-Konzept fasst alle Speicher im Netz (beispielsweise unternehmensweit) zu einem virtuellen Massenspeicher zusammen. Dabei werden zwei Ebenen unterschieden: Die Blockebene, bei der einzelne Speicherblöcke zu logischen, größeren Blöcken zusammengefasst werden, und der Dateiebene, bei der für jede Datei die virtuelle Speicheradresse, der Dateiname, die Zugriffsrechte usw. verwaltet werden („Metadaten“). Die für die Verwaltung, Wartung und Konfiguration dieses virtuellen Speichers nötigen Programme lassen sich als Betriebssystem des virtuellen Speichers auffassen.

Die verschiedenen Ebenen können wir mithilfe eines Schichtenmodells in der Funktionalität besser fassen als mithilfe eines Verbindungsgraphen. In [Abb. 6.27](#) ist eine solche SAN-Architektur gezeigt, wie sie von der *Storage Network Industry Association* SNIA vorgeschlagen wurde.

**Abb. 6.26** Ein Speicher-  
netzwerk



**Abb. 6.27** Das SAN Schich-  
tenmodell nach SNIA.org



Die Vorteile eines spezialisierten Speichernetzes auf Netzebene sind klar:

- Ein spezialisiertes Speichernetz stört nicht den normalen Datenverkehr
- Trotzdem wird die Flexibilität eines allgemeinen Netzwerks behalten, das ermöglicht, Geräte unterschiedlicher Hersteller zu integrieren („IT-Konsolidierung“)
- Eine zentrale Sicherheitsverwaltung vereinheitlicht Authentifikation, Zugriffskontrolle und Verschlüsselung, etwa um Datendiebstahl im Netz unmöglich oder unwirksam zu machen.
- Die Datenströme können besser der gewünschten Anforderung (*Quality of Service*) angepasst und ihre Topologie zentral konfiguriert werden.
- Die Netzüberwachung für Ereignisbehandlung, Wartung und Diagnose wird vereinfacht.
- Es ermöglicht ein zentrales Speichermanagement.



Besonders der letzte Punkt ist interessant. Das Speichermanagement beinhaltet nämlich

- Eine effiziente Lastverteilung und Kapazitätsmanagement, etwa durch die Regruppierung von Speicherclustern oder der Migration von Daten zwischen Bandlaufwerken und Plattenlaufwerken.
- Management der Datensicherung (Kopien, Backup, Rollback und *Recovery* im Schadensfall)
- Konfiguration und Erweiterung (*upgrade*) der Einzelkomponenten

Üblicherweise sind die Dateiserver, die mit virtuellen Speicheradressen, also Speicherblockadressierung (Block-I/O) auf die Speichermedien zugreifen, durch schnelle Datenleitungen verbunden, die mit einem schnellen Protokoll versehen sind. Üblicherweise werden dazu Glasfaserleitungen (*Fiber Channel FC*) mit dem FC-Protokoll verwendet.

Es gibt aber auch Nachteile des SAN: Durch die Verteilung der Funktionen auf mehrere Rechner ist das Zusammenspiel der Komponenten mehrerer Hersteller ziemlich komplex. Benutzt eine Softwarekomponente spezielle, in Hardware implementierte Funktionen, etwa *file copy* oder *backup*, so können die Bausteine anderer Hersteller dafür nicht verwendet werden und führen zu Problemen. Es ist in diesem Fall besser, bei SAN-Komponenten nur auf einfachen, einheitlichen Funktionen aufzusetzen und alle „Optimierungen“ zu vermeiden.

Im Gegensatz zum SAN-Konzept kann man die Speichereinheiten auch zentral bei einem einzigen Server anordnen, etwa als RAID-System, und die FC-Verbindungen des Speichernetzes nur auf diesen Server beziehen. In Abb. 6.28 ist ein solcher **Network Attached Storage NAS** gezeigt und seine Einordnung im Schichtenmodell.

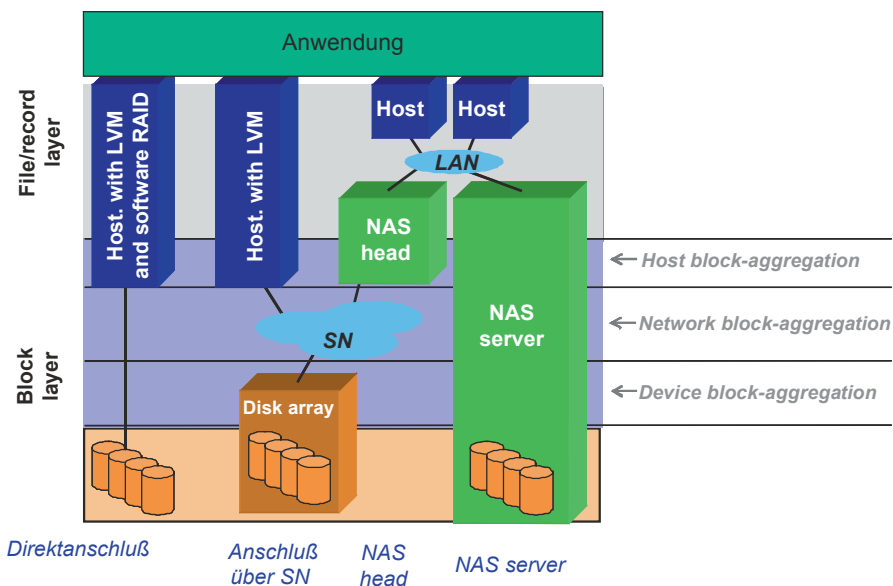


Abb. 6.28 Das NAS-Modell

In diesem Fall garantiert das Speichernetz einen schnellen Zugriff auf die Daten und der Server (*host*) eine Netzwerkanbindung. Ein solcher NAS enthält neben dem Speichernetz alle Funktionen des Speichermanagers wie Zugriffsverwaltung, Backup, Rollback usw. die sonst im SAS existieren. Wie in Abb. 6.28 gezeigt, lässt sich die Schicht „Blockaggregation“ aus Abb. 6.27 unterteilen in drei Unterschichten: Die Aggregation der Blöcke mehrerer Geräte auf Geräteebe, die Aggregation mehrerer Geräte auf Netzwerkebene und die Aggregation von Blöcken im Betriebssystem auf Hostebene. Abbildung 6.28 zeigt dabei mehrere mögliche Systeme nebeneinander: Ein konventionelles System, bei dem die Geräteansteuerung direkt vom Rechner ausgeht, ein System, bei dem ein Netzwerk zur flexiblen Konfiguration zwischen Rechner und Speichermedien geschaltet ist und ein NAS-Server, der diese unteren Schichten bereits enthält. Der NAS-Betriebsteil ohne Speicher kann auch als Extrarechner (NAS-Head) ausgelagert sein.

Wird ein NAS innerhalb einem SAN als ein Speichermedium unter vielen betrieben, so kann man es als ein „SAN im SAN“ ansehen oder ein „SAN-in-a-box“.

---

## 6.5 Arbeitsmodelle im Netz

Im Unterschied zu Multiprozessorsystemen, die eine leichte Arbeitsaufteilung von parallel ausführbarem Code und den Prozessoren gleichen Typs ermöglichen, sind vernetzte Computer mit verschiedenen Problemen konfrontiert.

- Die Übermittlung von Daten übers Netz kostet Zeit, so dass sich nur Codestücke zur parallelen Abarbeitung lohnen, die keine oder wenig Kommunikation haben.
- Auch die Verlagerung von Code „außer Haus“ ist nur sinnvoll, wenn es sich um größere Arbeitspakete handelt, also Aufwand und Ertrag in vernünftigen Verhältnis zueinander stehen.
- Die Netze sind meist inhomogen aus Computern verschiedener Hersteller und verschiedenen Betriebssystemen zusammengesetzt. Ein lauffähiges Programm auf einem Rechner muss deshalb nicht auf allen Rechnern im Netz ablauffähig sein.

Ein Jobmanagementsystem, das den Wahlspruch „Nicht der Einzelrechner, sondern das Netz ist der Computer“ verwirklichen will, muss deshalb auf die obigen Probleme ganz konkret eingehen.

### 6.5.1 Jobmanagement

Es gibt verschiedene Jobmanagementsysteme, die es erlauben, ungenutzte Rechenzeit in Workstation-pools zu nutzen. Derartige Systeme werden besonders in Luft- und Raumfahrt, Automobil- Entwicklungsabteilungen usw. genutzt, wo viele hochwertige Rechner versammelt sind. Die Anforderungen an ein solches *Load Sharing Facility*-System sind folgende:

- Die *Lastverteilung* für alle Jobarten und Leistungsklassen soll ausgewogen sein.
- Für unterschiedliche Anforderungen an Betriebsmittel wie Rechenzeit und Speicherbedarf sollen verschiedene *zentrale Warteschlangen* geführt werden.
- Die *Jobdurchlaufzeiten* sollen optimiert werden.
- Die *Lizenzen* von Programmen sollen transparent (Workstation-unabhängig) verwaltet werden.
- Trotz der zusätzlichen Belastung soll der *normale, interaktive Betrieb* der Workstations störungsfrei möglich sein.
- Die Workstations sollen im Lastverbund möglichst in der *Nacht*, an Wochenenden und im Urlaub genutzt werden können.
- Die Gesamtkonfiguration soll übersichtlich sein und *leicht gewartet* werden können.

Diese Forderungen sind nicht einfach zu erfüllen, da ein Jobmanagementsystem nicht auf alle Faktoren Einfluss hat. So kann es zwar mit einer guten Benutzeroberfläche und einfachen Konfigurationsmechanismen die Wartung des Systems vereinfachen, durch *batch*-Möglichkeiten die Nacht- und Feiertagsstunden nutzen und über Netzwerkkommunikation Jobs auf Rechner verschieben, aber die Zuordnung von Jobs zu Rechnern ist nicht beliebig. So sind nicht nur Speicherbedarf, Prozessortyp, Betriebssystem und lokaler Plattenbedarf für temporäre Dateien zu beachten, sondern die Lizenzen mancher Programme sind auch an ganz bestimmte Rechner gebunden (*node locking*) oder in der Gruppe nur auf einer bestimmten, maximalen Anzahl von Rechnern gleichzeitig ausführbar (*floating licence*).

Auch die Priorität der verlagerten Jobs muss so gewählt sein, dass der eigentliche Benutzer des Rechners immer Vortritt hat und deshalb die zusätzliche Nutzung „seines“ Rechners nicht bemerkt.

## 6.5.2 Netzcomputer

Ein völlig anderes Konzept für verteilte und vernetzte Systeme stellt das Netzcomputerkonzept dar, das von der Firma Oracle, Sun, IBM, Apple usw. entwickelt wurde. Enthält ein „normaler“ Rechner noch alle Betriebssystemkomponenten selbst, um autonom mit Platten, Drucker etc. arbeiten zu können, so sind in Systemen mit verteiltem Betriebssystem die verschiedenen Betriebssystemfunktionen als spezielle Server (Dateiserver, Prozessserver etc.) auf spezialisierte Rechner verlagert.

Dieses Client-Server-Konzept eines verteilten Betriebssystems wird nun beim Netzcomputer NC auf die Spitze getrieben: Außer einem Hauptspeicher, einem Prozessor, einem Netzanschluss und einem Bildschirm enthält er nichts weiter. Selbst die Software des Betriebssystems (Dienstprogramme, BS-Kern usw.) ist auf einen Mikrokern, bestehend aus dem nackten BS-Kern mit den Schichten des Netzanschlusses sowie den Funktionen zum Laden von Programmen, reduziert. Alle Programme, die sonst auf der lokalen Festplatte liegen, kommen aus dem Netz; selbst das Betriebssystem kann über einen fest eingebauten

ROM-*bootstrap loader* beim Einschalten des Geräts übers Netz geladen werden. Alle Leistungen wie Speicherung von Dateien, Druckservice usw. werden von speziellen Servern vollbracht.

Die Managementvorteile eines solchen Mikrokernkonzepts gegenüber den üblichen Netzen aus PCs in Firmen liegen auf der Hand:

- *Aktuelle Dateien*

Durch die zentrale Wartung der benutzten Daten sind diese immer aktuell und konsistent. Dies bezieht sich sowohl auf die Teile des Betriebssystems, die über das Netz geladen werden (Treiber, Dienstprogramme usw.) als auch auf die Benutzerprogramme und die verwendeten Dateien.

- *Billigere Hardware*

Der NC benötigt weniger Hardware, da bestimmte Betriebssystemteile und lokaler Plattenplatz für die Dienstprogramme entfallen. Dies macht den NC billiger in der Anschaffung.

- *Billigere Wartung*

Die Pflege der Systemsoftware, die Lizenzvergabe und die Konfiguration auf dem NC werden nur einmal zentral durchgeführt und gepflegt. Dies macht den NC billiger in der Wartung, was nicht unerheblich ist: Man rechnet für die Software- und Konfigurationspflege eines PCs den gleichen Betrag pro Jahr wie für den Kauf.

Auch die Hardwarereparatur ist einfacher, da die Computerteile beliebig ausgetauscht werden können: Alle Benutzerdateien und -profile liegen auf dem Zentralrechner und gehen dabei nicht verloren.

- *Höhere Datensicherheit*

Durch die zentrale Datensicherung kann bei einem Rechnerausfall die Arbeit auch bei unzuverlässigen oder vergesslichen Mitarbeitern weitergehen. Auch der Datendiebstahl ist ohne Peripherielaufwerke nicht mehr so einfach möglich, ebenso wie das Installieren von Spielen oder unabsichtliche Verbreiten von Viren.

- *Bessere Ausnutzung von Ressourcen*

Neben der besseren Nutzung von Druckern, Fax-Anlagen usw., die allgemein durch die Anbindung an das Netzwerk möglich wird, ist speziell beim NC die bessere Ausnutzung der Massenspeicher im zentralen *pool* möglich. Die notwendigen Erweiterungen können sich besser am jeweiligen Bedarf orientieren.

Aber auch die Nachteile eines solchen Konzepts sind deutlich:

- *Erhöhter Netzaufwand*

Laufen alle Applikationen über das Netz, so müssen das firmeninterne Netz (*Intranet*) und die dabei verwendeten Server in der Leistung (Durchsatz und Speicherkapazität) deutlich gesteigert werden. Dies kostet zusätzlich Geld.

- *Erhöhter Pufferaufwand*

Fügt man zur Pufferung von Daten und häufig benötigten, großen Programmen noch zusätzliche Hardware (Hauptspeicher, Massenspeicher) in den NC ein, um die Ladezeiten

und die Netzbelastung klein zu halten, wie dies schon bei den UNIX-Grafikschirmen (X-Terminals) nötig wurde, so werden die Kostenvorteile des NC wieder relativiert.

- *Bevormundung der Benutzer*

Durch die zentrale Wartung der Software und das Management der Hardware fühlen sich die Anwender durch die EDV-Zentrale wie früher bevormundet: Nur die Zentrale entscheidet, welche Programme benutzt werden können und wieviel Speicherplatz dem einzelnen zugewiesen wird.

- *Eingeschränkter Datenschutz*

Befindet sich der Server nicht innerhalb der Firma oder Arbeitsgruppe, sondern außerhalb bei einem Dienstleister, so können Personal- oder Firmendaten leicht dort abgegriffen werden, ohne Schutzmöglichkeit durch die eigene Firma. Ist durch Unachtsamkeit, Korruption oder Angriff durch Schadsoftware (Viren etc.) beim Server der Datenschutz ausgehebelt, liegen alle Daten der Arbeitsgruppe oder Firma frei zur Verwertung vor. Durch die zentrale Speicherung sind Wirtschaftsspionage und Erpressung durch Verschlüsselung aller Daten (*Ransomware*) leichter möglich.

Die Akzeptanz und damit die Zukunft der Netzcomputer ist deshalb zurzeit noch ungewiss. Ein kommerzielles Beispiel dafür ist das bekannte *Chromebook* von Google, das auf *Chrome OS* beruht, einer Entwicklungsvariante des quelloffenen *Chromium*-Betriebssystems. Es besteht im Wesentlichen aus einem Webbrowser, der auf einem Linux-Kern läuft. Alle Anwendungen und Nutzerdaten befinden sich auf Servern der Firma Google.

Eine interessante Alternative ist zweifelsohne ein Computernetzwerk, das jedem seinen individuell konfigurierten Arbeitsplatz gestattet, aber durch zentrale Wartung für die Standardprogramme die meisten Konfigurationsprobleme am Arbeitsplatz verhindert und für aktuelle Dateien sorgt, siehe [Abschn. 6.5.3](#).

## Java-Applets

Ein deutliches Problem in diesem Konzept ist die Inkompatibilität der Hardware und Software bei verschiedenen Rechnern verschiedener Hersteller. Um dieses Problem zu umgehen, wird eine einheitliche Programmiersprache benutzt: Java. Die objektorientierte Sprache Java (Sun 1997) ähnelt in der Syntax sehr C++, hat aber als Vereinfachung keine Zeiger, keine Mehrfachvererbung und ein verbessertes Schnittstellenkonzept.

Jeder NC erhält als Programm die Folge von Maschinenbefehlen einer virtuellen Maschine: der **Java Virtual Machine**, die von der Firma Sun, den Entwicklern von JAVA, spezifiziert wurde. Aufgabe des NC-Prozessors (und aller anderen Maschinen, die den JAVA-Code ausführen sollen) ist es nun, diese virtuelle Maschine zu emulieren. Die Aufgabe eines Betriebssystems eines solchen NC besteht darin, die Ablaufumgebung für über das Netzwerk geladene Java-Programme (**applets**) bereitzustellen.

Insbesondere ist dies

- die **Hauptspeicherverwaltung** durch automatische Speicherreservierung für Objekte sowie Aufspüren und Beseitigen von gelöschten bzw. nicht benutzten Objekten (*garbage collection*)

- die **Isolation** verschiedener, gleichzeitig ablaufender Programme voneinander und zum restlichen System. Um dies schnell und einfach zu ermöglichen, gibt es in Java keine Zeiger und Adressen, und die Aktionsmöglichkeiten der Netzprogramme und Applets auf dem NC sind sehr beschränkt.
- die **Interpretation** des Byte-Code der *Java Virtual Machine*, falls sie nicht direkt emuliert wird. Insbesondere müssen die elementaren JAVA-Datentypen auf die Wortlänge der Zielmaschine angepasst werden.
- die Bereitstellung der **Standardfunktionen** für Grafik, Ein- und Ausgabe, soweit dies dem Netzprogramm oder Applet überhaupt gestattet ist. Generell haben Netzprogramme keinen Zugriff auf lokale Platten, Drucker etc.

Die Notwendigkeit einer einfachen, sicheren Programmiersprache für Netze ergab sich bei der explosiven Ausbreitung und Anwendungen der Hypertextsysteme im World Wide Web (WWW). Vielen Anbietern reichte die Funktionalität einer einfachen Textseite nicht mehr aus. Anstatt aber immer neue Spezialfunktionen in die Hypertext-Präsentationsprogramme (*web browser*) aufzunehmen, ist es besser, den Code für eine Spezialfunktion vom Anbieter direkt zu laden und sie lokal durch einen im *web browser* integrierten Java-Interpreter auszuführen. Der Java-Code erfüllt diese Anforderungen durch die normierte, festgelegte Maschinenbefehlsspezifikation und die normierte, standardisierte Laufzeitbibliothek. Dies ermöglicht den *web browsern* auch, ihre eigentlichen Funktionen flexibler zu erfüllen – neue, unbekannte Protokolle und Funktionen können so durch Laden spezieller *Java content handler* installiert werden, ohne den *browser* neu schreiben zu müssen.

Der Nachteil eines solchen Ansatzes liegt darin, dass durch die begrenzten Möglichkeiten eines Applets keine großen Aufgaben bewältigt werden können. Alle Dateizugriffe, temporäre Dateien und andere lokalen Ressourcen sind aus Sicherheitsgründen verboten – und beschränken damit die Applets auf ihre Rolle als kleine Hilfsfunktionen für Browser etc.

### 6.5.3 Schattenserver

Eine häufige Konfiguration eines Netzdateisystems besteht aus einem zentralen Dateiserver und vielen Satellitenrechnern, also PCs, Laptop-Computer oder andere Kleincomputer wie Notebooks oder Organizer. Bei dieser Konfiguration, in der die Satellitenrechner eigene, nur lose gekoppelte Dateisysteme besitzen, ist die konsistente Aktualisierung aller Dateien und Programme im System ein ernstes Problem. Dies tritt meist in zweierlei Formen auf:

- *Roaming*  
Manche Benutzer, beispielsweise Außendienstmitarbeiter, Telearbeiter oder Mitarbeiter einer Firma, die zwischen Arbeitsgruppen an verschiedenen Geschäftsstellen wechseln, müssen Dokumente und Dateien ohne dauernde Netzverbindung zur Zentrale

editieren und beim Aufbau der Netzverbindung ihre neue Dateien überspielen sowie eigene aktualisieren. Dabei verlangen diese Benutzer überall die gleiche Arbeitsumgebung, egal wo sie andocken.

- *Wartung*

Jeder Computernutzer möchte einerseits alle möglichen Programme nutzen und Daten erzeugen, andererseits aber für Konfigurationspflege, Programmaktualisierung und Datensicherung möglichst keinen Aufwand betreiben. Die Administration von Rechnernetzen verursacht aber viele Kosten: man rechnet ungefähr den Anschaffungspreis pro Jahr pro PC.

Für die Lösung der Problematik kann man zwischen zwei Konzepten unterscheiden:

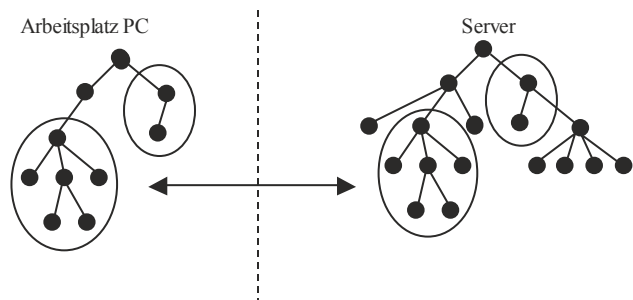
- *Zentrale Aktualisierung*

Üblicherweise hat der Netzwerkmanager die Möglichkeit, aktiv die Konfiguration eines am Netz hängenden Computers zu verändern. Dies erfordert aber das Wissen um eine nötige Veränderung sowie die benötigten Daten für jeden einzelnen Computer. Aus diesem Grund sind jedem Netzwerkmanager inhomogene Systeme aus unterschiedlicher Hardware und Software ein Graus und Quelle ständigen Ärgers.

- *Dezentrale Aktualisierung*

Statt dessen kann auch jeder einzelne Arbeitsplatzcomputer neben seinen speziellen Programmen und Nichtstandard-Dienstleistungen einen vom Benutzer nicht beeinflussbaren Teil (Dateisystem) enthalten, der regelmäßig (z. B. nach dem Anschalten des Rechners oder in festen Zeitabständen) mit der Konfiguration bzw. den Datenbeständen eines oder mehrerer, benutzerspezifisierten Server automatisch durch ein besonderes Programm (Dämon, Agent usw.) abgeglichen wird. Die benötigten Dateibestände und Server (Programmserver, Objektserver) werden dazu einmal beim initialen Einrichten des Rechners festgelegt. Der Rechner arbeitet also damit wie ein Netzcomputer, der große Teile des Betriebssystems und der benötigten Anwenderprogramme im Cache hält, um die Zugriffszeit zu senken. Der Agentenprozess der Aktualisierung hat dabei die Funktion eines Cache-Snoopers (siehe [Abschn. 3.5](#)), der den Cache aktuell hält. Der Arbeitsplatzcomputer ist also „wie ein Schatten“ des eigentlichen Dateiservers, wobei nur die tatsächlich benutzten Dateien und Verzeichnisstrukturen auf dem Arbeitsplatzcomputer vorhanden sind. In [Abb. 6.29](#) ist ein solches Konzept gezeigt. Der gestrichelt umrandete

**Abb. 6.29** Das Konzept des Schattenservers (*shadow server*)



Teil des Dateisystems auf dem Server wird direkt auf den PC abgebildet durch PC-Server-Kommunikation konsistent gehalten. Wird eine Datei auf einem der beiden „Spiegel“ verändert, so muss sie möglichst rasch auf den anderen kopiert werden. Ebenso müssen Löschanforderungen oder Dateiumbenennungen immer bei beiden durchgeführt werden.

Ein solcher Ansatz hat viele Vorteile:

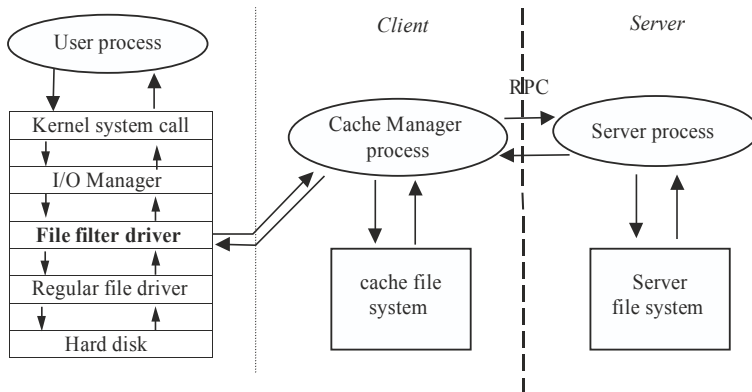
- *Aktuelle Versionen*  
Es sind immer alle aktuellen Versionen verfügbar, ohne dass der Benutzer sich darum kümmern muss oder der Administrator sie ihm überspielt.
- *Datensicherung*  
Die Benutzerdaten werden automatisch auf dem Server gesichert.
- *Netzunabhängigkeit*  
Bei Ausfall des zentralen Servers oder bei Netzwerksstörungen kann (im Unterschied zum NC!) jeder netzunabhängig weiterarbeiten.
- *Adaptive Konfiguration*  
Die sich zeitlich ändernde Konfiguration der Benutzerprogramme ist immer auf dem neuesten Stand; die auf dem Rechner existierenden Programme sind auch tatsächlich die benötigten.
- *Rechnerunabhängige Konfigurationen*  
Im Unterschied zur zentralen Wartung muss beim Wechsel eines Benutzers zu einem anderen Arbeitsplatz (anderen Rechner) die neue Konfiguration nicht aufgespielt werden, sondern stellt sich automatisch ein.
- *Geringere Wartungskosten*  
Sowohl die Softwarewartung, die zentral und damit kostengünstig abgewickelt werden kann, als auch die Hardwarewartung, die ein unterschiedliches Spektrum an Rechner einsetzen und ersetzen kann, werden preiswerter.

### Beispiel Linux

Ab Version 2.2 gibt es bei der freien Unix-Variante LINUX die Möglichkeit, das Dateisystem „Coda“ zu benutzen (CODA 1999). Das Coda-Dateisystem (Satyanarayanan et al. 1990) ist ursprünglich für das *mobile computing* an der Carnegie Mellon University geschaffen worden und verwendet eine Architektur ähnlich der in Abb. 6.30.

Die Implementation benutzt wie beim Netzdateiserver in Abb. 6.23 einen Betriebssystemtreiber, der alle Dateiaufrufe (CreateFile, OpenFile, CloseFile, RenameFile usw.) abfängt und umleitet. Dieser spezielle Dateisystemtreiber hängt im Kern und leitet die Dateiaufrufe (*File System Calls*) aus dem *kernel mode* an einen Prozess („Venus Cache Manager“) im *user mode* um. Dieser Prozess sieht nach, ob die gesuchte Datei im lokalen Dateisystem (Coda-Dateisystem, implementiert mittels eines Unix *raw device special file*) vorhanden ist oder nicht. Wenn ja, so wird die Information über Blocknummer usw. der lokalen Platte normal ausgelesen und wieder an den *user process* zurückgegeben. Dieser merkt nichts von der Umleitung; er erhält direkt die gewünschte Datei.





**Abb. 6.30** Struktur des Coda-Dateisystems

Anders dagegen, wenn die Datei nicht vorhanden ist oder als „verändert“ markiert wurde. In diesem Fall wendet der Manager sich mittels RPC an den Server und fragt nach der neuesten Version, die dann kopiert wird. Erst die Blockangaben der kopierten Datei werden sodann über den Filtertreiber an den Benutzerprozess zurückgegeben.

Man beachte, dass zwar alle Veränderungen bei beiden Partnern notiert werden, alle Dateien aber erst dann kopiert werden, wenn sie wirklich gebraucht werden (*lazy evaluation*).

Die gesamte Coda-Software enthält übrigens noch weitere Komponenten wie fehler-tolerante Backup-Server. Auch hier wirkt wieder das Schatten- oder Spiegelprinzip: Ist ein Server ausgefallen und das Dateisystem per Hand völlig neu initialisiert worden, so bemerkt das Coda-System daraufhin die Inkonsistenz beider Server und bringt den ausgefallenen automatisch auf den neuesten Stand.

Die Idee der Schattenserver wird übrigens auch für NFS genutzt, etwa in Linux, bei dem der NFS-Server durch einen transparenten Dateisystem-Cache (*CacheFS*) entlastet wird. Hier werden Dateien vom NFS-Server in einem lokalen, separaten Dateisystem gehalten.

### Beispiel Windows

In Windows NT 5, genannt Windows 2000 gibt es verschiedene Mechanismen, die ein solches Konzept unterstützen. Neben dem Konzept des aktiven Verzeichnisses (*active directory*), das alle Dateiänderungen im gesamten Netz von unten nach oben propagiert und damit das gesamte hierarchische Dateisystem aktuell hält, stützen sich die Kopie-mechanismen des Schattenservers auf ein „IntelliMirror“ genanntes Produkt, das durch Konsistenzmechanismen die lokalen und globalen Dateisysteme beim *roaming* abgleicht.

Seit Windows NT 5.1, genannt Windows XP, ist dieses Konzept allgemein zugänglich und wird als „Offline-Aktualisierung“, bezeichnet. Dazu werden alle zu synchronisierenden

Dateien bzw. Ordner, die vom Server im Netz freigegeben sind, beim Client in einen speziellen Ordner „Offline-Dateien“ aufgenommen. Loggt sich der Client-Benutzer ein oder aus, so kann eine Synchronisierung aller Objekte des Offline-Ordners erfolgen. Je nach Optionen, die bei der Einrichtung der Synchronisation einmal festgelegt werden, überschreibt die neue Client-Datei die alte Datei gleichen Namens auf dem Server, oder sie wird selbst von der alten Server-Datei überschrieben, oder die neue Datei wird mit einer neuen Versionsnummer neben die alte auf dem Server gespeichert. Die jeweilige Politik kann der Benutzer entweder bei jedem Konfliktfall neu auswählen oder einmal für immer festlegen.

### 6.5.4 Aufgaben zu Arbeitsmodellen im Netz

#### Aufgabe 6.5-1 Offlinedateien

- a) Geben Sie einen Ordner auf einem beliebigen Gerät via SMB-Protokoll als Netzlaufwerk frei (z. B. als Windows-Dateifreigabe oder unter Linux mit Samba) und speichern Sie eine Textdatei in diesem. Verwenden Sie nun ein anderes Gerät. Dieses sollte Windows als Betriebssystem verwenden und Schreibrechte für das Netzlaufwerk haben. Machen Sie den Ordner des Netzlaufwerkes auf diesem Gerät als Offline-datei verfügbar. Trennen Sie anschließend die Verbindung zum Netzwerk, ändern Sie den Inhalt der Datei und speichern Sie diese. Stellen Sie anschließend die Netzwerkverbindung wieder her und starten Sie Ihren PC neu.

Wie hat sich der Inhalt der Datei auf dem Netzlaufwerk verändert?

*Hinweise:*

- Die Verwaltung der Offlinedateien finden Sie unter Systemsteuerung/*Alle Systemsteuerungselemente/Synchronisierungscenter/Offlinedateien verwalten*
  - Sollte Ihnen nur ein einziger PC zur Verfügung stehen, so können Sie mit Virtual-Box einen zweiten PC simulieren. Hierzu müssen Sie den virtuellen PC mit einer Netzwerkbrücke verbinden. Die zugehörige Option finden Sie unter *Ändern/Netzwerk/Angeschlossen an*.
- b) Welchem aus diesem Kapitel bekannten Prinzip entspricht das in a) dokumentierte Verhalten bzw. die Funktionsweise von Offlinedateien unter Windows? Erklären Sie die Funktionsweise dieses Prinzips!

#### Aufgabe 6.5-2 Schattenserver und Netzwerkcomputer

- a) Wie funktioniert ein Netzwerk mit Schattenserver (SS), und wie ein Netzwerk mit Netzwerk-Computern NC?
- b) Was unterscheidet einen Client im SS-Netz von einem Client im NC-Netz funktionell, hardware- und softwaremäßig?
- c) Welche Vorteile bietet die jeweilige Konfiguration?

## 6.6 Middleware und SOA

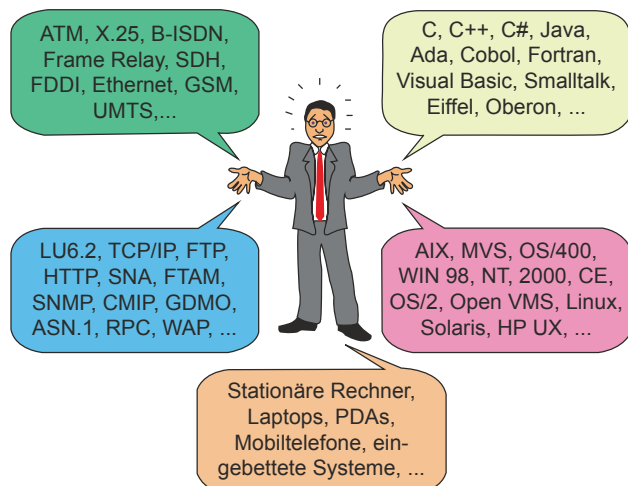
In vielen Firmen habt sich im Laufe der Jahre eine bunte Mischung aus diversen Rechnerfabrikaten und -formen, Anwendungen aus verschiedenen Programmiersprachen und Kommunikationsnormen und Netzkomponenten verschiedener Hersteller angesammelt und stellen ein massives Problem dar.

- Sie können nicht zusammenarbeiten, da sie unterschiedliche Schnittstellen haben: Datenformate, Befehle und Funktionen sind inkompatibel.
- Sie ergänzen sich nicht, da die Funktionalität nicht aufeinander abgestimmt ist meist Überschneidungen oder Lücken existieren.
- Sie können nicht einfach weggeworfen werden, da meist die Alltagsarbeit darauf beruht.
- Sie sind schwer wartbar, da kaum Dokumentation oder Quelltexte existieren.

Dies ist in [Abb. 6.31](#) visualisiert. Was kann man tun, um dieses kostspielige Chaos zu beseitigen?

Eine mögliche Lösung für eine IT-Konsolidierung liegt auf der Hand und wird gern von Herstellerfirmen propagiert: Alles aus einer Hand. Diese Lösung ist möglich, wenn man ganz neu anfängt, aber bei einer Firma, die im laufenden Betrieb „das Rad wechseln“ muss, schafft eine solche Monokultur neue Probleme: Neben der gefährlich starken Abhängigkeit von einem einzigen Hersteller und seiner Modell- und Lizenzpolitik ist eine solche „Rundumerneuerung“ auch nicht konfliktfrei, ist teuer und dauert lange, meist zu lange.

**Abb. 6.31** Heterogenität als Problem



6.6.1 Transparenz durch Middleware

Es gibt aber noch eine zweite Lösung: Statt einheitlichen Komponenten fordert man nur ein reibungsloses Zusammenspiel, die **Interoperabilität**, der Komponenten. Trotz unterschiedlicher Hardware, unterschiedlicher Betriebssysteme, unterschiedlicher Kommunikationsprotokolle und unterschiedlicher Programmiersprachen kann durch Einbau einer Vermittlungsschicht, der sog. Middleware, zwischen Anwendung und Ziel ein Zusammenwirken erreicht werden. In Abb. 6.32 ist dies am Beispiel von kommunizierenden Applikationen visualisiert, die unterschiedlich auf verschiedene Datenbanken zugreifen wollen.

Rein technisch gesehen hilft hier wieder unser Schichtenmodell mit seinen virtuellen Maschinen: Die Middleware verhält sich für jede der mögliche Anforderungen wie die gewünschte Datenbank und das gewünschte Protokoll, siehe Abb. 6.33.

Die Anwendung muss auf keine Transportprotokolle oder andere Dinge Rücksicht nehmen; alle notwendigen Einstellungen werden in der Middleware vorgenommen. Umgekehrt sieht die Datenbank dadurch nur noch Standardapplikationen; alle abweichenden Zugriffsarten

Abb. 6.32 Middleware als Vermittlungsschicht

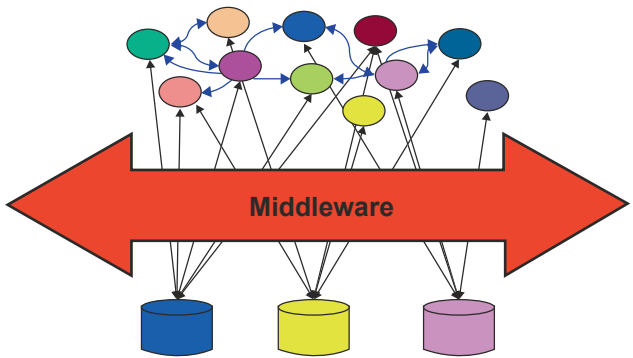
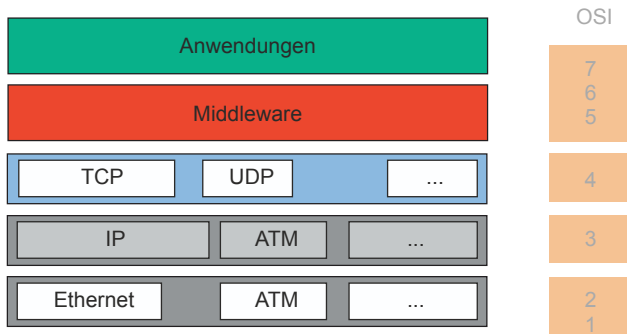
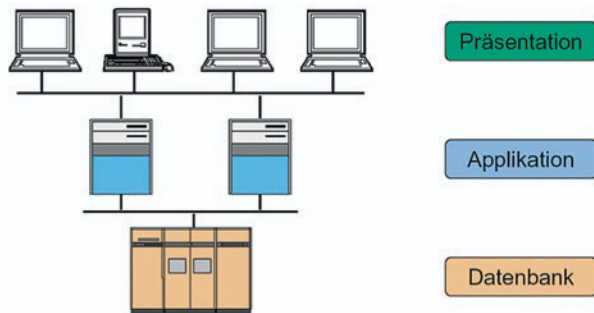


Abb. 6.33 Schichtung der Middleware



**Abb. 6.34** Beispiel Middleware: SAP/R3 3-tiers Architektur



und – protokolle werden von der Middleware verdeckt. In [Abb. 6.34](#) ist die dreigegliederte Architektur von SAP/R3 gezeigt, bei der Applikationen die Aufgabe übernommen haben, zwischen Präsentationsprogrammen und den dazu nötigen Daten zu vermitteln.

### 6.6.2 Vermittelnde Dienste

Neben proprietären Middleware-Diensten einzelner großer Herstellerfirmen gibt es auch systematische Ansätze und Architekturen, die Erstellung von Middleware zu vereinfachen und billiger zu machen. Einer der ersten Ansätze war die Konzeption einer allgemeinen, objektorientierten Dienstarchitektur durch die *Object Management Group* OMG 1989 mit ca. 800 Mitgliedern. Aufgabe der beteiligten Prozesse sollte es sein, alle Anfragen an Drucker, Datenbanken usw. so zu vermitteln, dass der anfragende Prozess nicht wissen muss, welcher Drucker im Pool noch Kapazität frei hat und ausreichend Toner besitzt, sondern nur, was für eine Art von Druck er will. Der Dienstvermittler (Broker) nimmt den Auftrag entgegen, verpackt ihn mit seinen Parametern, schickt ihn an den betreffenden Service, nimmt die Antwort (Ergebnis) entgegen und gibt es an den betreffenden Auftraggeber zurück. Ein solcher **Object Request Broker** ORB ist sehr praktisch; allerdings ist es nicht einfach, einen solchen Dienstvermittler zu erstellen. Deshalb konzipierte das OMG eine allgemeine Architektur, die *Common ORB Architecture* **CORBA**, und führte eine Referenzimplementierung durch.

#### Beispiel Java-Technologie

Auch im Bereich der Programmiersprache Java wuchs die Erkenntnis, dass die dedizierte, anwendungsabhängige Client-Server-Architektur, die für die Programmiersprache Java als Bibliotheken zur Verfügung gestellt wird, nicht ausreicht. Deshalb wurde von der Fa. SUN als eine anwendungsunabhängige, dienstvermittelnde Schicht die Middlewarearchitektur **JINI** entwickelt. In [Abb. 6.35](#) ist ein Überblick gezeigt.

Jini ist eigentlich eine Menge von Schnittstellen (API), die die Java2-Plattform für verteilte Programmierung erweitert. Als Abstraktionsschicht ermöglicht sie dynamische

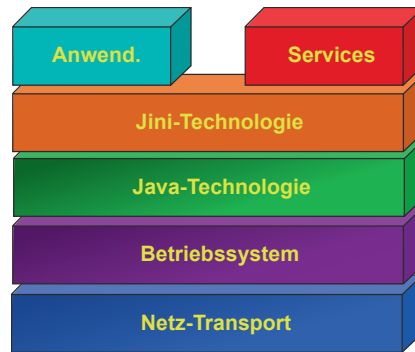


Abb. 6.35 Architekturschichten von Jini

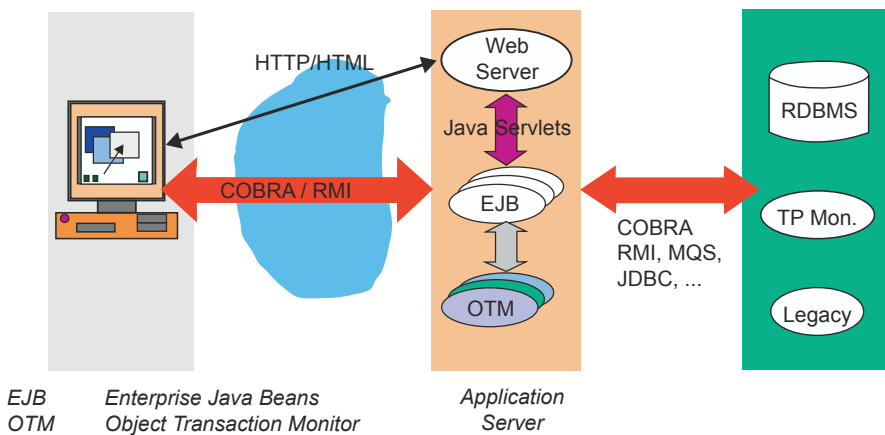


Abb. 6.36 Thin clients/Thick server durch Middleware

Systeme mit wechselnder Anzahl von Servern und Clients, die über den *namings*-Service sich registrieren, suchen und finden, und auch wieder entfernen können. Die Architektur baut auf RMI zur Kommunikation auf.

Als Anwendungsbeispiel ist in Abb. 6.36 ein *Thin client-Thick Server* Modell gezeigt, das die verschiedenen vorgestellten Techniken einsetzt, um datenbankbasierte Anwendungen in Middleware-technik zu realisieren.

#### Beispiel Windows NT

Unabhängig davon konzipierte Microsoft ein System zur Kopplung verschiedener Anwendungen, das Common Object Model **COM**. Es definiert einen binären, proprietären, aber sprachunabhängigen Standard für den Zugriff auf Objekte durch eine Tabelle von verfügbaren Funktionen (Methoden) eines Objekts. Allerdings stellte sich

bald heraus, dass in Netzen weitere Funktionalität nötig ist. Dies führte zur Entwicklung von Distributed COM **DCOM**, das mit der Microsoft Interface Definition Language MIDL programmiert werden muss und im Gegensatz zu CORBA nur *Remote Procedure Calls* zur Datenübermittlung im Netz benutzt.

Für die Anwendung von DCOM bei Datenbanken ist aber noch die Unterstützung von Transaktionen nötig. Deshalb integrierte Microsoft einen Transaction Server MTS in das COM/DCOM-System. Diese Middleware, genannt **COM+**, nimmt dabei noch zusätzliche Aufgaben wie Cachefunktionen der Datenbank, Lastbalancierung zwischen den Objekt-Servern und Unterstützung asynchroner Aufrufe (*Queued Components*). Portierungen von DCOM auf andere, nicht-Windowsbasierte Systeme (etwa auf Solaris und OS/390 durch EntireX) ermöglichen eine Interoperabilität auch zu anderen Betriebssystemen und Rechnern.

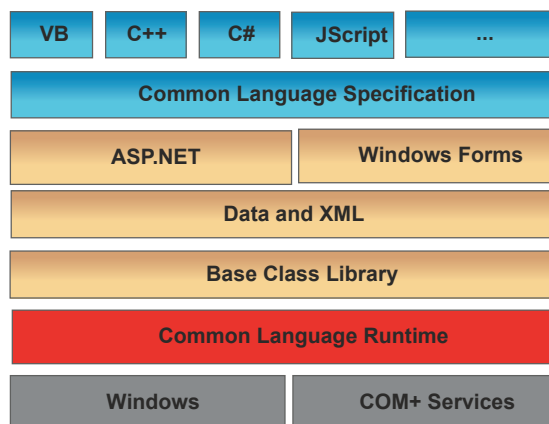
Das COM+-Modell ist noch stark von den verwendeten Sprachen und Kommunikationsmechanismen der Komponenten abhängig. Aus diesem Grund wurde das Modell zu einem allgemeineren Rahmenwerk **.NET** weiterentwickelt. Das .NET-Konzept beinhaltet im Wesentlichen ein Entwicklungssystem, das VisualStudio.NET, das die Komponentenentwicklung mit einer Vielzahl von Programmiersprachen ermöglicht und die dazu benötigten Bibliotheken und Laufzeitsysteme enthält. In [Abb. 6.37](#) ist ein Überblick der Architekturschichten gezeigt.

Zum Datenaustausch wird ein offenes Format, die Datenbeschreibungssprache XML, verwendet.

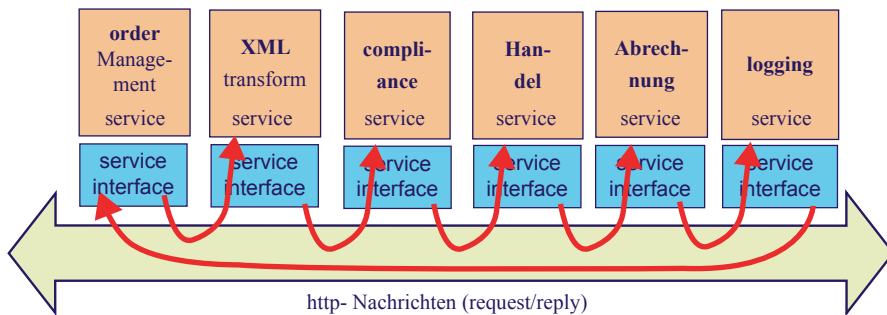
### 6.6.3 Service-orientierte Architektur

Wie anfangs erwähnt, existiert in größeren Betrieben eine Vielzahl von Anwendungen, die historisch entstanden und gewachsen sind und deren Anzahl mehrere Tausend

**Abb. 6.37** Gesamtschichtung der .NET-Architektur



### Asynchrone Abwicklung einer Bestellung



**Abb. 6.38** Ablauf einer Bestellung durch *enterprise service bus* (IBM)

umfassen können. Diese unterschiedlichen Komponenten haben Probleme beim Zusammenspiel, hohe Kosten bei Wartung und massive Problem bei notwendigen Änderungen und Neukonfigurationen.

Im vorigen Abschnitt sahen wir, dass die Konzeption einer Middleware hier neue Möglichkeiten bietet. Dieses Konzept kann man nun verallgemeinern: alle alten Applikationen werden durch eine Zwischenschicht gekapselt und neu zusammengeführt. Die Idee besteht darin, alle alten Applikationen über Zusatzteile auf einheitliche Schnittstellen zu bringen und sie in ein gemeinsames Rahmenwerk einzubinden. Hier wird das bewährte Konzept, unabhängige Softwaremodule flexibel über eine gemeinsame Schnittstellendefinition zusammenzuschalten, neu belebt und als „Service-orientierte Architektur“ (SOA) in Betriebe eingeführt.

Im Wesentlichen besteht es aus einer gemeinsamen Schnittstelle (z. B. „Enterprise Service Bus“ ESB, s. Keen 2004) sowie einigen mehr oder weniger genormten Regeln, wie die Servicedienste (Module) miteinander kommunizieren sollen (Krafzig 2004; Woods 2004). In Abb. 6.38 ist dies am Beispiel eines Bestellungsablaufs visualisiert.

Entscheidend für den Erfolg einer SOA-Architektur ist allerdings ein systematisches Vorgehen bei der Aufgabenverteilung unter den Diensten. Dazu muss zuerst ein Geschäftsmodell (Ablauf und Interaktion von Geschäftsprozessen) formuliert werden, deren fachliche Programmteile dann in modularisierter Form als SOA umgesetzt werden. Typisch und erforderlich für die Servicedienste ist eine Informationskapselung durch die Schnittstellendefinitionen, wie sie für abstrakte Datentypen üblich ist:

- Die Dienste implementieren bei fest definierten Funktionszusicherungen bestimmte Leistungen, ohne dabei anzugeben, auf welche Art dies geschieht.
- Ihr Aufruf wird durch einen für alle Module einheitlichen, losen (d. h. nachrichtenbasierten) Kommunikationsmechanismus (SOA-Protokoll SOAP) sichergestellt.



- Die Datenformate müssen einheitlich sein (z. B. XML) und ermöglichen so die Weiterverarbeitung, Dokumentation und Auditing.
- Obwohl die Dienste und Services unabhängig voneinander installiert, konfiguriert und gewartet werden, ist ihre Administration, also die Installation, Überwachung und Wartung, zentralisiert.

Der Nutzen einer solchen SOA auf Basis von Geschäftsprozessmodellierung („*Business Process Management*“ BPM) und Spezifikationen („*Business Process Execution Language*“ BPEL) ist vielfältig:

- Neue oder geänderte Geschäftsprozesse können schneller und damit preisgünstiger durch Kombination bestehender Dienste realisiert werden.
- Durch die Modularisierung, klare Aufgabentrennung und Funktionskapselung werden die Systeme beherrschbarer und leichter wartbar.
- Damit ist auch eine Auslagerung unwirtschaftlicher Teile an Fremdanbieter (*outsourcing*) wird durch die Modularisierung leichter möglich.
- Bewährte, ältere Systeme können weiter genutzt werden, ohne Neuentwicklungen zu blockieren oder sie mit Kompatibilitätsforderungen zu belasten (Investitionsschutz). Die älteren Einzeldienste können dann Stück für Stück durch moderne Versionen (z. B. Hardware-Software-Kombinationen) ersetzt werden.

---

#### Beispiel

Ein typisches Beispiel einer SOA ist die Nutzung Web-basierten Dienste über einen Browser (Web-orientierte Architektur WOA) [Erl04]. Die durch die Anfragen über das zustandslose Client-Server-Protokoll *http* beauftragten Dienste nutzen dann gemeinsame Standards für Kommunikation (*http*), Darstellung (z. B. *Xhtml*), Datenrepräsentation (z. B. *XML*), Sprachen (*Asynchronous Javascript and XML*: *Ajax*) und externe Dienste. Dies zielt allerdings eher auf die gemeinsame Darstellung verschiedener Datenquellen auf dem Bildschirm ab, weniger auf eine verzahnte SOA-Funktionalität, wie sie mit Hilfe von *CORBA* innerhalb einer Firma implementiert werden kann.

Beispiele für WOA sind *NetWeaver* von SAP, *WebSphere* von IBM oder *.NET* von Microsoft.

### 6.6.4 Aufgaben zu Middleware

---

#### Aufgabe 6.6-1 Middleware

- a) Angenommen, Sie haben 100 verschiedene Programme, die über ein Netzwerk auf einer von fünf verschiedenen Datenbanken arbeiten. Dabei wird jeweils eine bestimmte Netzwerkfunktionalität (Protokoll) von zwei möglichen vorausgesetzt. Wie

viele Programmversionen können Sie abdecken, wenn Sie dafür eine Middleware einführen und wie viele Versionen müssen damit nicht neu programmiert werden?

- b) Was sind die Unterschiede und die Gemeinsamkeiten von einem RPC-Aufruf und einer CORBA-Anfrage?

---

## Literatur

CODA: siehe <http://www.coda.cs.cmu.edu> im World Wide Web (1999)

Levy E., Silberschatz A.: Distributed File Systems: Concepts and Examples. ACM Computing Surveys 22(4), 322–374 (1990)

Satyanarayanan M., Kistler J.J., Kumar P., Okasaki M.E., Siegel E.H., Steere D.C.: Coda: A Highly Available File System for a Distributed Workstation Environment. IEEE Trans. On Computers 39(4), 1990

Sinha A.: Network Programming in Windows NT. Addison-Wesley, Reading, MA 1996

Sun: siehe <http://java.sun.com> im World Wide Web (1997)

Keen, M. et al.: Patterns: Implementing an SOA Using an Enterprise Service Bus, IBM Redbooks, s. <http://www.redbooks.ibm.com/redbooks/pdfs/sg246346.pdf> 2004

Krafzig D., Banke K., Slama D.: Enterprise SOA: Service-Oriented Architecture Best Practices. The Coad Series, Prentice Hall PTR 2004

Woods D.: Enterprise Services Architecture. Galileo Press, Bonn 2004

Inhaltsverzeichnis

7.1 Vorgeschichte..... 324

7.2 Passworte..... 324

7.2.1 Passwort erfragen..... 325

7.2.2 Passwort erraten..... 325

7.2.3 Passwort abhören..... 326

7.3 Trojanische Pferde..... 326

7.4 Der buffer overflow-Angriff..... 327

7.5 Viren..... 328

7.5.1 Wie kann man einen eingeschleppten Virus entdecken?..... 330

7.5.2 Was kann man dagegen tun, wenn man einen Virus im System entdeckt hat? . 330

7.5.3 Aufgaben zu Viren..... 331

7.6 Rootkits..... 332

7.6.1 User mode-Rootkits..... 332

7.6.2 Kernel mode-Rootkits..... 333

7.6.3 Ring -1, Ring -2, Ring -3 Rootkits..... 334

7.6.4 Entdeckung und Bekämpfung der Rootkits..... 335

7.7 Zugriffsrechte und Rollen..... 336

7.7.1 Zugriffsrechte im Dateisystem..... 337

7.7.2 Zugriffsrechte von Systemprogrammen..... 337

7.7.3 Zugriffslisten und Fähigkeiten..... 338

7.7.4 Aufgaben..... 339

7.8 Vertrauensketten..... 340

7.9 Sicherheit in Unix und Windows NT..... 341

7.10 Sicherheit im Netz: Firewall-Konfigurationen..... 343

7.11 Die Kerberos-Authentifizierung..... 345

7.11.1 Das Kerberos-Protokoll..... 346

7.11.2 Kerberos Vor- und Nachteile..... 348

7.11.3 Aufgaben zur Netz-Sicherheit..... 349

Literatur..... 350

Sicherheitsfragen in Computersystemen sind keine technischen Probleme, sondern menschliche Probleme. Ein Computer funktioniert auch ohne Sicherheitseinrichtungen technisch einwandfrei. Besonders aber wenn ein Computer nicht mehr isoliert, mit wenigen, vertrauten Benutzern betrieben, sondern in ein Netz mit vielen, unbekannten Benutzern eingegliedert wird, tauchen Sicherheitsprobleme auf. Eines der wichtigsten Aufgaben eines Systemadministrators besteht darin, diese Probleme zu erkennen und Zeit und Geld dafür aufzuwenden, um sie zu beheben oder mindestens entsprechende Maßnahmen dafür zu ergreifen. Viele der notwendigen Vorkehrungen betreffen die Sicherheitsmechanismen des Betriebssystems. Aus diesem Grund werden wir in diesem Abschnitt näher auf die möglichen Gefahren und Gegenmaßnahmen eingehen.

---

## 7.1 Vorgeschichte

Nachdem im November 1988 ein sich selbst replizierendes Programm einen Großteil aller ans Internet angeschlossenen UNIX-Computer an der Carnegie-Mellon-Universität (USA) infiziert und lahmgelegt hatte, wurde dort das *Computer Emergency Response Team* CERT gegründet, das systematisch versucht, Sicherheitslücken in Computersystemen zu entdecken, zu dokumentieren und zu beseitigen. Für 1996 rechnete das CERT mit ca. 4000 erfolgreichen Einbruchversuchen durch ca. 30.000–40.000 Angreifer (*Hacker*), die immer neue Sicherheitslücken in den Systemen entdecken oder die bekannten schneller ausnutzen, als die Systemadministratoren sie beseitigen können (oder wollen). Obwohl inzwischen „der moderne Dieb mit seinem Computer mehr stehlen kann als mit vorgehaltener Waffe“ (*National Research Council*) und „der Terrorist von morgen keine Bomben mehr legt, sondern nur eine Taste drückt und damit viel mehr Schaden anrichtet“ (Wissenschaftsrat des US-Kongresses), wird bisher wenig getan, um sichere Rechnersysteme zu konstruieren.

Im Zusammenhang damit ist es interessant zu wissen, dass auch das US-Verteidigungsministerium (Pentagon) Arbeitsgruppen betreibt, die Kriegführung durch maximale Schädigung feindlicher Computersysteme vorbereiten. Dies schließt nicht nur den Einbruch in Computer über eine Netzwerkverbindung und das Manipulieren oder Löschen wichtiger Dateien ein, sondern auch den gezielten Hardwaredefekt von Computern am Netz. Man kann beispielsweise durch wiederholtes Ausführen bestimmter Instruktionen bestimmte Teile eines Mikroprozessors, die dafür nicht ausgelegt sind, überhitzen und damit den Prozessor generell schädigen.

Voraussetzung für die Angriffe auf Computersysteme über das Netzwerk ist das Eindringen von Daten in einen Computer über das Netz. Dies kann auf unterschiedlichen Wegen geschehen.

---

## 7.2 Passworte

Eine der naheliegenden Möglichkeiten, über ein Netz einzudringen, ist die Computerbenutzung („*Einloggen*“) über das Netz, beispielsweise mit einem Programm wie *Telnet*.

Das Problem, die Zugangskontrolle über ein Schlüsselwort (*Password*) zu umgehen, wird von den Einbrechern auf verschiedene Weise gelöst:

### 7.2.1 Passwort erfragen

Ein in der Effizienz bisher unerreichte Angriffsart vermag auch gut geschützte Systeme zu durchdringen: Der logistische Angriff. Hierbei ruft der potentielle Einbrecher einfach telefonisch bei einem Mitarbeiter an, gibt sich als Mitarbeiter des Rechenzentrums aus und fragt „aus Kontrollgründen“ das Passwort ab. Viele Menschen fallen leider darauf herein. Auch die elektronische Version mittels Email hat Erfolge: So genannte *Phishing*-Mails („Fishing“ ausgesprochen) täuschen eine seriöse Herkunft vor – meist von Banken, Kreditkarteninstituten, Online-Auktionshäusern- und Bezahlendiensten – und fordern den Empfänger zur Eingabe persönlicher Daten, Passwörter, Kreditkartennummern und PIN-Codes auf. Dazu wird der Anwender entweder auf eine präparierte Webseite geleitet oder ein entsprechendes HTML-Formular in der Mail nimmt die Daten auf. Es ist klar, dass die eingegebenen Daten „zum Datenabgleich“ nicht bei der Bank, sondern beim Betrüger landen.

### 7.2.2 Passwort erraten

Einer der typischen Fehler besteht darin, die Standardpasswörter der Herstellerfirma des Betriebssystems für Systemadministration, Service usw. nach Inbetriebnahme des Rechners nicht zu ändern, beispielsweise, weil vergessen wurde, das entsprechende Manual zu lesen. Sind die Standardpasswörter bekannt, so ist es kein Problem, von außen in ein System zu kommen.

Ein weiterer, oft begangener Fehler legaler Benutzer besteht darin, nur normale Worte (Vornamen, Bäume, Tiere, Städte, Länder usw.) zu verwenden. Ist die Datei der verschlüsselten Passwörter frei lesbar (wie in UNIX-Systemen unter */etc/passwd*), so kann ein harmloser Besucher wie *guest* (der häufig vorhandene Name für unbekannte Besucher) die Datei kopieren und mit dem Verschlüsselungsmechanismus alle Einträge eines Lexikons (bei 250.000 Einträgen und 1 ms pro Eintrag dauert dies nur 4 Minuten!) verschlüsseln und mit den Einträgen der Passwortdatei vergleichen – meist sind ein oder mehrere Benutzer dabei, die so einfache Passwörter haben, dass sie im Lexikon stehen.

Auch der Name, die Zimmer- oder Telefonnummer sind als Passwörter leicht zu erraten; über das Netz werden diese Daten von den Programmen *finger* und *who* geliefert.

Gegenmittel wie aufgezwungene Passwörter (z. B. Zufallszahlen) helfen auch nicht: Die Benutzer vergessen sie oder schreiben sie auf am Terminal befestigte Zettel, deutlich sichtbar für jeden zufällig Vorbeikommenden. Auch der regelmäßig erzwungene Wechsel des Passwortes hilft nicht: Benutzer wehren sich dagegen, neue Passwörter zu lernen und überlisten das System, um die alten zu verwenden.

Eine Möglichkeit, zu simple Passwörter zu verhindern, besteht darin, gleich bei der Eingabe das neue Passworts daraufhin zu prüfen, ob es leicht zu erraten ist.

### 7.2.3 Passwort abhören

Hat ein Eindringling Kontrolle über einen Übermittlungsrechner des Netzwerks gewonnen, so kann er den Nachrichtenverkehr, beispielsweise eines legalen Benutzers beim Einloggen, mitverfolgen und aufzeichnen. Damit kann er sich später selbst einloggen, selbst wenn das Passwort vom Benutzer vorher verschlüsselt wurde.

Abhilfe schafft hier nur entweder ein zeitabhängiges, verschlüsseltes Passwort des Benutzers (z. B. wenn der Benutzer sich über einen Satellitenrechner in den Netzwerkcomputer einloggt) oder ein Sicherheitsprotokoll, das zwischen den Computer (bzw. der „intelligenten“ Zugangskarte) des Benutzers und den Netzwerkcomputer geschaltet wird und dessen Parameter (und damit die Protokolldaten) ständig wechseln.

---

## 7.3 Trojanische Pferde

Aus der griechischen Sage wissen wir, dass die Stadt Troja nicht durch Überwindung ihrer starken Mauern eingenommen wurde, sondern dadurch, dass die Einwohner ein vermeintlich harmloses, großes hölzernes Pferd in ihre Stadt rollten. Die im Innern verborgenen Angreifer bemerkten die Trojaner nicht, und wurden so in der Nacht innerhalb ihrer starken Mauern im Schlaf überrascht.

Genau dieser Strategie, bekannt als „Trojanisches Pferd“, bedienen sich verschiedene Eindringlinge, um in ein System zu kommen. Dazu werden Lücken in normalen Diensten, die im Netz existieren, benutzt, um eigene Programme in einen Computer zu schmuggeln. Typische Lücken gibt es beispielsweise bei folgenden Diensten:

- *E-mail-Programme*

Eine Botschaft eines unbekannten Absenders kann alles mögliche enthalten – beispielsweise auch nicht druckbare Zeichen (Escape-Sequenzen), die beim Auflisten des Textes der Botschaft nicht ausgedruckt werden, sondern das Terminal dazu veranlassen, unter einer Funktionstaste Befehle zu speichern.

Passiert dies dem Systemadministrator (*super user*) eines UNIX-Systems, so werden beim nächsten Betätigen der Funktionstaste die Befehle abgeschickt mit der Befehlsgewalt des *super users*. Löscht der letzte Befehl das Zeilenecho und die Belegung der Funktionstaste, so bemerkt der *super user* nichts von seiner unfreiwillig gegebenen Anweisung, beispielsweise bestimmte Zugriffsrechte auf bestimmte Dateien zu verändern, so dass der Eindringling sie später ungehindert als Gast manipulieren kann.

Eine Gegenmaßnahme dafür besteht darin, in *E-mail*-Daten prinzipiell alle nicht druckbare Zeichen auszufiltern.

- *World-Wide-Web-Dienste*

Die Browser des Hypertextsystems WWW benutzen meist verschiedene Hilfsprogramme, um besondere Dienste auszuführen. Beispiele dafür sind Bild- und Tondisplayprogramme für verschiedene Dateiformate. Ist etwa eine Postscriptdatei im Netz vorhanden und der Benutzer möchte sie ansehen, so wird die Datei übers Netz geladen und einem speziellen Programm, meist *ghostview*, übergeben, das die darin enthaltenen Texte und Bilder auf dem Bildschirm darstellt. Leider ist aber Postscript nicht nur eine Seitenbeschreibungssprache, sondern auch eine Programmiersprache und kann ausführbare Programme enthalten, die von *ghostview* im Namen des Benutzers ohne dessen Wissen sofort ausgeführt werden. Diese Eigenschaft kann man bei der Konfiguration abstellen, wenn das Wissen um die Gefahr vorhanden ist und der Administrator die Dokumentation gut gelesen hat ...

Eine andere Möglichkeit des Eindringens bietet die Active-X-Technologie der Fa. Microsoft: Die Objekte, die über das Netz geladen werden und Code enthalten, haben die Rechte des ausführenden Programms und damit prinzipiell Zugriff auf den gesamten Rechner. Dies kann dazu führen, dass unbemerkt nach Besuch einer Webseite Spionageprogramme auf dem Rechner installiert werden, die Passworte, Bankzugangsdaten u. ä. protokollieren und an Kriminelle weiterleiten.

- *FTP file transfer-Dienste*

Auf jedem vernetzten Rechner existiert i. d. R. ein Dateitransferdienst *ftp*, der von außen aufgerufen werden kann. Dies kann als Einstiegspunkt genommen werden. Der *ftp*-Prozess auf einem Rechner hat normalerweise nur sehr wenig Rechte. Sind diese aber falsch konfiguriert oder die Zugriffsrechte für Dateien von anderen Benutzern falsch gesetzt, so kann der Angreifer von außen manipulierte Dateien, beispielsweise eine Passwortdatei, in einen Computer kopieren und so den Angriff von außen vorbereiten.

Aus diesem Grund sollte der *ftp*-Prozess praktisch keinerlei Zugriffsrechte haben und seine Aktionsmöglichkeiten sollten auf einen kleinen, genau festgelegten Dateibaum begrenzt werden.

- *Dateien aus dem Internet*

Leider sind es nicht nur die kostenlosen Spielprogramme aus dem Internet, die Trojanische Pferde enthalten, sondern auch „normale“ Dateien, die ausführbar sein können. Hierzu zählen alle Textdateien mit Makros (z. B. .doc oder .docx vom MS-Word), oder zur Ausführung bestimmte Dateien, etwa die MS Office-Dateiarten mit den Extensionen .ade, .adp, .mde, .vbe, .vbs, oder die Windows Systemdateien mit .wsf, .wsh, .scr, .sct, .reg, .pcd, .jse, .hta, .crt. Sie sollten alle nicht automatisch ausgeführt werden, sondern bei Ausführung nur einen Editor öffnen.

---

## 7.4 Der buffer overflow-Angriff

Einer der ältesten und bisher durch die Jahre erfolgreichster Angriffsmechanismus ist das Erzeugen eines *buffer overflow*. Hierbei wird beim Aufruf eines Programms über das

Netzwerk ein Argument (Zeichenkette) übergeben, das viel zu groß ist für den Zeichenpuffer des Programms. Ein Beispiel dafür ist der Aufruf einer CGI-Funktion (Funktion auf einem Webserver, die von jedem, der die Webseite im Internet besucht, angesprochen werden kann) oder die Eingabe bei einem Datenbankrecherche-Programm. Um zu verstehen, wieso man mit einer harmlosen Eingabe einen Rechner direkt übernehmen kann, müssen wir nur uns die typischen Programmiermechanismen vergegenwärtigen. Beim Einlesen einer Zeichenkette landet sie meist früher oder später auf dem Stack, entweder weil die Stringvariable dynamisch und nicht statisch ist, oder weil der Eingabestring als Argument einer Funktion übergeben wird. In beiden Fällen wird beim Kopieren der Eingabepuffer eine *string copy*-Prozedur `strcpy(Quelle, Ziel)` verwendet, die die gesamte Eingabekette kopiert, egal, wie lang sie ist. Auf dem Stack wird üblicherweise ein Standardplatz (ca. 80 Zeichen) pro String reserviert. Ist die Zeichenkette länger, so werden alle anderen Daten auf dem Stack ebenfalls überschrieben, etwa die Rückkehradresse der Kopierprozedur. Sind die überschreibenden Bytes vom Angreifer so gewählt worden, dass sie den Maschinencode eines Angriffsprogramms repräsentieren, so wird statt beim Rücksprung nicht die korrekte Rücksprungsadresse vom Stack gelesen, sondern die Gefälschte, die hinein in das Angriffsprogramm führt. Statt des Systemprogramms wird nun das Angriffsprogramm, ausgestattet mit den Rechten des Systemprogramms, ausgeführt.

Wie kann man sich vor solchen Angriffen schützen? Der wirkungsvollste Versuch ist sicherlich, auf Betriebssystemebene das Stacksegment für ausführenden Code zu sperren, also die Prozessrechte für das Stacksegment auf Lesen und Schreiben zu beschränken. Da inzwischen diese Angriffsart auch auf das Heap (Daten) -Segment ausgedehnt wird, sollte nur noch das reine Programmcodesegment Ausführungsrechte erhalten, wobei das Setzen reiner Lese- und Ausführungsrechte auf dem Programmsegment das Ändern und Überschreiben verhindert.

Versucht man, auf Programmiererebene den Pufferüberlauf zu verhindern, so kommt hier zuerst die Benutzung von Programmiersprachen in Frage, die den Index bei Feldern ständig überprüfen und bei Überlauf einen Fehler melden, etwa die Sprache Java. Möchte man C verwenden, so lassen sich spezielle Funktionen benutzen, etwa der Gebrauch von `strncpy()` anstelle von `strcpy()`, bei der das Ziel des Kopierens der Zeichenkette vor Gebrauch überprüft wird. Bei reinen Binärversionen kann man noch zwischen Programm und Bibliothek eine Zwischenschicht schalten, die die Argumentüberprüfung übernimmt.

---

## 7.5 Viren

Ein anderer Ansatz besteht darin, den Angriff gegen das System nicht per Hand auszuführen, sondern als Programm ablaufen zu lassen. Ist darin noch zusätzlich eine Möglichkeit zum Kopieren (Vervielfältigen) des Einbruchprogramms enthalten, so kann sich dieses Programm wie eine Infektion von Computer zu Computer weiterverbreiten. Allerdings benötigt es dazu meist andere Programme (z. B. Netzwerk- oder Systemprogramme) als Überträger, so dass diese Art von Programmen **Viren** genannt werden.



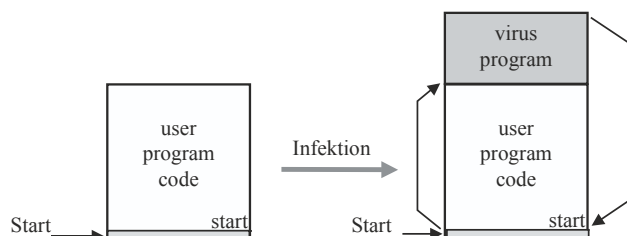
Es gibt verschiedene Arten von Viren. Beherrschten früher noch Viren das Feld, die auf Wechselmedien (Floppy-Disk, USB-Sticks) vorhanden waren und beim Starten (*booten*) des Systems oder bei der automatischen Ausführung des Startprogramms auf dem USB-Stick aktiviert wurden, so kommt die heutzutage auf Windows und Unix-Systemen am meisten verbreitete Form von Viren aus dem Internet durch Anklicken eines *email*-Anhangs oder eines Internet-Links. Der Virus siedelt sich dabei in den ausführbaren Dateien der Benutzer an. Dazu fügt der Virus einen zusätzlichen Codeteil an das Programmende an, der eine Viruskopie enthält. In [Abb. 7.1](#) ist das Prinzip gezeigt.

Der Virus klinkt sich so in die Startsequenz ein, dass er zuerst ausgeführt wird, sein dunkles Tagewerk vollbringt und erst danach die Kontrolle an das Originalprogramm weitergibt. Das läuft ab, ohne dass der Benutzer dabei etwas von den anderen Dingen ahnt, die er mit der Aktivierung des Programms verursacht hat. Infiziert der Virus nur langsam und unregelmäßig das Dateisystem, so kann man sicher sein, dass keiner etwas davon bemerkt. Näheres darüber ist in (Spafford et al. 1990) zu finden.

Da Anti-Virenprogramme die Viren durch typische Befehlssequenzen am Programmende erkennen, gibt es mittlerweile Viren, die ihren Code nicht mehr direkt an die Programme anhängen, sondern nur aus dem verschlüsselten Viruscode sowie einer Entschlüsselungsprozedur bestehen. Erst zur Laufzeit wird der Code zu dem eigentlichen Programm expandiert und ausgeführt. Da auch diese Spielart an typischen (codierten) Befehlssequenzen erkannt werden können, versuchen die polymorphen Viren durch dauernde Mutation des Schlüssels ihren Befehlscode zu bei jeder Infektion zu verändern.

Die dritte Art von Viren besteht aus Prozeduren, die einem Objekt beigefügt werden, ohne dass der Benutzer dies merkt. Ein typisches Beispiel dafür sind Textdateien, die mit entsprechenden benutzerdefinierten Funktionen zusammen abgespeichert werden, beispielsweise die in einer hardwareunabhängigen Hochsprache (Makrosprache) geschriebenen Funktionen für den Microsoft-Word-Editor. Bezieht man eine Word-Datei von einem fremden Rechner, so kann ein in der Makrosprache geschriebener Virus darin enthalten sein (**Makrovirus**), der im Dokument nicht als Text sichtbar ist. Im Unterschied zu einem herkömmlichen Virus wird er nicht direkt mit dem Wirtsprogramm gestartet, sondern erst beim Laden der Daten. Durch seine Unabhängigkeit von der Betriebssystem- und Compilerumgebung ist ein Makrovirus extrem portabel und ansteckend. Unbekannte Textdateien sollten also zuerst mit einem Virens Scanner auf unbekannte Funktionen (Makros) abgesehen und dann erst geladen werden.

**Abb. 7.1** Die Infektion eines Programms



Ein wichtiger Schutz ist deshalb die Typisierung von Dateien und die Registrierung im Betriebssystem, welche Programme auf welchen Typ mit welchen Rechten zugreifen dürfen. Ist beispielsweise registriert, dass ausführbare Dateien nur vom Linker erzeugt werden können, so kann ein Virus nicht Programme beschreiben und damit sich nicht in andere Programme einschleusen.

Leider ist die Ausdehnung dieser Systematik auf die Makros ausführenden Programme wie Word oder Excel nicht möglich, da die Ausführungseigenschaften vom Makros nirgendwo spezifiziert werden und deshalb auch nicht beachtet werden können. Hier besteht Verbesserungsbedarf.

### 7.5.1 Wie kann man einen eingeschleppten Virus entdecken?

Wie entdeckt man, dass ein Virus sich im System angesiedelt hat? Eine Möglichkeit ist, von allen ausführbaren Dateien die Quersumme (Prüfsumme) zu bilden und sie in extra Dateien abzuspeichern. Ist das Format dieser Prüfsummendatei geheim, so kann der Virus sie nicht fälschen, und eine regelmäßige Überprüfung der Quersummen kann zur Entdeckung führen.

Eine andere Idee besteht darin, eine genau definierte Testdatei (Lockvogel) zu installieren und regelmäßig zu überprüfen. Ist sie vom Virus verändert worden, so kann man damit nicht nur die Anwesenheit des Virus feststellen, sondern auch den Code extrahieren und den Virus im ganzen Rechnersystem gezielt suchen und löschen.

Ein dritter Test besteht darin, den Datenstrom ins Netz durch eine *firewall* laufend zu überwachen. Meist versuchen Viren, mit einem unbekannten Server Kontakt aufzunehmen. Muss jede neue Verbindung vom Benutzer genehmigt werden, so kann das auffallen. Nachteilig ist bei diesem Konzept, dass der Benutzer laufend eingreifen und entscheiden muss. Dies kann leicht zum Überdross und Abschalten der *firewall*-Funktion führen.

### 7.5.2 Was kann man dagegen tun, wenn man einen Virus im System entdeckt hat?

Das bewährte Mittel dazu heißt: Erst einmal Ruhe bewahren (*Don't panic*). Haben wir ein kleines System (z. B. einen PC), so ist die Strategie klar:

Gibt es auf einem separaten Datenträger eine saubere Kopie des Betriebssystemkerns (z. B. die Original-Systemdiskette) sowie ein vertrauenswürdiges und aktuelles Antivirusprogramm, das den fraglichen Virus „kennt“ (neuere Datenbasis!), dann reicht es, den Rechner auszuschalten (Löschen aller Viren im Hauptspeicher). Nach dem Einschalten und einem Systemstart kann das Virensuchprogramm laufen und die Massenspeicher „reinigen“.

Findet das Virensuchprogramm den Virus nicht oder haben wir gar kein solches Programm, so müssen wir anders vorgehen. Zuerst müssen alle ausführbaren Benutzerprogramme gelöscht werden. Dann müssen wir in einer „logischen Kette von vertrauenswürdigen Maßnahmen“ alle Systemprogramme ersetzen. Dazu booten wir das System von einer sauberen

BS-Kopie (die wir hoffentlich noch haben) und installieren alle wichtigen Systemprogramme neu. Dabei verwenden wir nur die Originalprogramme der Systemdiskette. Dann werden alle weiteren Hilfsprogramme und Systeme erneut installiert (bzw. von den Originalmedien kopiert), bis alles wieder wie vorher war. Zum Schluss werden alle Benutzerprogramme erneut kompiliert.

Bei einem großen Computersystem (z. B. *main frames*) kann man aber leider nicht so vorgehen, da der Betrieb weiterlaufen soll. Hier hilft ein Scannerprogramm, das alle in Frage kommenden Dateien systematisch auf den Virus hin prüft und sie gegebenenfalls „reinigt“. Allerdings ist der Erfolg eines solchen Ansatzes nicht garantiert: Ist das Reinigungsprogramm langsamer als der Virus oder ebenfalls infiziert, so ist dem kein Erfolg beschieden. Man kann zwar analog zu der biologischen Abwehr spezielle Antiviren schreiben („Antikörper“), aber ihr Erfolg und der Zeitpunkt, wann sie sich selbst abschalten und vernichten sollen, sind höchst unklar.

Das einzige und probate Mittel in einem solchen Fall (und bei anderen Störfällen!) ist und bleibt die logische Vertrauenskette. Man geht von einem ganz sicher funktionierenden Minimalsystem aus und erweitert es mit vertrauenswürdigen Schritten solange, bis das Gesamtsystem wieder vertrauenswürdig ist.

### 7.5.3 Aufgaben zu Viren

#### Aufgabe 7.5-1 Sicherheit

Ein beliebtes Mittel zum Abhören von Passwörtern besteht in einem Programm, das „login:“ auf den Bildschirm schreibt. Jeder Benutzer, der sich an den Rechner setzt, wird seinen Namen und sein Passwort eintippen. Registriert dieses Programm die Daten, erscheint eine Meldung wie „Passwort falsch, bitte wiederholen Sie“ oder ähnliches, und es beendet sich. So fällt es nicht einmal auf, dass hier ein Passwort gestohlen worden ist. Wie können Sie als Benutzer oder Administrator den Erfolg eines solchen Programms verhindern?

#### Aufgabe 7.5-2 Viren-was tun?

- Angenommen, Sie bemerken einen Virus auf Ihrem PC. Unter welchen Umständen reicht es nicht aus, die Massenspeicher mit einem Virensuchprogramm zu „reinigen“?
- Recherchieren Sie: Wie kann es sein, dass ein Virus in einem Server vorhanden ist, obwohl er weder im Hauptspeicher noch auf der Festplatte sich als Programm nachweisen lässt? Erklären Sie jede der Möglichkeiten ausführlich.

#### Aufgabe 7.5-3 Anti-Virenprogramme

- Was sind die gängigen Strategien, die Antivirenprogramme zum Erkennen von Viren und Ähnlichem nutzen? Wie funktionieren diese und wann versagen sie?
- Beurteilen Sie die Aussage „Je mehr verschiedene Antiviren Programme ich auf meinem Computer installiert habe, desto sicherer wird dieser.“

## 7.6 Rootkits

Eine der größten Herausforderungen für die Erkennung und Beseitigung von Malware sind in den letzten Jahren die sog. **Rootkits** geworden. Ihr Name rührt von den Administratorrechten (*root*-Rechten unter Unix) her, die diese Software benötigt, um sich zu installieren. Ihr Zweck ist einfach: Sie soll verhindern, dass Malware (Viren, Würmer, Trojaner, ...) entdeckt und beseitigt werden. Sie bildet also so eine Art Schutzprogramm für alle „bösen“ Programme. Die Art und Weise, wie Rootkits dies tun, greift tief in die Funktionen der Betriebssysteme ein und ist deshalb für uns sehr interessant. Die Grundidee dabei ist relativ einfach: Fange alle Aufrufe ab, die Dateien, Ordner oder Prozesse auflisten können und manipulierte sie so, dass die „bösen“ Dateien, Ordner, Prozesse und Netzwerkverbindungen ausgeblendet werden getreu nach dem Motto: Was nicht gesehen wird, kann auch nicht vernichtet werden.

Die Rootkit-Angriffe kann man in zwei verschiedene Kategorien einteilen, entsprechend der Art und Weise, sich im Betriebssystem einzuklinken: die *user mode*-Rootkits und die *kernel mode*-Rootkits.

### 7.6.1 User mode-Rootkits

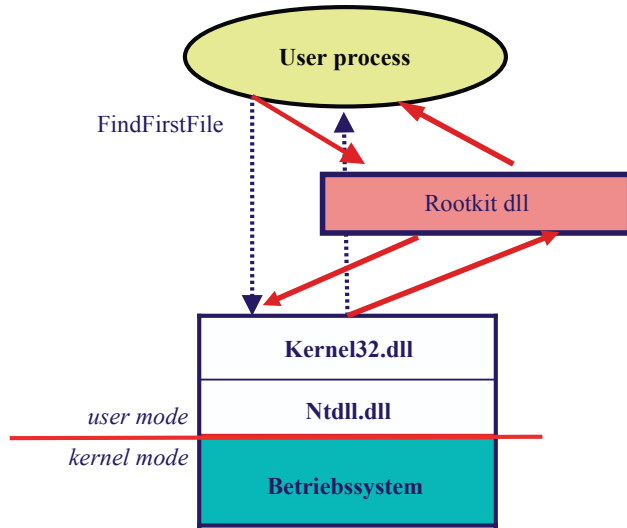
Die Grundidee bei diesem Angriff besteht darin, alle Anfragen eines Programmes für Dateien, Ordner und Prozesse über die eigenen Prozeduren umzulenken und dabei zu zensieren. Dazu wird beim *dll import hooking* beim dem Systemprogramm die Einträge in der Import-Tabelle der benutzten Prozeduren umgebogen auf die manipulierten Prozeduren des Rootkits. Beispielsweise wird in Windows NT beim Anzeigen von Ordnerinhalten die Methode „FindFirstFile“ benutzt. Wird der Zeiger zu der User-API-Bibliothek „Kernel32.dll“ umgebogen auf das Rootkit, so werden alle Anfragen vom Rootkit untersucht, bevor sie an Kernel32.dll weitergeleitet werden. In Abb. 7.2 ist eine solche Konfiguration gezeigt, beispielsweise durch das „Vanquish rootkit“.

Damit können alle Dateien, die auf der „unsichtbar“-Liste des Rootkit stehen, ausgeblendet und so nicht mehr gefunden werden. Dies gilt analog auch für Prozesse und Netzwerkverbindungen (TCP/IP-Ports).

Möchte man diese Zensur auch für alle Systemprogramme, etwa den Windows Task Manager oder für ein Antiviren-Programm durchführen, so wird bereits auf allerunterster Ebene bei den low-level-Systemaufrufen angesetzt, in Windows NT etwa bei „Ntdll.dll“.

Eine dritte Angriffsmethode geht noch einen Schritt weiter: Sie modifiziert wie ein Virus den Zielprozess so, dass zuerst das Rootkit angesprochen wird, dann erst der eigentliche Prozesscode. Ein Beispiel dafür ist der „Hacker-Defender“ von [www.rootkit.com](http://www.rootkit.com).

**Abb. 7.2** Angriff eines *user mode*-Rootkits unter Windows NT

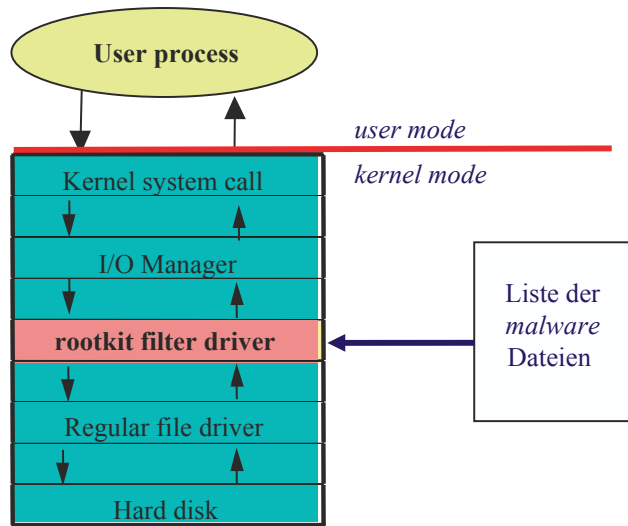


### 7.6.2 Kernel mode-Rootkits

Am schlimmsten sind die kernel mode rootkits: Sie können sich, einmal installiert, fast perfekt verbergen. Es gibt drei verschiedene Ansätze dafür:

- Sie klinken sich in der *system call*-Verteilertabelle ein und simulieren bestimmte *system calls*, so dass sie dazu aufgerufen werden („*system call hooking*“). Die dazu gehörenden Rootkit-Programmzeilen befinden sich in einem Treiber, der geladen wird.
- Eine andere Methode besteht darin, die Datenstrukturen des Kernels (z. B. Liste aller Prozesse) direkt zu fälschen, ohne, dass dabei das Dispatchen leidet. Diese Methode ist riskant für die Autoren, da bei Fehlern in undokumentierten Datenstrukturen des Kerns etwa bei Windows NT das System abstürzt und das Rootkit in Gefahr läuft, entdeckt zu werden. *Beispiel*: „FU-rootkit“.
- Eine dritte Methode installiert sich explizit als Filter-Treiber in das Dateisystem, so dass bei allen Anfragen die *malware*-Dateien ausgeblendet werden, siehe [Abb. 7.3](#). *Beispiel*: „NT Rootkit“.
- Eine vierte, sehr raffinierte Methode geht noch einen Schritt weiter. Um die Entdeckung der Malware zu verhindern werden keine verdächtigen Dateien geschrieben, Prozesslisten gefälscht oder Treiber geladen, sondern der Code wird direkt im Hauptspeicher in einem laufenden Systemprozess abgelegt, beispielsweise durch den „Code Red“-Wurm im Microsoft Internet Information Server. Natürlich überlebt diese Angriffsart einen Reboot-Vorgang nicht, aber dies ist bei den meisten Servern unwichtig: Sie werden fast nie abgeschaltet. Solange die Infektionsrate von Server zu Server höher ist als die Abschaltrate wird ein solcher Angriff Erfolg haben.

**Abb. 7.3** Der Rootkit-Angriff als Filtertreiber



- Eine weitere Methode besteht darin, die Firmware von Geräten (/Festplattencontroller, Netzwerkcontroller, ...) zu infizieren. Wird eine solche Festplatte gebooted, so kann die Bootanforderung (Lesen von Block0 bzw. des MBR) bereits zur Installation des Virus im MBR bzw. im Hauptspeicher führen; Teile des Hauptspeichers können so für den normalen Betriebssystemzugang unbemerkt gesperrt werden. Eine solche „Spezialbehandlung“ ist nach den Enthüllungen von Snowden bereits von der NSA an Geräten vorgenommen worden, die von der USA in bestimmte Länder exportiert wurden.

### 7.6.3 Ring -1, Ring -2, Ring -3 Rootkits

Angeichts der neueren Entwicklungen bei Rootkits stellen die *kernel mode*-Rootkits noch einen harmloseren Ansatz dar. Entsprechend der verschiedenen Sicherheitsringe im Zwiebelschalen-Modell aus [Abschn. 3.6.3](#) kann man nun den innersten Ring 0 noch in weitere Ringe (oder Basisschichten) untergliedern. Sie unterscheiden sich von den CPU-Instruktionsrechten dadurch, dass sie entweder noch privilegierter sind als die „normalen“ CPU-Befehle und noch weniger kontrolliert werden können. Jede einzelne Schicht ist als Angriffsziel von Malware im Visier. Die Angriffsschichten sind

- **Ring -1:** Üblicherweise setzen die Betriebssysteme, um gut portierbar zu sein, auf eine abstrakte Hardware (HAL) auf. Eine solche gut strukturierte Schnittstelle ermöglicht es leicht, eine Zwischenschicht (virtuelle Maschine) zwischen das Betriebssystem und die Hardware zu schieben. Eine solche Schicht wird als Hypervisor bezeichnet und wird

allgemein zur Lastverteilung auf Großrechnern benutzt, siehe [Abschn. 1.8](#). Geschieht dies nun durch Malware, so kann das Schadprogramm alles vorspiegeln, was es will: Infizierte Speicherbereiche im RAM und auf Festplatten werden verborgen, Kommunikation zwischen Programmen und Hardware, etwa Tastatur oder Netzwerken, kann abgehört werden usw. Ein prominentes Beispiel für einen solchen Code ist ein *Blue Pill* genannter Code (*proof of concept*).

Dieser Angriff ist auch auf bereits laufende Hypervisoren auf Großrechnern möglich, so dass auch ein Hypervisor, etwa XEN, unterwandert werden kann (Wojtczuk et al. -1,-3 [2008](#)).

- **Ring -2:** Noch tiefer treten die Rootkits ein, die den System Management Mode (SMM) der Intel x86 Prozessoren ausnutzen (Embleton et al. [2013](#)). Dieser historisch ursprünglich zur Verwaltung gedachte Modus kann einen eigenen, von außen unsichtbaren privaten Speicherbereich aufmachen, jede Instruktion ohne Sicherheitsüberprüfung ausführen und läuft im non-präemptiven Modus ohne Speicherbereichsüberprüfung. Ursprünglich nur für low-level-Aufgaben gedacht wie Temperaturkontrolle und Energiemanagement im BIOS gedacht, können damit beispielsweise Tastaturangaben abgefangen (*key logging*) und mittels Direktzugriff durch eine Netzwerkkarte verschickt werden.

Und das alles, ohne dass es das Betriebssystem bemerken kann.

- **Ring -3:** Ein davon unabhängiger, noch tieferer Eingriff ist möglich, wenn außer dem Hauptprozessor noch ein Nebenprozessor existiert, der einen eigenen Speicherbereich und eine Direktverbindung zu den Netzwerkchips hat. Dies gibt es nicht? Weit gefehlt! Die *Intel Active Management Technologie* AMT (memory controller hub MCH chipset „northbridge“) auf Mainboards ist ein solcher Fall und ist als Chipsatz zur zentralen Verwaltung von Rechnern gedacht. Wird sie gehackt, so kann dies weder durch Antivirenprogramme noch durch andere Software bemerkt werden. Weder ein Reset noch eine Neuinstallation des Betriebssystems helfen hier weiter.

### 7.6.4 Entdeckung und Bekämpfung der Rootkits

Die wichtigste Aufgabe besteht zunächst darin, die Rootkits überhaupt erst einmal zu finden. Kaum ein Virens Scanner überprüft die Systeme auf alle bekannten Rootkits, obwohl das Problembewusstsein dafür wächst.

Möchte man ein System näher untersuchen, so sollte man den Systemstatus mit möglichst unterschiedlichen Programmen testen und die verschiedenen Systemsichten miteinander vergleichen. Beispielsweise kann ein Diagnoseprogramm das Dateisystem sowohl auf oberster Zugriffsebene durch die User-API (z. ReadFile) als auch durch die System-API (z. B. *Read raw data*) zugreifen (*High level* vs. *low level scan*). Ergeben sich dabei Unterschiede in den Dateiangaben, so lässt sich dies auf ein Rootkit zurückführen (z. B. Russinovich [2006](#)).

Bei der *kernel mode* Rootkit-Angriffsmethode haben wir aber Probleme, den Angriff zu bemerken: Die Malware existiert, ohne dass sie im Dateisystem, der Prozessliste oder Netzliste explizit auftaucht. Hier kann man nur noch die Größe des Codes eines Programms auf dem Massenspeicher mit der Größe im Hauptspeicher vergleichen und feststellen, ob es Unterschiede gibt. Allerdings wird dieser Diagnoseansatz in Windows NT dadurch erschwert, dass vom Hersteller keine veröffentlichten Schnittstellen angeboten werden, um die Speicherbelegung eines Programms im Hauptspeicher abzufragen. Hier hilft nur der Trick, eine Zwischenschicht zwischen Betriebssystem und Hardware zu legen, und die Wirkung von direkten und „normalen“ Hardwareoperationen zu vergleichen.

Bei den *Ring -1*-Angriffen ist es noch schwieriger, dies zu bemerken. Die einzige Möglichkeit besteht darin, die Umschaltzeiten des Hypervisors zwischen den Virtuellen Maschinen zu messen und mit den Originalzeiten zu vergleichen. Sind sie zu lang, so ist eine Schadprogramm-Zwischenschicht eingebaut worden (Wojtczuk et al. 2008). Auch die Installation eines vertrauenswürdigen Hypervisors von Anfang an, der Angriff auf diese Zwischenschicht verhindert, kann helfen (Rutkowska 2007).

*Ring-2* Angriffe mittels der Befehle im *System Management Modus* können nur indirekt bemerkt werden. Da z. B. *key logging* über die Umleitung von Interrupts funktioniert, muss die Interrupt-Tabelle auf Konsistenz zur Standardtabelle geprüft werden. Allerdings ist dies nicht unproblematisch, da Änderungen auch von normalen Ereignissen (z. B. Anpassung von USB-Geräten an den Tastaturinput) bewirkt sein können, oder auch nicht. Abhilfe schafft hier, in der CPU ein Sperr-Bit *D\_LCK* so früh wie möglich beim Booten zu setzen, um zu verhindern, dass der Modus durch einen infizierten Treiber ausgenutzt wird (Embleton et al. 2013).

Einen *Ring -3*-Angriff ist sehr schwer festzustellen und zu unterbinden. Hier geht die Aufmerksamkeit auf das Unterbinden einer Infektion, beispielsweise durch besser geschützte, signaturgesicherte Flash-Speicher. Dies ist aber eine Aufgabe der Hardware-Hersteller und nicht der Anwender.

Die beste Möglichkeit der Bekämpfung von Rootkits besteht also darin, sie gar nicht erst zur Ausführung zu bringen. Da ihre Aktion von den root-Administratorrechten bei der Infektion abhängt, ist es wichtig, die Zugriffsrechte sauber zu konfigurieren und bei der Systembenutzung möglichst wenig als Benutzer mit Administratorrechten tätig zu sein.

---

## 7.7 Zugriffsrechte und Rollen

Hat sich ein Eindringling erst einmal Zugang zu einem Computersystem verschafft, so kann er auf verschiedene Weise versuchen, die Administratorenrechte darauf zu erlangen. Gerade auf UNIX-Rechnern, die keine abgestuften Administratorenrechte kennen, bedeutet die Übernahme eines wichtigen Systemprogramms meist auch die Übernahme der gesamten Kontrolle über den Rechner. Wie ist es nun möglich, vom Status eines



„normalen“ Benutzers zu Administratorrechten zu gelangen? Dazu gibt es verschiedene Wege, die folgende Schwachstellen nutzen:

### 7.7.1 Zugriffsrechte im Dateisystem

Sind die Schreib- und Leseberechtigungen für Systemdateien falsch gesetzt, so kann ein Eindringling dies leicht ausnutzen.

---

#### Beispiel Kommandopfad

Üblicherweise werden Kommandos in UNIX so abgearbeitet, dass an verschiedenen Stellen im Verzeichnisbaum danach gesucht wird und die erste ausführbare Datei mit dem Kommandonamen, die gefunden wird, auch als Programm ausgeführt wird. Die Reihenfolge der Stellen ist in der Zeichenkette mit dem Namen PATH gespeichert. Möchte ich z. B. den Befehl *cmd* ausführen lassen, so wird nach */bin/cmd*, */usr/bin/cmd*, */usr/local/bin/cmd*, und *./cmd* gesucht. Hat jemand Zugriffsrechte auf eines dieser Verzeichnisse im Dateibaum, so kann er unter dem offiziellen Kommandonamen seine eigene Version des Kommandos dort hineinstellen, beispielsweise der Kommandos *ssh*, *su* oder auch nur *ls* zum Anzeigen der Dateien.

Führt ein Benutzer nun ein solches Programm aus, so kann es die gewünschten Ergebnisse liefern – und zusätzlich (bei *ssh* und *su*) die Passwörter in einer Datei des Eindringlings speichern (Trojanische Pferde). Auch ein *login*-Programm ist denkbar, das ein Standardpasswort des Eindringlings kennt und ihn ohne Kennung einlässt.

### 7.7.2 Zugriffsrechte von Systemprogrammen

Viele Dienstprogramme für Arbeiten am Betriebssystem haben umfangreiche Rechte. Beispielsweise muss ein Editor auf alle Dateien eines beliebigen Benutzers schreiben und lesen können; ein *E-Mail*-Programm muss die elektronischen Briefdateien allen Benutzern in ihren Briefkasten schreiben dürfen und ein Prozessstatusmonitor (z. B. *ps* in UNIX) muss Zugriff auf Tabellen des Betriebssystemkerns haben. Gelingt es, die Aktionen des Systemprogramms so auszunutzen, dass es andere als vorgesehene Dinge tut, so hat der Angreifer sich im System eingenistet.

---

#### Beispiel Emacs-Fehler

Im GNU-Emacs-Editor hatte man die Möglichkeit, mit *movemail* eine *E-Mail* in ein Verzeichnis zu schreiben. Allerdings prüfte der Editor (obwohl Systemprogramm) nicht nach, ob das Zielverzeichnis auch dem aktuellen Benutzer gehört und ermöglichte

so, andere Systemdateien zu überschreiben. Dies war eine der Lücken, die deutsche Hacker 1988 nutzten, um im Auftrag des KGB in militärische Computer der USA einzudringen und auf Militärgeheimnisse hin zu untersuchen.

Meist sind von diesen Problemen neue Versionen oder inkorrekt installierte Systemprogramme betroffen. Einen Sicherheitstest auf Zugriffsrechte und andere Schwachstellen wie zu einfache, ungültige oder fehlerhafte Passwörter bietet das Softwarepaket COPS von CERT für UNIX, das kostenlos bezogen werden kann (COPS 1997). Weitere Informationen sind auch z. B. in (SAT 1997) über das Werkzeug Satan sowie über neuere Sicherheitspatches in (DFN 1997) zu finden.

### 7.7.3 Zugriffslisten und Fähigkeiten

Bisher war das Beste, was wir an Sicherheitsmechanismen betrachtete hatten, die Zugriffslisten, kurz ACL (Access Control Lists) genannt. In diesen Listen werden für ein Objekt alle Nutzer und ihre Rechte aufgeführt, sowohl als Positivliste, was sie dürfen, als auch als Negativliste, wer etwas nicht darf (obwohl alle Welt es sonst darf). Es gibt nun einen zweiten Blick auf die Dinge: die Sicht der Benutzer. Anstatt jedem Objekt eine Liste mitzugeben kann man auch jedem Nutzer eine Liste zuordnen, wo alle drin steht, was er darf und was nicht, entsprechend seinen **Fähigkeiten** (*Capability Oriented Approach*). Die beiden Sichten sind zwei Seiten der selben Medaille: einer Zugriffsmatrix aus Nutzern und Objekten, in der für jedes Paar aus (Nutzer, Objekt) die Zugriffsmöglichkeiten vermerkt sind. In Abb. 7.4 ist ein Beispiel dafür gezeigt.

Dabei bildet die Liste der Fähigkeiten jeweils eine Zeile der Matrix, die Liste der ACL jeweils eine Spalte. Betrachten wir allerdings ein großes System etwa ein Netz aus 1000 Computern und 1000 Nutzern, so kommen wir schnell an die Grenze einer solchen Matrix. Anstelle für jeden Nutzer jede Zugriffsart für jedes existierende (und neu erstellte!) Objekt zu spezifizieren, ist es sinnvoll, die Nutzer in Kategorien wie „Administrator“, „Benutzer“, „Gast“ usw. einzuteilen und nur jeder dieser Kategorien oder Rollen ein differenziertes Profil zuzuordnen. Damit reduziert sich die zu spezifizierenden Situationen erheblich und damit auch die Arbeit des Administrators.

Abb. 7.4 Die Matrix der Zugriffsrechte

|          | Objekt 1 | Objekt 2 | Objekt 3 | Objekt 4 |
|----------|----------|----------|----------|----------|
| Nutzer 1 | r w      | r        | r w x    |          |
| Nutzer 2 |          | r        | r x      | r x      |
| Nutzer 3 | x        |          | r x      |          |

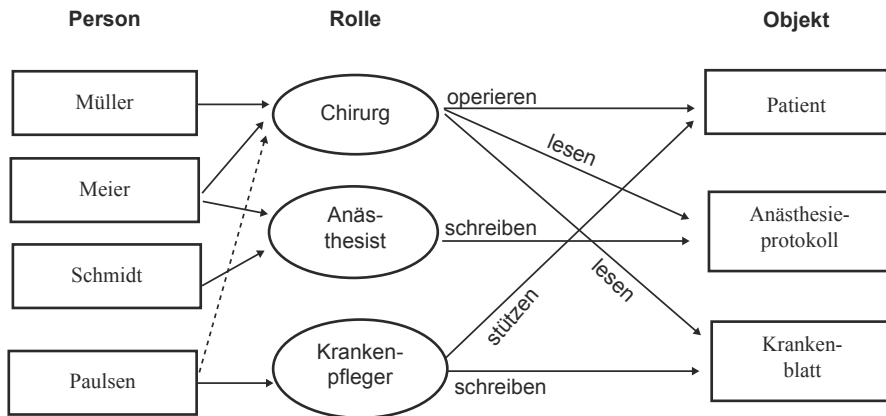


Abb. 7.5 Rollen und Zugriffsrechte

### Beispiel: Krankenhausrollen

In einem Krankenhaus hat jeder Mitarbeiter seine Rolle, die sich nach seinen Fähigkeiten richtet. Sinnvollerweise können aber einige Mitarbeiter durch Zusatzqualifikation verschiedene Rollen einnehmen, je nach den Umständen, siehe [Abb. 7.5](#).

Dabei dürfen hier die Chirurgen die Anästhesieprotokolle nur lesen; sie schreiben darf aber nur das dafür zuständige Fachpersonal, die Anästhesistin. Allerdings hat Herr Paulsen auch eine Sonderausbildung absolviert, so dass er auch Frau Müller bei der Chirurgie assistieren darf. Wie wir sehen, kann man bei der Erstellung von Rollen die Zugriffsrechte genau auf die Arbeit zuschneiden und muss dies nicht explizit an einzelne Nutzer binden. Dieselbe Person kann also bei der Ausübung unterschiedlicher Rollen (und damit unterschiedlicher Arbeiten) auch unterschiedliche Zugriffsrechte für die jeweilige Arbeitszeit bekommen.

Ein wichtiges Beispiel für ein solches *Capability-oriented System* ist das Authentifizierungssystem Kerberos.

## 7.7.4 Aufgaben

### Aufgabe 7.7-1 ACL und Rollen

- Wie funktionieren *access control lists* ACL und wie das *capability-oriented model* COM?
- Was sind die Vorteile von ACL und von COM?
- Welche Vorteile bietet darüber hinaus das Rollen-Modell?

## 7.8 Vertrauensketten

Im vorigen Abschnitt wurde ein System vorgestellt, das bei allen Aktionen die Authentizität und Rechtmäßigkeit des Benutzers sicherstellen soll. Sind damit alle Viren- und Trojanerangriffe zum Scheitern verurteilt? Leider nein. Stellen wir uns ein trojanisches Programm vor, das sich auf allen Rechner eingenistet hat und alle Schlüssel bei Bedarf kommuniziert, so wird das Sicherheitssystem ausgehebelt. Hier hilft nur, bei jedem Rechner von Startbeginn aus die Grundlagen für einen vertrauenswürdigen Rechnerbetrieb zu schaffen, indem eine sog. Vertrauenskette (*chain of trust*) aufgebaut wird. Eine solche Kette beginnt beim Start des Rechners. Ausgehend von einem Kern (Boot-ROM) errechnet die CPU einen Hash-Wert des BIOS. Anschließend wird dieser Wert mit einem intern in einen Chip (Trusted Platform Modul TPM) gespeicherten Wert verglichen. Sind sie ungleich, so wird der Bootstrap abgebrochen; andernfalls startet der Kern das BIOS. In Abb. 7.6 ist dies gezeigt.

Der gleiche Vorgang wird nun vom BIOS beim Laden des Betriebssystems und von diesem beim Betriebssystem selbst durchgeführt. Am Schluss prüft das Betriebssystem jede einzelne Anwendung, ob sie vertrauenswürdig ist und führt sie dann erst aus.

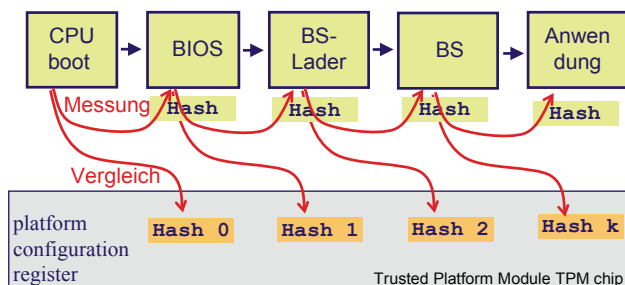
Ein solcher „sicherer Start“ ist als *secure startup* in Windows Vista integriert und wurde von der „*trusted Computing*“-Initiative angeregt.

Der Kryptochip TPM dient dabei nicht nur zur Speicherung und Überwachung der Quersummen und Verschlüsselung, sondern auch der Generierung von Zufallszahlen und anderen Schlüsseln für kryptografische Verfahren, etwa dem Kerberos-System oder bei der Datenverschlüsselung von Massenspeichern zum Schutz bei Diebstahl.

Allerdings ist eine solche Hardwareüberwachung von Programmen nicht unproblematisch: Sind beliebige Programme zur Registrierung möglich, so reduziert sich der Virenschutz; schränkt man den Zugriff auf wenige, vertrauenswürdige Programme ein, so liegt ein Missbrauch nahe, bei dem nur Programme bestimmter Hersteller zugelassen werden. Zusammen mit der Möglichkeit, den TPM als Basis für die digitale Rechteverwaltung (*Digital Rights Management* DRM) zu benutzen, hat die „*Trusted Computing*“-Initiative einiges Misstrauen bei verschiedenen Nutzergruppen erregt.

Zusätzlich gibt sie auch keine Antwort auf die gängigsten Angriffsarten: alle Angriffe durch *buffer overflow*-Mechanismen oder andere Angriffsarten, die offiziellen, aber schlecht programmierten Code benutzen, können so nicht abgewehrt werden.

**Abb. 7.6** Die Vertrauenskette beim Starten



## 7.9 Sicherheit in Unix und Windows NT

Im Folgenden sollen nun die Sicherheitsmechanismen in UNIX und Windows NT beschrieben werden. Für eine genauere Beschreibung der notwendigen UNIX-Sicherheitskonfiguration siehe z. B. Groll (1991) und Farroes (1991).

### Beispiel UNIX Authentifizierung (Benutzeridentifizierung)

Die grundsätzliche Personenzugangskontrolle erfolgt in UNIX mit einem Benutzernamen, der allgemein bekannt sein kann, und einem Passwort, das nur der Benutzer persönlich kennt. Bevor eine Sitzung eröffnet wird, prüft das *login*-Programm, ob der Benutzername und das dazu eingetippte Passwort registriert sind. Ist dies der Fall, so wird für den Benutzer ein besonderer Prozess (die Shell) erzeugt, der als Kommandointerpreter fungiert und die Zugriffsrechte und Identifikationen des Benutzers hat. Alle weiteren, vom Benutzer erzeugten Prozesse zum Ablaufen seiner Programme erben die Rechte der *shell* und haben damit die Rechte des Benutzers. In Abb. 7.7 ist der *login*-Ablauf gezeigt.

Wie bereits in Abschn. 4.3.1 beschrieben, ist jedem Benutzer eine *Benutzerkennung* *uid* und eine *Gruppenkennung* *gid* zugeordnet, für die bei jeder Datei Zugriffsrechte definiert werden. Diese Kennungen stehen mit dem verschlüsselten Passwort und dem Namen des Benutzers (getrennt durch „:“) in der Datei */etc/passwd*, dessen Einträge beispielsweise lauten

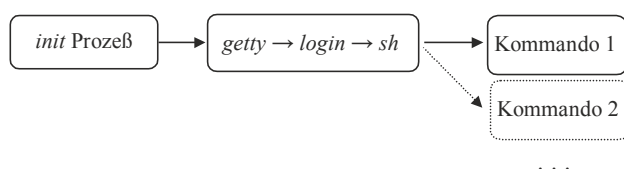
```
root:Igl6derBr45Tc:0:0:The Superuser:/:/bin/sh
brause:ntkyblioøkk3j:105:12:&Brause:/user/user2/NIPS:/bin/csh
```

Der erste Eintrag ist der Benutzername, der zweite das verschlüsselte Passwort, der dritte die Benutzerkennung *uid*, dann die Gruppenkennung *gid*, der Name des Benutzers, sein Startverzeichnis und der Name des Startprozesses (der Shell). Die Gruppennamen sind in der Datei */etc/group* definiert, beispielsweise

```
staff:*:12:boris,peter,brause
```

wobei das erste der Gruppenname ist, dann folgt das verschlüsselte GruppenPasswort (hier mit „\*“ leer) dann die Gruppenkennung *gid* und dann die Benutzernamen, die zusammen die Gruppe bilden. Ein Benutzer kann dabei in verschiedenen Gruppen gleichzeitig sein.

**Abb. 7.7** Ablauf beim login



---

**Beispiel Linux Authentifizierung (Benutzeridentifizierung)**

Die öffentliche Speicherung der Nutzerdaten, insbesondere des (verschlüsselten) Passworts, bietet eine gute Angriffsfläche. Aus diesem Grund wurde in Linux die Datei in nur in den von der *root* kontrollierten Bereich */etc/shadow* verlagert und für alle anderen Nutzer gesperrt. Auch das Format der Einträge hat sich geändert. Es ist *Name:Pass:DOC:MinTag:MaxTag:Warn:DeaktivIn: DeaktivSeit:Frei* mit dem Beispiel *brause:ntkyb1ioøkk3j:12952:0: 99999:7:::*

Der Benutzer heißt hier »brause«, das verschlüsselte Passwort lautet »ntkyb1ioøkk3j«, das Passwort wurde am 12952. Tag nach 1.1.1970 das letzte Mal verändert und kann frühestens in 0 Tagen (also heute) und muss spätestens in 99999 Tagen (ca. 274 Jahren) geändert werden – in dem Fall also immer. 14 Tage vor Ablauf des Passwortes (sofern Sie diesen Tag überleben) soll eine Warnung ausgegeben werden. Die anderen Werte wurden vom Administrator nicht definiert und sind leer.

Dies gilt übrigens auch für die Gruppenrechte; die Datei */etc/group* wurde nach */etc/gshadow* verschoben und gegen den Zugriff aller anderen Nutzer gesichert.

---

**Beispiel UNIX Autorisierung (Vergeben von Rechten)**

Jeder Prozess in UNIX hat zwei verschiedene Zugriffsrechte: die Zugriffsrechte seines Erzeugers, genannt „reelle Rechte“ *ruid* und *rgid*, und damit die des Benutzers sowie die effektiven Rechte *euid* und *egid*, die der Prozess tatsächlich zur Laufzeit hat. Wird ein Programm ausgeführt, das von einem anderen Benutzer und einer anderen Gruppe erzeugt wurde, so werden die bei der Datei vermerkten Angaben nicht beachtet, und es wird *euid* := *ruid* sowie *egid* := *rgid* gesetzt. Auch wenn das Programm vom *super user* erzeugt wurde und ihm gehört, so hat es doch nur die Zugriffsrechte des Benutzers.

Manche Programme (wie *E-Mail*-Programme etc.) müssen allerdings auch auf Dateien (z. B. *mailbox*) anderer Benutzer schreiben. Um dem Programm die stärkeren Rechte von *root* zu geben, kann man nun zusätzlich bei einer Datei vermerken, dass ihre Zugriffsrechte (*uid* bzw. *gid*) auf den Prozess (auf *euid* und *egid*) übertragen werden und damit *ruid* und *rgid* nicht beachtet werden (*set user id*- bzw. *set group id*-Attribute setzen).

---

**Beispiel UNIX Authentifikation in Netzen**

In lokalen UNIX-Netzen kann man sich mit dem *ssh*-Protokoll einfach und sicher in fremden Netzen anmelden und eine verschlüsselte Verbindung (VPN) erzeugen. Die Schlüssel bestehen dabei aus einem Paar, bei dem der öffentliche Schlüssel einmal mittels *ssh-keygen* erzeugt und mittels *ssh-copy-id* zum Zielrechner kopiert und dort beim Nutzer unter *~/.ssh/authorized\_keys* gespeichert wird. Wird nun eine aktuelle Verbindung mit dem geheimen Schlüssel per SSH aufgebaut, so wird beim Zielrechner geprüft, ob der geheime und der öffentliche Schlüssel zusammenpassen und dann die Verbindung ohne Passwortabfrage aufgesetzt.

---

**Beispiel Windows NT Benutzerauthentifikation**

Für die Funktion als Server wurden bei Windows NT besondere Konzepte implementiert. Über die lokalen *Access Control Lists* ACL der lokalen Dateisysteme hinaus wurde die Kontrolle von verschiedenen, sich global und lokal überschneidenden Benutzergruppen (*domain control*) eingerichtet. Damit gibt es also insgesamt drei verschiedene Möglichkeiten für einen Benutzer, sich im Rechnersystem anzumelden:

- eine *lokale Anmeldung*, bei der sich der Benutzer mit seinem lokalen Benutzernamen auf seinem Computer anmeldet
- eine *Netzwerk-Anmeldung*, um Zugriff auf die Dateien eines Dateiservers zu erhalten
- eine *Anmeldung in einer Domäne*, mit der der Zugriff auf die Rechner und die Dienste der Domäne geregelt wird.

Obwohl jede *Windows NT Workstation* NTW als kleiner Server fungieren kann, hat jede Domäne einen besonderen Server: den *Windows NT Server* NTS, der als *domain controller* DC fungiert. Um Verfügbarkeit und Leistung zu erhöhen, kann es in einer größeren Domäne nicht nur einen, sondern mehrere DCs geben. Von diesen fungiert aber immer einer als Hauptcontroller (*primary domain controller* PDC), die anderen als Nebencontroller (*backup domain controller* BDC), die vom Hauptcontroller aktualisiert werden. So ist die Benutzerauthentifizierung auf einem BDC möglich, aber nicht die direkte Modifizierung der Account-Daten. Dies kann nur indirekt von den Systemadministratoren erreicht werden, die die Daten der Domäne verwalten. Dabei ist die Abbildung auf die PDC und BDC transparent.

Der Austausch zwischen Benutzern und Daten verschiedener Domänen kann durch Einrichtung von Vertrauensrelationen (*trust relationship*) zwischen den Domänen erleichtert werden, so dass ein Benutzer von einer Domäne in einer anderen automatisch bestimmte Zugriffsrechte erhält. Welche Rechte dies sind, muss explizit beim Einrichten der Vertrauensrelationen festgelegt werden. Diese Konstruktion erleichtert beispielsweise die Zusammenarbeit zwischen verschiedenen Firmen oder den Abteilungen einer Firma, ohne zu große Privilegien und zu großen Datenzugriff gewähren zu müssen.

---

## 7.10 Sicherheit im Netz: Firewall-Konfigurationen

Ein wichtiges Konzept, um die Ausbreitung von Feuer auf mehrere Häuser zu verhindern, ist die Isolierung der Häuser oder Hausabschnitte durch spezielle Brandschutzmauern und Brandschutztüren. Dieses Konzept stand Pate bei den Bestrebungen, den Schaden bei einem Systemeinbruch in einem Computernetz auf ein Gebiet zu begrenzen. Die Brandschutzmauer (**fire wall**) besteht in diesem Fall aus einem speziellen Vermittlungsrechner (**router**), der als Bindeglied zwischen einem lokalen Netz und der feindlichen Außenwelt fungiert, beispielsweise dem Internet, siehe Kienle (1994). Dieser *firewall router* hat die Aufgabe, alle Datenpakete zu untersuchen (*screening*) und Pakete mit unerwünschter Internetadresse oder Portadresse auszusondern und zu vernichten. Dazu verwendet er eine

Positivliste und eine Negativliste von Portnummern, die entweder ausdrücklich erlaubt oder verboten sind. Weiterhin kann er Datenpakete von Rechnern des internen Netzes (Intranet) mit IP-Adressen, die nach außen unbekannt sind, in den „offiziellen“ IP-Adressraum abbilden. Dies erlaubt nicht nur, alte Netze problemlos ans Internet anzubinden, sondern unterbindet auch den direkten Durchgriff von Hackern des Internets auf das interne Netzwerk. Da all dies sehr schnell gehen muss, um den Datenverkehr nicht unnötig zu behindern, wird das *screening* meist als Kombination von Software- und Hardwaremaßnahmen verwirklicht.

Allerdings haben wir dabei ein Problem: Woher soll der *fire wall router* wissen, welche Internetadressen (Rechner) und Portadressen (Dienste) problematisch sind? Meist ist dies benutzer- und kontextabhängig: Portnummern wechseln, dem System unbekannte Benutzer antworten aus dem Internet, eine Folge von Portbenutzungen ist erlaubt, aber nicht die Einzelbenutzung. Dies führt bei starren Listen bald zu Problemen. Aus diesem Grund wird dem *fire wall router* noch ein zweiter Rechner zur Seite gestellt: der *relay host*. Er befindet sich außerhalb des LAN und ist beispielsweise über einen allgemeinen Vermittlungsrechner (*external router*) mit dem Internet verbunden. In Abb. 7.8 ist eine solche klassische *fire wall*-Konfiguration gezeigt. Auf dem *relay host* befinden sich nun alle wichtigen Programme (*application relay software*) und Netzwerkdienste.

Ihre Ausführung ist dabei bestimmten Restriktionen unterworfen, die mit programmabhängigen Zugangskontrolllisten (ACL) überprüft werden. In diesen Listen sind nicht nur die Benutzer und ihre Zugriffsrechte verzeichnet, sondern auch wer welche Funktionen des Programms benutzen darf. Zusätzliche Programme (*wrapper*) können Kommunikationsverbindungen überprüfen und Datenpakete auch auf dem *relay host* unterdrücken.

Die Sicherheitskriterien, die der *relay host* mit seinen ausgewählten, gesicherten Programmen und Internetverbindungen erfüllt, kann man nun leicht auf den *fire wall router* übertragen. Dazu erlaubt man dem *fire wall router* nur noch, Datenpakete zwischen den Rechnern des LAN und dem *relay host* durchzulassen; alle anderen Daten werden ausgefiltert und gelöscht. Alle Kommunikationsverbindungen laufen damit zwangsweise über den *relay host*; vom Internet gibt es für Eindringlinge in dieser Konfiguration zwei Sicherheitsbarrieren: den *external router* und den *relay host*.

Sparen wir uns den externen Router und kombinieren ihn mit dem *relay host*, so sparen wir zwar Geld, senken aber die Sicherheitsschranke. Eine weitere Möglichkeit besteht

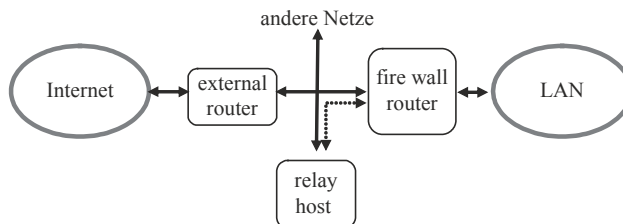
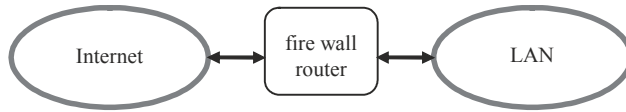


Abb. 7.8 Die klassische fire-wall-Konfiguration





**Abb. 7.9** Die dual-homed-host-Konfiguration

darin, alle drei Funktionen in einem einzigen *fire wall*-Rechner zu integrieren. In [Abb. 7.9](#) ist diese *dual homed host* genannte Sparversion gezeigt.

Auch hier leidet die Sicherheit: Ist es einem Angreifer gelungen, in den *dual homed host* einzubrechen, so ist die Brandschutzmauer vollständig durchbrochen. Aus diesem Grunde gibt es Systeme, die ausgefeiltere Mechanismen zur Benutzerkontrolle anbieten. Eines der bekanntesten ist das Kerberos-System.

---

## 7.11 Die Kerberos-Authentifizierung

Hat man ein sehr großes Netz von Rechnern, etwa 1000 Workstations wie sie beim US-Forschungsinstitut MIT bereits 1988 vorhanden waren, so ist die Strategie von Unix oder Windows NT, sich bei jedem Dienst (Email, Drucker, *remote login*, Dateiserver, etc.) mit Passwort und Nutzernamen anmelden zu müssen, sehr hinderlich. Kann man den Dienst ohne Anmeldung nutzen, so ist das auch nicht besonders attraktiv: Niemand mag es, wenn seine Email von anderen gelesen wird oder seinen Dateien „aus Versehen“ gelöscht werden. Den Kompromiss zwischen beiden Vorgehensweisen stellt beispielsweise ein zentraler Vermittler dar, der alle Nutzer und ihre Passworte sowie alle Dienste und ihre Passworte kennt. Möchte man einen Dienst nutzen, so wendet man sich an ihn und bezieht von ihm die nötigen Passworte und Ausweise gemäß der über einen selbst dort gespeicherten Fähigkeiten und Rollen-Berechtigungen. Eines der am weitesten verbreiteten Systeme ist das Kerberos-Authentifizierungssystem.

Das Kerberos-Sicherheitssystem (Steiner et al. [1988](#)) ist Teil des Athena-Projekts, das für das Netz am MIT eine systematische Verwaltung entwickelte. In einem solchen Netz tritt die Frage auf, ob der anfragende Prozess, der eine Dienstleistung haben will, auch der ist, der er zu sein vorgibt. Das Problem der Authentizität ist also nicht nur auf das Einloggen eines Benutzers beschränkt, sondern gilt für alle Dienstleistungen im Netz. Da es sehr problematisch ist, ein offenes, nicht verschlüsseltes Passwort über mehrere (eventuell angezapfte) Rechner zu schicken, wurden im Athena-Projekt drei verschiedene Protokolle spezifiziert, um das Problem der Sicherheit zu lösen.

Die zentrale Idee besteht darin, alle Anfragen zu verschlüsseln und den Inhalt (Absender usw.) an die Kenntnis des involvierten Schlüssels zu binden. Bei der Verschlüsselung wird im Allgemeinen eine Einwegfunktion  $f$  (*trap function*) verwendet, die nicht einfach umgekehrt werden kann. Der eigentliche Inhalt von  $y = f$  (Nachricht, Schlüssel  $s_1$ ) ist deshalb nicht einfach mit der Kenntnis von  $f$  und  $y$  zu ermitteln. Üblicherweise gibt es dazu eine

Dekodierungsfunktion  $g(\cdot)$ , die mit Hilfe eines zweiten Schlüssels  $s_2$  die ursprüngliche Nachricht wiederherstellen kann:

$$\begin{aligned}\text{Nachricht} &= g(y, s_2) = g(f(\text{Nachricht}, s_1), s_2) \\ &= g_{s_2}(f_{s_1}(\text{Nachricht}))\end{aligned}$$

Gilt für Schlüssel  $s_1 = s_2$  und Kodierung/Dekodierung  $f = g$ , so spricht man von *symmetrischer* Kodierung, ansonsten von *asymmetrischer* Kodierung. Für einen Überblick über solche Verfahren siehe Preneel et al. (1993). Im Kerberos-System wird mit dem DES (Data Encryption Standard) ein symmetrisches Verschlüsselungsverfahren verwendet.

### 7.11.1 Das Kerberos-Protokoll

Die drei Protokolle kommen folgendermaßen zum Einsatz:

- *Benutzerauthentifizierung (Single Sign on Protocol)*

Beim Einloggen des Benutzers mit dem Kerberos läuft alles zunächst wie gewohnt ab: Der `kinit()`-Prozess wird gestartet und der Benutzer tippt sein geheimes Passwort ein. Dies wird in einen DES-Schlüssel  $S_0$  umgewandelt und zunächst lokal gespeichert. Dann schickt `kinit()` den Benutzernamen im Klartext an einen speziellen Prozess, den *Authentication Server* AS. Dieser sucht in seiner zentralen Datenbank nach und ermittelt das Passwort des Benutzers und damit auch den dazu gehörenden DES-Schlüssel  $S_0$ . Sodann stellt er ihm einen Sitzungsausweis  $T_G$  (*ticket*) aus (Zeit, NutzerName etc., verschlüsselt mit seinem geheimen Schlüssel  $S_T$ ) und erwürfelt sodann einen Zufallsschlüssel  $S_1$  für alle weitere Kommunikation mit dem Benutzer, den er aber mit Hilfe von  $S_0$  verpackt (verschlüsselt). Beides wird zusammen als Nachricht an den Benutzer (bzw. den lokalen `kinit()`-Prozess) geschickt. Die Nachricht besteht also aus zwei Teilen

$$[\text{User}, S_1]_0, T_G = [\text{User}, \text{Zeit}]_T$$

- wobei eine Verschlüsselung jeweils mit Klammern und dem Schlüsselindex notiert sei. Mit  $S_0$  dekodiert `kinit()` den ersten Teil der Nachricht und speichert den Schlüssel  $S_1$  ab; der Schlüssel  $S_0$  kann sofort vernichtet werden und wird nicht mehr benutzt. Wozu enthält der erste Teil redundante Information? Nun, zum einen weiß der Benutzer, wenn er seinen Namen richtig herausbekommt, dass er richtig dekodiert hat, zum anderen, dass der Schlüssel valide ist, da nur der echte AS den Namen mit  $S_0$  verpacken kann. Der zweite Teil, der Sitzungsausweis  $T_G$ , kann nicht entschlüsselt werden; so wie er ist wird er einfach nur lokal abgespeichert. Damit ist der Benutzer so lange zentral registriert wie der Sitzungsausweis gilt. In Abb. 7.10 ist dies verdeutlicht.

Die Rahmenschattierung eines Objekts entspricht der Schlüsselschattierung, mit der das Objekt verschlüsselt wurde.

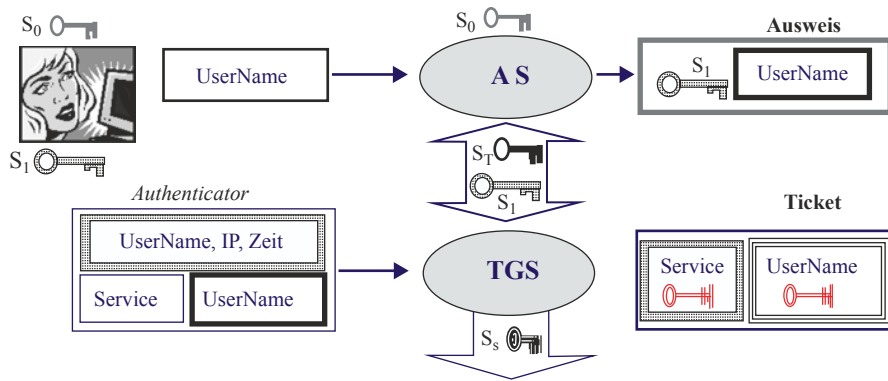


Abb. 7.10 Sitzungsausweis und Transaktionsausweis

- *Schlüsselverteilung (Key Distribution Protocol)*

Möchte nun der Benutzer einen Dienst, etwa *remote login*, nutzen, so setzt er den Befehl „rlogin“ oder ähnliches ab. Der so entstandene Benutzerprozess schickt kein „rlogin“ im Klartext zum Server; einen solchen Dienst gibt es nicht mehr. Stattdessen müssen alle Anfragen fälschungssicher gemacht werden. Allerdings schickt der Prozess auch nicht sofort eine verschlüsselte Anfrage zum Server: Dieser weiß ja gar nicht, mit welchem Schlüssel die ankommende Nachricht verschlüsselt ist. Stattdessen fragt der Benutzerprozess zuerst bei einer neutralen Instanz, dem *Transaction Grant Server* TGS (dem Bruder vom AS) nach einer Erlaubnis bzw. dem Passwort des Dienstes mittels einer Anfrage

$$\text{Service}, [\text{User}, \text{Zeit}]_1, T_G$$

zum TGS. Dieser entschlüsselt zuerst den Ausweis  $T_G$  mit seinem Standardgeheimsschlüssel und sieht so, dass ein „User“ etwas von ihm will. Der User hat vom AS in der mit dem TGS gemeinsamen Datenbank den Vermerk „Sitzungsschlüssel  $S_1$ “ bekommen, so dass der TGS auch den Abschnitt davor, den sog. „Authentifikator“ entschlüsseln kann. Er vergleicht den darin befindlichen Benutzernamen mit dem in Frage kommenden aus  $T_G$ ; stimmen sie überein, so hat sich der Benutzer ausgewiesen. Ist der verlangte Service auch für ihn freigegeben, so stellt der TGS eine Antwort der Form

$$[\text{Service}, S_2]_1, T_T = [\text{User}, S_2, \text{Zeit}]_s$$

aus. Die Antwort enthält wieder in verschlüsselter Form den Namen des verlangten Dienstes zur Kontrolle sowie den Schlüssel für die Kommunikation sowie ein Ticket  $T_T$ , das in nicht fälschbarer Form den Kontext enthält.

- *Transaktion durchführen (Authentication Protocol)*

In dem durch die kurze Lebensdauer der Transaktion (ca. 5 Minuten) spezifizierten Zeitraum kann nun der Client eine Transaktion mit dem Ticket  $T_T$  einleiten:

$$\text{Anfrage, } [\text{User, Zeit}]_2, T_T$$

Der Server entschlüsselt mit seinem geheimen Serverschlüssel  $S_s$  das Ticket  $T_T$ , kann daraufhin mit dem daraus entnommenen Schlüssel  $S_2$  den Authentikator des Benutzers entschlüsseln und beide Namen „User“ miteinander vergleichen. Nun weiß er, dass die Anfrage wirklich von „User“ stammt und zum Service berechtigt (**Authentifikation**). In Abb. 7.11 ist dies visualisiert. Seine Antwort mit den Daten verschlüsselt er deshalb mit dem gleichen Transaktionsschlüssel:

$$[\text{Ergebnis}]_2$$

Eventuell legt er auch noch einen weiteren Schlüssel  $S_3$  bei, mit dem die darauf folgende Kommunikation, etwa alle Kommandos für das *rlogin*, verschlüsselt werden.

In obigen Beispielen steht übrigens „User“ stellvertretend für den Benutzernamen, IP-Adresse usw.

### 7.11.2 Kerberos Vor- und Nachteile

Die Authentifikation in obiger Form hat verschiedene Vorteile:

- Eine Kopie der Ausweise oder Tickets ist zwar möglich, aber sie lässt sich nicht zur Täuschung einsetzen, da für jede Anfrage der Authentikator fehlt. Dieser kann aber nur gefälscht werden, wenn der entsprechende Schlüssel bekannt ist, was aber ohne den entsprechenden Schlüssel, also das Passwort, nicht einfach ist.
- Eine Fälschung der Ausweise oder Tickets ist nicht möglich, da die Schlüssel von TGS und Dienst unbekannt sind.
- Alle Tickets und Authentifikatoren lassen sich auch vom Benutzer selbst nicht beliebig aufbewahren und verwenden – durch die Zeitvalidierung kann man die Benutzerrechte einschränken, ohne dass ihm alte Ausweiskopien weiterhelfen.

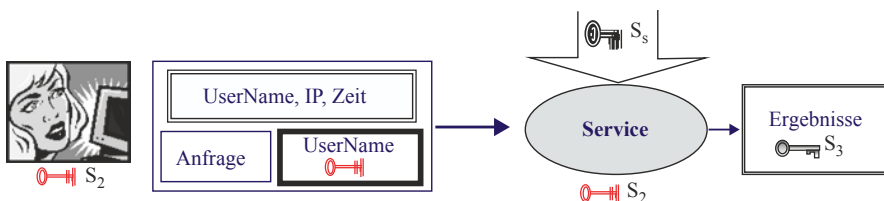


Abb. 7.11 Transaktions- (Service-) durchführung

- Da das Benutzerpasswort nur einmal eingegeben werden muss, bemerkt der Benutzer von dem ganzen Authentifikationsprotokoll nichts weiter – eine wichtige Voraussetzung für die Akzeptanz.

Die oben skizzierten drei Protokolle werden in mehreren gekoppelten LANs (**Domänen**) nun durch ein viertes ergänzt: Möchte ein Client eine Leistung eines Servers nutzen, der in einem anderen LAN arbeitet, so schickt er zuerst eine Anfrage an den TSG „seines“ LANs für eine geschützte Transaktion mit dem TSG des Nachbar-LANs, dem vertraut wird. Ist dies erfolgreich, so wird in der Transaktion nach einem Transaktionsticket für den weiter benachbarten TSG gefragt, und so fort, bis der Client schließlich ein Ticket für den TSG des LANs mit dem gewünschten Server hat. Nun kann er die gewünschte Leistung direkt beim Server mit dem erhaltenen Ticket und Schlüssel durchführen, als ob dieser direkt im LAN wäre.

Das Kerberos-System ist zwar ziemlich sicher; es gibt aber einige Schwierigkeiten dabei zu überwinden:

- Der Dienst funktioniert nur, wenn die Uhren aller Rechner synchron gehen und damit die Zeitschranken überprüft werden können. Dies erfordert in Gegenden ohne Funkzentraluhr (z. B. USA) ein Zeitauthentifizierungssystem, um eine Manipulation der Uhren zu verhindern – ein Unding, da ja auch dieses auf einem solchen Dienst aufgebaut sein müsste.
- Die eingebauten festen Zeitschranken für die Transaktionsschlüssel und Sitzungsschlüssel (z. B. 5 Minuten und 8 Stunden) erzeugen Probleme, wenn Transaktionen länger dauern oder Benutzer länger arbeiten. Hier wird eine Ticket-Eigenschaft „verlängerbar“ wichtig, die aber schon existiert.
- Transaktionsschlüssel und -tickets werden in Dateien abgelegt, die in Multi-user-Umgebungen von anderen evtl. gelesen werden könnten. Lokale Zugriffsbeschränkungen sind deshalb besonders hier wichtig.

Es ist deshalb noch ein langer Weg zu einem sicheren System, besonders wenn mehrere Systeme mit unterschiedlichen Sicherheitsmechanismen zusammengekoppelt werden. Aus diesem Grund gibt es Bestrebungen der X/Open-Gruppe, mit einheitlichen Sicherheitsrichtlinien XBSS (*X/Open Baseline Security Specifications*) lückenlose Sicherheit in Netzen zu erzielen.

### 7.11.3 Aufgaben zur Netz-Sicherheit

#### Aufgabe 7.11-1 Firewall

Was sind die Nachteile eines *firewall*-Systems, das alle Datenpakete untersucht?

#### Aufgabe 7.11-2 Recherchieren Sie: Anonymität im Netz

- a) Nennen Sie mindestens vier verschiedene Kriterien, anhand deren man beim Surfen im Internet identifiziert werden kann. Erklären Sie, wie bzw. warum anhand dieser Kriterien eine Identifikation des Nutzers möglich ist.

- b) Erklären Sie Gegenmaßnahmen, um eine Identifikation anhand der unter a) genannten Kriterien zu verhindern oder zu erschweren.

---

#### Aufgabe 7.11-3 Recherchieren Sie: Netzattacken

- a) Erklären sie, worum es sich bei einem so genannten *Honey Pot* handelt und wie ein solcher funktioniert.
- b) Wobei handelt es sich bei den so genannten DoS Attacken? Wie funktionieren diese und wie kann man sich gegen sie schützen?
- c) Erklären sie das Prinzip des „*Port Knocking*“.

---

#### Aufgabe 7.11-4 Kerberos

- a) Was ist der Unterschied zwischen Autorisierung und Authentifikation?
- b) Wozu werden in Kerberos zwei Schlüssel  $S_1$  und  $S_2$  statt einem benötigt? Was ist der Unterschied zwischen den Schlüsseln?

---

## Literatur

COPS: siehe <ftp://ftp.cert.org/pub/cops> im World Wide Web

DFN: siehe <http://www.cert.dfn.de> im World Wide Web

Embleton S., Sparks S., Zou C.: SMM Rootkits: A New Breed of OS Independent Malware, Security and Communication Networks, 12(6) 1590–1605, December 2013, <http://www.eecs.ucf.edu/~czou/research/SMM-Rootkits-Securecom08.pdf> [10.1.2016]

Farroes R.: UNIX System Security. Addison-Wesley, Reading, MA 1991

Groll M.: Das UNIX Sicherheitshandbuch. Vogel Verlag, Würzburg 1991

Kienle M.: Firewall Konzepte: Sicherheit für lokale Netze ohne Diensteeinschränkung. iX, Verlag Heinz Heise, Hannover, Juli 1994

Preneel B., Govaerts R., Vandewalle I. (eds): Computer Security and Industrial Cryptography. LNCS 741, Springer-Verlag Berlin, 1993

Russinovich M., Cogswell B.: RootkitRevealer, 2006, <http://www.sysinternals.com> (freeware)

Rutkowska J.: IsGameOver() Anyone? Presented at *Black Hat, USA*. Aug 2007, <http://invisiblethingslab.com/resources/bh07/IsGameOver.pdf> [10.1.2016]

SAT: siehe <http://www.trouble.org> im World Wide Web

Spafford E., Heaphy K., Ferbrache D.: A Computer Virus Primer. In: P. Denning (ed.): Computers Under Attack. Addison-Wesley, Reading, MA 1990

Steiner J. G., Neumann B. C., Schiller I.: Kerberos: An Authentication Service for Open Network Systems. Proc. Winter USENIX Conference, Dallas, TX, pp. 191–202, 1988

Wojtczuk R., Rutkowska J., Tereshkin A.: Proc. Black Hat conf. 2008

1. Subverting the Xen Hypervisor,  
<http://invisiblethingslab.com/resources/bh08/part1.pdf> [10.1.2016]
2. Detecting & Preventing the Xen Hypervisor Subversions,  
<http://invisiblethingslab.com/resources/bh08/part2-full.pdf> [10.1.2016]
3. Bluepillling the Xen Hypervisor,  
<http://invisiblethingslab.com/resources/bh08/part3.pdf> [10.1.2016]

Inhaltsverzeichnis

8.1 Das Design der Benutzeroberfläche..... 352

8.2 Die Struktur der Benutzeroberfläche..... 355

8.2.1 Eingaben ..... 356

8.2.2 Rastergrafik und Skalierung..... 360

8.2.3 Fenstersysteme und Displaymanagement ..... 362

8.2.4 Virtuelle Realität ..... 365

8.2.5 Das Management der Benutzeroberfläche..... 365

8.2.6 Aufgaben ..... 367

8.3 Das UNIX-Fenstersystem: Motif und X-Window..... 368

8.3.1 Das Client-Server-Konzept von X-Window..... 369

8.3.2 Das Fensterkonzept von X-Window ..... 370

8.3.3 Dialogfenster und Widgets..... 371

8.3.4 Ereignisbehandlung ..... 373

8.3.5 Sicherheitsaspekte ..... 374

8.4 Das Fenstersystem von Windows NT ..... 375

8.4.1 Das Konzept der Benutzerschnittstelle ..... 375

8.4.2 Die Implementierung..... 377

8.4.3 Aufgaben ..... 379

Literatur..... 380

Der Benutzer eines Programms kann nicht sehen, was sich im Rechner abspielt. Deshalb ist es für die Bedienung eines interaktiven Programms wichtig, die Ein- und Ausgabe so zu konzipieren, dass der Benutzer immer weiß, was von ihm erwartet wird. Die Art und Weise, wie die Aktionen zwischen den Programmen eines Rechners und dem Benutzer gestaltet werden, wird als **Benutzeroberfläche** (Bedienoberfläche, Bedieneroberfläche, *user interface*) bezeichnet. Dabei spielen Eingabe sowie Darstellung und Ausgabe der

verarbeiteten Daten eine wichtige Rolle. Neben den traditionellen, menschenlesbaren Ausgabegeräten wie Drucker und Plotter ist heute vor allem die grafische Darstellung (*Visualisierung*) der Daten am Bildschirm in den Vordergrund gerückt.

Allerdings stellt die grafisch-interaktive Benutzeroberfläche heutiger Computer hohe Anforderungen an die Rechnerressourcen und ist deshalb sowohl vom Codeumfang als auch vom Ressourcenverbrauch (CPU, Speicher usw.) einer der Hauptteile eines personenbezogenen Rechners geworden, die fest im Betriebssystem verankert sein müssen, um effizient zu funktionieren. Dazu werden auf der Hardwareseite spezialisierte Grafikkarten mit Grafikprozessoren und schnellem Bildwiederholtspeicher vorgesehen. Auch im Hauptprozessor sollen zusätzliche Maschinenbefehle für Grafik (z. B. Intel MMX) helfen, die Programmabarbeitung zu beschleunigen.

Die Unterstützung des Betriebssystems für die grafisch-interaktive Benutzeroberfläche ist deshalb ein wichtiges Anliegen an die Architektur aller modernen Betriebssysteme. Aus diesem Grund wollen wir uns in diesem Abschnitt näher mit den Anforderungen, Konzepten und Implementierungsfragen von Benutzeroberflächen im Hinblick auf die traditionellen Betriebssystemteile und Rechnerressourcen beschäftigen.

---

## 8.1 Das Design der Benutzeroberfläche

Die heutigen grafischen Benutzeroberflächen basieren auf den grundlegenden Arbeiten der Xerox-Forschungsgruppen um den STAR-Computer und das Smalltalk-80-Projekt Anfang der 80er Jahre. Sie konzipierten nicht nur einen Einpersonencomputer mit einem seitengroßen Rastergrafik-Bildschirm (für damalige Verhältnisse revolutionär und viel zu kostspielig, weshalb der Xerox-STAR und seine Kopie, das Apple-Lisa-System, keine kommerziellen Erfolge wurden), sondern entwickelten auch systematisch sowohl die radikal objektorientierte Sprache als auch die Benutzeroberfläche dazu. Dabei modellierten sie auf dem Bildschirm die Büroumgebung dadurch, dass jedem Gegenstand wie Papierseite, Papierkorb, Drucker, Ablage, Ordner etc. ein symbolisches Bild (**Ikon**) zugeordnet wurde, aus dem man seine Bedeutung erkennen kann. Die normalen Aktionen (wie Mappe öffnen, Dokument ablegen usw.) wurden als Aktionen zwischen den Ikonabbildern der Realität auf einer imaginären Schreibtischoberfläche modelliert (**Schreibtischmetapher**).

Motiviert durch experimentelle Erfahrungen, stellten sie folgende Leitideen auf:

- Statt sich viele Tastensequenzen für die Aktionen zu merken und sie jeweils einzutippen, soll der Benutzer die Aufgabe direkt zeigen können, die er erledigen will. Dazu entwickelten sie ein **Zeigedepot**, die „Maus“. Die Anzahl ihrer Tasten ergab sich in Experimenten optimal zu zwei; drei waren zu kompliziert zu bedienen, und eine war zu wenig.
- Das **Selektieren** von Text durch Ziehen der Maus oder Mehrfachklicken zum Selektieren unterschiedlich großer Textteile wurde hier zum ersten Mal verwirklicht.



- Statt sich alle möglichen Kommandos zu merken und einzutippen, sollten sie von der jeweiligen Anwendung als Auswahlliste (**Menü**) angeboten werden. Diese kann auch hierarchisch mit Untermenüs strukturiert sein („progressiver Ausschluss“ möglicher Befehle).
- Die Anzahl der Kommandos sollte möglichst gering sein, die Kommandos selbst möglichst universal und unabhängig (**orthogonal**) voneinander in ihren Auswirkungen. Folgende Befehle wurden als ausreichend angesehen: MOVE, COPY, DELETE, UNDO, HELP und ShowProperties. Beispielsweise kann man mit COPY sowohl eine Textzeile im Editor als auch ein ganzes Dokument oder ein Bild kopieren, obwohl dies jeweils unterschiedliche Funktionen impliziert. Bei „ShowProperties“ handelt es sich um die Präsentation eines interaktiv änderbaren Datenblatts, das die Aus- und Eingabe von Kontextparametern gestattet, etwa von Schriftgröße, Stil, Art etc. bei einem Textobjekt.
- Die Universalkommandos wurden sowohl speziell reservierten Funktionstasten zugeordnet (**shortcut**) als auch als Menü am rechten Mausknopf angeboten.
- Alle Daten (Properties) und Dokumente entstehen durch Veränderung bestehender Daten und Anpassung an die spezielle Aufgabe, ähnlich einer Objektvererbungshierarchie. Deshalb existierte auch kein CREATE-Kommando; die Autoren sahen es als zu komplex für Menschen an.
- Der Benutzer sollte nur immer auf einer Programmebene sein, von der aus er alle Aktionen starten kann, und nicht in einem Modus landen, bei dem er nicht mehr weiß, wo in der Eingabehierarchie er sich befindet und wie er wieder herauskommt („Don't mode me in“). Dazu wurden z. B. bei Smalltalk alle Operationen in polnischer Notation erfasst: Zuerst werden die Argumente ausgewählt, dann die Funktion für die Argumente.
- Das Aussehen der Dokumente auf dem Bildschirm entspricht dem tatsächlichen Ausdruck (*What You See Is What You Get*: **WYSIWYG**). Dazu wurde der Bildschirm wie ein Blatt Papier hochkant gestellt und verschiedene Buchstabenfonts und -größen eingeführt.

Als problematisch wurde die Abbildung der physikalischen Geräte auf ihre elektronische Simulation (Ikon) empfunden: Was sollte z. B. mit einem Dokument geschehen, wenn man es auf ein Druck-Ikon zieht? Soll es auf dem Schreibtisch verbleiben und eine Kopie davon im Drucker? Dies gibt es nicht in der Wirklichkeit. Oder soll es im Drucker verschwinden? Dann müsste es danach auch gelöscht sein S oder soll es nach dem Druck wieder „herauskommen“?

Es ist erstaunlich, wie selbstverständlich vieles von der Schreibtischmetapher heutzutage Eingang in die Benutzeroberfläche von Computern gefunden hat, ohne dass ihr konkreter Nutzen experimentell belegt wird. Statt optimale Einstellungen für die Fenster und Ikone (z. B. Komplexität, Farbe etc.) auf fundierter Grundlage zu bestimmen, wird es dem Benutzer überlassen, selbst das Beste für ihn herauszufinden (*Benutzeranpassung der Oberfläche*).

Allerdings haben sich auch einige allgemeine Designprinzipien herauskristallisiert, die unverbindlich als *style guides* vorgeschlagen für jede konkrete Benutzeroberfläche

mit Inhalt gefüllt werden müssen. Sie sollen verhindern, dass ein bestehendes Element in seinem Verhalten geändert wird und neue Elemente hinzugefügt werden, wenn die gewünschten Funktionen schon von der Oberfläche unterstützt werden. Beispielsweise sollen gelten (Microsoft 1995):

- *Benutzerkontrolle (user in control)*

Der Benutzer muss immer die Kontrolle über den Rechner haben und nicht nur reagieren. Dies bedeutet, auch automatische Arbeitsgänge kontrollierbar zu machen sowie keine Zustände (*modes*) in der Benutzerschnittstelle zu verwenden (s.o.). Falls es sich aber nicht umgehen lässt, sollte dieser Zustand sichtbar sein und einfach abgebrochen werden können.

- *Rückkopplung (feedback)*

Die Benutzeroberfläche sollte ein Eingabeecho oder eine Reaktion bereitstellen für den Benutzer, beispielsweise mit visuellen oder auditiven Mitteln, um ihm zu zeigen, dass alles seinen Wünschen entsprechend funktioniert. Nichts ist frustrierender als ein „toter“ Bildschirm, der auf alle Eingaben scheinbar nicht reagiert.

- *Beispiele*

- Der Mauszeiger kann seine Form ändern in einem Fenster oder während der Bearbeitung.
- Das Objekt kann sein Aussehen charakteristisch ändern.
- Ein Menüeintrag „verblasst“, wenn er gerade gesperrt ist.
- Eine Statuszeile zeigt numerische oder textuelle Fortschritte an.
- Ein spezielles Fenster visualisiert als Fortschrittsanzeiger (*progress indicator*) den Fortgang der Bearbeitung.
- Ein Nachrichtenfenster (*pop-up message box*) erscheint.

- *Visualisierung (directness)*

In Programmen, die Daten bearbeiten, zeige man ein visuelles Äquivalent der Daten und der auf ihnen möglichen Funktionen. Dabei ist es sinnvoll, dem Benutzer geläufige Metaphern (z. B. Schreibtischmetapher) zu verwenden. So kann sein vorhandenes Vorwissen gut eingesetzt werden; er „weiß“, was er von der Oberfläche erwarten kann und wie sie funktionieren sollte. Man kann auch andere Metaphern verwenden, wenn dies angebracht ist. Beispiele dafür sind die Metapher „Arbeitsbuch“ (*work book*), in dessen Seiten man blättern und Bilder und Texte hineintun oder herausnehmen kann, und die Metapher „Pinnwand“ (*pin board*), um Nachrichten und Texte zum Austausch zwischen Menschen zu präsentieren.

- *Konsistenz (consistency)*

Die Benutzeroberfläche sollte nicht plötzlich anders reagieren, als in ähnlichen Situationen gewohnt: Ähnliche Operationen sollten eine ähnliche Visualisierung und ähnlichen Ablauf haben.

*Beispiele*

- Ein nicht auswählbares Menü sollte nicht weggelassen werden, sondern aus Konsistenzgründen mitgezeigt werden.

- Wenn bei der Operation „Daten sichern“ bei einer Applikation die Daten sofort gesichert werden, bei der anderen aber zuerst ein Dateiname in einem Dateiauswahlmenü angegeben werden muss, so ist dies nicht konsistent.
- Ein Mausklick selektiert in allen Programmen immer ein einziges Objekt oder eine einzige Koordinate. Wird der Mausknopf gedrückt gehalten und die Maus verschoben (*dragging*), so soll immer ein ganzer Bereich selektiert werden, beispielsweise in einem Text oder auf einem grafischen Feld.
- *Einfachheit (simplicity)*

Bei der visuellen Präsentation muss immer ein Kompromiss gefunden werden, um nicht zuviel und nicht zuwenig Information zu geben. Techniken dafür sind

  - nur kurze und prägnante Mitteilungen und Kommandos verwenden,
  - *progressive disclosure*: Information erst dann zeigen, wenn sie nötig ist; Sachverhalte werden zuerst grob und dann (bei Bedarf) immer feiner detailliert dargestellt.

Ein Beispiel für das Designprinzip „Einfachheit“ ist das Design der Gefahrenmeldungen beim Airbus A340. Im Unterschied zum traditionellen Design, bei dem alle wichtigen und unwichtigen Fehlerlampen nebeneinander in Feldern im Cockpit aufgereiht sind, werden bei diesem modernen, computergesteuerten Flugzeugtyp die Fehlermeldungen auf dem Hauptmonitor eingeblendet. Dabei wird nicht jeder Fehler aufgelistet, sondern nur immer der zum Zeitpunkt wichtigste und kritischste. In Übersichtszeichnungen kann der Pilot sich dann anzeigen lassen, an welcher Stelle im Flugzeug, in welchem System und in welchem Kontext der Fehler aufgetreten ist.
- *Ästhetik (Aesthetics)*

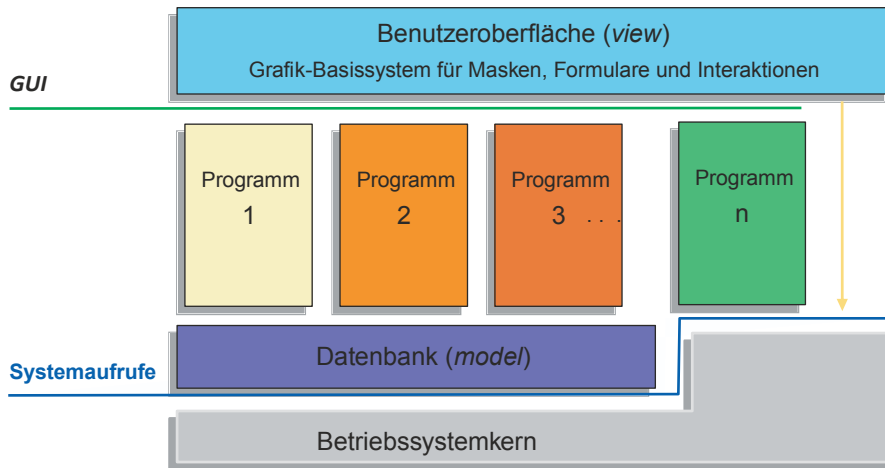
Eine Oberfläche sollte nicht nur funktionalen Erfordernissen genügen; für den Benutzer sind vielmehr auch ästhetische Aspekte wichtig: Ein farbiger Bildschirm sieht hübscher aus als ein grauer. Allerdings muss man darauf achten, nicht die Einfachheit dabei preiszugeben: Barocke Bildschirme verwirren nur. Die grafischen Elemente sollten sinnvoll und übersichtlich gegliedert sein. Da alle Elemente um die Aufmerksamkeit des Benutzers konkurrieren, sollte man sich gut überlegen, worauf man Aufmerksamkeit lenken will und warum.

---

## 8.2 Die Struktur der Benutzeroberfläche

Eine komplexe grafische Benutzeroberfläche kann es einem geübten Benutzer leichter machen, das Anwendungsprogramm zu bedienen; sie kann aber auch eine zusätzliche Hürde darstellen und die Bedienung erschweren. Aus diesem Grund ist es zum einen besonders wichtig, die Benutzeroberfläche sehr sorgfältig zu konzipieren, und zum anderen, die einmal eingeführte Oberfläche möglichst nicht zu ändern.

Diese Forderungen werden am ehesten dadurch erfüllt, dass nicht jedes Programm seine eigene, individuell und damit teuer programmierte Oberfläche erzeugt, wie es in der Vergangenheit üblich war, sondern dass von dem Betriebssystem bereits eine einheitliche



**Abb. 8.1** Benutzeroberfläche und Gesamtsystemstruktur

Benutzeroberfläche zur Verfügung gestellt wird. Die Funktionalität sollte möglichst viele Anwendungen unterstützen und besteht deshalb meist nur aus allgemeiner Ein- und Ausgabeunterstützung wie Funktionstasten, Maus, Fenster, Ein- und Ausgabemasken etc. Diese Funktionen werden in einer Schnittstelle (*Graphical User Interface GUI*) zusammengefasst, die vom Anwendungsprogrammierer über eine Standardschnittstelle (*Application Programming Interface API*) mit Hilfe von Bibliotheksprozeduren angesprochen werden können. In der folgenden [Abb. 8.1](#) ist ein Übersichtsschema in der Schichtennotation für die Einordnung einer solchen Benutzerschnittstelle gezeigt.

Ein besonderes Problem liegt vor, wenn man eine grafikorientierte Benutzeroberfläche durch ein Terminal mit beschränkten grafischen Möglichkeiten darstellen will.

Dies wurde beispielsweise mit dem POINT-Projekt (Verband Deutscher Maschinen und Anlagenbau VDMA) versucht, das die Benutzeroberfläche nicht nur auf einem vollgrafischen Farbbildschirm-Arbeitsplatz, sondern auch auf einem rein Schwarzweiß-ASCII-Terminal unterstützt. Spezielle Displaymanager übernehmen dann die (unvollständige) Abbildung der Ausgabefunktionen wie Fensteraufbau und Aktualisierung einer Statuszeile.

## 8.2.1 Eingaben

Die klassische Art und Weise, Texte und Befehle in einen Computer zu transferieren, ist die Tastatur. Üblicherweise existieren neben den reinen Buchstaben des Alphabets zusätzliche Tasten, die spezielle Kontrollfunktionen ausüben. Neben den Tasten ↑, ↓, →, ← zum Verschieben des Aufmerksamkeitspunktes (Eingabemarkierung, *cursor*) auf dem Bildschirm

|    | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8  | 9  | A   | B   | C  | D  | E  | F   |
|----|-----|-----|-----|-----|-----|-----|-----|-----|----|----|-----|-----|----|----|----|-----|
| 0X | NUL | SOH | STX | ETX | EOT | ENQ | AC  | BEL | BS | HT | NL  | VT  | NP | CR | SO | SI  |
| 1X | DLE | DC1 | DC2 | DC3 | DC4 | NA  | SYN | ETB | CA | EM | SUB | ESC | FS | GS | RS | US  |
| 2X | SP  | !   | „   | #   | \$  | %   | &   | ‘   | (  | )  | *   | +   | ,  | -  | .  | /   |
| 3X | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8  | 9  | :   | ;   | <  | =  | >  | ?   |
| 4X | @   | A   | B   | C   | D   | E   | F   | G   | H  | I  | J   | K   | L  | M  | N  | O   |
| 5X | P   | Q   | R   | S   | T   | U   | V   | W   | X  | Y  | Z   | [   | \  | ]  | ^  | _   |
| 6X | `   | a   | b   | c   | d   | e   | f   | g   | h  | i  | j   | k   | l  | m  | n  | o   |
| 7X | p   | q   | r   | s   | t   | u   | v   | w   | x  | y  | z   | {   |    | }  | ~  | DEL |

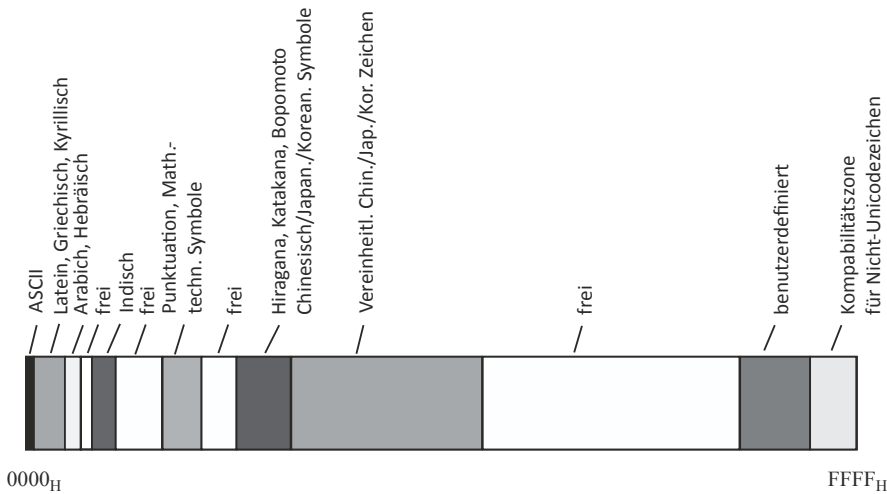
Abb. 8.2 Die ASCII-Buchstabenkodierung

gibt es noch Tasten für das Umschalten von Klein- auf Großbuchstaben (SHIFT), von einem Buchstabensatz auf einen anderen (ALTERNATE) und von Buchstabeneingabe auf Befehle (CONTROL).

Die Gesamtmenge der so erzeugten Tastenkombinationen (symbolische Eingaben) wird nun vom Tastaturprozessor auf einen internen Zahlencode abgebildet, der vom Betriebssystem-Gerätetreiber in den offiziellen, international genormten Zahlencode für die Buchstaben umgesetzt wird. Bedingt durch die Herkunft der ersten Computer ist dies das amerikanische Alphabet, standardisiert in 128 Zeichen. Der derart mit 7 Bits beschriebene Zahlencode, der *American Standard Code for Information Interchange* ASCII, ist in [Abb. 8.2](#) zu sehen. Außen an der Tabelle ist der Zahlencode in hexadezimaler Schreibweise notiert. Der Wert der ersten vier Bits ist am linken Rand vertikal eingetragen, der der zweiten vier Bits horizontal am oberen Rand. Die beiden ersten Zeilen der Tabelle enthalten nur Kontrollzeichen wie Eingabeende EOT, Horizontaltabulator HT, Zeilenrücklauf CR usw. Nun besteht die Welt nicht nur aus Nordamerika, so dass sich bald die Notwendigkeit ergab, für eine verständliche Ausgabe auch europäische Zeichen wie ü, ä, æ und ê in den bestehenden Code aufzunehmen. Dies führte zu einem 8-Bit-ANSI-Code, der von der ISO (*International Standards Organization*) genormt wurde, beispielsweise *Latin-1* (ISO 8859-1) für Westeuropa und *Latin-2* für Osteuropa.

Allerdings hört die Welt auch nicht bei Europa auf, so dass die Notwendigkeit, verschiedene Softwarepakete wie Texteditoren auch für den arabischen, chinesischen und indischen Markt zu schreiben, bald dazu führte, die Zeichenkodierung auf mehr als 8 Bit auszudehnen, siehe (Madell et al. 1994).

Ein wichtiger Versuch in diese Richtung ist die Entwicklung eines „universellen“ Codes, des **Unicode**, in dem alle Schriftzeichen der Weltsprachen enthalten sind (UNI 1997). Zusätzlicher, freier Platz in der Codetabelle garantiert ihre Erweiterbarkeit. In [Abb. 8.3](#) ist die Auslegung des Unicode, beginnend mit dem 16-Bit-Code 0000<sub>H</sub> und endend mit FFFF<sub>H</sub>, gezeigt. Man kann erkennen, dass der Unicode als Erweiterung des ASCII-Codes konzipiert wurde, um Verträglichkeit mit den bestehenden Standardsystemen zu



**Abb. 8.3** Auslegung des Unicodes

ermöglichen. Obwohl es sehr viele Schriftarten (*Fonts*) gibt, kann man beim Unicode mit einer 16-Bit-Kodierung auskommen, da die Information, *welches* Zeichen verwendet wird, streng von der Information getrennt wird, *wie* es verwendet wird. Alle Formatierungsinformationen (Schriftart, Darstellungsart fett/ schräg usw.) werden vom Zeichencode getrennt verwaltet und sind typisch für den jeweiligen Texteditor. Der Unicode ist genormt (ISO 10646) und wird ständig weiterentwickelt.

Ein weiterer wichtiger Code ist der 32-Bit-(4-Byte)-**Extended-UNIX-Code** EUC, der eine Multibyte-Erweiterung des UNIX(POSIX)-US-ASCII-Zeichensatzes darstellt. Asiatische und andere komplexe Schriftzeichen werden als Folge mehrerer ASCII-Buchstaben auf der normalen Tastatur eingegeben und als landesspezifischer EUC abgespeichert.

#### Beispiel UNIX Eingabecode

Seit dem Beginn wird in UNIX bzw. POSIX ausschließlich der US-ASCII-Zeichensatz verwendet. Erst in neueren Versionen wird auf nationale Besonderheiten eingegangen. Beispielsweise installierte die Firma Hewlett-Packard eine spezielle Erweiterung, das *Native Language Support* NLS, die auch in Linux verwendet wird. Sie besteht zum einen aus einer Reihe von Umgebungsvariablen (LANG, LC\_XX), die in der Prozessumgebung eines Benutzers, wenn sie korrekt gesetzt werden, den lokalen Teil der Bibliotheksroutinen und sprachspezifischen Kommandos (*ed*, *grep* usw.) des Betriebssystems steuern. Hierzu gehören nicht nur die sprachspezifischen Besonderheiten wie die richtige Verwendung von Multibyte-Codes bei String-Vergleichen (z. B. ss = ß im Deutschen, LLL = LL im Spanischen) oder die Identifizierung von Buchstaben-codes als Großbuchstaben, Zahlen, Kontrollzeichen usw., sondern auch die korrekte,

landesübliche Anwendung von Punkt und Komma bei numerischen Ausdrücken, (z. B. 1.568,74, Datumsformat) sowie die Bezeichnung der Landeswährung.

### **Beispiel Windows NT *Eingabecode***

Im Gegensatz zu UNIX wurde in Windows NT bereits beim Design ein Multinationaler Zeichencode für das Betriebssystem festgelegt, der Unicode. Dies bedeutet, dass alle Zeichenketten, Objektnamen, Pfade usw. nur Unicode-Zeichen verwenden. Es sind also auch chinesische Objektnamen in indischen Pfaden auf deutschen Computern möglich!

Die landestypischen Gegebenheiten wie Zeitzone, Währungsnotation, Sprache usw. wird unabhängig davon bei der Einrichtung des Systems festgelegt und zentral abgespeichert.

Neben der Tastatur für die Eingabe von Buchstaben gibt es heutzutage noch weitere, menschengerechtere Eingabemöglichkeiten.

- *Funktionstasten*

Eine einfache Erweiterung bietet die Möglichkeit, Tasten für besondere Funktionen bereitzustellen. Dies ist ein konzeptionell wichtiger Schritt, da hiermit neben der Eingabe alphanumerischer Daten auch das Ansprechen von Funktionen ermöglicht wird. Es ist deshalb logisch wichtig, die konzeptionell getrennten Daten und Funktionen im Programm auch beim Eingabemedium getrennt zu halten, um Verwechslungen zu vermeiden. Nur eine schlechte Benutzeroberfläche vermischt beide Funktionen, etwa das Anhalten der Ausgabe mit CTRL S oder, noch schlimmer, die Steuerung von Programmen durch einzelne Buchstabentasten.

- *Zeigegeräte*

Einen wichtigen Schritt zur analogen Eingabe bedeutete die Einführung von Zeigegegeräten wie „Maus“ oder „Trackball“. Hier kann die Position eines Zeigers auf dem Bildschirm leichter kontrolliert werden als mit speziellen Funktionstasten (Cursortasten).

- *Grafische Tablett*s

Eine große Hilfe bedeutet auch die direkte Übermittlung der Position eines Stifts auf einem speziellen elektronischen Eingabebrett. Obwohl seine prinzipielle Funktion die eines Zeigegegeräts ist, kann man damit auch Formen direkt in den Computer übertragen, z. B. Daten von biomedizinischen oder architektonischen Vorlagen, Funktionen und Kurven, oder es für Unterschriften zur Scheckverifikation verwenden.

- *Scanner*

Das grafische Tablett ist allerdings in den letzten Jahren bei den meisten Anwendungen durch hochauflösende, optoelektronische Abtastgeräte, die Scanner, ersetzt worden. Zusammen mit der Tendenz, Ergebnisse nicht mehr auf Papier zu speichern, sondern elektronisch und damit auch direkt verarbeitbar zu machen, haben sie sich auf diesem Gebiet durchgesetzt.

- *Spracheingabe*

Eine der benutzerfreundlichsten Eingabeschnittstellen ist zweifelsohne die Spracheingabe. Obwohl die benutzerabhängige Spracherkennung einzelner Worte in leiser Umgebung gut gelingt, lassen die Systeme für eine sprecherunabhängige, störteste Spracherkennung allerdings noch viel zu wünschen übrig. Auch ihr Nutzen ist sehr umstritten: Zwar ist die Spracheingabe bei allen Menschen, die bei der Rechnerbedienung die Hände zur eigentlichen Arbeit frei haben müssen (Chirurgen etc.) sehr beliebt, doch findet sie bei Büroangestellten mäßige Resonanz, da die Sprache zum einen nicht unbedingt genauer ist als ein Zeigeelement, und zum anderen dauerndes Reden beim Arbeiten störend sein kann.

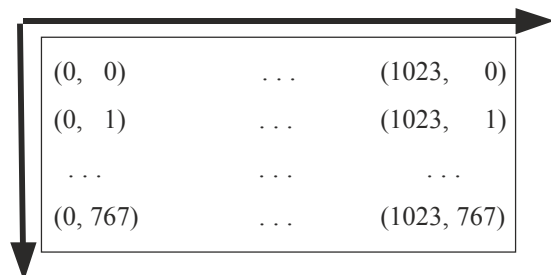
## 8.2.2 Rastergrafik und Skalierung

Für die Grafikausgabe wird oft ein spezielles Modell benutzt: das Modell einer Rastergrafik. Es benutzt ein Koordinatensystem für Zeichnungen, wie es bei einem Bildschirmraster entsteht. Ausgangspunkt (0,0) der (x,y) Koordinate ist dabei die linke obere Ecke, wobei die Indizes der Bildpunkte (Pixel) ähnlich der einer transponierten Matrix angeordnet sind. Für einen Bildschirm mit  $1024 \times 768$  Punkten ist dies in [Abb. 8.4](#) zu sehen.

Das Rastergrafikmodell weist im Unterschied zu einer Vektorgrafik, die die Koordinaten nur als geometrische Punkte, (also z. B. als Anfangs- und Endpunkte von Linien) betrachtet, jedem Punkt einen definierten Farbwert zu, der als Zahl gespeichert wird. Der Bildwiederholungspeicher der Rastergrafik muss also anders als bei der Vektorgrafik für jeden Bildpunkt (Koordinate) des gesamten Bildschirms eine Speichereinheit vorsehen. Aus diesem Grund bevorzugte man früher Vektorgrafik, die sehr wenig Speicherplatz benötigt. Allerdings dauert das Neuzeichnen (*refresh*) komplizierter Grafiken durch das Neuzeichnen aller Grafikelemente in der Liste bei Vektorgrafiken sehr lange, was bei älteren Bildschirmen (phosphorbeschichtete Scheibe!) und komplexen Vektorgrafiken zu Flackern führte.

Die moderne Technologie der Rasterbildschirme (Fernseh- oder Flüssigkristallbildschirme mit Bildwiederholungspeicher) ist stark mit dem Programmiermodell der Bildschirme verbunden und damit in die Konzeption der Grafiksoftware eingeflossen. Jeder Bildpunkt wird

**Abb. 8.4** Die Pixelkoordinaten der Rastergrafik



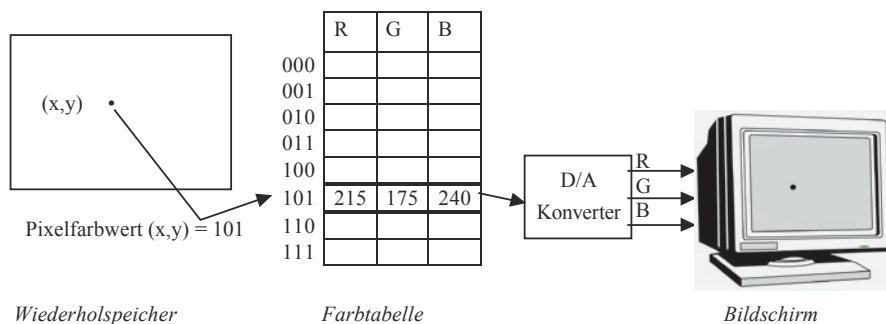


in seiner Farbe durch einen Farbwert (eine Zahl) aus mehreren Bits ( $b_0, b_1, \dots, b_n$ ) beschrieben. Vielfach können die Bits mit gleichem Index vom Displaycontroller hardwaremäßig extra zu einem Bild zusammengefasst werden. Ein solches Bild wird auch als **Ebene** bezeichnet.

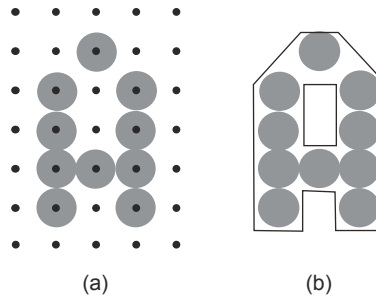
### Beispiel Bildschirmeneben

Ein Bild habe 8 Bits = 1 Byte pro Bildpunkt. Teilen wir dieses in 6 + 2 Bits auf, so ist es möglich, softwaremäßig einen Vordergrund aus  $2^6$  Farben (z. B. eine Figur) vor einem Hintergrund (z. B. ein Sternhimmel) aus  $2^2$  Farben zu stellen. Über spezielle Hardwarebefehle können wir die Daten einer Ebene (z. B. den Vordergrund) verschieben, so dass der Eindruck einer bewegten Figur vor unbewegtem Hintergrund entsteht, ähnlich einer Kulissenlandschaft. In der Hardware für Videospiele gibt es meist noch die Möglichkeit, mehrere kleine, rechteckige Ebenenausschnitte (*sprites*) für die Visualisierung von Figuren über eine extra Ansteuerung auf dem Bildschirm zu bewegen.

Die Farben selbst werden meist als Mischung aus den Farben Rot, Grün und Blau (RGB-System) angegeben. Da die Zusammenfassung aller drei Farbintensitäten (z. B. je 8 Bit) zu einer einzigen Zahl (24 Bit!) zu viele Bits beansprucht, ist bei jedem Pixel nicht die RGB-Zahl notiert, sondern nur der Index (Adresse) der Farbe in einer besonderen Tabelle, der **Color Lookup Table** (CLUT). Das resultierende Zugriffsschema für den Farbwert eines Pixels beim Display ist damit der einstufigen Adressübersetzung aus [Abschn. 3.3](#) sehr ähnlich. In [Abb. 8.5](#) ist als Beispiel der Datenfluß für einen hellvioletten Punkt mit dem Farbwert (R,G,B) = (215,175,240) an der Stelle (x,y) gezeigt. Man beachte, dass zwar jeder Pixel nur 3 Bits Speicherplatz beansprucht, aber über eine Farbgenauigkeit („Farbtiefe“) von 24 Bits verfügt. Die Begrenzung der Pixelbeschreibung auf 3 Bit wirkt sich dabei nicht auf die Farbtiefe, sondern auf den maximalen Index in der CLUT, also auf die Anzahl der gleichzeitig darstellbaren Farben aus. Zur Darstellung der digitalen RGB-Farbwerte müssen die digitalen Werte erst in ihr analoges Äquivalent mittels eines D/A-Konverters gewandelt werden, also z. B. die Werte 0–255 in Spannungen 0,0–1,0 Volt, mit denen die Farberzeugungsquellen (z. B. Elektronenstrahlen) angesteuert werden.



**Abb. 8.5** Farbwertermittlung über eine Color Lookup Table



**Abb. 8.6** Ein Buchstabe (a) im 5×7-Raster und (b) als Umrandung

Allerdings hat das Denken in Pixelbereichen einen entscheidenden Nachteil: Die Auswirkungen der Funktionen sind sehr von der tatsächlich verfügbaren Hardware (Bildschirmauflösung etc.) abhängig. Beispielsweise werden Vierecke und Fenster der Bildschirmgröße entsprechend skaliert, nicht aber Schriften, die als feste Pixelblöcke (*pixmap*s) definiert werden können. Mischen wir nun Grafik (z. B. Fenster) und Text (z. B. Bezeichnungen), so ist die Lage des Textes in der Grafik von der Bildschirmauflösung abhängig, was zu sehr unschönen Effekten führen kann und eine hardwareunabhängige Benutzerschnittstelle sehr erschwert.

Eine wichtige Alternative zu pixelorientierten Schriften sind die *skalierbaren* Schriften. Bei Ihnen wird ein Buchstabe nicht durch seine Bildpunkte (Pixel) sondern durch den Umriss und die Farbe bzw. Struktur (*Textur*) der darin enthaltenen Fläche charakterisiert. Dies ist zwar zunächst umständlich und ineffizienter bei der Speicherung und Darstellung, aber die Umrissbeschreibung lässt sich im Unterschied zu den Pixelfeldern beliebig vergrößern und verkleinern. In [Abb. 8.6](#) ist links eine pixelorientierte und rechts ein skalierbare Beschreibung des Buchstabens „A“ gezeigt. Die tatsächlich zu sehenden Bildpunkte sind jeweils grau schraffiert.

Eine solche Beschreibungsart aus Umriss und Textur ist zwar aufwendiger, aber sie lässt sich für alle wichtigen grafischen Elemente der Benutzeroberfläche (Balken, Kreise, Ikone usw.) einheitlich vorsehen und damit sehr effizient in den Grafikroutinen des Betriebssystems verankern sowie leicht Spezialprozessoren übertragen.

### 8.2.3 Fenstersysteme und Displaymanagement

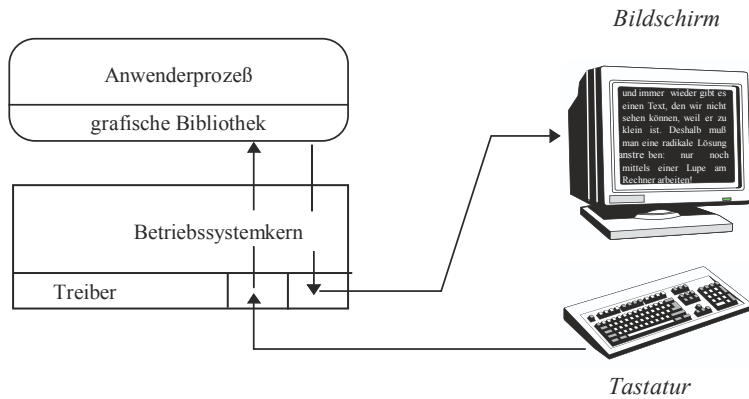
Eines der wichtigsten Darstellungsmittel, das vom Xerox-STAR-System übernommen wurde, ist die Anzeige aller Daten in rechteckigen Bildschirmausschnitten, den Fenstern. Diese Technik, die heutzutage auf fast allen Rechnern selbstverständlich geworden ist, ergab sich aus der Möglichkeit, den Bildwiederholpeicher einer Darstellungseinheit (z. B. Terminal, grafischer Bildschirm) direkt zu adressieren und so die textuelle Ausgabe eines Prozesses auf einen bestimmten, dafür reservierten Teil des Bildschirms zu lenken. Reichten früher noch spezielle, mit einem Sonderzeichen (z. B. ESC) eingeleitete Zeichenketten (*Escape-Sequenzen*) als Befehle für den Displayprozessor aus, so wurde dies

bei den heutigen Fenstersystemen durch komplexe Funktionen ersetzt, bei denen die Fensterdaten (Größe, Position, Ausschnitt usw.) interaktiv durch die Eingabegeräte (Maus, Joystick usw.) verändert werden können.

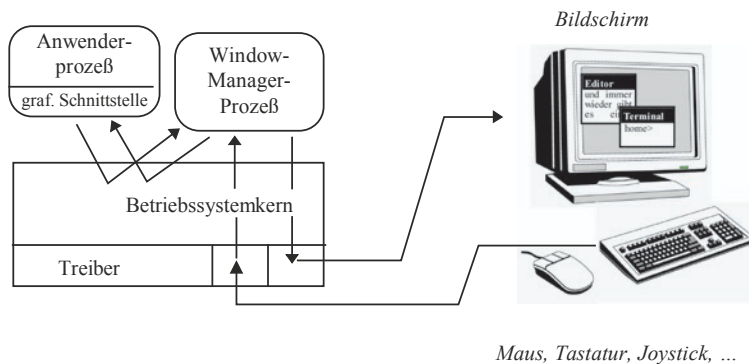
Die Prozessstruktur einer grafischen Anwendung hat sich dadurch gewandelt: Enthielt das Anwendungsprogramm früher noch alle grafischen Ausgabeprozeduren als Bibliothek (s. Abb. 8.7), so ist dies heutzutage meist abgetrennt.

Die Eingaben wie Mausposition, Mausklicks, Texteingabe usw. werden von einem extra Prozess verwaltet, dem Fenstermanager (*window manager*), siehe Abb. 8.8. Beide Prozesse sind durch eine Client-Server-Beziehung miteinander verbunden: Der Anwenderprozess als Client beauftragt den Fenstermanager als Server mit grafischen Darstellungswünschen.

Dabei ist der Manager grundsätzlich als Programm mit Endlosschleife (*Dämon*) konzipiert, wobei als Reaktion auf Benutzereingaben Aktionen durchgeführt werden:



**Abb. 8.7** Systemstruktur einer traditionellen Benutzeroberfläche



**Abb. 8.8** Systemstruktur einer fensterorientierten Benutzeroberfläche

```

InitProcess
LOOP
    waitForEvent                      (Mausklick,                      Tastatur,
                                     AusgabeDesBenutzerprogramms,..)
    executeNecessaryProcedures;
END (*LOOP*)

```

Das Fensterprozesssystem ist also meist ereignisgesteuert. Als asynchrone Ereignisse werden nicht nur die Eingaben von außen (Mausklicks usw.) verarbeitet, sondern auch die grafischen Aufträge des Anwenderprogramms. Hier hilft ein nachrichtenbasiertes Ereignissystem, das alle Eingaben und Aufträge in ein einheitliches Nachrichtenformat bringt und in die zentrale Warteschlange des Window-Managers hängt.

Die Funktionsaufteilung zwischen Anwendungsprogramm und Window-Manager bietet außer der sauberen Aufgabentrennung, bei der zwischen Standardaufgaben der Fensterverwaltung (Aufgaben der Benutzeroberfläche) und den speziellen grafischen Elementen des Anwenderprogramms unterschieden wird, auch noch andere Vorteile:

- Die Ergebnisse mehrerer, unabhängiger Prozesse können in einem gemeinsamen Fenstersystem zusammengefasst und damit überschaubar gemacht werden, beispielsweise
  - Ergebnisse verschiedener Sensoren in einer Industriesteuerung,
  - parallele Präsentation der Ergebnisse bei mehreren Programmen mit gleicher Funktionalität zur diversitären Softwarefehlertoleranz.
- Das grafische Displaysystem kann auch auf einen anderen Rechner in einem Netz verlagert werden, s. [Abschn. 8.2.5](#). Dies ermöglicht Funktionen wie
  - *Netzmonitoring*: Auf jedem Rechner befindet sich ein Stellvertreterprozess (*Agent*), der unabhängig von einem Zentralprogramm den Status seines Rechners in einem eigenen Fenster anzeigt.
  - *Netzmanagement*: Softwareverteilung und -wartung kann von einem zentralen Arbeitsplatz aus organisiert werden, indem z. B. auf jedem Rechner als Stellvertreterprozess ein Terminal-Emulatorprozess existiert, der seine Ausgabe auf das zentrale Display gibt.
- Das Fenstermanagement ist zentral geregelt und die Oberfläche, das *look-and-feel*, damit für alle Anwendungen gleich.
- Die Ereignisbearbeitung kann einem Subsystem (dem Server) übertragen werden und entlastet somit den Prozessor der eigentlichen Anwendung.

Dieses Konzept bedeutet aber auch, dass der aktuelle Zustand eines Fensters dem Anwenderprogramm nicht mehr bekannt ist, sondern die Werte der Zustandsvariablen (Fensterposition, Fenstergröße usw.) beim Server explizit erfragt werden müssen.

Fenster können auch übereinanderliegen und sich gegenseitig teilweise verdecken. Für nicht gezeichnete Teile von Fenstern gibt es zwei Konzepte:

- Entweder wird das gesamte Fenster vom Server gespeichert, so dass beim Offenlegen bisher verdeckter Fenster Teile (z. B. beim Vergrößern eines Fensters oder Verschieben

darüberliegender Fenster) automatisch der Fensterinhalt ergänzt wird. Dies erfordert mehr Speicher für den Serverprozess, erspart aber zusätzliche Prozeduren zum Ergänzen fehlender Fensterinhalte im Anwenderprogramm.

- Oder aber das Anwendungsprogramm muss fehlende Fensterteile im Bedarfsfall immer neu zeichnen. Dies erfordert zwar weniger Speicher beim Server, aber dafür spezielle Prozeduren beim Anwenderprogramm und ergibt eine erhöhte dynamische Last des Anwenderprozessors.

Die tatsächliche Implementierung hängt stark vom verwendeten Fenstersystem ab. Überlässt man die Entscheidung darüber der zur Laufzeit vorhandenen Konfiguration des Servers (Speicherausbau etc.) und dem Anwendungsprogrammierer, so kann der Fall auftreten, dass sowohl bei der Anwendungsprogrammierung als auch beim Serverspeicher gespart wurde und die Anwendung deshalb in der gewünschten Fensterumgebung nicht fehlerfrei läuft, obwohl kein Programmierfehler vorliegt. Es ist deshalb sinnvoll, feste Konventionen für den Display vorzuschreiben.

### 8.2.4 Virtuelle Realität

In neuerer Zeit sind für eine perfekte Simulation von bewegten Szenen spezielle Ein- und Ausgabegeräte dazugekommen, die die dritte Dimension in der Benutzeroberfläche ermöglichen. Beispielsweise gibt es Mäuse und Zeiger, die es gestatten, eine 3D-Position anzusteuern oder Eingabegeräte, welche die Position des menschlichen Kopfes (Tracker) oder Gliedmaßen (Datenhandschuh) feststellen. Fortgeschrittene Systeme ermitteln den Zustand der menschlichen Ausdrucksfähigkeiten durch Fernsehkameras und Abstandssensoren. Dadurch werden zusätzliche Eingabearten wie Gestik oder Mimik möglich. Auch die grafische Darstellungstechnik wurde derart ergänzt, dass durch spezielle Brillen oder Displaygeräte jedes Auge einzeln angesteuert werden kann und dadurch ein räumlicher Eindruck beim Benutzer entsteht.

Abgesehen von den zur Zeit noch unvollkommenen Möglichkeiten und ihrem hohen technischen und finanziellen Aufwand bringt die virtuelle Realität allerdings keine prinzipiell neuen Überlegungen zur Benutzerschnittstelle, sondern perfektionieren sie und ermöglichen darüber hinaus neue Anwendungen, wobei noch zusätzliche Probleme (wie physische Übelkeit bei Telemetriesteuerungen etc.) dazukommen. Alle in den vorigen Abschnitten beschriebenen Möglichkeiten und Probleme lassen sich direkt in den 3D-Kontext übertragen, so dass auch für die 3D-Schnittstellen die oben beschriebene Systemgliederung in eine Client-Server-Architektur sinnvoll ist.

### 8.2.5 Das Management der Benutzeroberfläche

Eine besondere Rolle bei den grafischen Oberflächen nimmt die Verwaltung der Objekte, das *User Interface Management System UIMS*, ein. Die Verwaltung ist als Softwarepaket

noch zusätzlich unterteilt in eine Unterstützung der Ein- und Ausgabe mit verschiedenen Geräten wie Maus, Grafiktablets, Lautsprecher usw. sowie die reinen Grafikprozeduren auf der einen Seite und der Verwaltung der Eingabeereignisse und Ausgabewünsche auf der anderen Seite. Das typische Erscheinungsbild einer Benutzeroberfläche (*look-and-feel*), also die grafische Präsentation und das Verhalten der grafischen Objekte bei Aktionen (z. B. Verkleinern eines Fensters, Benutzung von Ikonen usw.) wird in dieser Verwaltungsschicht erreicht. Sie ist unabhängig von der Schnittstelle zum Anwenderprogramm, dem *Graphical User Interface GUI*, das durch eine Liste von Prozeduren, Objekten und Protokollen beschrieben wird.

Es gibt verschiedene Ansätze, eine solche Verwaltung zu strukturieren. Eine Möglichkeit besteht darin, für die Ereignisse und daraus folgenden Aktionen Regeln aufzustellen und diese in der Programmierung des UIMS zu etablieren.

---

#### Beispiel Ausdrucken

Wenn ein Ikon, das für ein Dokument steht, selektiert wird (Maustaste drücken), auf ein Drucker-Ikon geschoben und dann deselektiert wird (Maustaste loslassen), so bewirkt diese **drag-and-drop**-Handlung beim UIMS, dass die Dokumentdatei zum Drucker-spooler transferiert und ausgedruckt wird.

Die Folge der Betriebssystemaufrufe sowie der grafischen Animation der Ikone (das Dokumenten-Ikon verschwindet im Drucker, das Drucker-Ikon geht in den Druckstatus, ein gezeichnetes Papierblatt kommt langsam aus dem Drucker-Ikon usw.) werden von dem UIMS übernommen und sind damit bei allen Anwenderprogrammen gleich.

Ein derartiges Regelsystem kann dabei in einer eigenen logischen Spezifikations- oder Programmiersprache geschrieben werden. Der Vorteil eines solchen UIMS besteht darin, dass es leicht zentral änderbar ist, so dass notwendige Funktionsanpassungen, Korrekturen usw. sich konsistent und gleichartig bei allen Anwenderprogrammen auswirken.

Im Unterschied zu den Komponenten der Standardbenutzeroberfläche, die für alle Anwendungsprogramme gleich sind und meist vom Betriebssystemhersteller mitgeliefert werden, sieht die Situation bei den anwendungsspezifischen grafischen Komponenten anders aus. Hier sind besondere Programmieranstrengungen nötig, die bei jedem Programm extra unternommen werden müssen. Um diese Programmierung ebenfalls zu vereinfachen, gibt es deshalb verschiedene Ansätze, interaktiv ohne explizite Programmierung die grafische Funktionalität der Benutzeroberfläche zu gestalten:

- *Erzeugung von speziellen Dateien (resource files)*

Mittels eines speziellen grafischen Editors (eines *resource construction tool kit* für das UIMS) kann man Elemente wie Menüs, Ikone, Auswahlknöpfe, Mausereignisse, Tonfolgen usw. geeignet zusammenfügen. Die Ergebnisse werden in sog. *Ressourcendateien* als Datenstrukturen abgelegt und zur Laufzeit vom UIMS geladen und verwendet. Die beim Editor vereinbarten Namen der Objekte gestatten es den Anwenderprogrammen, die entsprechenden Objekte mit Hilfe des GUI zu benutzen.

- *Erzeugung von Programmcode*

Parallel zur visuell-interaktiven Erzeugung von grafischen Aktionen und Reaktionen mit Hilfe eines grafischen Editors ist es möglich, die entsprechenden Programmaufrufe des GUI einer Programmiersprache in eine Datei zu schreiben. Wird diese dann kompiliert, so erhalten wir eine komplette, anwendungsspezifische, zusätzliche UIMS-Schicht. Ein populäres Beispiel für ein solches Programmiersystem ist das Delphi-System der Fa. Borland, das den Programmcode in objektorientiertem Pascal erzeugt. Weitergehende Informationen zu diesem Thema sind z. B. in den Büchern von Foley und van Dam (1982) oder Shneiderman (1987) oder im Buch von Fähnrich et al. (1996) zu finden.

## 8.2.6 Aufgaben

### Aufgabe 8.2-1 Benutzerschnittstelle und visuelle Programmierung

Wenn der zentrale Lösungsmechanismus eines Problems in analoger Form in der Wirklichkeit bekannt ist, so kann man ihn in der Benutzerschnittstelle grafisch danach modellieren.

- a) Dabei sollen einfache, konsistente Beziehungen zwischen dem Modell und dem Algorithmus aufgebaut werden. Die Visualisierung soll sehr einfach konzipiert werden, wobei Vieldeutigkeit zu vermeiden ist, z. B. durch Benutzung von Ikonen statt Symbolen.
- b) Muss der Benutzer die interne Logik des Programms erlernen, so soll dies als aufbauendes Lernen konzipiert werden, bei dem die Komplexität zunächst versteckt wird und erst im Laufe der Benutzung deutlich wird.

Verwirklichen Sie diese Grundsätze an einem einfachen Beispiel. Für die Verwaltung von Daten kann man als Programmiermetapher die Vorgänge in einem Lagerhaus verwenden. Hier ist das Liefern, Einsortieren von Paketen, Lagerung, Aussondern etc. gut bekannt und als Referenz beim Benutzer vorhanden.

Konzipieren Sie dazu ein Schema, bei dem die Dateiobjekte (Attribute und Methoden) den visuellen Objekten zugeordnet werden.

### Aufgabe 8.2-2 Color Lookup Table

- a) Angenommen, wir beschreiben für eine anspruchsvolle Grafik die Farbe eines Pixels direkt ohne CLUT mit 24-Bit-Genauigkeit (Farbtiefe). Wie groß muss der Bildwiederholtspeicher für ein 1024×768 großes Bild mindestens sein? Wie groß, wenn er drei Ebenen davon abspeichern soll?
- b) Wie groß muss der Bildwiederholtspeicher für das Bild mindestens sein, wenn gleichzeitig 65536 = 16 Bit Farben sichtbar sein sollen und eine einstufige Umsetzung mit einer CLUT möglich ist?

- c) Verallgemeinern Sie und stellen Sie eine Formel auf für den Speicherbedarf  $s$  ohne CLUT bei der Farbtiefe von  $f$  Bits pro Pixel und der Bildpixelzahl  $N$  sowie dem Speicherbedarf  $s_{\text{CLUT}}$  mit CLUT, wenn gleichzeitig  $n$  Farben sichtbar sein sollen. Bilden Sie das Verhältnis  $s_{\text{CLUT}}/s$ , und zeigen Sie, dass eine Voraussetzung dafür, dass es kleiner als eins ist, in der Bedingung  $N > n$  liegt (was durchaus plausibel erscheint).

---

### Aufgabe 8.2-3 Benutzeroberflächen in verteilten Systemen

Im obigem Abschn. 8.2.3 wurde das Konzept des Display-Servers eingeführt. Als Alternative gibt es dazu Programme, die Texte und Bilder über das Internet laden und anzeigen können, sog. Hypertext-Browser wie Netscape und Internet Explorer. Zusätzliche Programme (sog. cgi-Programme) auf dem Internet-Datenserver sorgen dafür, dass Suchanfragen und ähnliche Aufgaben interaktiv mit dem Browser durchgeführt werden können. Damit kann man sehr viele Aufgaben, die mit einem *application client/display server*-System durchgeführt werden, auch mit einem derartigen Browsersystem durchführen.

- a) Vergleichen Sie die Client-Server-Beziehungen in beiden Systemen.  
b) Welche Arten der Kommunikation herrschen zwischen Client und Server?

---

## 8.3 Das UNIX-Fenstersystem: Motif und X-Window

Die Benutzeroberfläche von UNIX wurde im *Common Desktop Environment* CDE standardisiert (UNIX-98). Die für UNIX-Systeme mit Abstand bedeutendste Implementation des CDE beruht auf einem speziellen Fenstersystem, dem **X-Window**-System.

Alternativ dazu wurde für das Linux-Betriebssystem zuerst das K Desktop Environment (KDE, s. <http://www.kde.org>) entwickelt, was in die *KDE Plasma workspaces* (Werkzeugsammlung) überging. Diese Oberfläche versucht, die Unix-Idee der Bausteine auch auf der Benutzeroberfläche beizubehalten. Im Unterschied zu herkömmlichen Büroanwendungssystemen, bei denen in verschiedenen Applikationen ähnliche Funktionen unterschiedlich realisiert werden und damit den Benutzer verwirren, sind alle Grundfunktionen wie Texteditor, Rechentabellen und Zeichenprogramme parallel in allen Anwendungen verfügbar. Diese können dann in den Programmen als normale Funktionen benutzt werden. Gemeinsames Datenaustauschformat in KDE ist XML, wobei die Kommunikation über eine gemeinsame Kommunikationsfunktion, den D-Bus, geht und von einem speziellen Prozess, den *message bus demon* `dbus-demon`, vermittelt wird. Mit diesem Ansatz bleiben die Programme trotz großer Funktionalität klein und die Benutzeroberfläche auf vertraute Elementarfunktionen beschränkt, wobei die Benutzeroberfläche *Plasma Desktop* sich je nach verwendetem Gerät automatisch anpasst.

Auch das KDE beruht auf dem X-Window-System. Wie funktioniert nun das? Eines der wichtigsten Projekte in den 80er Jahren war das ATHENA-Projekt am MIT (*Massachusetts*



*Institute of Technology*) in Boston, bei dem versucht wurde, eine Arbeitsumgebung für sehr viele, miteinander vernetzte Computer zu schaffen. Ein wichtiger Teil des Projekts, das von der Fa. Digital unterstützt wurde, bestand in der Schaffung einer Benutzeroberfläche, die im Netz verteilt unabhängig vom Rechner der Anwendung existieren sollte. Der Nachfolger eines einfachen, dort vorher entwickelten *window system* (W-System) wurde ein System, abgekürzt mit dem nächsten Buchstaben im Alphabet: das X-Window-System.

In den folgenden Abschnitten wollen wir uns dieses System näher anschauen. Dabei reicht es uns, die grundsätzlichen Modelle und Konzepte zu beleuchten; die detaillierten Schnittstellenbeschreibungen und programmtechnischen Tricks sollen den Handbüchern und Nachschlagewerken vorbehalten bleiben. Eine tiefere Einführung bietet beispielsweise Gottheil et al. (1992).

### 8.3.1 Das Client-Server-Konzept von X-Window

Das X-Window-System besteht aus einer grafischen Bibliothek **Xlib**, die von der Applikation benutzt wird, und einem Serverprozess, der den Fenstermanager enthält und der die gesamte Ein- und Ausgabe der Benutzeroberfläche vornimmt. In Abb. 8.9 ist dies visualisiert.

Das Client-Server Konzept ist in der Xlib sehr einfach verwirklicht. Um eine Verbindung zum Server herzustellen, muss der Client nur zu Anfang der Sitzung einen Aufruf *XOpenDisplay()* durchführen, der als Argument den Rechnernamen und die Bildschirmnummer enthält; alle weiteren Ausgaben gehen automatisch an diese Adresse. Führen mehrere Prozesse auf verschiedenen Rechnern diesen Aufruf durch, so geht die Ausgabe der verschiedenen Prozesse auf ein und denselben Server. Die verteilten Informationen im Netz können so auf einem gemeinsamen Bildschirm dargestellt werden, ohne vorher zentral vom Anwenderprogramm koordiniert werden zu müssen.

Die Informationen über Lage und Größe der Fenster, die vom Benutzer interaktiv auf dem Bildschirm arrangiert werden, sind allerdings nur dem Server bekannt. Möchte eine Applikation die Daten seiner Fenster erfahren, so muss sie erst eine Anfrage *XGetWindow()* starten.

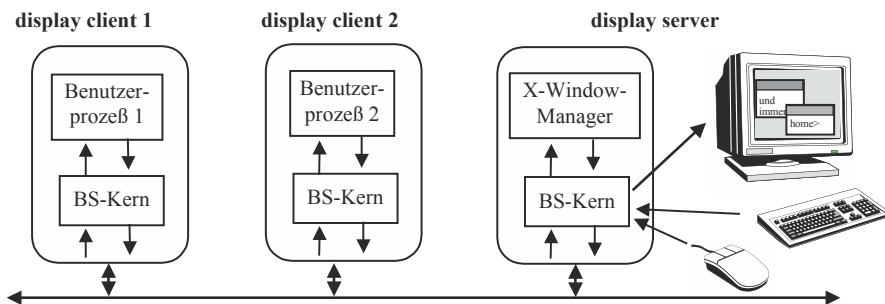


Abb. 8.9 Die netzübergreifende Darstellung durch das Client-Server-Konzept von X-Window

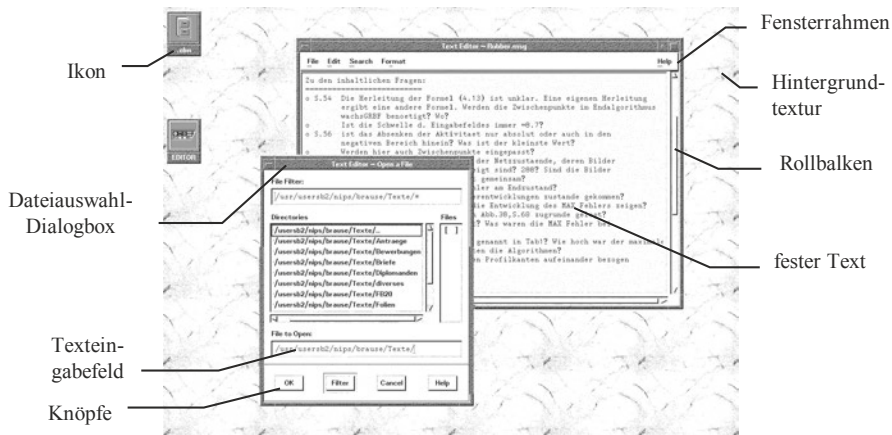


Abb. 8.10 Beispiel einer fensterorientierten Interaktion in Motif

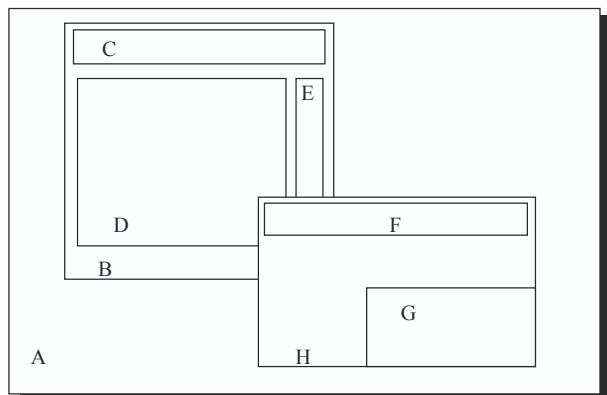
### 8.3.2 Das Fensterkonzept von X-Window

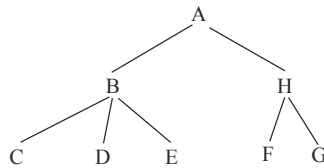
Die Xlib enthält nur sehr einfache Grafikfunktionen und Eingabeereignisse. Für ein komplexes Fenster mit Rollbalken, wie es in Abb. 8.10 zu sehen ist, benötigt man eine große Anzahl von Aufrufen dieser elementaren Funktionen.

Das logische Konzept der Fenstererzeugung und -bezeichnung für das X-Window-System ist sehr klar und einfach. Ausgehend von einem Grundfenster werden alle darin definierten Fenster nur im Rechteck des Grundfensters gezeigt; alles, was darüber hinausragt, wird „weggeschnitten“ (*clipping*).

Als „Fenster“ ist dabei ein elementares Rechteck zu verstehen, das nur einen Rand und ein Hintergrundmuster besitzt und auf der Xlib-Ebene definiert ist. In Abb. 8.11 ist ein solches System von Fenstern und Unterfenstern gezeigt, wobei jedes Fenster mit einem Buchstaben gekennzeichnet ist.

Abb. 8.11 Fenster und Unterfenster in X-Window





**Abb. 8.12** Der Hierarchiebaum der Fenster aus [Abb. 8.11](#)

Da jedes Fenster ein Unterfenster (*subwindow*) enthalten kann, lässt sich die sich daraus ergebende Hierarchie durch einen Baum darstellen, dessen Wurzelknoten mit dem Grundfenster, dem *root window*, identisch ist, siehe [Abb. 8.12](#).

Das Grundfenster ist dabei meist der ganze Bildschirm des Monitors.

### 8.3.3 Dialogfenster und Widgets

Auf der Grundlage der einfachen Xlib-Funktionen müssen neue, höhere Schichten mit größerer Funktionalität errichtet werden. Die Zwischenschicht zwischen den allgemeinen Aufrufen des Anwenderprogramms und den detaillierten Funktionen der Xlib wird **Xtoolkit** genannt und kann sehr verschieden implementiert sein. In [Abb. 8.12](#) ist die Schichtung der Software gezeigt.

Die grafischen Objekte dieser Schicht, die Dialogobjekte, sind sehr unterschiedlich gestaltet, je nach Programmierer. Um den Vorteil einer einheitlichen Benutzerschnittstelle auch auf der höheren Ebene zu bewahren und das Aussehen und Verhalten (*look-and-feel*) der Dialogobjekte zu normieren, wurde von der OSF (*Open Software Foundation*), einem Zusammenschluß verschiedener Hersteller, nach einer Ausschreibung eine einheitliche Benutzeroberfläche für UNIX ausgezeichnet: das *Motif toolkit*. Neben der beschreibenden Festlegung des Erscheinungsbildes von Motif im *Motif Style Guide* gibt es noch eine Beschreibung des speziellen, dafür nötigen Fenstermanagers *Motif Window Manager* (*wmf*) und als Menge der Dialogobjekte die *Motif Widgets*. Neben diesem klassischen System stehen als aktuelle *toolkits* zum Erzeugen von graphischen Benutzeroberflächen inzwischen GTK+ und Qt zur Verfügung.

Allerdings liegt den Funktionen der Xlib direkt das Bild- und Farberzeugungsmodell der Rastergrafik zugrunde: Sie arbeitet mit den nicht-skalierbaren Pixelfonts. Aus diesem Grund erstellte die Fa. Sun das Netzwerk-Fenstersystem NeWS, das auf einer Erweiterung der Seitenbeschreibungssprache Postscript basiert und nur skalierbare Grafikobjekte und Fonts verwendet.

Bei den Dialogobjekten in X-Window existieren (vgl. [Abb. 8.10](#))

- simulierte Knöpfe, die man drücken kann (*XmPushButton*)
- fester Text, der dargestellt werden kann (*XmText*)
- Rollbalken, um den Fensterinhalt zu verschieben (*XmScrollBar*)

- Ausgabefelder für Grafik (*XmDrawingArea*)
- Dateiauswahlfenster (*XmFileSelectionBox*)

Die Schnittstelle zum Anwendungsprogramm wird in Motif durch eine genormte Sprache beschrieben: die *User Interface Language* UIL. Mittels dieser Sprache können die Elemente der benötigten Benutzeroberfläche angegeben werden, also z. B. Funktion und Geometrie der *widgets*.

Das *toolkit* ist dabei für verschiedene Aufgaben zuständig:

- dynamische Erzeugung und Vernichtung von *widgets*,
- Veränderung der *widgets* zur Laufzeit (Fenstermanagement!),
- zentrale Verwaltung der Ein- und Ausgabe, Erzeugung des Eingabeechos usw.,
- Bereitstellen von Kommunikationsmechanismen zwischen den Applikationen bzw. ihren Fenstern. Dies ermöglicht die Einrichtung eines zentralen Puffers (*cut-and-paste-Clipboard-Mechanismus*).

Diese Aufgaben werden im Motif-*toolkit* (wie im GTK + *toolkit*) mit den Mitteln der objektorientierten Programmierung gelöst. Das *toolkit* enthält komplexe Klassenhierarchien und Vererbungsmechanismen, die der Anwendungsprogrammierer nutzen kann. Ausgenommen davon sind nur die einfachen grafischen Ausgaben, deren simple Xlib-Funktionalität ohne Mehrwert direkt zur Applikation durchgereicht wird.

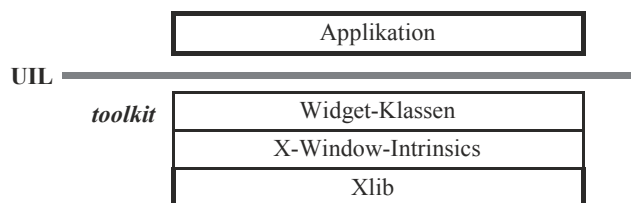
Jedes *widget*, das Instanz einer Klasse ist (d. h. dem Speicherplatz zugewiesen wurde), besitzt Werte (z. B. Zahlen) in seinen Attributen (d. h. Variablen). Diese Speicherbelegung kann man zum Initialisieren des Objekts auch aus einer Datei einlesen, der sog. *Ressourcedatei*.

Es gibt zwei Arten von *widgets*: einfache und zusammengesetzte. Die einfachen *widgets* sind grafische Primitiven, die aus der Klasse *XmPrimitives* abgeleitet werden und nur nebeneinander auf dem Bildschirm gezeichnet werden können. In Abb. 8.13 ist ein kleiner Ausschnitt aus der *widget*-Hierarchie der Primitiven gezeigt.

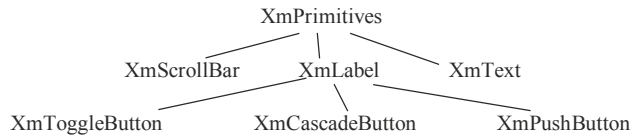
Man beachte, dass jede abgeleitete Klasse eine Spezialisierung der Überklasse durch zusätzliche Attribute (Variablen) und Methoden (Prozeduren) bedeutet.

Die zusammengesetzten *widgets* fungieren als Behälter (*container*) und können andere *widgets* (z. B. Primitiven) enthalten. Die Container bestimmen im Wesentlichen die geometrische Anordnung (*layout*) der enthaltenen *widgets*. In Abb. 8.14 links ist als Beispiel das Aussehen eines Fensters gezeigt, das zur Ausgabe von Nachrichten an den Benutzer

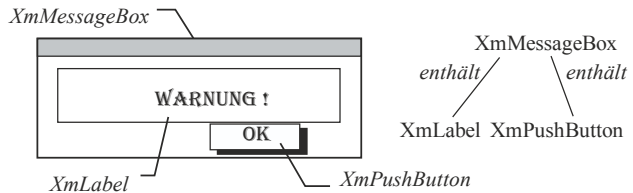
**Abb. 8.13** Die Schichtung des X-Window-Motif-toolkits



**Abb. 8.14** Ausschnitt aus der Hierarchie der Motif-widget-Primitiven



**Abb. 8.15** Beispiel eines *container widgets*



dient. Diese Art von *container-widgets*, die aus der *composite*-Klasse der *intrinsics* abgeleitet sind, werden auch *composite-widgets* oder *layout-widgets* genannt.

Rechts im Bild ist eine Hierarchie für die Bestandteile des *container-widgets* angegeben, die auf der „enthält“-Relation beruht. Das *container-widget* (hier: *XmMessageBox*) wird dabei als **parent-widget** und das darin enthaltene als **child-widget** bezeichnet. Die so entstehende Hierarchie ist direkt mit der Fensterhierarchie (s. [Abb. 8.15](#)) gekoppelt: Normalerweise hat jedes *widget* ein Fensterrechteck, das es umschließt. Dadurch entsteht mit der Relation „enthält“ der *widget*-Hierarchie auch eine Fensterhierarchie; zum *parent-widget* gehört das *parent-window*, das ein *child-window* hat, in dem das *child-widget* enthalten ist.

Das oberste Fenster (*top-level window*) in der Hierarchie der von der Anwendung definierten *widgets* ist das *shell-widget*. Es enthält genau ein *composite-widget*, in dem alle anderen enthalten sind, und dient zur Kommunikation zwischen dem Fenstermanager und der Hierarchie im *composite-widget*. Es kann aber auch noch weitere *shell-widgets* enthalten, etwa die der *popup-childrens*. Dies sind Fenster (z. B. mit Hinweisen oder Fehlermeldungen), die bei gegebenem Anlaß „plötzlich“ erscheinen und dann wieder verschwinden können.

### 8.3.4 Ereignisbehandlung

Der X-Window-Serverprozess befindet sich, ähnlich wie in [Abschn. 8.2.3](#) beschrieben, in einer Schleife *XtMainLoop()*, in der auf externe Ereignisse gewartet wird. Angenommen, wir klicken mit der Maus auf ein Dialogobjekt. Was passiert dann?

Der X-Window-Fenstermanager weist alle Eingaben dem Dialogobjekt zu, über dem gerade der Mauszeiger steht und das aktiviert (an oberster Stelle der Displayliste) ist. Interessiert sich das Fenster nicht für das Ereignis, so wird es in der Hierarchie nach oben an das Elternfenster gereicht und so weiter, bis es entweder beim *root window* ankommt oder aber ignoriert wird.

Zusätzlich zu dieser Möglichkeit, die durch Festlegung von interessanten Ereignissen (Aufsetzen einer Ereignismaske eines Fensters) vertikal innerhalb der Hierarchie gesteuert wird, ist es für ein Fenster möglich, auch an Ereignisse von Nachbarfenstern (horizontal in der Hierarchie) durch sog. *grabbing* heranzukommen. Dies ist vorteilhaft, wenn man beispielsweise eine Linie in den verdeckten Teil eines Fensters hineinziehen will und das Linienziehprogramm den zweiten Punkt in seinem (verdeckten) Fenster benötigt, um die Eingabe abzuschließen. Nachteilig ist dies zur Datensicherheit: Ein fremder Prozess kann so eine Eingabe mithören, etwa ein Passwort!

Wie können wir nun eine Ereignisbehandlung für unsere Applikation definieren? Dazu muss die gewünschte Reaktion als Aktionsprozedur (**Callback-Routine**) programmiert und mit einer speziellen Prozedur *XtCallback()* zur Laufzeit eingehängt werden. Tritt nun ein Ereignis in *XtMainLoop()* auf, so wird das Ereignis (*event*) an die *event handler* der Fenster weitergereicht. Ein solcher *event handler* kann entweder selbst das Ereignis behandeln, oder aber er sieht in einer Tabelle (*translation table*) nach, welche Aktionsprozedur dafür aufzurufen ist. An dieser Stelle wird nun erst unsere Callback-Routine aufgerufen.

Ein Problem dieses Schemas besteht darin, dass bei einem Fehler in der Callback-Routine keine manuelle Korrektur durch den Benutzer möglich ist: Jede Eingabe kann nur in *XtMainLoop()* vorgenommen werden, was die Fortsetzung in der Callback-Routine verhindert. Eine Möglichkeit, das Problem zu umgehen, besteht darin, nur den Fehler zu notieren und die Callback-Routine zu beenden. An geeigneter Stelle im Programm muss dann auf eventuell aufgetretene Fehler abgefragt werden.

### 8.3.5 Sicherheitsaspekte

Das X-window-System ist historisch gewachsen und sehr groß. Wie alle derartigen Systeme ist der Code weder klar durchstrukturiert noch nachvollziehbar systematisch aufgebaut. Damit ergeben sich automatisch viele Sicherheitsprobleme. Da der X-Server mit *root*-Rechten läuft, ist dies auch kritisch und ein Einfallstor für viele Angriffe. Wie erwähnt, ist es beispielsweise leicht möglich, die Eingabe in ein Fenster eines anderen Prozesses abzugreifen, so dass Passwörter mitgelesen werden können. Ein anderes Beispiel ist die Implementation eines *screen savers* in Ubuntu, der lange Zeit und viele Implementationszyklen lang immer neue Lücken aufwies, mit denen der Rechner übernommen werden konnte, da das X-window – System keine Rechteverwaltung für Fenster besitzt.

Ein solches Rechtesystem nachträglich aufzusetzen bei einem historischen Code ist sehr schwierig und teuer. Aus diesem Grund gibt es starke Bestrebungen, ein neues Fenstersystem in der Linux-Welt einzuführen. Ein starker Kandidat dafür ist das Weyland-Protokoll, das neben dem Sicherheitsproblem auch andere graphischen Probleme wie etwa das Verschmieren, Verzögerungen und Flackern von bewegter Grafik sowie die Hardwarebeschleunigung beheben soll. Es regelt die Kommunikation zwischen einem *display server* („Wayland compositor“) und seinen *clients*. Dabei verwaltet der *display server* die Fenster: Er prüft, von welchem Fenster eine Eingabe kommt oder für welches Fenster

eine Ausgabe bestimmt ist, transformiert sie auf die internen Fensterkoordinaten, gibt die Daten weiter und veranlasst auch die nötigen Hardware-Displayoperationen dafür (Setzen der Auflösung, Farbtiefe, Bildwiederholrate etc.) via *ioctl*-Aufrufen an den Linux-Kern.

Im Unterschied zum X-Server ist bei Weyland der Client für die Darstellung der Fenster zuständig, etwa für die Schrift und die Widgets. Der zentrale *display server* wird so entlastet und der Umfang von Code mit *root*-Rechten vermindert. Außerdem wird die Ein- und Ausgabe für jedes Fenster extra und von den anderen Fenstern getrennt behandelt und so die Sicherheit vergrößert.

---

## 8.4 Das Fenstersystem von Windows NT

Obwohl Windows NT ein *multi-tasking*-System ist, ist es noch kein *multi-user*-System. Dies bedeutet, dass nicht mehrere Benutzer gleichzeitig auf einem Rechner arbeiten können. Konsequenterweise ist damit auch die Existenz von Stellvertreterprozessen bestimmter Benutzer, beispielsweise von Displayservern, ursprünglich nicht vorgesehen. Im Unterschied zu X-Window wird zurzeit in Windows NT keine verteilte Benutzeroberfläche unterstützt. Jede Applikation, die eine grafische Ausgabe vorsieht, muss sie auf ihrem lokalen Rechner durchführen. Eine Ausgabe auf einem anderen Rechner, etwa einem zentralen Display, ist nicht möglich. Will man trotzdem die Ausgabe verschiedener Prozesse unterschiedlicher Rechner auf einem zentralen Display erreichen, so muss auf diesem Rechner ein benutzerprogrammierter Displayprozess gestartet werden, der seine anwendungsabhängigen Daten über das Netzwerk (z. B. mittels *named pipes* oder anderen Kommunikationskonstrukten, siehe [Kap. 6](#)) holt und sie erst auf dem Displayrechner an das grafische System übergibt.

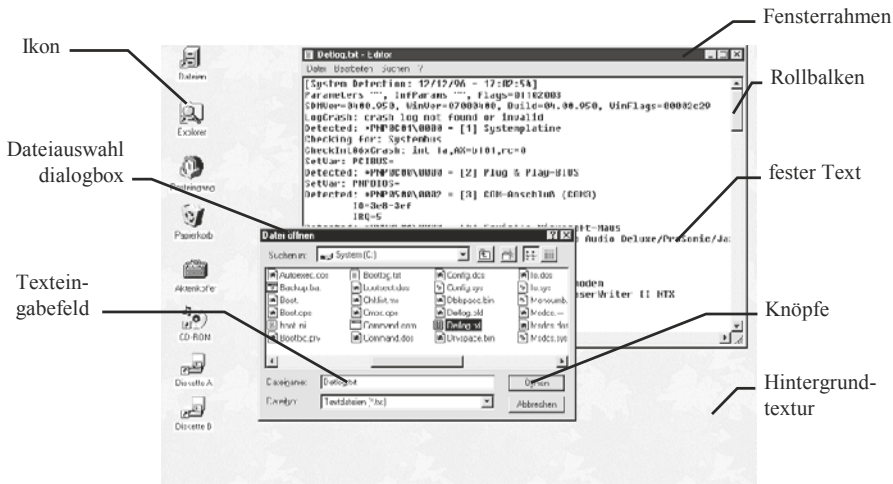
### 8.4.1 Das Konzept der Benutzerschnittstelle

Im Gegensatz zu X-Window kennt das Windows-NT-Fenstersystem nur wenige, komplexe Fensterarten. Im Wesentlichen werden zwei Arten von Fenstern unterschieden: **Hauptfenster** (*primary windows*) und **Nebenfenster** (*secondary windows*). In [Abb. 8.16](#) ist eine typische Konfiguration gezeigt.

Im Folgenden wollen wir nun einige grundsätzliche Interaktionstechniken betrachten, die von der Win32-API mit entsprechenden Objekten und Methoden unterstützt werden. Wie wir in [Abb. 8.16](#) sehen, hat jedes Hauptfenster in Windows NT (ebenso wie die *Motif*-Fenster von X-Window)

- einen *Rahmen*, der beim Anklicken zum Vergrößern/Verkleinern benutzt werden kann,
- eine *Titelleiste*, in der der Namen der Applikation stehen soll,
- ein *Ikon*, das die Applikation visualisiert,
- *Rollbalken*, falls Teile des Fensterinhalts über den Rahmen hinausgehen würden,





**Abb. 8.16** Beispiel einer fensterorientierten Interaktion in Windows NT

- eine applikationsspezifische *Menüleiste*,
- *Kontrollknöpfe* zum Minimieren, Maximieren und Schließen des Fensters,
- eine *Statusleiste*, auf der diverse Information angezeigt werden kann.

Die Fenster können sich gegenseitig überlappen und verdecken, wobei das aktive Fenster unverdeckt als „oberstes Fenster auf dem Stoß“ (oberstes Element in der Displayliste) mit seinem Rahmen in der speziellen „aktives Fenster“-Farbe gezeichnet ist.

Die Rollbalken werden auch hier an die relative Größe des Dokumentenausschnitts, den das Fenster bietet, in der Länge angepasst. Je kleiner der sichtbare Anteil der Daten im Fenster am Gesamtdokument ist, desto kürzer wird der Rollbalken gezeichnet.

Solche Hauptfenster haben noch weitere interessante Eigenschaften. So kann man auf verschiedene Weise Unterfenster darin definieren und verwenden. Im Unterschied zu X-Window gibt es dabei jedoch nur wenige Möglichkeiten.

Da auch bei Windows NT die datenzentrierte Sichtweise propagiert wird, soll man für jeden Datensatz (für jede Datei, jedes Dokument) ein eigenes Fenster (*child window*) eröffnen, wobei auf der Titelleiste jeweils nicht das Icon des Applikationsprogramms, sondern das der Daten erscheinen soll. Das Verhalten der Daten-Hauptfenster im Applikationsprogramm-Hauptfenster (*parent window*) wird durch die Multidokumentenschnittstelle (*Multi Document Interface MDI*) geregelt. Diese Schnittstelle sieht vor, dass ein Datenfenster (*child window*), das maximiert wird, sich mit seinen Außenmaßen den Innenmaßen des Applikationsfensters anpasst. Außerdem wird die Titelleiste des Datenfensters weggelassen; der Titel verschmilzt mit dem Applikationstitel im Format {Datentitel}-<Applikationstitel>, und das Icon und die Kontrollknöpfe der Daten-Titelleiste erscheinen in der Menüleiste der Applikation. In [Abb. 8.17](#) ist links Applikation und Datenfenster getrennt, rechts verschmolzen gezeigt. Die Datenfenster (Kindfenster) sind dabei direkt





**Abb. 8.17** Getrennte und verschmolzene MDI-Fenster

mit dem Applikationsfenster (Elternfenster) verbunden: Wird das Elternfenster geschlossen, so geschieht dies auch automatisch mit den Kindfenstern.

Gehören die Datenfenster zu einer einzigen Datei, so sollte eher die Möglichkeit genutzt werden, das Hauptfenster in Teilfenster (*panes*) zu zerteilen. Dazu kann man einen grafischen Knopf (*split box*) bei den Rollbalken vorsehen, dessen Betätigung eine Trennlinie (*split bar*) erscheinen lässt. Die Visualisierung der Daten in beiden Fensteranteilen ist aber Aufgabe der Applikation.

Im Unterschied zu den Hauptfenstern haben die Nebenfenster eine feste Größe und werden z. B. zum Darstellen interner Parameter (*property sheet window*) oder für Mitteilungen verwendet. Die Nebenfenster behalten ihre relative Lage innerhalb der Hauptfenster immer bei und sind fest an sie gekoppelt. Wird beispielsweise ein Hauptfenster zum aktiven Fenster und damit voll sichtbar, so wird auch sein Nebenfenster automatisch sichtbar. Beispiele für fertig aufrufbare Nebenfenster sind Dialogboxen zum Suchen und Ersetzen (*find/replace dialog box*), Drucken (*print dialog box*), Auswahl einer Schriftart (*font dialog box*), Auswahl einer Farbe (*color dialog box*), rechteckige Felder aus Funktionsknöpfen (*palette windows*) und Nachrichtenfenster (*message box*).

Das Win32-API ermöglicht über Hauptfenster und Nebenfenster hinaus eine Vielzahl verschiedener Dialogobjekte wie Kontrollknöpfe, exklusive Auswahlknöpfen (*option buttons*), nichtexklusive Auswahlknöpfe (*check buttons*), statische oder dynamische Auswahllisten (*list box*), die wie *pull-down*-Menüs arbeiten oder die Dateien wie ein Dateimanager mit Ikonen und Texten präsentiert (*list view control*). Außerdem gibt es noch statische und editierbare Textfelder und die zu Leisten zusammengesetzten Funktionsknöpfe (*toolbars*).

## 8.4.2 Die Implementierung

Die Benutzeroberfläche von Windows NT wurde weitgehend aus der älteren Oberfläche von Windows 3.1 entwickelt. Insbesondere wurde die alte 16-Bit-Schnittstelle zur Programmierung der grafischen Aus- und Eingabe (*Application Programming Interface API*) auf 32 Bit erweitert (*Win32 API*) und mit zusätzlichen Betriebssystemfunktionen versehen. Beispielsweise konnte nun der Programmierer auf ein sicheres, preemptives,

multithreaded multi-tasking Betriebssystem zugreifen, das über ein lineares 32-Bit-Speichersystem verfügt.

Zwar wurden möglichst alle Namen vom alten System übernommen, aber alle Zeiger und Prozeduren beziehen sich nicht mehr auf ein segmentbasiertes, sondern auf ein lineares, virtuelles Speichermodell. Zusätzliche Funktionen zur Synchronisation, zu Objektsicherheitsmechanismen und Speicherverwaltung erlauben den Zugriff auf große Teile des Betriebssystemkerns, der Windows-NT-Executive.

Das resultierende Win32-API-Modul wurde zunächst als eines der Subsysteme eingefügt, s. Abb. 1.9. Das *look-and-feel* der Oberfläche wurde dabei stark an das gewohnte Windows 3.1-Aussehen angepasst, um den Übergang auf NT für die Benutzer zu vereinfachen. Konsequenterweise wurde in Version 4.0 von Windows NT nicht nur die Funktionalität des Win32-API aus Effizienzgründen in den Betriebssystemkern verlegt, sondern auch das *look-and-feel* an Windows95, den Nachfolger von Windows 3.1, angepasst.

Das Win32-Subsystem gliedert sich in fünf Module, die mit der Win32-API angesprochen werden können, s. Abb. 8.18.

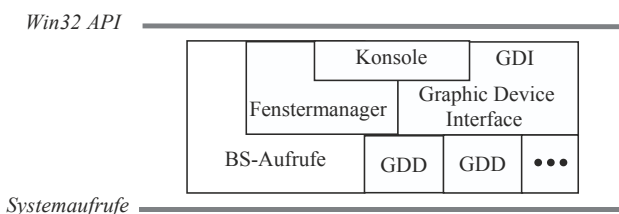
Dies sind im Einzelnen

- der Fenstermanager DWM (User32.dll),
- die grafische Schnittstelle *Graphic Device Interface GDI*,
- die grafischen Treiber *Graphic Device Driver GDD*,
- Betriebssystemfunktionen (Kernel32.dll),
- Konsolenfunktionen für allgemeine Textein- und -ausgabe.

Die Module sind in Bibliotheken (*Dynamic Link Library DLL*) enthalten, die das Subsystem beim Starten lädt.

Die **grafische Schnittstelle** GDI erlaubt das Zeichnen und die Manipulation einfacher und komplexer grafischer Objekte wie Punkte, Linien, Kreise, Fenster, Rollbalken usw. Die Funktionen setzen dabei auf der Funktionalität der grafischen Treiber auf, die ihrerseits wieder Treiber des Kerns aufrufen und so die Hardware steuern. Da jeder grafische Aufruf des Benutzerprogramms die Kette GDI-GDD-BS-Kern in Gang setzt und so Zeit kostet, wurde zur Effizienzsteigerung eine Technik namens „*attribute caching*“ eingesetzt: Mehrere Aufrufe des Benutzerprogramms, die alle das gleiche Objekt betreffen (z. B. Hintergrundfarbe wählen, Vordergrundfarbe wählen etc.), werden beim Anwenderprogramm gesammelt und als eine einzige Nachricht an die GDI weitergeleitet.

**Abb. 8.18** Die Struktur der Window-Subsystemschicht



Für die GDI-Bibliothek sorgt eine ähnliche Technik, das *batching*, für schnellere Reaktionen. Alle Funktionsaufrufe werden in einer Warteschlange so lange gepuffert, bis die Warteschlange voll ist oder der Benutzer Daten eingibt. Der gesamte Puffer wird dann in einer einzigen Nachricht dem Win32-Subsystem übergeben. Dieser Mechanismus ist so schnell, dass keine Ruckeleffekte usw. beim Grafikdisplay eines Standardrechners entstehen.

Der **Fenstermanager** *Desktop Window Manager DWM* vermittelt das *look-and-feel*, indem er die Fenstermechanik, also die Knöpfe für Vergrößern/Verkleinern der Fenster, Bedienung der Rollbalken, Verwaltung der Displayliste (welches Fenster verdeckt welches) usw. bedient sowie die Eingabe für ein Fenster in den Eingabepuffer des entsprechenden Programms stellt (vgl. [Abschn. 5.4](#)). Darüber hinaus stellt er noch allgemeine Ein- und AusgabeprozEDUREN zur Verfügung, um das Anwendungsprogramm unabhängig von den vorhandenen Hardwaregeräten (z. B. Maustypen, Grafikkarte) zu machen, etwa bei der Erzeugung von Transparenz bei Grafiken.

Da MS-Windows ursprünglich nur für Systeme mit wenig Speicher gedacht war, pufferte der Fenstermanager die unsichtbaren Fensterteile nicht, sondern übernahm die Aufgabe, einer Anwendung zu signalisieren, wenn das Fenster neu gezeichnet werden muss, da vorher unsichtbare Fensterteile nun sichtbar werden. Inzwischen wird seit Windows 7 der ebenfalls integrierte *Media Integration Layer* dazu genutzt, um die Grafikdarstellung komplett unabhängig von den Anwendungen und dem Displaybuffer zu puffern (*double buffering*) und zu managen (*retained mode*). Der DWM entscheidet also nun, welches Fenster und was davon zu sehen ist, und nicht mehr die Anwendung. Auch die Verwaltung eines systemweiten Zwischenspeichers für Objekte (*clip board*) wird von ihm durchgeführt.

Die **Konsolenfunktionen** sind zentrale Funktionen zur zeichenweisen Ein- und Ausgabe und werden von allen Subsystemen (POSIX, OS/2, MS-Windows usw.) benutzt. Die Ein- und Ausgabe der Texte, die von alten Programmen, die nur zeichenorientierte und nicht grafikunterstützte Benutzerschnittstellen besitzen, wird dabei zweckmäßigerweise in besonderen Fenstern („Konsolen“) durchgeführt, die vom System automatisch geöffnet werden.

### 8.4.3 Aufgaben

#### Aufgabe 8-4.1 Client-Server-Architektur

Welche Vor- und Nachteile hat das lokale Konzept von Windows NT gegenüber dem verteilten X-Window-System? Denken Sie dabei an Applikationen wie die Prozesssteuerung eines Stahlwerks mit einem Rechnernetz, die Datenauswertung (*data mining*) verschiedener Abteilungen über das Intranet eines Betriebs, die Softwareinstallation und Wartung für einen vernetzten Rechnerpool, usw.

## Literatur

- Fährnrich K., Janssen C., Groh G.: Werkzeuge zur Entwicklung graphischer Benutzungsschnittstellen. Oldenbourg Verlag, München 1996
- Foley J. D., van Dam A.: Fundamentals of Interactive Computer Graphics. Addison-Wesley, Reading, MA 1982
- Gottheil K., Kaufmann H.-J., Kern, Zhao R.: X und Motif. Springer-Verlag, Berlin 1992
- Madell T., Parsons C., Abegg J.: Developing and Localizing International Software. Prentice Hall, Englewood Cliffs, NJ 1994
- Microsoft: The Windows Interface Guidelines for Software Design. Microsoft Press, Redmond, WA 1995
- Shneiderman B.: Designing The User Interface. Strategies for Effective Human-Computer Interaction; Addison-Wesley, Reading, MA 1987
- UNI: siehe <http://www.unicode.org> im World Wide Web

Inhaltsverzeichnis

9.1 Lösungen zu Kap. 1 ..... 381

9.2 Lösungen zu Kap. 2 ..... 386

9.3 Lösungen zu Kap. 3 ..... 416

9.4 Lösungen zu Kap. 4 ..... 428

9.5 Lösungen zu Kap. 5 ..... 439

9.6 Lösungen zu Kap. 6 ..... 448

9.7 Lösungen zu Kap. 7 ..... 455

9.8 Lösungen zu Kap. 8 ..... 460

Literatur ..... 463

9.1 Lösungen zu Kap. 1

Aufgabe 1-1 Betriebssystem

*Der Zweck eines Betriebssystems besteht beispielsweise in der Verteilung von Betriebsmitteln auf sich bewerbende Benutzer.<sup>1</sup>*

- a) *Wie ist der grobe Aufbau eines Betriebssystems?*
- Wesentliche Bestandteile eines Betriebssystems sind
    - Prozessorverwaltung
    - Prozessverwaltung
    - Speicherverwaltung
    - Geräteverwaltung.

<sup>1</sup> verkürzt für Benutzerinnen und Benutzer

- Weiterhin ergänzen Dienstprogramme das Betriebssystem. Hier finden sich Lader, Compiler, Binder, Editoren, Systemprogramme, Sortierprogramme.  
Eine andere Sichtweise ist durch die Schichten Hardware, Kernel, Anwendung und Benutzer gegeben.
  - Den Zugriff auf die Hardware ermöglicht der Betriebssystemkern (Kernel). Die Dienste werden über Systemaufrufe angefordert.
  - In der Anwendungsschicht befinden sich elementare Dienstprogramme, z. B. Compiler und Systemprogramme.
  - Die Benutzerschicht bilden diverse Benutzerprogramme, etwa Textverarbeitung, Zeichenprogramme usw.
- b) *Welche Betriebsmittel kennen Sie?*  
Betriebsmittel sind die Menge aller Hard- und Softwarekomponenten eines Rechnersystems, die zur Ausführung und Steuerung von Programmen benötigt werden. Dies sind beispielsweise
- Prozessor
  - Geräte: Speicher, Drucker, Monitor
  - Dateien
  - Compiler, weitere Dienstprogramme.
- c) *Welche Benutzer könnten sich bewerben? (Dabei ist der Begriff „Benutzer“ allgemein gefasst!)*  
Außer einem oder mehreren menschlichen Benutzern können sich alle möglichen Prozesse (Benutzer- oder Systemprozesse) um Betriebsmittel bewerben.
- d) *Welche Anforderungen stellt ein menschlicher Benutzer an das Betriebssystem?*  
Ein menschlicher Benutzer erwartet eine schnelle Antwortzeit. Wenn er z. B. ein Zeichen per Tastatur eingibt, möchte er es nahezu sofort auf dem Bildschirm sehen. Ein Systemadministrator ist mehr am Durchsatz, also der geleisteten Arbeit pro Zeit interessiert, nicht so sehr an der Antwortzeit. Auch Zuverlässigkeit, Datensicherheit (*Backup, Recovery* im Fehlerfall) und Datenschutz bei Mehrbenutzerbetrieb (Zugriffsberechtigungen) sind wünschenswerte Eigenschaften. Weiterhin ist eine gut gestaltete grafische Oberfläche heutzutage Standard.

### Aufgabe 1-2 UNIX System calls

- a) *Finden Sie heraus, wie viele System Calls auf einem aktuellen x86\_64-Linux-System definiert sind. Beispielsweise können Sie dazu die entsprechenden Header-Dateien, mit denen der Kernel übersetzt wurde, inspizieren. Auf vielen Rechnern finden Sie diese im Verzeichnis `/lib/modules/[aktuelle Kernelversion]/build`, wobei Sie die [aktuelle Kernelversion] in einem Terminalfenster mit den Befehl `uname -r` herausfinden können.*  
*Alternativ können Sie auch eine aktuelle Linux-Distribution in der 64-Bit-Variante, wie beispielsweise Ubuntu (<http://www.ubuntu.com/>) in einer virtuellen Maschine installieren. Beachten Sie jedoch, dass unter Umständen das Paket `linux-headers-generic` installiert werden muss, damit die Header-Dateien bereitstehen.*

*Geben Sie die Anzahl der definierten Systemaufrufe an und erläutern Sie, wie Sie auf diese gekommen sind.*

Die Tabelle ist unter `/lib/modules/[aktuelle Kernelversion]/build/arch/x86/include/generated/asm/` in der Datei `syscalls_64.h` zu finden. Insgesamt existieren darin 342 Einträge, also 342 System Calls. `syscalls_32.h` enthält die Tabelle für 32-Bit-Systemaufrufe, darin gibt es nur 337 Einträge.

Die Zahl kann ferner leicht variieren, da manche *system calls* beim Kompilieren deaktiviert worden sein könnten und die verfügbaren *system calls* von der Kernelversion abhängen.

- b) *In welche Funktionsgruppen lassen sie sich einteilen?*

Speicher-, Dateiverwaltung, I/O, Prozess- und Signalbehandlung, Verwaltung der Ein- und Ausgabegeräte.

- c) *Welcher Assemblerbefehl sorgt bei x86-Prozessoren für die Auslösung der Trap bei einem Systemaufruf, und wie wird dabei die Information übergeben, welcher Systemaufruf ausgeführt werden soll?*

**Unix-artige** Systeme (Unix, Linux, OS X, etc.) nutzen traditionell Interruptvektor 0x80. Hier sind die System Call-Nummern über die verschiedenen Versionen stabil. Kompatibilitätsbrechende Änderungen (also Ändern der Nummer oder gar Entfernen von System Calls) gibt es nur *sehr* selten, weshalb diese Variante funktioniert. (Trotzdem verwenden viele Programme stattdessen eine Library, die die System Calls auf high-level-Funktionen abstrahiert, was einfach bequemer ist.)

**Linux** hat drei Möglichkeiten:

- `int 0x80` für 32-Bit-Programme, älteste Variante.
- `sysenter` für 32-Bit-Programme, neuere Variante. Hier wird von der Hardware einiges an Overhead gegenüber `int 0x80` vermieden, was für schnellere Systemaufrufe sorgt.
- `syscall` für 64-Bit-Programme.

Unter **MS-DOS** wird stattdessen `int 0x21` (neben ein paar anderen, weniger wichtigen) benutzt.

Auf **Windows** sieht das anders aus: Hier werden explizit keine über die verschiedenen Windowsversionen stabilen System-Call-Nummern garantiert (vergleiche <http://j00ru.vexillum.org/ntapi/>: die Nummern ändern sich über die Releases). Man kann zwar via `int 0x2e` bzw. `sysenter` mit System Call-Nummer in Register `eax` einen System Call initiieren, aber portabel ist das wegen erwähnter Nummerninkonsistenz nicht.

Deshalb werden für den normalen Gebrauch sog. *Wrapper* benutzt: jeder Prozess bekommt gezwungenermaßen die `kernel32.dll` in den Adressraum geladen, die Funktionen bereitstellt, die sich um das Aufrufen der System Calls (mit den richtigen Nummern) kümmern. Viele Calls werden davon allerdings auch einfach weiterdelegiert an die `ntdll.dll`, die die Low-Level System Calls übernimmt (`kernel32.dll` kümmert sich um die Win32-API). Daher kann man, wenn man will, auch direkt über `ntdll.dll` seine System Calls absetzen, wenn man diese dynamisch einbindet. Unter der Haube passiert dabei aber auch nichts anderes, als dass die

entsprechenden Nummern in `eax` (bzw. `rax` bei 64 Bit) sowie die Argumente in die übrigen Register gelegt werden und `int 0x2e, sysenter` bzw. `syscall` ausgeführt werden, um in Ring 0 zu wechseln.

Zu den Quellen darüber: Zum einen findet man Informationen bei <http://www.vivid-machines.com/shellcode/shellcode.html>, zum anderen bei <http://www.hick.org/code/skape/papers/win32-shellcode.pdf>. Es scheint sich tatsächlich kaum jemand damit zu beschäftigen, unter Windows in *asm* manuell System Calls auszulösen, weshalb die einzigen Informationen dazu sich um Exploitprogrammierung kümmern...

Die Information, *welcher* Systemaufruf ausgeführt werden soll, wird bei 32-Bit im Register EAX, bei 64-Bit in RAX (64-Bit-Erweiterung von EAX) als Zahl übergeben. (Zumindest bei Linux/UNIX/OSX und Windows in diesem Register; wahrscheinlich bei allen auf x86 wegen ABI von *sysenter*, *syscall* etc.)

### Aufgabe 1-3 Schichtenmodell und Kernel

- a) *Beschreiben Sie mit Hilfe eines Beispiels das Schichtenmodell. (Das Beispiel sollte dabei kein Betriebssystem sein.)*

#### **Beispiel:** Digitales Telefon

Beim Telefon gibt es drei Schichten: Die Benutzeroberfläche, die eine elektronische Funktionsschicht benutzt, und die benutzte Hardware.

Die Benutzeroberfläche hat als Schnittstelle verschiedenen Tasteneingaben (Nummertaste, Sprechaste, Rückholtaste etc.) sowie ein Display.

Die Funktionsschicht hat als Importschnittstelle von oben die Tasten und Sensoren der Benutzeroberfläche, als Export nach oben die Displaydarstellung und den Hörer.

Die Leistungen der Hardware zur Funktionserfüllung werden von unten nach oben exportiert, also Rufnummernaufbau, Kommunikation, Kodierung und Dekodierung der Sprache durch die Chips. Als Importschnittstelle (und Export der Funktionsschicht) sind die Befehle zum Nummernaufbau bzw. Sprachaktivierung vorhanden.

- b) *Erläutern Sie kurz die Begriffe „virtuelle Maschine“ und „Schnittstelle“.*

Eine „virtuelle Maschine“ VM ist eine Einheit, die gewisse, festgelegte Dienstleistungen zur Verfügung stellt (Export). Diese Dienstleistungen werden durch eigenen Arbeit und Anfordern von Dienstleistungen aus untergeordneten VM oder auch physikalischen Maschinen erbracht (Import). Da die Dienstleistungen nicht alle von der Maschine selbst erbracht werden, nennt man sie „virtuelle Maschine“.

Das Verhalten der VM wird nur durch die „Schnittstellen“ spezifiziert, so dass die genaue Implementierung der VM für die benutzende Schicht transparent (also unsichtbar und verborgen) ist. Eine Schnittstelle besteht dabei aus Daten, den benutzten Funktionen bzw. Methoden und die Art und Weise, wie diese Funktionen zu benutzen sind (Protokolle).